

2nd
Edition

Learning Go Programming

Explore automation and data analysis with Golang



Shubhangi Agarwal



2nd
Edition

Learning Go Programming

Explore automation and data analysis with Golang



Shubhangi Agarwal

bpb

Learning Go Programming

2nd Edition

*Explore automation and data analysis
with Golang*

Shubhangi Agarwal



www.bpbonline.com

Second Revised and Updated Edition 2025

First Edition 2021

Copyright © BPB Publications, India

ISBN: 978-93-65890-051

All Rights Reserved. No part of this publication may be reproduced, distributed or transmitted in any form or by any means or stored in a database or retrieval system, without the prior written permission of the publisher with the exception to the program listings which may be entered, stored and executed in a computer system, but they can not be reproduced by the means of publication, photocopy, recording, or by any electronic and mechanical means.

LIMITS OF LIABILITY AND DISCLAIMER OF WARRANTY

The information contained in this book is true to correct and the best of author's and publisher's knowledge. The author has made every effort to ensure the accuracy of these publications, but publisher cannot be held responsible for any loss or damage arising from any information in this book.

All trademarks referred to in the book are acknowledged as properties of their respective owners but BPB Publications cannot guarantee the accuracy of this information.

To View Complete
BPB Publications Catalogue
Scan the QR Code:



www.bpbonline.com

Dedicated to

To my pillars,

My father and mother,

Dinesh Chandra Agarwal

and

Pushpa Agarwal

About the Author

Shubhangi Agarwal is a passionate technology enthusiast and currently serves as a Technical Lead at Capgemini. With a rich professional background, she has previously contributed her expertise at esteemed organizations like TCS & IBM. She holds a B.Sc. degree from Kumaun University and an MCA from Gurukul Kangri University, showcasing her strong academic foundation.

Her journey into technical content creation began with blogging, which later blossomed into authoring her first book, *Learning Go Programming*, published by BPB Publications in 2020. She continues to share her knowledge and insights through her website and her YouTube channel, empowering others to excel in the dynamic world of technology.

Beyond her professional life, she enjoys cooking, exploring new destinations, and cherishing moments with her family and friends, reflecting her vibrant and balanced personality.

About the Reviewers

- **Eray Ates** is a software developer specializing in Go programming, and distributed systems. With a degree in Electrical-Electronic Engineering with a Master's degree in Big Data and Business Analytics, he discovered his true passion in telling computers what to do.

Eray's career journey includes a role as a Software Developer at Huawei Enterprise, where he contributed to the development of enterprise-level software solutions. After this he gained valuable hands-on experience in the intricacies of embedded systems as an Embedded Software Engineer at Rebuslabs. He then expanded his skillset as a Full Stack Engineer at Siemens, gaining a comprehensive understanding of the entire software development lifecycle.

Currently working at Worldline Global as a Go Developer, Eray spends his days building microservices, fine-tuning distributed systems, and ensuring APIs talk to each other without starting an argument. When he is not immersed in code, he enjoys mentoring developers, contributing to technical communities.

- **Deev Chandra** is a distinguished Software Engineer professional with over 8 years of experience in the active computer science industries. He is passionate about adaptive back-end development with a strong track record in creating efficient, scalable, and reliable product services. Along with proficiency in Golang, he also has pragmatic experience in data engineering, using data to drive business decisions.

As a Senior Software Engineer at Cisco, Deev is currently focused on NPI AI strategies, innovative contributions, and enhancing his expertise in SaaS, being an integral part of Cisco Unified Computing System. He aims to deepen his expertise in software engineering to tackle complex problems and is well known for his versatility and creativity.

- **Satish Prahalad Gururajan** is a passionate and highly skilled Solutions Architect with nearly two decades of professional experience in the tech industry. He has a strong foundation in application development, with a focus on Java technologies, and has played pivotal roles in software design, development, and delivery. Over the course of his career, Satish has excelled in diverse positions, ranging from developer to architect, product owner, and designer, which has allowed him to build a comprehensive understanding of both technical and business needs. Satish's expertise goes beyond development; he has consistently demonstrated a strong ability to bridge the gap between technical teams and business stakeholders, ensuring that solutions align with both strategic goals and user requirements. His experience spans a variety of industries, and he has a proven track record of successfully leading teams through the development lifecycle, from ideation and design to implementation and deployment. As a certified professional, Satish holds multiple industry-recognized certifications, including SAFe® Product Owner/Product Manager, CSPO® (Certified Scrum Product Owner), and CSM® (Certified Scrum Master), which have honed his skills in agile methodologies and product management. These certifications have not only enriched his technical acumen but also enhanced his leadership and project management capabilities, enabling him to guide teams in delivering high-quality software in a collaborative and efficient manner. Satish is especially passionate about problem-solving and has a natural talent for distilling complex technical concepts into easily understandable terms. His ability to simplify intricate topics allows him to communicate effectively with both technical and non-technical audiences.

Acknowledgement

I would like to express my sincere gratitude to everyone who has contributed to the creation of this second edition of ‘Learning Go Programming’.

First and foremost, I am deeply grateful to my parents whose love and wisdom formed the constant foundation for everything I have learned and created. A special thanks to my four wonderful nephews — *Gaurang*, *Sahaj*, *Aarush* and *Amay* — who, with much enthusiasm, insisted that I add their names to this acknowledgment. Your joy, energy, and laughter have been a source of inspiration throughout this process, and I am delighted to include you in this work.

My sincere thanks go to my technical reviewers, whose insights, feedback, and suggestions have significantly improved the quality and accuracy of this book. Your attention to detail and dedication to excellence made this second edition stronger than the first.

I also want to acknowledge the editor and the team at BPB for their professionalism and hard work in ensuring this edition is ready for publication. Your guidance and support throughout both editions have been indispensable.

Finally, to the readers of the first edition—thank you for your feedback and continued interest. Your insights have driven the improvements in this new edition, and I hope it continues to meet your needs and expectations.

Preface

Welcome to the second edition of this comprehensive guide to **Go Programming**. In this updated edition, we have expanded the content to provide a deeper exploration of Go and its practical applications. This edition introduces **advanced topics** relevant to modern software development, including **Web Scraping, Automation, Data Analysis, Machine Learning**, and **CI/CD pipelines**. These additions reflect Go's versatility and growing importance across diverse fields, from cloud services and data science to automation and continuous deployment.

Throughout the book, we continue to emphasize real-world examples, **best practices**, and hands-on techniques, providing you with the tools to use Go effectively in a wide range of projects. Whether you are working on concurrency, error handling, machine learning, or building scalable applications, this book is designed to help you deepen your understanding and gain practical skills.

With detailed coverage of Go's core features, such as **concurrency, reflection, interfaces**, and **error handling**, alongside powerful new chapters on **CI/CD, data analysis**, and **automation**, this edition provides you with everything you need to take your Go programming skills to the next level.

I hope you find this edition both engaging and informative as you continue your journey with Go. Whether you are just starting or looking to enhance your existing knowledge, this book will guide you through building efficient, high-performance applications.

Happy coding, and enjoy exploring Go programming!

General introduction to the book:

Chapter 1: Introduction to Go - This chapter introduces the Go programming language, exploring its evolution, core characteristics, and the factors behind its success. We examine Go's object-oriented features and draw analogies with other languages to highlight its unique approach. The chapter also addresses some of the pain points of Go and discusses its diverse applications in fields like cloud computing, microservices, and web development.

Chapter 2: Environment Setup - In this chapter, we walk you through the process of setting up your Go development environment. You will learn about system requirements, explore different text editors and cloud-based IDEs, and get hands-on instructions for installing and uninstalling the Go compiler to ensure a smooth setup.

Chapter 3: Beginning with Go - This chapter introduces you to the basics of writing a Go program. We cover the structure of a simple Go program, the key language keywords, and how identifiers are used to name variables, functions, and other elements within your code.

Chapter 4: Variables, Data Types and Constants - This chapter covers Go's **variables**, **data types**, and **constants**. You will learn how to declare and initialize variables, understand Go's type system, and work with basic and complex data types. It also explains how constants are used for fixed values and introduces format specifiers for output formatting.

Chapter 5: Operators - This chapter covers **operators** in Go, focusing on their role in performing operations on values. We explore different **types of operators** such as arithmetic, relational, logical, and bitwise. Additionally, we discuss operator precedence and how the order of evaluation affects expressions.

Chapter 6: Control Structures - This chapter introduces control structures in Go, which guide the flow of program execution. We cover branching statements like **if**, **else**, and **switch**, allowing decisions based on conditions. The chapter also explores looping statements such as **for** for repeated tasks, and jumping statements like **break**, **continue** which control the flow within loops and blocks.

Chapter 7: Functions - This chapter explores functions in Go, covering their declaration, arguments, and return types. We discuss **call by value** and **call by reference**, along with **named return values** and **anonymous functions**. The chapter also introduces **closures**, **recursive functions**, the **defer** keyword, and **variadic functions**, which handle a variable number of arguments, offering a comprehensive understanding of function usage in Go.

Chapter 8: Packages in Go - This chapter covers packages in Go, starting with an introduction to their role in organizing code. We discuss the main package and the widely used `fmt` package for formatted I/O. You will learn how to import packages, create custom packages, and work with nested packages. The chapter also explores installing third-party packages and handling circular imports and highlights some of the top Go libraries that enhance development.

Chapter 9: Arrays and Slices - This chapter explores **arrays** and **slices** in Go. We begin with arrays, covering their declaration, initialization, and use of multidimensional arrays. You will then learn about slices, their declaration, initialization, and how to create slices from arrays, other slices, and structs. The chapter also delves into slice internals, using the `make` function for slices, and working with two-dimensional slices. Finally, we discuss various **slice functions** to effectively manipulate slices in Go.

Chapter 10: Strings - This chapter introduces **strings** in Go, starting with their basic usage and the importance of the `rune` data type for representing characters. We explore how to handle multiline strings and leverage the **strings package** to manipulate and work with strings effectively. The chapter also covers key functions in the `strings` package, providing you with the tools to perform common string operations such as searching, replacing, and splitting.

Chapter 11: Pointers - This chapter introduces **pointers** in Go, explaining their role in directly accessing and manipulating memory. We discuss the **need for pointers**, how to **declare** and **initialize pointers**, and how to dereference them to access the values they point to. The chapter also covers advanced pointer concepts like creating pointers with the `new` keyword, working with **nested pointers**, and using an array of pointers. Finally, we

explore how to **pass pointers to functions** for efficient memory handling and manipulation.

Chapter 12: Structures - This chapter introduces structures in Go, which allow you to group related data together. We begin with declaring and initializing structures, followed by how to access and modify their fields. The chapter covers using pointers to structures, creating **structs with a new function**, and handling anonymous struct fields. We also explore nested structures, passing structures to functions, and the process of **copying structs**. Finally, we discuss how to compare structs for equality in Go.

Chapter 13: Composition - This chapter introduces **composition** in Go, a key concept for building flexible and reusable code. We explore how to **embed pointers** and **slices of structs** within other structs to create complex data structures. By leveraging composition, you can enhance code modularity and reuse, promoting cleaner and more maintainable designs.

Chapter 14: Interfaces and Polymorphism - This chapter explores **interfaces** and **polymorphism** in Go, fundamental concepts for writing flexible and reusable code. We cover how to declare and implement interfaces, including the use of the **empty interface** for any type. The chapter also discusses implementing multiple interfaces, **type assertion**, and **type switch** to handle dynamic types.

Chapter 15: Maps - This chapter covers **maps** in Go, a powerful data structure for storing key-value pairs. We begin with an introduction to maps, followed by how to **declare** and **initialize** them. The chapter explores various **map operations**, such as adding, updating, and deleting key-value pairs. We also highlight how **maps are reference types**, meaning changes to one map affect others that share the same reference.

Chapter 16: Concurrency with Go - This chapter introduces concurrency in Go, an essential concept for writing efficient, multitasking programs. We begin by differentiating parallelism versus concurrency, helping you understand their roles in concurrent programming. The chapter covers the use of goroutines to run functions concurrently and how to manage their execution. Finally, we explore techniques for waiting for goroutines to finish, ensuring proper synchronization and handling of concurrent tasks.

Chapter 17: Mutex and Channels - This chapter covers **channels** and **mutexes** for managing concurrency in Go. We discuss declaring channels, performing **channel operations**, and avoiding deadlock and race conditions. The chapter introduces the **select** keyword for handling multiple channels, methods to check for data races, and how to use mutexes to protect shared resources in concurrent programs.

Chapter 18: Error Handling - This chapter explores **error handling** in Go, focusing on Go's unique approach to errors. We begin with an introduction to error types and the **errors package**, which provides tools for error management. The chapter also covers the use of the **defer** statement for delayed execution and how **panic** and **recovery** mechanisms handle critical errors and ensure program stability.

Chapter 19: Reflection - This chapter introduces **reflection** in Go, a powerful feature for inspecting and manipulating types at runtime. We explore the **reflect package** and its core functions, such as **Type**, **Value**, and **Kind**, to examine types and values. The chapter also covers **NumField()** and **Field** for accessing struct fields, as well as **String()**, **Int()**, and **Float()** for converting values. Finally, we discuss the **laws of reflection** that govern its usage in Go.

Chapter 20: Web Scraping in Go - This chapter covers **web scraping** in Go, focusing on how to extract data from websites. We introduce popular web scraping libraries in Go, with a detailed overview of the **Colly** library, known for its simplicity and efficiency. The chapter guides with **headless scraping** without a browser UI.

Chapter 21: Automation with Golang - This chapter explores **automation** in Go, highlighting its power in streamlining various tasks. We cover key areas of automation, including **Infrastructure as Code (IaC)**, **container orchestration**, and **workflow automation** using Go scripts. The chapter also delves into using Go for cloud services development, network automation, and testing, as well as its role in **CI/CD** pipelines for continuous integration and deployment.

Chapter 22: Data Analysis and Machine Learning - This chapter introduces **machine learning** and **data science** with Go, highlighting its applications in the field. We discuss key aspects like **data handling** and

manipulation, as well as performing **Exploratory Data Analysis (EDA)** in Go. The chapter explores the **machine learning ecosystem** in Go, featuring popular Go machine learning libraries. Additionally, we cover techniques for **time series analysis**, demonstrating how Go can be used for data-driven insights and building machine learning models.

Chapter 23: Build CI/CD pipeline with Golang - This chapter focuses on building a **CI/CD pipeline** using Go, streamlining the development and deployment process. We begin with an introduction to CI/CD and the Golang project structure. The chapter guides you through setting up the CI/CD environment and integrating it with Go projects. We explore automated testing within pipelines, effective deployment strategies for Go applications, and the importance of **monitoring** and **logging** for maintaining application performance and stability in production environments.

Chapter 24: Wrap-up and Takeaways - This chapter recaps the **key learnings** from the Go programming course, highlighting essential concepts and **common pitfalls**. We discuss practical applications of Go and provide guidance on the **next steps** to advance your skills and explore more advanced topics.

Code Bundle and Coloured Images

Please follow the link to download the
Code Bundle and the *Coloured Images* of the book:

<https://rebrand.ly/a0b8c7>

The code bundle for the book is also hosted on GitHub at <https://github.com/bpbpublications/Learning-Go-Programming-2nd-Edition>. In case there's an update to the code, it will be updated on the existing GitHub repository.

We have code bundles from our rich catalogue of books and videos available at <https://github.com/bpbpublications>. Check them out!

Errata

We take immense pride in our work at BPB Publications and follow best practices to ensure the accuracy of our content to provide with an indulging reading experience to our subscribers. Our readers are our mirrors, and we use their inputs to reflect and improve upon human errors, if any, that may have occurred during the publishing processes involved. To let us maintain the quality and help us reach out to any readers who might be having difficulties due to any unforeseen errors, please write to us at :

errata@bpbonline.com

Your support, suggestions and feedbacks are highly appreciated by the BPB Publications' Family.

Did you know that BPB offers eBook versions of every book published, with PDF and ePub files available? You can upgrade to the eBook version at www.bpbonline.com and as a print book customer, you are entitled to a discount on the eBook copy. Get in touch with us at :

business@bpbonline.com for more details.

At www.bpbonline.com, you can also read a collection of free technical articles, sign up for a range of free newsletters, and receive exclusive discounts and offers on BPB books and eBooks.

Piracy

If you come across any illegal copies of our works in any form on the internet, we would be grateful if you would provide us with the location address or website name. Please contact us at business@bpbonline.com with a link to the material.

If you are interested in becoming an author

If there is a topic that you have expertise in, and you are interested in either writing or contributing to a book, please visit www.bpbonline.com. We have worked with thousands of developers and tech professionals, just like you, to help them share their insights with the global tech community. You can make a general application, apply for a specific hot topic that we are recruiting an author for, or submit your own idea.

Reviews

Please leave a review. Once you have read and used this book, why not leave a review on the site that you purchased it from? Potential readers can then see and use your unbiased opinion to make purchase decisions. We at BPB can understand what you think about our products, and our authors can see your feedback on their book. Thank you!

For more information about BPB, please visit www.bpbonline.com.

Join our book's Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

<https://discord.bpbonline.com>



Table of Contents

1. Introduction to Go

- Introduction
- Structure
- Objectives
- Software essence
- Evolution of Go and its success
- Characteristics of Go
- Object-oriented nature
- Analogy with other languages
- Pain points of Golang
- Applications of Go language
- Conclusion
- Questions

2. Environment Setup

- Introduction
- Structure
- Objectives
- System requirements
- Text editors*
- Cloud-based IDEs*
- Compiler*
- Installing Go

Uninstalling Go

Conclusion

Questions

3. Beginning with Go

Introduction

Structure

Objectives

Introduction to Go program

Keywords

Identifiers

Conclusion

Questions

4. Variables, Data Types and Constants

Introduction

Structure

Objectives

Variables

Data types

Constants

Format specifiers

Conclusion

Questions

5. Operators

Introduction

Structure

Objectives

Understanding operators

Types of operators

Operator precedence

Conclusion

Questions

6. Control Structures

Introduction

Structure

Objectives

Introduction to control structures

Branching statements

Looping statements

Jumping statements

Conclusion

Questions

7. Functions

Introduction

Structure

Objectives

Introducing functions

Declaring functions

Function arguments and return types

Calling a function

Call by value

Call by reference

Named return values

Blank identifier

Using a function as a type

Anonymous functions

Return anonymous functions

Closures

Recursive functions

Defer keyword

Variadic function

Conclusion

Questions

8. Packages in Go

Introduction

Structure

Objectives

Introduction to packages

Main package

fmt package

Importing a package

Creating a custom package

Nested packages

Installing third-party package

Circular import

Top Go libraries

Conclusion

Questions

9. Arrays and Slices

Introduction

Structure

- Objectives
- Introduction to array
- Declaring an array
- Initializing an array
- Need for an array*
- Multidimensional arrays
- Functions of an array
- Length of an array*
- Array comparison*
- Copying an array*
- Passing an array to a function*
- Iteration over an array
- Introduction to slices
- Declaring slices
- Initializing slices*
- Slice from slice
- Slice from structs
- Slice internals
- Slice using the make function
- Extending slices*
- Two-dimensional slices
- Slice functions
- Sort a slice*
- Compare slices*
- Searching in slice*
- Reverse slice*
- Appending slice element*
- Conclusion
- Questions

10. Strings

Introduction

Structure

Objectives

Introduction to strings

Functions with strings

Rune data type

Iterate using range keyword

Multiline string

Converting a slice of runes to string

Converting a slice of bytes to string

Immutable nature of the string

Strings package

Functions with strings package

Conclusion

Questions

11. Pointers

Introduction

Structure

Objectives

Introduction to pointers

Need for pointers

Declare pointers

Initializing pointers

Dereferencing pointers

Manipulating pointers

Creating pointers with new

Nested pointers
Array of pointer
Passing pointers to a function
Conclusion
Questions

12. Structures

Introduction
Structure
Objectives
Introducing structures
Declaring struct
Initializing struct
Accessing struct's field
Pointer to struct
Struct with a new function
Anonymous struct fields
Nested struct and embedded struct
Passing structs to functions or copying structs
Comparing structs
Conclusion
Questions

13. Composition

Introduction
Structure
Objectives
Introducing composition
Embedding pointers

Embedding slice of structs

Conclusion

Questions

14. Interfaces and Polymorphism

Introduction

Structure

Objectives

Introduction to interface and polymorphism

Declaring interface

Implementing interface

Empty interface

Implementing multiple interfaces

Type assertion

Type switch

Embedding interfaces

Interfaces as polymorphism

Conclusion

Questions

15. Maps

Introduction

Structure

Objectives

Introduction to maps

Use of maps

Declaring a map

Initializing a map

Map operations

Length of map

Check for keys

Iteration over maps

Delete an element from the map

Comparison of maps

Map as a reference type

Conclusion

Questions

16. Concurrency with Go

Introduction

Structure

Objectives

Introduction to concurrency

Parallelism versus concurrency

Thread

Goroutines

Multiple Goroutines

Goroutines at multiple cores

Waiting for Goroutine to finish

Conclusion

Questions

17. Mutex and Channels

Introduction

Structure

Objectives

Declaring channels

Channel operations

Deadlock

Using multiple Goroutines

Closing a channel

Direction of a channel

Select keyword

Data race in Go

Race condition

Check for data race

Mutex

RWMutex

Conclusion

Questions

18. Error Handling

Introduction

Structure

Objectives

Introduction to Golang errors

Error types

Error package

Defer statement

Panic

Recovery

Conclusion

Questions

19. Reflection

Introduction

Structure

Objectives

Introduction to reflection

Reflect package

Type

Value

Kind

NumField()

Field

String(), Int() and Float()

Laws of reflection

Conclusion

Questions

20. Web Scraping in Go

Introduction

Structure

Objectives

Introduction to web scraping

Web scraping

Web Scraping libraries in Go

Overview of Colly

Scrap websites in Go

Get scraped data to CSV

Headless scraping

How to avoid blocking

Conclusion

Questions

21. Automation with Golang

Introduction

- Structure
- Objectives
- Introduction to automation
- Automation key areas
- Infrastructure as Code*
 - IaC with Golang*
- Container orchestration*
 - Container orchestration platform*
- Kubernetes with Golang*
- Workflow automation or automation scripts*
- Cloud services development*
- Network automation*
- Testing*
- Database automation*
- Continuous Integration and Continuous Deployment*
- Conclusion
- Questions

22. Data Analysis and Machine Learning

- Introduction
- Structure
- Objectives
- Machine learning and data analysis overview
- Application of machine learning
- Data handling and manipulation
- Exploratory Data Analysis in Go
- Machine learning ecosystem with Go
- Golang machine learning libraries
- Time series analysis
- Conclusion

Questions

23. Build CI/CD pipeline with Golang

Introduction

Structure

Objectives

Introduction to CI/CD

Golang project structure

Setting up CI/CD Environment

Integration of CI/CD in Go

Integrating automated testing into CI/CD pipelines

Deployment strategies for Golang applications

Monitoring and logging

Conclusion

Questions

24. Wrap-up and Takeaways

Introduction

Structure

Summary of key learnings

Common pitfalls in Go programming

Practical applications

Next step

Conclusion

Index

CHAPTER 1

Introduction to Go

Introduction

Programming languages enable us to communicate with computers by developing various programs and applications. Our technical world has multiple programming languages like C, C++, PHP, Python, Ruby, etc. Golang (Go) is a growing programming language that addresses issues like slow build times, complex concurrency models, and excessive dependencies by offering simplicity, speed, and built-in concurrency support. Let us go through the evolution of the Go language, its features, scope, and use in the technical field.

Structure

In this chapter, we will discuss the following topics:

- Software essence
- Evolution of Go language and its success
- Characteristics of Go language
- Object-oriented nature of Go language
- Analogy with other languages
- Pain points of Go language
- Applications of Go language

Objectives

This chapter's goal is to give readers a thorough rundown of the Go programming language, also referred to as Golang. By the end of this chapter, readers should have a firm grasp of the history, guiding principles, and distinctive characteristics of the language. Readers will also understand the motivation behind Golang's development and its function in the modern software development industry.

Software essence

At the time of Go's initiation, complexity was the major concern in software development. Usually, development used to be done in C++ or Java, but there were some challenges, such as security vulnerabilities, high memory usage, slower performance, inefficiency, and difficulties in debugging. Software should have the power to function correctly against potential risks produced by malicious attacks. Dead objects should not exist to avoid resource blocking and improve software efficiency.

Evolution of Go and its success

By looking at the problems with software development, it was decided that with the rise of networked and multicore computing, a language must support some sort of concurrency and parallelism, garbage collection, and should automate memory management. All that was addressed in a discussion thread was a bunch of ideas, a decent strategy, and a great development. With Golang, efficient compilation, execution, and ease of programming are all possible with one language.

Go language is sponsored by Google and created by *Robert Griesemer*, *Rob Pike*, and *Ken Thompson*, who have depicted the goals of Golang. It finally came out in public on November 10, 2009. It is an open-source programming language. Go language's creators and the community have built many tools natively supported by the language core, which usually do not need any third-party library. For example, Go language comes with built-in support for HTTP, JSON, and HTML templates natively. This allows developers to build highly complex API services without needing to

search for additional libraries on GitHub. Millions of programmers are working on Go, and many more are coming. Following is a glimpse of how the Go logo evolved in 2018, reflecting the language's growth and modernization:



Figure 1.1: Logo change in April 2018

Characteristics of Go

Go is a statically typed and fast-compiled language that uses multicore architecture for efficient memory utilization. Here are some features of Golang:

- **Automatic garbage collection:** With this feature, the programmer does not need to worry about how the memory will be managed. Currently, a mark-and-sweep collector is used in Go. It tracks created objects and handles garbage collection concurrently with running processes, managed by Go's built-in executor package in every application.
- **Concurrency:** To run more than one task simultaneously is called concurrency. Golang provides a different approach to executing multiple functions at the same time. That is possible with goroutines and channels. Goroutines can be considered functions executed concurrently with other functions. On the other hand, the channel provides communication functionality between multiple goroutines.
- **Multi-core support:** As stated above, Golang has a concurrency feature, and these goroutines use multiple CPU cores for execution. Now the question is how many CPUs are to be used by goroutines to perform the concurrency.

Golang has a runtime variable "GOMAXPROCS" that controls the

number of CPUs to be executed simultaneously. By default, the value of this parameter is equal to the number of CPU cores identified on the hardware. The developer has the flexibility to change its value as required to improve the performance.

- **Statically typed and compiled:** Golang is built as a compiled language, which means go programs are compiled through a compiler that converts source code to machine/binary code. The operating system executes this machine code without using any run time engine.

This language is statically typed, where every variable is bound to a specific type that cannot be changed once defined. It helps the compiler detect type-related errors early before run time, improving performance. These two aspects make Golang execution faster as compared to other languages.

- **Generics:** Go introduced generics in the 1.18 version, which is nothing but a type that can be used collectively with other data types. Before generics, using multiple types together was possible by using interfaces or `io.Reader`, but that was a bit challenging. Generics help carry out varied behaviors of single variables with different data types.
- **Ease testing:** Go language offers a rich library that supports automated testing. Here is a package called **testing** that helps to run the go **test** command, which makes the execution of any testing function possible. There are some benchmarks such as function and memory benchmarks, to assess performance. Many forms of testing functions are executed by go **test** commands.
- **Standard library::** The important part of good programming is **code reusability**. The standard library of the Go language contains many packages that provide great functionality for Go applications. It provides packages like OS, runtime, HTTP, XML, and JSON that connect with operating systems and give support for runtime.
- **Cross-platform:** Go is a cross-platform language that can run on Linux, Mac, Free BSD, Windows, iOS, Android, and others.
- **Documentation:** Golang has a command called Godoc that concentrates on package documentation and serves it as HTML or plain text. With the help of this documentation, the programmer can steer

from illustration to the implementation of any function. It defines a package's functions, constants, variables, and even the types that Go supports.

Object-oriented nature

A standard definition of object-oriented language doesn't work with Go, but it provides the same functionality in some other manner. There is a defined data structure **struct** that integrates fields and methods. Golang has no inheritance but has a different approach to establishing the relation between structures. Object composition establishes an IS-A relationship, whereas interfaces are used to form a Has-A relation.

Furthermore, methods in the Go language are more transparent. They are not bound with structs. Go does not support the feature of overloading. In this way, the Go language makes things simpler and more convenient.

Analogy with other languages

In C++, Java, and other object-oriented languages, inheritance and overloading are the key features. Here, the Go team focuses on concurrency, garbage collection, and fast compilation to work well with parallel systems. Let us go through some nuts and bolts of the programming language:

- **Type-safe:** This refers to a feature to prevent “**type**” errors. Go is also a type-safe language like C, C++, Java, and Python. The Go compiler meets a certain level of validity during compilation. It ensures that if there is a “+” operator, both operands are numeric.
- **Object-oriented:** Go can be considered a lightweight, object-oriented language. However, it supports types and methods that can be defined for any data but have no classes or inheritance. Not everything is an object associated here. Go syntax resembles the C language. It is statically typed and scalable to a larger systems like C++ and Java.
- **Inheritance:** Go is not an object-oriented language. It does not support inheritance but has a feature of composition and interfaces. With composition, it is possible to embed more than two structs together. Multiple methods can be added to the interface, and thus, it

has a chance to grow. This new concept is more powerful than the class-based inheritance.

- **Overloading:** Go does not have a concept of method and operator overloading, unlike C++ and Java. Identifying methods by their name removes complexity and makes it simple to code.
- **Exception handling:** Unlike C++ and Java, there are no try-catch-finally idioms in Go. It has some built-in functions to signal and recover exceptional conditions. Go functions return the error as the last value. Here is a simple mechanism: the last value is compared to “nil.” If it is nil, there is no error; otherwise, there would be an error.
- **Concurrency:** In Java, there is an option for threads for concurrent programming. Java has a lot of utilities to manage those threads, but it is still considered quite expensive and hard to understand. To simplify it, Go has a simple technique for concurrency with the features of Go routines and channels. Go routines are lightweight threads, and the channel helps to communicate between Go routines.
- **Garbage collection:** Generally, there are two ways of memory management. One is automatic reference counting, and the other one is garbage collection. Like C# and Java, Go also has a garbage collector. With Objective-C, Swift uses the reference counting technique to manage the resources. The current GC “mark-and-sweep” is used with the Go language. It runs on a separate CPU, parallel with main programs, thus, it helps with multicore architectures.

Pain points of Golang

With multiple features, Golang has some disadvantages. The limitations of Go language are as follows:

- **Lack of demanding domain support:** However, Golang is a rising programming language, supporting standard, custom, and some third-party libraries, and there is a long way to cover in terms of wide-spreading areas like scientific computing artificial intelligence compared to JAVA or Python.
- **Tedious error handling:** Go language lacks a robust approach to error handling. There is no execution carried out by the compiler to handle

the error. It depends on the programmer how the functions can return the error whenever it occurs.

Applications of Go language

Go has been gaining popularity since it was first announced in 2009. A few companies such as IBM, Twitter, Facebook, YouTube, Apple, BBC, Google, & others are working on the Go language. While the future is uncertain, there are several factors that suggest Golang will continue to grow.

Here are some applications that are developed in the Go language:

- **Docker:** It is a set of tools to create, deploy, and run the distributed application. It is an open-source platform that allows the extension of its features if needed.
- **OpenShift:** An open-source cloud computing platform provided as a service by Red Hat. It provides the service for container-based software deployment and management.
- **Kubernetes:** It is an extensible open-source container cluster management system that promotes seamlessly automated deployment processes. Google designed this tool in mid-2014
- **Dropbox:** Dropbox migrated from Python to Go language to scale its system more efficiently, which supports better concurrency and fast execution.
- **InfluxDB:** It is an open-source time-series database that fetches time-series data. It supports the Internet of Things sensor data, application matrices, and real-time analytics.

Conclusion

In this chapter, you learned the pain points of software development and how the evolution of the Go language resolved them. This open-source programming language makes software development easy, efficient, and reliable. We saw some applications of various organizations developed with the Go language. We discussed some disadvantages of the Go language that we need to remember while programming in Golang. This is all the

theoretical knowledge we need to know. Let us proceed with installing the Go language in the next chapter.

Questions

1. Who developed the Go language?
2. How does garbage collection work in the Go language?
3. Which command illustrates Go language documentation?
4. In what ways is the Go language different from other programming languages?
5. How are fields and methods integrated into Golang?
6. Go is a concurrent language. What does that mean?
7. Which feature was introduced in Go 1.18.?

CHAPTER 2

Environment Setup

Introduction

After becoming aware of the Go language, let us move on to the Golang system requirements and installation process. This chapter defines the system requirements to install Golang, text editors to write Golang programs, and all about the installation and uninstallation process with various operating systems. The first Go program is shown to verify the proper installation once Go has been installed.

Structure

In this chapter, we will discuss the following topics:

- System requirements
- Installing Go
- Uninstalling Go

Objectives

The goal of this chapter is to equip readers with the information and abilities needed to set up a stable environment for Go programming development. It focuses on helping users to install the Go programming language, defining environment variables to support efficient development workflows, and setting up necessary tools including text editors and the Go compiler. By the

end of the chapter, readers ought to be able to properly set up their systems for Go development and be able to create, compile, and execute Go programs on a variety of platforms and operating systems.

System requirements

Go targets platforms like Linux, Mac, FreeBSD, and Windows of 32-bit and 64-bit x86 processor architectures. In the case of any other operating system, one can use gccgo, the compiler of the Go language.

Go1.20 is the latest version of Go and was released in February 2023. This version supports FreeBSD/RISC-V, Darwin, iOS, and Windows. The latest version of Go that will work with *macOS 10.13 High Sierra* or *10.14 Mojave* is 1.20. This is the last version that will run on any Windows 7, 8, Server 2008, and Server 2012 release. After this version, Go will need at least Windows 10 or Server 2016. To run a simple Go program, there is a need for a text editor and a Go compiler. Let us discuss the most popular editors.

Text editors

Go supports a variety of editor plugins and IDEs to make Go development more productive and seamless. Most well-known text editors either facilitate directly or through some plugins. Text editors can vary with different operating systems.

Here are some text editors:

- **Atom:** It is a free, open-source text editor for Windows, Linux, and macOS. It is a cross-platform java-script-oriented desktop application from GitHub.
- **BBedit:** It is a commercial text editor developed for macOS. Go-bbpackage includes clippings, ctags standard library completion, better syntax highlighting, and tools.
- **Eclipse IDE:** It is the most widely used open-source, cross-platform IDE. GoClipse is an Eclipse extension that adds IDE functionality to the Go language.
- **Emacs:** It is free and loaded by multiple extensible and customizable text editors. Emacs' additional features are code auto-completion, code analysis and refactoring, error checking, playground, and code snippets.

- **Gedit:** It is a default text editor for the GNOME desktop. Go-gegit-plugin is a Go programming plugin for the gedit editor.
- **Goland:** It is a commercial cross-platform IDE by JetBrains. The IDE analyzes the code, the connection between symbols, quick navigation, error analysis, formatting, and refactoring.
- **Notepad++:** It is a distributed source code text editor for Microsoft Windows. The GOnpp plugin provides code completion, function call tips, Go import integration, and keyboard shortcuts for common Go commands.
- **Sublime Text:** It is a free cross-platform source code editor that supports auto-save, customizable key binding, macros, and many others. GoSublime is a Golang plugin collection that provides the code completion feature from Go code, automatically adding/removing package imports, etc.
- **Vim and Neovim: VI improved (Vim)** is a free open-source text editor for Unix. It is designed to be used both from a command-line interface and as a standalone application in a graphical user interface. Neovim allows switching between user-defined panels along with a set of components and extensible API. Since Neovim is backward compatible with Vim, you can use Neovim with the same plugins and configuration files as Vim.
- **Visual Studio:** Visual Studio is a commercial IDE for Windows to develop websites, web apps, web services, and mobile apps. With Visual Studio, the Go extension supports features like IntelliSense code auto-completion, hover information, signature help, snippets, etc.

Cloud-based IDEs

The development of cloud IDEs is a great innovation in the software industry. In such IDEs, the developer's atomic services like syntax analysis, build, run, and debug are operated on separate horizontally scalable microservices. These are some cloud IDEs:

- **Cloud9:** It is an online, open-source, multi-language cloud-based IDE. It connects with GitHub, Bitbucket, and Google Cloud platform repositories. It has MySQL, other databases, terminal, and immediate windows.

- **CodeEnv:** CodeEnv is also an open-source, multi-language cloud-based IDE. It is a browser editor with a feature to edit, build, run, and debug the project.
- **Wide:** It is a web-based IDE for the Go language. It uses a code mirror-based themed programming editor with Gocode-assisted syntax highlighting, auto-completion, and real-time collaboration.
- **Micro:** It is a toolkit for cloud-native development. It provides an opinionated framework for developing applications with a pluggable architecture.
- **Gitpod:** Gitpod is GitHub and GitLab integrated cloud IDE with full Go support. It offers collaborative coding and enables several developers to work on the same codebase parallelly in real-time. Once you open the Gitpod workspace, it instantly copies your Git repository and installs all required dependencies.

Compiler

Several compilers are running for the Go language that help to set up the Go environment. Two major implementations of Go compiler areas:

- **Gc:** Gc is a default compiler with a recursive descent parser and uses a custom loader. It supports popular processors like x86 (32/64bit) and ARM.
- **Gccgo:** Gccgo is part of GCC, the GNU compiler collection that supports the front end. It supports all the processors that GCC supports. Gccgo is slower than Gc but supports more processors and has more powerful optimization. Therefore, a CPU-bound program built by gccgo will run fast. A third Go compiler, known as GopherJS, also exists. It compiles Go code into java-script code, enabling Go for front-end development in all browsers.

Installing Go

The following procedures prepare an environment to install Go on various platforms:

- **Windows installation:** Here are the steps to install Go on the Windows machine:

1. One can visit the Go page at <https://go.dev/doc/install> and download Go:

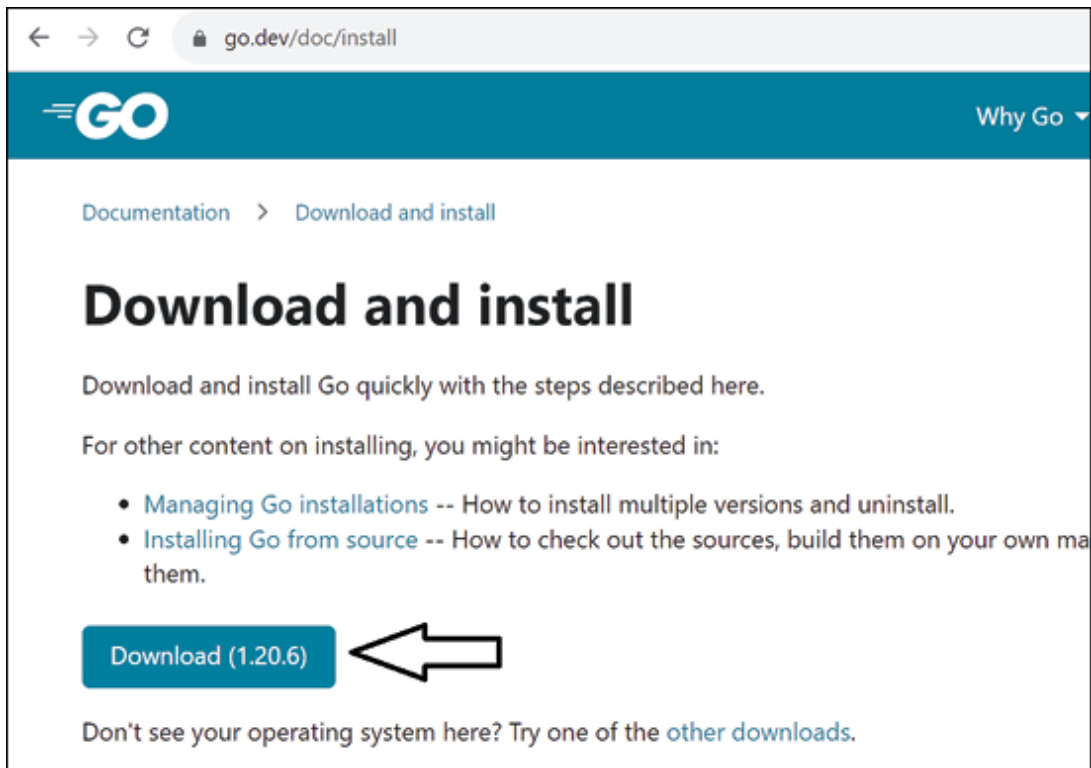


Figure 2.1: 'golang.org' home page

2. Download MSI or zip file as per the system requirement:

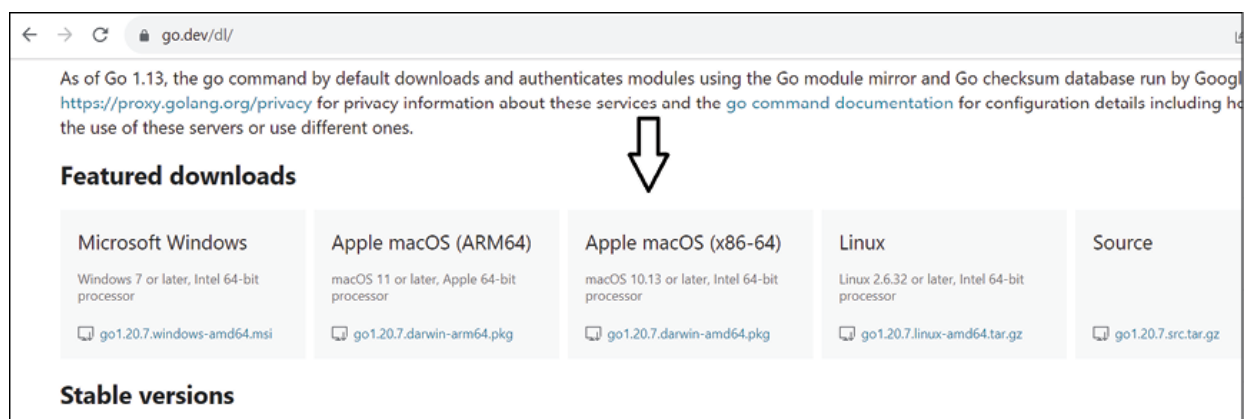


Figure 2.2: Go for multiple OS

3. Once the file is downloaded, execute it and follow the prompts: **Next**
Refer to the following figure:

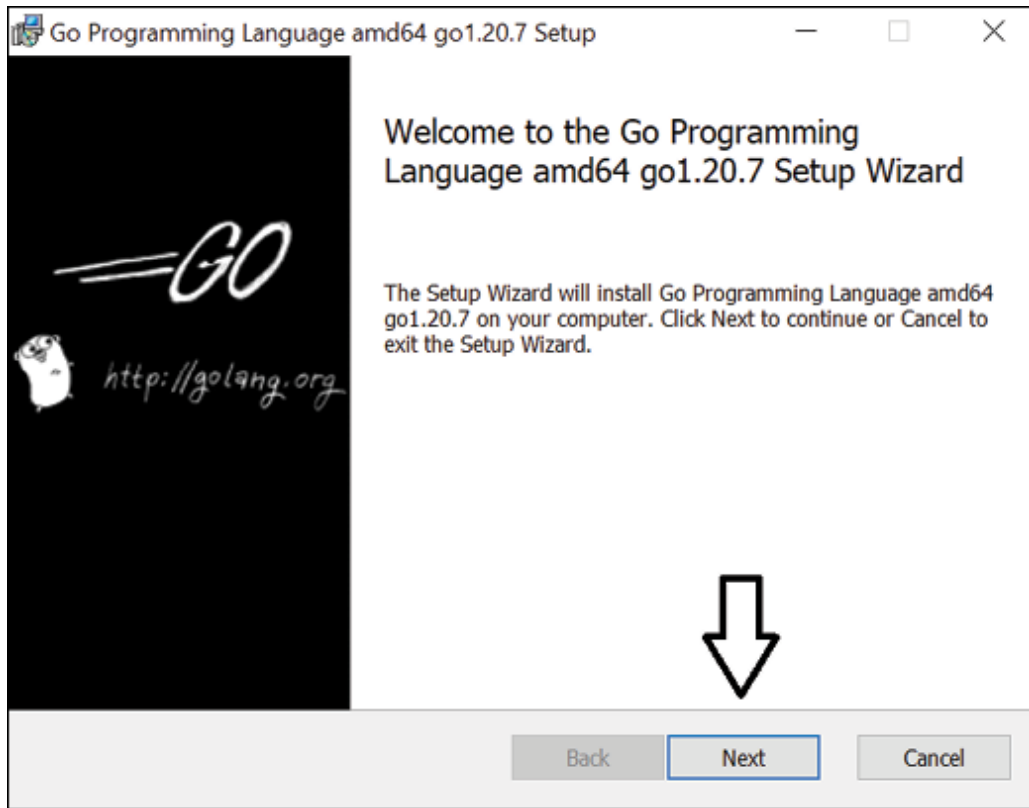


Figure 2.3: Run file

4. Here are some license agreements that need to be accepted to install Golang:

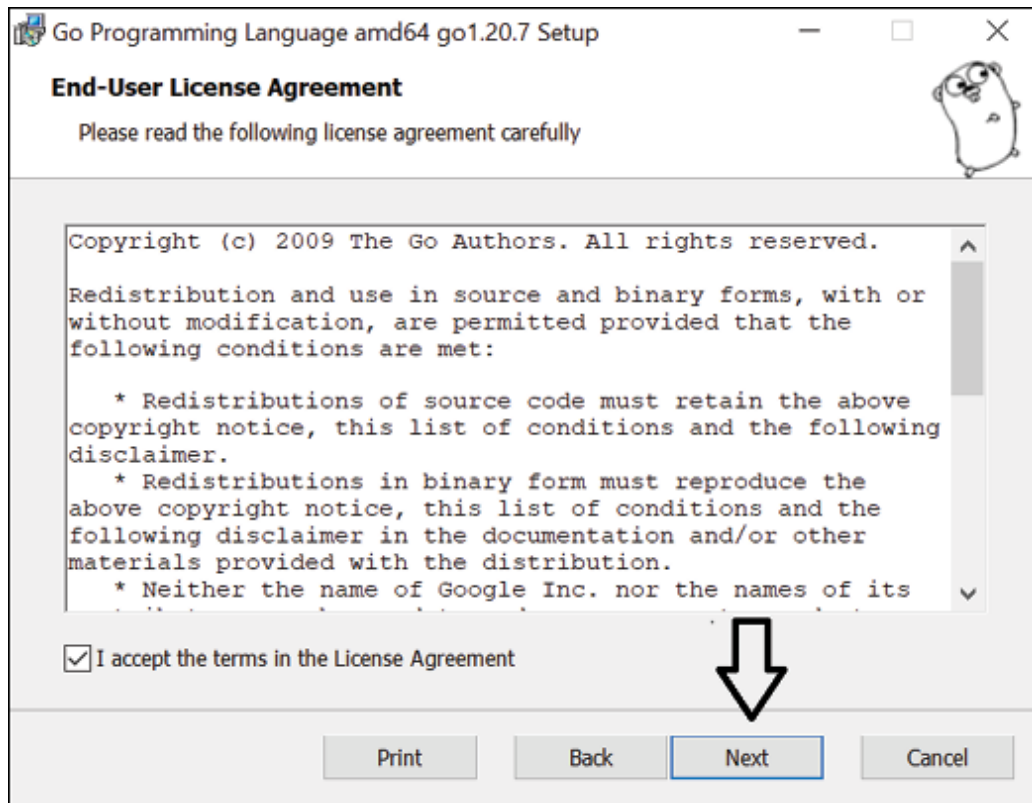


Figure 2.4: Accept license agreement

5. You can keep the directory as it is "**C:\Program Files\Go**" and go next:

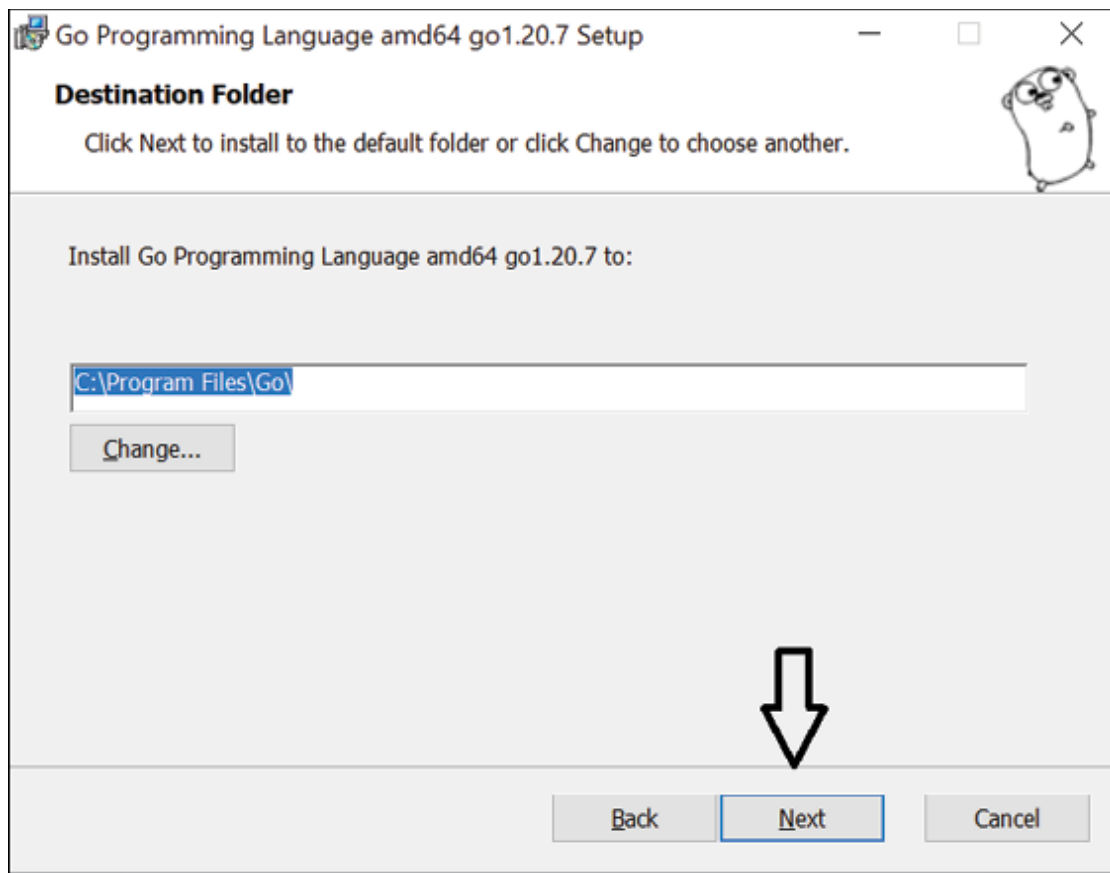


Figure 2.5: Check the directory

6. Or you can change the directory as shown in the following figure:

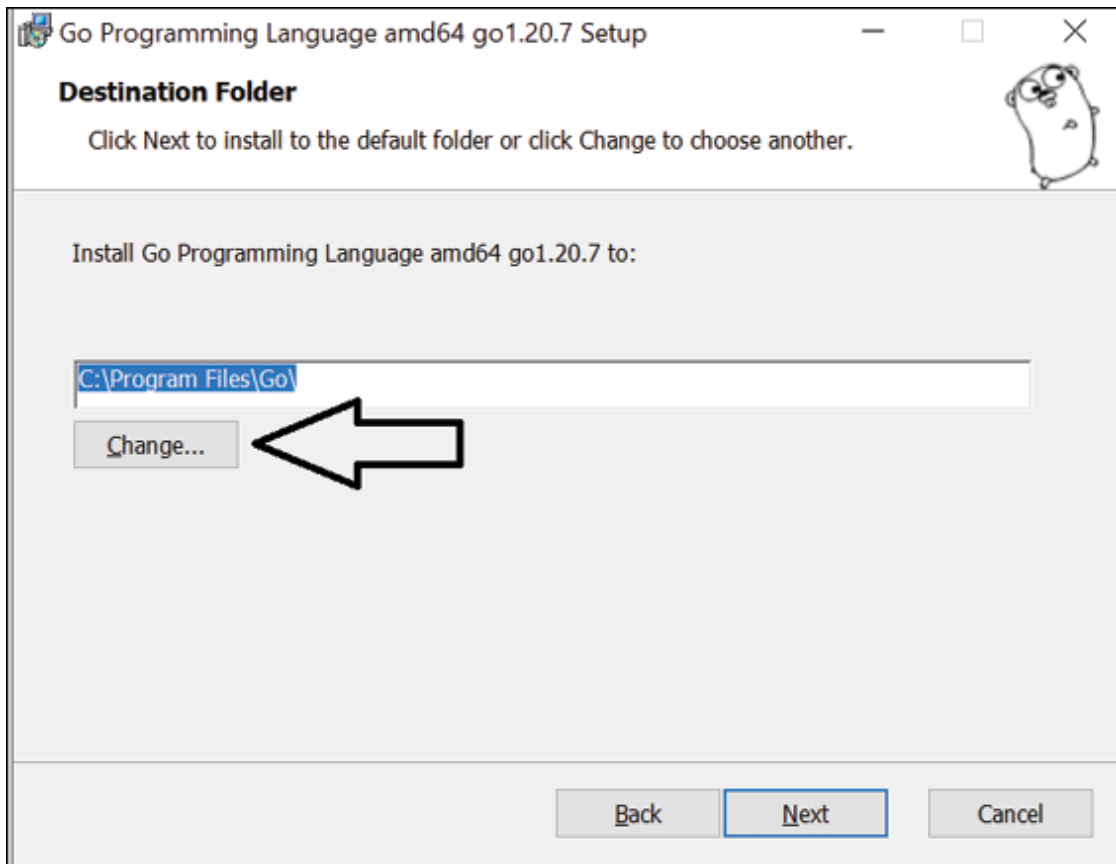


Figure 2.6: Click on the change button

Refer to the following figure:

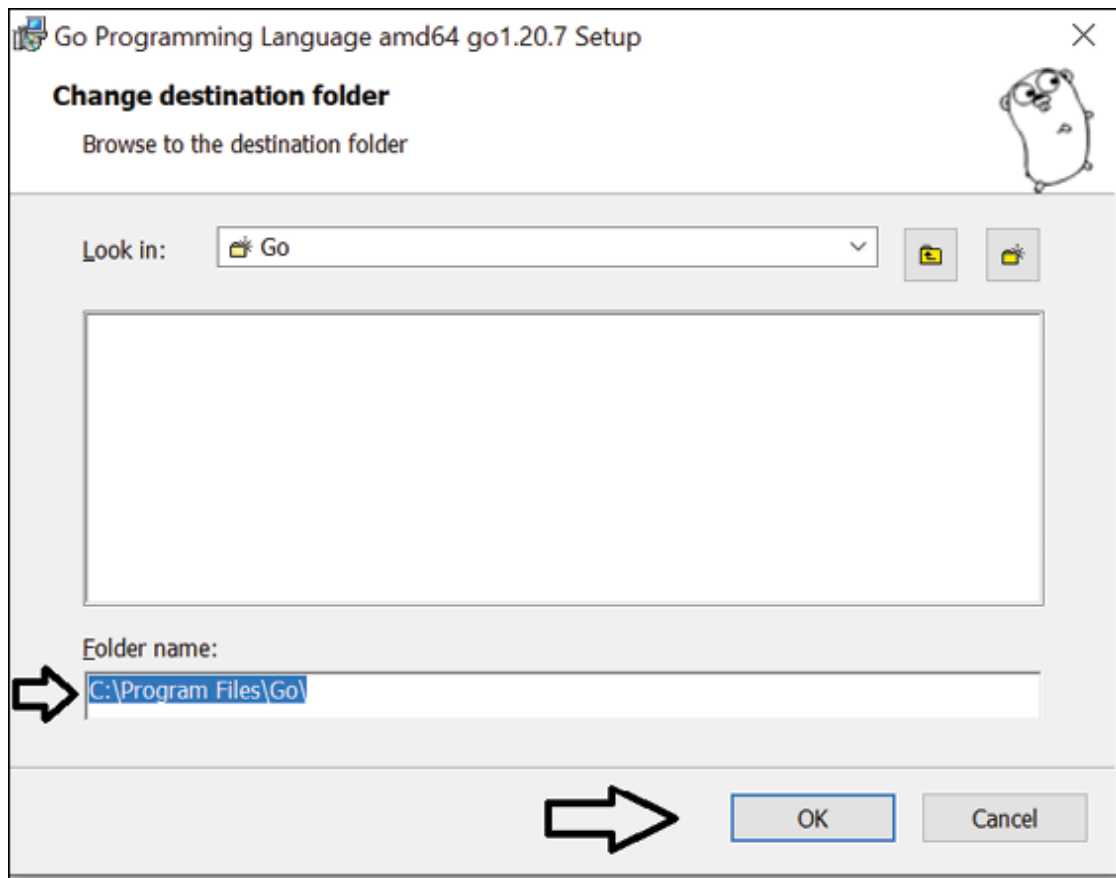


Figure 2.7: Set the required directory path

7. Ready to install:

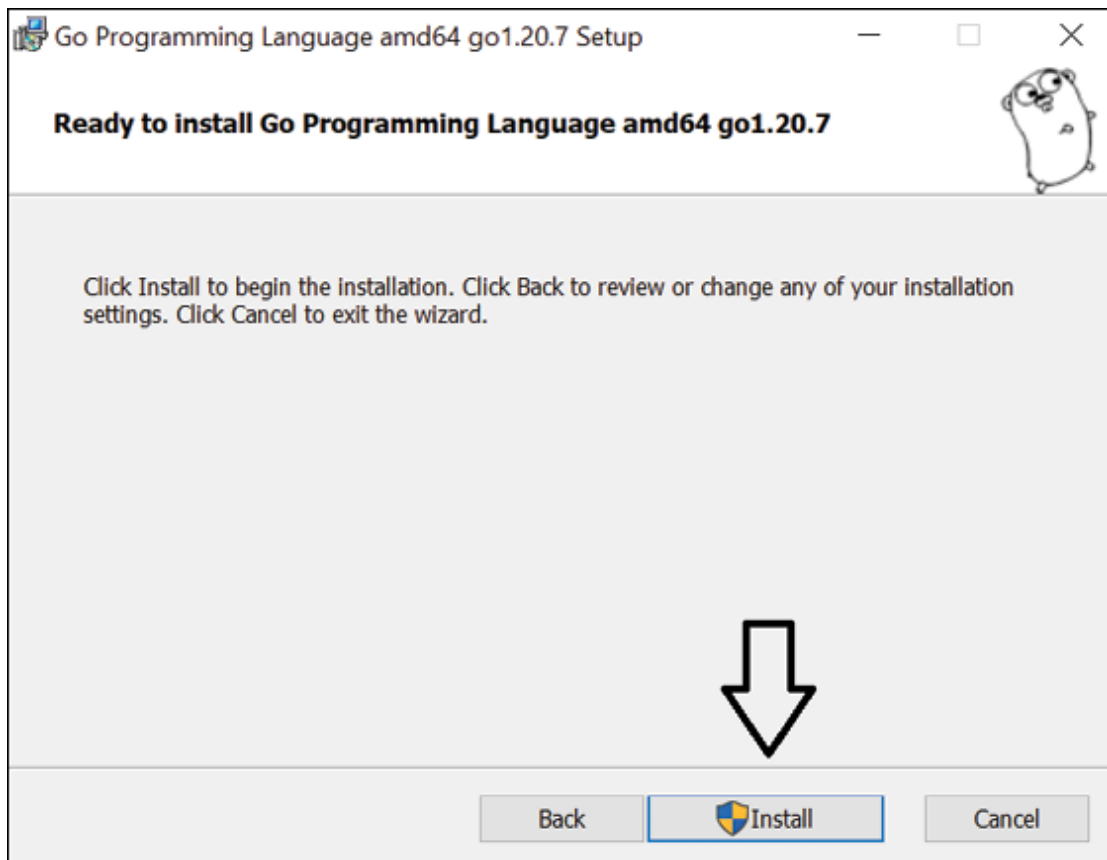


Figure 2.8: Install

8. Go folder is created in the default directory once the installation is completed, as shown in the following figure:

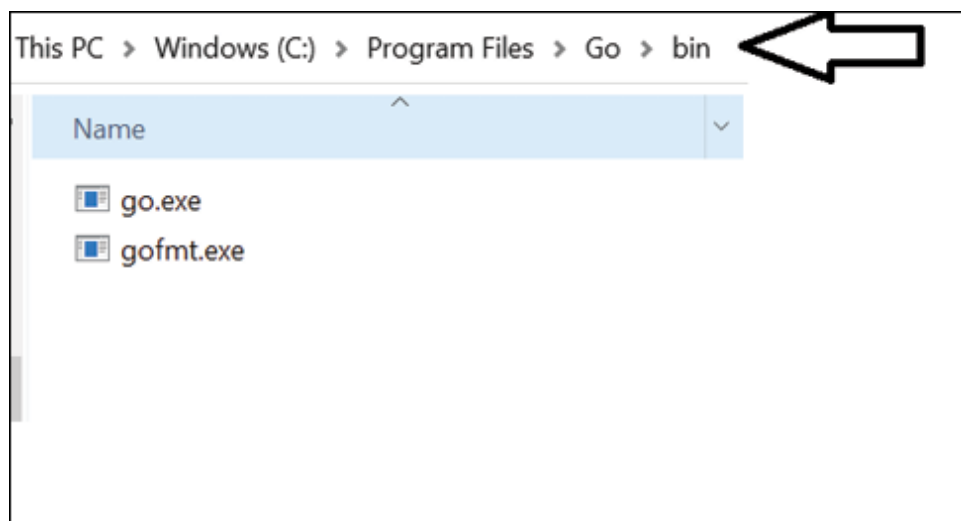


Figure 2.9: Go's default directory after installation

9. If we changed the directory path at the time of installation, it would

be as shown in the following figure:

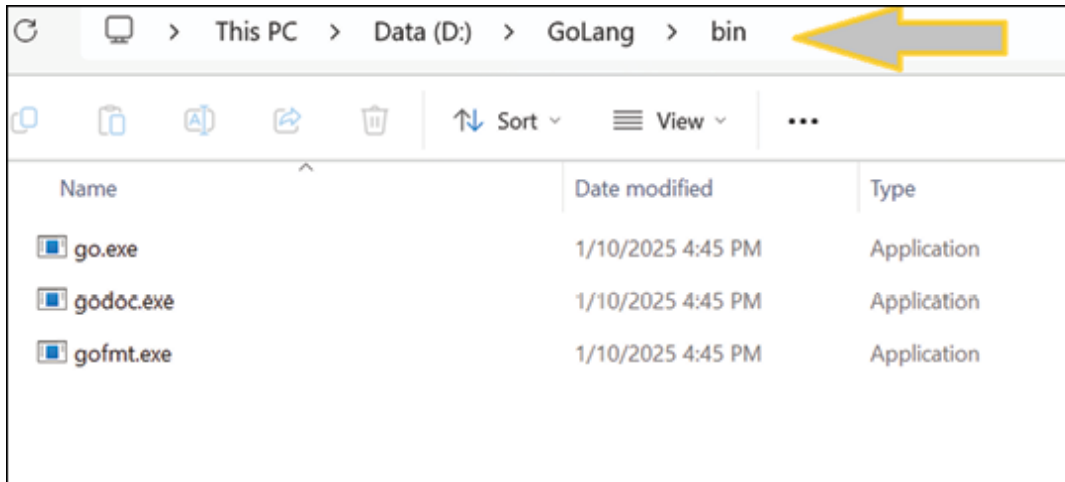


Figure 2.10: Go's directory (depends on the given path)

10. At the installation time, it sets the bin path as an environment variable; hence, we do not need to set this explicitly. Open the terminal and verify the Go version by using the **go version** command:

```
(c) Microsoft Corporation. All rights reserved.  
  
C:\Users\      >go version  
go version go1.20.7 windows/amd64
```

Figure 2.11: Verify the Go version

11. The path is all set. Now, create and run the Go sample program. It is advisable to create a workspace for Go projects. It can be in the same directory but must differ from the Go folder.
12. Install sublime or any supported text editor and write your first Go program as follows:

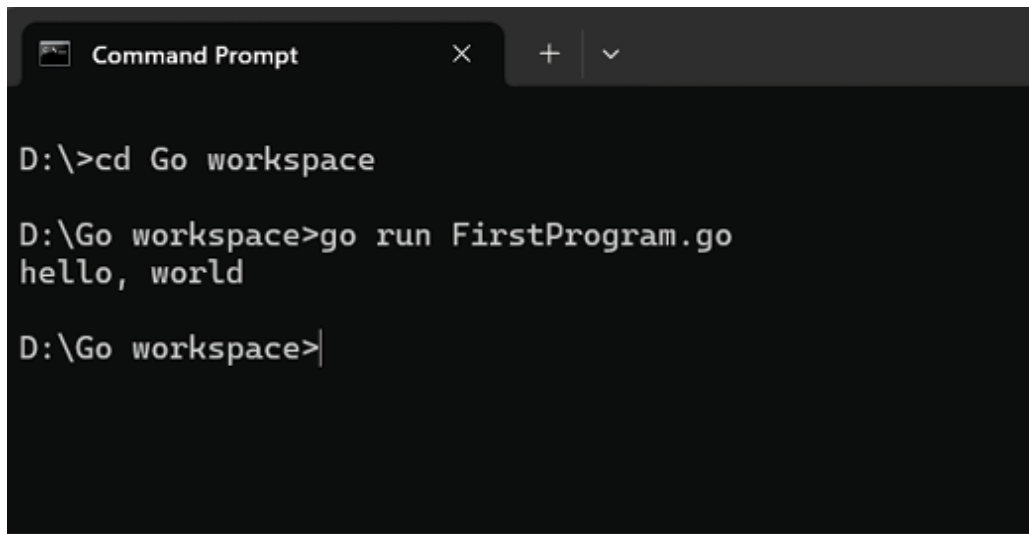
```
package main  
import "fmt"  
func main(){  
    fmt.Printf("hello, world\n")  
}
```

Note: Explanation of this program is in the next chapter.

13. Save this file with the go extension as **file_name.go** in your workspace. Now, let us run the first Go program. Open the command-line interface and navigate to the created workspace. Run command as:

go run file_name.go

Refer to the following figure:

A screenshot of a Windows Command Prompt window. The title bar says "Command Prompt". The command prompt shows the following sequence of commands and output:

```
D:\>cd Go workspace
D:\Go workspace>go run FirstProgram.go
hello, world
D:\Go workspace>
```

Figure 2.12: Run test program

- **Linux/macOS X and FreeBSD installation:** Download the package file from the same source described above and follow the prompts to install Go tools. Go tree is installed in **/usr/local/go**. If not added during installation, add the bin path to the system path environment variable. There could be a need to restart the open terminal session for the change to reflect. After that, you are ready to run the Go program.

Uninstalling Go

Delete the Go directory to remove an existing Go installation from your system. This is usually in **/usr/local/go** in Linux, macOS, and FreeBSD or **C:\Program Files\Go** in Windows.

Conclusion

In this chapter, we saw that Go is available for various platforms like FreeBSD, macOS, Windows, and Linux on its 32-bit and 64-bit processor architectures. We can check our system configuration and install the suitable version of Go. To upgrade, you can install the new version directly, as it will overwrite the existing one. However, if issues arise, uninstalling the old version may help. At last, we can verify installation by running a test program. For common issues like the 'go command not being recognized', ensure the Go binary path is added to your system's environment variables.

In the next chapter, we will discuss the basic structure of the Go program in depth.

Questions

1. Which platforms is Go language supported on?
2. Which compiler works for the Go language?
3. Specify the default compiler in the Go language.
4. Which file extension is used to write Go programs?
5. Specify the command to run Go programs.

Join our book's Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

<https://discord.bpbonline.com>



CHAPTER 3

Beginning with Go

Introduction

In the last chapter, we discussed the installation of the Go language. By now, we hope that Go is all set on your system. In this chapter, we will see the basic structure of the Go program, the components it is divided into, and the purpose of these components. In every programming language, keywords and identifiers play an important role. Here, we will discuss Golang's supported keywords and identifiers as well.

Structure

In this chapter, we will discuss the following topics:

- Introduction to Go program
- Keywords
- Identifiers

Objectives

The goal of this chapter is to walk through the process of writing basic Go programs, with a focus on the significance of identifiers, comments, and keywords. Readers will have a thorough grasp of how to write concise and clear Go code, identify and use important keywords, use comments for

documentation and readability, and use identifiers to improve code clarity and maintainability by the end of this chapter.

Introduction to Go program

Let us begin with the basic structure of the Go program. A simple Go program consists of defining the program as a package, importing packages, classes, functions, variables, comments (if needed), and the required statements. Here, we are going to use Visual Studio for Go programming. You can use any other compatible editor.

In Visual Studio, set “**go.useLanguageServer: true**” in your VS code Go settings to activate the language server. A prompt to install gopls will appear.

Please use “**Go: Install/Update Tools**” command in VS command palette and choose gopls if you do not see a prompt.

Let us understand the Go program structure with the following code:

```
package main

import (
    "fmt"
)

/* This is a multiline comment */
//This is a single line comment

func main(){
    name := "Jitendra"

    fmt.Printf("Hello %s\n", name)
}
```

The first line defines the program as a package named main. This statement is necessary for every program that we are creating. As Go is a case-sensitive language, we need to make sure that the name of the package does not start with a capital letter.

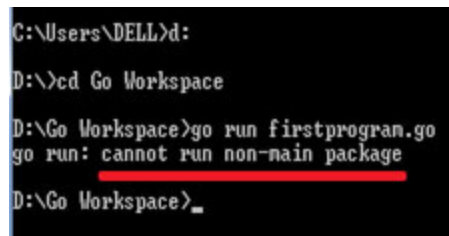
fmt is a package. It refers to a format that supports formatted I/O functions like **scan**, **printf**, and many more. Here, **fmt** is to be used as a string. That

is why we are writing it in double-quotes. **fmt** becomes the prefix for all those functions that are defined in **fmt** package.

The third line refers to comments that are ignored by the compiler. There are two ways to represent the comments. Either use `/*` or `//` to show the comments in Go.

func is a keyword that indicates the function declaration. The **function** is simply a piece of code to perform a particular task. It enhances the reusability of code. Every Go program that is written inside a main package has a function called **main()**. Execution begins from the main function here. It executes the statements defined within the function and exits when it returns from this function. Other than the main function, we can also create other functions, but all these functions would be called within the main function only.

If we do not keep the package or function name as per standard, it could throw an error as follows:

A screenshot of a Windows command prompt window. The prompt is at C:\Users\DELL>. The user enters 'd:' to change to the D: drive. Then they enter 'cd Go Workspace' to navigate to the Go workspace. Finally, they enter 'go run firstprogram.go'. The output shows an error: 'go run: cannot run non-main package'. The error message is underlined with a red line.

```
C:\Users\DELL>d:
D:\>cd Go Workspace
D:\Go Workspace>go run firstprogram.go
go run: cannot run non-main package
D:\Go Workspace>_
```

Figure 3.1: Non-main package error

It will show the following error if we do not use main function inside the main package in Go program:

A screenshot of a Windows command prompt window. The prompt is at D:\Go Workspace>. The user enters 'go run firstprogram.go'. The output shows an error: '# command-line-arguments runtime.main_main·f: function main is undeclared in the main package'. The error message is underlined with a red line.

```
D:\Go Workspace>go run firstprogram.go
# command-line-arguments
runtime.main_main·f: function main is undeclared in the main package
D:\Go Workspace>_
```

Figure 3.2: Undeclared function error

As described in the previous chapter, we can run this program on the command prompt or VS terminal by using the following command:

go run file_name.go

In the next line, we will declare the variables if required. As Go is a

statically typed language, variables cannot change their type at run time. We can define the variable as **name:= "Jitendra"**. It will identify the type of variable by its value.

"Printf" is a function defined in the **fmt** package. All the exported names should start with a capital letter.

Functions in Go are fundamental building blocks of the program. Program execution begins with a function call, and functions can be assigned to variables or passed as arguments. The Go compiler starts execution from the main package. It identifies the appropriate files, compiles them, converts them into a temporary executable, and runs the program. The execution strictly starts with the main function in the main package.

Keywords

Keywords are predefined reserved words specific to the programming language that have special meaning for the compiler. Keywords cannot be used as variables or identifiers. Golang supports 25 keywords. They are as follows:

- **import:** This keyword is used to import a package in a Go program.
- **type:** This is used to create a new type as per the requirement.
- **const:** This specifies a character, numeric, Boolean, or string value.
- **var:** This keyword is used to declare a variable containing a specific name and value.
- **func:** This keyword is used to declare functions with a specific name, parameter, result type, and body.
- **package:** This defines packages in Go program that contain variables, constants, and functions.
- **map:** This keyword declares the data of type map in a Go program.
- **chan:** The chan keyword is used to define a channel in a Go program.
- **struct:** A user-defined type **struct** is declared by using this keyword.
- **interface:** This keyword is used to define a custom data type **interface** in Golang.
- **if:** This keyword specifies **if statement** that evaluates the given condition.

- **else:** It defines an alternate process that needs to be executed if the corresponding 'if' condition is not satisfied.
- **for:** This keyword defines a loop that is used to repeat the execution of a specific block of code.
- **range:** This keyword is used with for loop to iterate over a specific given data.
- **break:** This defines the break statement inside a loop to terminate the current loop execution and resume the program control next to the loop.
- **continue:** This defines a continue statement inside a loop to jump the program control on the next iteration.
- **goto:** This executes the goto statement and transfers the program control to a labeled statement.
- **return:** This executes the return statement by returning the result to the caller.
- **switch:** This defines the switch statement in the Go program that is an alternate way to write sequence of multiple 'if' statements.
- **case:** This defines various switch cases/conditions within the switch statement.
- **select:** This keyword defines the select statement that works on multiple send/receive channel operations.
- **default:** This executes the default case in a switch statement.
- **fallthrough:** This executes the fallthrough statement that transfers the program control to the next case, irrespective of the given condition.
- **defer:** This executes the defer statement that delays the function execution.
- **go:** This keyword is used to handle Goroutines.

We will discuss every keyword in depth in the coming chapters. Here is a brief appearance of these keywords:

```
package main //use of package keyword

import (
    "fmt"
```

```

    "time"
) //use of import keyword

type person struct { //use of type keyword
    name string    age int
} // use of struct keyword

type interface_example interface{    interface_function_int() int interface_function_str() string
} // use of interface keyword

func interface_function_str() (string){
    return "returning from function" //use of return keyword
}

func main() { //func keyword
    var str = "hello world" //var keyword
    const num = 10 //const keyword
    arr := [6] int {10, 20, 30, 40, 50, 60}
    map_data := map[string]int{
        "hello":80,
        "world":45,
    } //declare map
    cs1 := make(chan string) // declare channel
    cs2 := make(chan string)

    go func() {
        time.Sleep(1 * time.Second)
        cs1<- "one"
    }()

    fmt.Println(str,num, map_data,cs1)

    if num>100{ //use of if keyword
        fmt.Println("num is greater than 100")
    } else { // use of else keyword

```

```

        fmt.Println("num is not greater than 100")
    }
    for pos, val := range arr { //use of for and range keyword
        if (pos==5){
            break //break keyword
        }
        if(pos==1){
            continue //continue keyword
        }
        fmt.Printf("value at index %d is %d\n", pos, val)
    }
    fmt.Println(interface_function_str())
    switch num { //switch keyword
        case 1: //use of case keyword
            fmt.Println("one")
        case 2:
            fmt.Println("two")
        case 3:
            fmt.Println("three")
        default: //default keyword
            fmt.Println("No match")
    }
    switch num {
        case 1:
            fmt.Println("one")
        case 2:
            fmt.Println("two")
        case 10:

```

```

        fmt.Println("three")

        fallthrough //use of fallthrough keyword
    default:
        fmt.Println("default statement")
}
defer interface_function_str() //use of defer keyword
go interface_function_str() // use of go keyword
select {
    case msg1 := <-cs1: fmt.Println("received", msg1)
    case msg2 := <-cs2: fmt.Println("received", msg2)
} //use of select keyword
goto gotolabel //use of goto keyword
gotolabel:
    fmt.Println("Inside goto statement")
}

```

Output:

```

hello world" 10 map[hello:80 world:45] 0x1183c0c0
num is not greater than 100
value at index 0 is 10
value at index 2 is 30
value at index 3 is 40
value at index 4 is 50 returning from function No match
three
default statement received one
Inside goto statement

```

Identifiers

The identifier is the name of the class, functions, statement label, constant, package name, or variables that uniquely identify an object. It is a fusion of

Unicode letters, digits, and underscore. The first letter of the identifier must be a letter or underscore. The name of the identifier is case sensitive in the Go language. We cannot use a reserved word as an identifier. Go provides a special type of identifier called **blank identifier** that is denoted as `_`. **Blank identifier** works as a placeholder that can have any value, but we cannot read it in our program. An identifier that has a capital letter as its first letter is called an exported identifier. These identifiers should be declared in package blocks as these are accessible from other packages also. On the other hand, the identifiers that do not have the first letter in capital are called non-exported identifiers.

Here are some valid and invalid identifiers:

- **Valid**

- o `_ = 12 //blank identifier`
- o `var Version = 4.0 //exported identifier`
- o `var name = "Ansh" //non-exported identifiers`
- o `var _12 = "hello world" //non-exported identifiers`

- **Invalid:**

- o `var 12 = "hello world" //should not start with digits`
- o `var count$ = 23 //can't have special characters`
- o `var default = 5.0 //can't use reserve keywords as identifiers.`

Conclusion

In this chapter, we saw the basic structure and components of the Go program. Writing and running a simple Go program is the first step in Go programming. We discussed the supported keywords in the Go language. By identifiers, we come to know the naming convention of packages, classes, functions, or variables. Let us move forward with Golang variables and data types in the next chapter.

Questions

1. What is the role of the `fmt` package in Go language?

2. How to comment lines in Go language?
3. Which package and function are necessary for the Go program?
4. Define the use of the fallthrough keyword in Go language.
5. What is the blank identifier?
6. Specify exported and non-exported identifiers in Go language.

CHAPTER 4

Variables, Data Types and Constants

Introduction

Data is a basic thing that programming works with. We can change some values and some we cannot. On the same concept, variables and constants work in programming languages and we will get into it in this chapter. In our lives, we see multiple types of values like our age in numeric terms, name as a combination of characters, amount as a decimal value, and many others. The same goes for programming data; it could be of any type, such as integer, string, float, etc. In this chapter, we will learn the Golang supported data types.

Structure

In this chapter, we will discuss the following topics:

- Variables
- Data types
- Constants
- Format specifiers

Objectives

With this chapter, we hope to provide you with a thorough understanding of variables, data types, and constants in Go programming language. We will examine the idea of variables as data value containers through concise explanations and useful examples. We will also look at how to declare variables using the **var** keyword and initialize them with certain values. We will go over the naming standards of variables while writing the code. In addition, we will explain several data types that Go supports, such as string, Boolean, integer, float, etc. By the time this chapter ends, readers will have a firm grasp of Go's variable and data type syntax, empowering them to produce clear, reliable code.

Variables

Variable is the name given to a specific type of value that is stored in memory at a particular location. Declaring a variable simply acquires some space in memory and tells the compiler the name and type of its value. As Go is a statically typed language, we cannot change the type of the variable at runtime. However, the value of a variable could vary as per our need with the same data type. Every variable should be necessarily defined in the Go language. While declaring a variable, it is required to declare its type if we are not assigning its value at the same time. By default, the Go compiler takes the value zero for numeric, blank for string, and false for Boolean type data.

Go facilitates multiple ways to declare and initialize a variable as below.

- `var name_of_variable type_of_variable`
- `var name_of_variable = value_of_variable`
- `name_of_variable := value_of_variable` //identifies the type of variable by its value
- `name_of_first Variable, name_of_second Variable: =value_of_firstVariable,value_of_secondVariable` //multiple declaration at the same time
- `var name_of_firstVariable, name_of_secondVariable type_of_`

```

variable //if all variables are of same type

var (
    name_of_firstVariable type_of_variable
    name_of_secondVariable type_of_variable
)

```

Go also has a blank identifier when it is necessary to declare a variable but there is no use of that variable programmatically. We simply put underscore (_) instead of the variable name to handle such worries.

Example:

```

var variable1 int //declare int type variable "variable1"

fmt.Println(variable1) // 0

var variable2 string

fmt.Println(variable2) // ""

variable1 = 123

variable2 = "hello world"

fmt.Println(variable1, variable2) // 123 hello world

var variable3 = 123.45

fmt.Println(variable3) //123.45

variable4 := false

fmt.Println(variable4) // false

variable5, variable6 := "multiple", 2

fmt.Println(variable5, variable6) // multiple 2

var variable7, variable8, variable9 int //declaring multiple variables of same
type

variable7, variable8, variable9 = 123, 456, 789

fmt.Println(variable7, variable8, variable9) // 123 456 789

var(
    variable10 int

```

```
variable11 string
variable12 bool

) //declaring multiple variables of different type at the same type.
fmt.Println(variable10, variable11, variable12) // 0 false variable10,
variable11, variable12 = 123, "multiple", true
fmt.Println(variable10, variable11, variable12) // 123 multiple true
variable13, variable14 := 123, 456
fmt.Println(variable13) //error "variable14 declared and not used"
variable15, _ := 123, 456 //use of "blank identifier"
fmt.Println(variable15) // 123
```

Data types

A data type is an attribute of data that defines what type of data can be stored. There are multiple variables and functions of different types. A data type tells the compiler the amount of space that would be occupied in storage by that particular variable. *Go* is a statically typed language in which the type of variable is declared at compile time and it cannot be changed later. In this way, a lot of small bugs are caught very quickly, preventing data type errors. Here is the categorization of Golang data types:

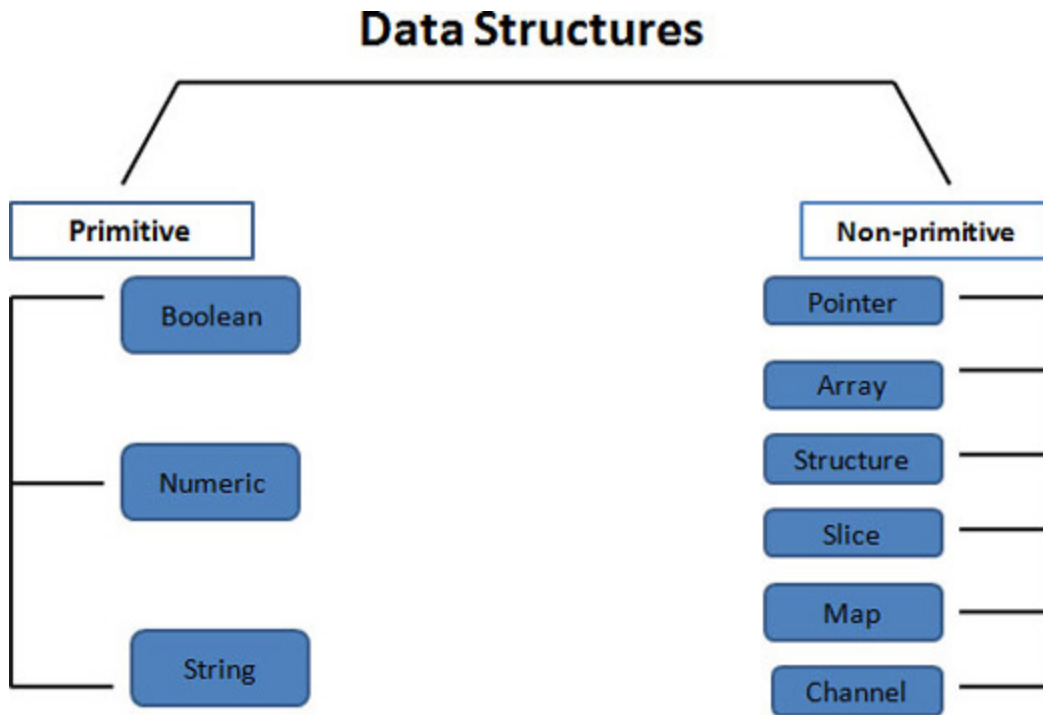


Figure 4.1: Data structure

There are two types of data - primitive and non-primitive:

- **Primitive data type:** Primitive data types are built with programming languages and always have a single value. They cannot be divided into sub-types. Boolean, numeric, and string come in primitive data type in the Go language. Let us talk about them in detail:

- o **Boolean:** A data type that can only have two values, either true or false, is the Boolean data type. It can be 1 as true or 0 as false.

Example:

```
var is_Red bool = true
fmt.Printf("%v\n", is_Red)
```

- o **Numeric:** The data type on which we can perform arithmetic and logical operations is called a numeric data type. It could be an integer or floating-point data type.

□ **Integer:** In the Go language, we have several integer data types:

- **uint8:** Unsigned 8-bit integers. They range from 0 to 255.
- **uint16:** Unsigned 16-bit integers. They range from 0 to 65535.
- **uint32:** Unsigned 32-bit integers. They range from 0 to

4294967295.

- **uint64:** Unsigned 64-bit integers. They range from 0 to 18446744073709551615.
- **int8:** Signed 8-bit integers. They range from -128 to 127.
- **int16:** Signed 16-bit integers. They range from -32768 to 32767.
- **int32:** Signed 32-bit integers. They range from -2147483648 to 2147483647.
- **int64:** Signed 64-bit integers. They range from -9223372036854775808 to 9223372036854775807.
- **complex64:** It has float32 real part and float32 imaginary part.
- **complex128:** It has float64 real part and float64 imaginary part.
- **byte:** It is similar to unsigned 8-bit integers having the same range.
- **rune:** It is similar to signed 32-bit integers with the difference of having Unicode code point representation.

Example:

```
var int32val int32 = 100
var valint int = 76 //treats as int32 or int64 depends on
implementation
fmt.Println(int32val, valint) // 100 76
var int16val int16 = int32val // error "cannot use int32val<type
int32> as type int16 in assignment"
var int16val int16 = int16(int32val) fmt.Println(int16val) // 100
var int64val int64 = int32val // error "cannot use int32val<type
int32> as type int64 in assignment"
var int64val int64 = int64(int32val) fmt.Println(int64val) // 100
var uint8val uint8 = int32val // error "cannot use int32val<type
int32> as type uint8 in assignment"
var uint8val uint8 = uint8(int32val) fmt.Println(uint8val) // 100
```

```
int32val = -100
```

```
uint8val = uint8(int32val)
```

```
fmt.Println(uint8val) // prints 156, as -100 is converting to unsigned integer (using 2's complement)
```

□ **Floating:** In the Go language, we have several floating-point data types.

□ **float32:** All 32-bit floating-point numbers.

□ **float64:** All 64-bit floating-point numbers.

□ **complex64:** complex64 has float32 real and imaginary parts.

□ **complex128:** complex128 has float64 real and imaginary parts.

□ **Error:** The error data type that is used to store error values (refer to [Chapter 18, Error Handling](#)).

- o **String:** In Go, String is not a simple collection of characters like in some other languages, but represents read-only byte slice. Each byte in the string represents part of a character in the UTF-8 encoding scheme, which is a variable-length encoding that can represent any Unicode character. Strings in Go are **immutable**, meaning that once a string is created, its contents cannot be modified directly.

It is also critical to realize that Go employs UTF-8 encoding when encoding Unicode characters into a string. However, the UTF-8 encoding can be broken if escape sequences (such as `\u` or `\x`) are used manually, because they may not match the way UTF-8 wants characters to be represented. When working with strings, this could result in errors or unexpected outcomes.

Example:

```
var strval string = "Grow with Golang"
```

```
var strcopied string = strval
```

```
fmt.Println("original string is:", strval)
```

```
fmt.Println("copied string is:", strcopied) //prints "copied string is: Grow with Golang"
```

```
fmt.Println("Address of strval:", &strval)
```

```
fmt.Println("Address of strcopied:", &strcopied) // above two lines  
prints the different memory address of variables
```

- **Non-primitive data type:** The data types that can store a batch of values are called non-primitive data types. They are also called derived data types. Primitive data types are a specific source of non-primitive data types. They are also called referenced data types as they contain the address of the dynamically created objects in memory. In the Go language, we have the following non-primitive data types:

- o **Pointer:** Every variable stored in memory has a particular location/address. A pointer is an object that stores the address of any value stored in memory. The type of pointer depends on the type of element that has been stored by the pointer. It allows direct access to memory and thus, reduces storage space of the program. Pointer uses two operators, **address of operator** and **indirection operator** dictated by & and *, respectively.

Here is a simple way to declare a pointer:

```
var name_of_pointer *type_of_pointer
```

Example:

```
var valint int = 12
```

```
var pval = &valint
```

```
fmt.Println(pval) // prints address of intval
```

```
fmt.Println(*pval) // 12
```

```
*pval = 78 // change the value of valint through pointer
```

```
fmt.Println(valint) // 78
```

- o **Array:** The array is a data structure that stores the same type of data elements, which can be accessed by their index number. In array, the index number starts at 0 so that the maximum index is one less than the length of the array. The type of array and the space occupied by the array depends on the elements of the array, whether it is an integer, string, or of any other type. We cannot put a different type of

element in a single array in the Go language. Here, the array behaves like a value. If we assign an array to a variable, it will copy all of its elements into another array. In the same way, on passing an array to a function, it receives the value of the array and not the reference of the array. In Go, at the time of declaration, we have to define the size of the array and how many elements can be stored.

It is declared as:

```
var name_of_array[size_of_array] type_of_array_elements  
or  
name_of_array      :=[size_of_array]      type_of_array_  
elements{elements_of_array}
```

Example:

```
var arrval[3] string  
arrval = ["abc","def","ghi"] // error "syntax error"  
arrval[0] = "abc" arrval[1] = "def"  
arrval[2] = "ghi" //will assign the values by accessing it's index  
fmt.Println(arrval) // [abc def ghi]  
arrval2 := [3]string{"abc","def","ghi"}  
fmt.Println(arrval2) // [abc def ghi]
```

□ **Structures:** This composite data type is similar to a database that can store multiple related items which point to a single entity. It groups the data of different types and creates a single type structure. The way you name struct fields and methods in Go (Golang) determines whether they are public or private. Identifiers that begin with a capital letter are exported (public), while those that begin with a lowercase letter are unexported (private).

□ Structures variables can be declared with the structure declaration or just like how other data types are declared.

The simple way to declare structures is as follows:

```
type StructName struct{
```



```

        field1 fieldType1
        field2 fieldType2
    }

```

In the given example, **StructName** is the name of the structure, and **field1** and **field2** are of types **fieldType1** and **fieldType2**, respectively. Here, the fields are accessed as:

```
StructName.field1
```

Example:

```

type company
    struct{
        companystring
        id string
        no_of_employees int16
    }

var comp company //creating variable of type company
comp.no_of_employees = 90 //accessing the members of
structure

var no_of_employees = comp.no_of_employees
fmt.Println(no_of_employees) // 90

var comp2 company
comp2 = company{ company:"abcpvt. ltd.",id:"SYRP7302",
no_of_employees:92} // initializing
structure

fmt.Println(comp2.no_of_employees) // 92

fmt.Println(comp2) // { abcpvt. ltd. SYRP7302 92}

```

□ **Slice:** Like arrays, slice also stores the same type of data collection. However, with slice, we are not bounded by its length. Slice acts as a strong interface with sequences. Unlike arrays, we can extend the length of slice by using the append function. On passing slices to a function, it creates a reference to an array and it

does not create a new copy. That is why slices are also of reference type. Slices will have the value when we assign a part of the array to them, and at that time, we can say that slice is referring to an array. The make function is used to declare slices which takes three component, Pointer, Length and Capacity. The total number of elements the slice can hold before it needs to allocate a new underlying array is called Capacity.

In Go, slices are declared as:

```
name_of_slice := make([] type_of_slice_elements, length_of_
slice, capacity_of_slice)
```

Example:

```
slice1 := make([]int,0)
fmt.Println(slice1) // []
slice2 := make([]int, 3)
slice2[0] = 123
slice2[1] = 456
slice2[2] = 789
fmt.Println(slice2) // [123 456 789]
slice3 := []string{"abc","def","ghi"}
fmt.Println(slice3) // [abc def ghi]
slice3 = append(slice3, "jkl")
fmt.Println(slice3) // [abc def ghi jkl]
slice4 := make([]string,5)
copy(slice4, slice3)
fmt.Println(slice4) // [abc def ghi jkl ]
slice2D := make([][]string, 2,3)
fmt.Println(slice2D) // [][]
slice2D[0] = []string{"a","b","c"}
slice2D[1] = []string{"d", "e"}
```

```
fmt.Println(slice2D) // [[a b c] [d e]]
```

□ **Map:** Maps can be defined by two elements: keys and values where key is a single element, while values can be an integer, string, array, structure, or any other map. Maps are like dictionaries in which the keys having the same name are not allowed. However, keys are case-sensitive, so we can keep the keys with the same names but with different cases if needed. In maps, every element is separated by a comma (,), and every key-value pair is separated by a colon (:). Maps are easy to extend as there is no need to define their size at the time of declaration. With maps, we can access any record by using keys. That is why this is also called a fast key lookup data structure. While declaring a map, we have to mention the data type of both the keys and the values. In Go, we have a function called make that is used to allocate and initialize slice, maps, and channels.

We can declare maps as:

```
var    name_of_variable    =    make(map[type_of_keys]
type_of_values)
```

or

```
name_of_variable    :=    make(map[type_of_keys]
type_of_values)
```

Example:

```
var mapval2 = make(map[int]string)
    mapval2[1] = "WORLD"
    mapval2[2] = "INDIA"
    fmt.Println(mapval2) // map[1:WORLD 2:INDIA]
    mapval := make(map[string]int)
    mapval["first"] = 16
    mapval["second"] = 32
    fmt.Println(mapval) // map[first:16 second:32]
```

□ **Channels:** Go is a multicore language and it supports concurrency and parallelism. By using channels, we can send the data to multiple Goroutines (functions) at the same time and thus avoid the problem of race condition, resource sharing, and many others that are difficult to debug. It allows the concurrent execution of multiple Goroutines where we can pass the value through the channel to one Goroutine and can get it again from the other Goroutine. Like maps, channels are also created with the use of the make function. We can also do buffering and iterating over the channels. We use the channel operator to transfer the values from the channel to the variable and vice-versa. Here is the syntax to create a channel below:

```
Channel_name := make(chan type_of_channel)
```

Example:

```
var channelval chan int
fmt.Println(channelval) //print undefined channel as <nil>
channelval2 := make(chan string)
go func() { channelval2<- "DONE" }() //passing value to
function
through channel
msg := <-channelval2 //reading channel
fmt.Println(msg) // DONE
```

Constants

Constant serves the values that remain fixed throughout the program. We cannot reassign the constant with another value. The value a constant has is called literal. In Go, it is required to have a constant value during the compile time. Any attempt to assign the value returned by a function call can cause an error since, because of function call, the compiler would not get the value of constant at compile time, and thus, it will throw an error. We have a keyword **const** to declare the constant. Like variables, constants also have multiple ways to declare and initialize the values as follows:

```

const name_of_constant = value_of_constant //untyped constant
const name_of_constant type_of_constant = value_of_constant // typed constant
const    name_of_firstConstant    name_of_secondConstant    =    value_of_    firstConstant,
value_of_secondConstant
const (
name_of_firstConstant = value_of_firstConstant name_of_firstConstant = value_of_secondConstant
)
const (
name_of_firstConstant  type_of_firstConstant firstConstant    =    value_of_
name_of_secondConstant    type_of_secConstant
secConstant    =    value_of_
)

```

- **Numeric constant:** A numeric constant can be an integer, float, complex, and rune. Integer constants represent decimal, octal, or hexadecimal values. They could be 32 bit or 64 bit. The floating-point constant values have an integer portion, a fractional portion, and an exponent. Complex constants are a combination of real and imaginary values. Runes are simply untyped 32-bit integers. They contain the value in the Unicode form or character. Runes give an integer value to each character, and that integer value is called the Unicode code point of a character. Here is an example to explain all about numeric constants:

Example:

```

const untypedConstInt = 123 //untyped integer constant

const untypedConstFloat = 3.97 //untyped floating-point constant
const typedConstFloat float64 = 4.87 //typed floating point constant
const untypedConstComplex = 56+2i //untyped complex constant

const typedConstComplex complex64 = 56+2i //typed complex
constant

```

```
const typedConstComplex1 complex64 = untypedConstInt //typed
complex constant
```

```
fmt.Println(typedConstComplex1) // 123+0i const untypedConstrune =
'G'
```

```
fmt.Println(untypedConstrune) // print unicode value of 'G' 71 const
untypedConstrune1 rune = untypedConstInt
fmt.Println(untypedConstrune1) // 123
```

- **String constant:** Any value enclosed within double quotes and declared with **const** keyword is a **string constant**. Mixing data types is not allowed in Go language. We can convert untyped data to other data types but we cannot convert the types of typed data.

Example:

```
const untypedConstString = "Golang is a great language" //untyped
string constant
```

```
const typedConstString string = "Golang is a great language"//
typed string constant
```

```
type myString string    //creating new type having same structure as
string
```

```
const customName myString = untypedConstString    //assigning
untyped data
```

```
const customName myString = typedConstString // error in assigning
typed data "conversion not allowed"
```

- **Boolean constant:** Boolean constants can have two values, either true or false that remain the same in the program. Like string constants, we cannot mix the data types with Boolean constants also.

Example:

```
const untypedConstBoolean = true // untypedboolean constant const
typedConstBoolean bool = false // typed boolean constant type myBool
bool //creating new type having same structure as bool
```

```
const customName myBool = untypedConstBoolean // assigning
untyped
```

```
data
```

```
const customName1 myBool = typedConstBoolean // error in
assigning typed data "conversion not allowed"
```

- **Iota:** Go supports the Iota identifier that increments with **const** declaration. Whenever compiler gets **const** block, the iota identifier resets to zero and increments by one after each non-blank line within **const** block.

Example:

```
const // iota resets to zero
(
C0 = iota // here iota is zero, increments after coming out from this line
_ // increments by one, iota is 1 here
C1 = iota // iota is 2
)
fmt.Println("C0:", C0) // C0: 0 fmt.Println("C1:", C1) // C1: 2 const //
resets iota to zero
(
//comments // doesn't consider this line so that iota would be the same.
C2 = iota
)
fmt.Println("C2:", C2) // C2: 0
```

Format specifiers

Format specifiers indicate the format of data contained by a variable during input and output operations. Format specifiers are used as operators with **Printf**, **Scanf**, and other similar functions for a specific type of data. Various formatting specifiers are used for specifying different data types. Each type of format specifier started with a percentage (%) sign, followed by a character that defined the type of data associated with the input/output function.

Here are the different types of format specifiers for each data type:

Common Format Specifiers:

%v	Prints the value of a variable in the default format
%#v	Prints the value of a variable in the form of Go-syntax
%T	Tells the type of value
%%	Dictates % sign

Table 1.1: Common Format Specifier

Format Specifier for Boolean:

Boolean	
%t	Prints the Boolean value of a variable

Table 1.2: Boolean Format Specifier

Format Specifiers for Integer:

Integer	
%b	Represents integer value in base 2
%c	Character corresponding to an integer value
%d	Represents integer value in base 10
%o	Represents integer value in base 8
%O	Represents integer value in base 8 with prefix O
%q	Represents a character in Go-syntax corresponding to an integer value

%x	Represents the lower case value in base 16
%X	Represents the upper case value in base 16
%U	Represents Unicode format of a value

Table 1.3: Integer Format Specifiers

Format specifiers for float:

Floating point	
%b	Decimal-less float value with the exponent power of two
%e	Scientific notation of value
%E	Scientific notation of value
%f	The decimal point with no exponent
%F	Same as %f
%g	%e for large exponents
%G	%E for large exponents
%x	Hexadecimal value with two exponents
%X	Hexadecimal value in upper case

Table 1.4: Float format specifier

Format specifiers for string:

String	
	Represents the value of string variable

%s	
%q	Represents the value with double quotes in Go-syntax
%x	Represents string value in base 16 lower case
%X	Represents string value in base 16 upper case

Table 1.5: *String format specifier*

Format specifier for pointers:

Pointer	
%p	Represents the value of pointer type

Table 1.6: *Pointer format specifiers*

Here is an example that explains the format specifiers well:

```
package main

import "fmt"

func main() {
    var str = "hello"
    var intval = 78
    var floatval = 56.8789
    var complexval = 67+6i
    var boolval = true
    var pointinterval = &str
    fmt.Printf("example of %%v, value is: %v \n", intval)
    fmt.Printf("example of %%#v, value is: %#v \n", str)
    fmt.Printf("example of %%T, value is: %T \n", intval)
    fmt.Printf("example of %% , used to print %%\n\n")
    fmt.Printf("example of %%t, value is: %t\n\n", boolval)
    fmt.Printf("example of %%b, value is: %b\n", intval)
```

```

fmt.Printf("example of %%c, value is: %c\n", intval)
fmt.Printf("example of %%d, value is: %d\n", intval)
fmt.Printf("example of %%o, value is: %o\n", intval)
fmt.Printf("example of %%O, value is: %O\n", intval)
fmt.Printf("example of %%q, value is: %q\n", intval)
fmt.Printf("example of %%x, value is: %x\n", intval)
fmt.Printf("example of %%X, value is: %X\n", intval)
fmt.Printf("example of %%U, value is: %U\n\n", intval)
fmt.Printf("example of %%b, value is: %b\n", floatval)
fmt.Printf("example of %%e, value is: %e\n", floatval)
fmt.Printf("example of %%E, value is: %E\n", complexval)
fmt.Printf("example of %%f, value is: %f\n", floatval)
fmt.Printf("example of %%F, value is: %F\n", floatval)
fmt.Printf("example of %%g, value is: %g\n", floatval)
fmt.Printf("example of %%G, value is: %G\n", floatval)
fmt.Printf("example of %%x, value is: %x\n", floatval)
fmt.Printf("example of %%X, value is: %X\n\n", floatval)
fmt.Printf("example of %%s, value is: %s\n", str)
fmt.Printf("example of %%q, value is: %q\n", str)
fmt.Printf("example of %%x, value is: %x\n", str)
fmt.Printf("example of %%X, value is: %X\n\n", str)
fmt.Printf("example of %%p, value is: %p \n", pointinterval)
}

```

Output:

example of %v, value is: 78

example of %#v, value is: "hello"

example of %T, value is: int

example of %, used to print %

example of %t, value is: true

example of %b, value is: 1001110 example of %c, value is: N

example of %d, value is: 78

example of %o, value is: 116

example of %O, value is: 0o116

example of %q, value is: 'N'

example of %x, value is: 4e

example of %X, value is: 4E

example of %U, value is: U+004E

example of %b, value is: 8004993526413866p-47

example of %e, value is: 5.687890e+01

example of %E, value is: (6.700000E+01+6.000000E+00i)

example of %f, value is: 56.878900

example of %F, value is: 56.878900

example of %g, value is: 56.8789

example of %G, value is: 56.8789

example of %x, value is: 0x1.c707fcb923a2ap+05

example of %X, value is: 0X1.C707FCB923A2AP+05

example of %s, value is: Hello

example of %q, value is: "hello"

example of %x, value is: 68656c6c6f

example of %X, value is: 68656C6C6F

example of %p, value is: 0x11806108

Conclusion

In this chapter, we learned how to name a specific value, declare a variable, and store it in the memory. We learned how various data types work with Go language. Each data type has its importance in the Go language. Some data types are built with a programming language, while some are referenced data types. We will discuss a few data types thoroughly in the

next chapters. Here, we learned the proper use of variables with input/output operations with the use of format specifiers. In further chapters, we will see various operators performing several operations on the specific types of data.

Questions

1. How is Go language a statically typed language?
2. What are primitive and non-primitive data types?
3. How to initialize the variables in structure?
4. What is the difference between array and slice in Go?
5. How to declare a map in Go language?
6. What are the channels in Go?
7. What is the format specifier in Go?

Join our book's Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

<https://discord.bpbonline.com>



CHAPTER 5

Operators

Introduction

In programming, there is a need to compute the data in various ways as per the requirement. Operators make this computation possible by performing a specific operation on a specific type of data. In this chapter, we will discuss these operators and how to use them with multiple types of data.

Structure

In this chapter, we will discuss the following topics:

- Understanding operators
- Types of operators
- Operator precedence

Objectives

We will look at operators in the context of Go (Golang) programming in this chapter. We will start with the operator's understanding and their importance in the programming language. The several kinds of operators that are accessible in Go, such as arithmetic, relational, logical, etc., will be further categorized and examined. Their use will be demonstrated through practical examples. We will also discuss operator precedence and

associativity in Go. You will understand how to use several operators in Go at the end of this chapter.

Understanding operators

An **operator** is a character that represents a specific mathematical, relational, or logical operation on a single or multiple operands to produce the result. Every programming language is incomplete without operators and operands. Operators can be built-in or user-defined, depending on the programming language. We have an almost similar set of built-in operators in every language. Golang has various operators to perform different types of operations.

Types of operators

Golang is rich in operators. The operators that we have in the Go language are as follows:

- **Arithmetic operator:** Arithmetic operators are used for performing arithmetic operations like addition, subtraction, multiplication, division, etc. These operators can be applied between one or two operands. According to the number of operands, we can divide the arithmetic operators into two types: binary and unary. Binary operators take two operands and produce a single result, while unary operators perform the operation on a single operand and return a single result.

Here is the list of arithmetic operators:

Operator	Description
+	Adds two operands
-	Subtracts the second operand from the first one
*	Multiplies both operands
/	Divides the first operand from the second one

%	Returns remainder on dividing the first operand from the second one
++	Increments integer/float value by one
--	Decrements integer/float value by one

Table 5.1: Arithmetic operators

Example:

```
var a int = 12
var b int = 3
fmt.Println("ex. of '+' operator:", a+b) // ex. of '+' operator: 15
fmt.Println("ex. of '-' operator:", a-b) // ex. of '-' operator: 9
fmt.Println("ex. of '*' operator:", a*b) // ex. of '*' operator: 36
fmt.Println("ex. of '/' operator:", a/b) // ex. of '/' operator: 4
fmt.Println("ex. of '%' operator:", a%b) // ex. of '%' operator: 0
a++;
fmt.Println("ex. of '++' operator:", a) // ex. of '++' unary operator:
13
b--;
fmt.Println("ex. of '--' operator:", b) // ex. of '--' unary operator: 2
```

- **Relational operator:** Relational operators stabilize a relation between two entities, divided on the basis of equality or inequalities. They always work between two operands. These operators are mainly used as conditions that provide a Boolean result. Here are some of the types of relational operators that we have in the Go language:

Operator	Description
==	Returns true if the value of two operands is same.
!=	Returns true if both operands have different values.

<	Returns true if the value of the first operand is less than the value of the second operand.
>	Returns true if the value of the first operand is greater than the value of the second operand.
<=	Returns true if the value of the first operand is less than or equal to the value of the second operand.
>=	Returns true if the value of the first operand is greater than or equal to the value of the second operand.

Table 5.2: Relational operators

Example:

```

var a int = 12
var b int = 3
fmt.Println("ex. of '==' operator:", a==b) // ex. of '==' operator:
false
fmt.Println("ex. of '!=' operator:", a!=b) // ex. of '!=' operator:
true
fmt.Println("ex. of '<' operator:", a<b) // ex. of '<' operator:
false
fmt.Println("ex. of '>' operator:", a>b) // ex. of '>' operator:
true
fmt.Println("ex. of '<=' operator:", a<=b) // ex. of '<='
operator: false
fmt.Println("ex. of '>=' operator:", a>=b) // ex. of '>='
operator: true

```

- **Logical operator:** Logical operators are used between two or multiple Boolean operands; they make an appropriate decision by returning a single Boolean result. If it returns true, the process will continue. Otherwise, it skips that particular process and continues. We have three types of logical operators in the Go language. These are as follows:

Operator	Description

&& (Logical AND)	Returns true if both the values are true, otherwise returns false.
 (Logical OR)	Returns true if any of the operands are true, otherwise returns false.
! (Logical NOT)	Works with the single operand. It returns true if the value of the operand is false and returns false if the value of the operand is true. It simply reverses the value of the operand.

Table 5.3: *Logical operators*

Example:

```
var a int = 12
var b int = 3
fmt.Println("ex. of '&&' operator:", a<b && a>b) // ex. of '
operator: false
fmt.Println("ex. of '&&' operator:", a>b && a<=12) // ex. of '
operator: true
fmt.Println("ex. of '||' operator:", a<b || a>b) // ex. of '
operator: true
fmt.Println("ex. of '||' operator:", a<b || a>30) // ex. of '
operator: false
fmt.Println("ex. of '!' operator:", !(a<b && a>b)) // ex. of '
operator: true
```

- **Bitwise operator:** Bitwise operators work as logical operators. The key difference is that a logical operator works on Boolean values while a bitwise operator works on bits, the binary representation of any value. While working with bitwise operators, operands are first converted into their binary form, and then they perform the specific bitwise operation. These operators work on an individual bit of an operand. Here are the

types of bitwise operators:

Operator	Description
& (Bitwise AND)	Performs AND operation on every bit of both operands and returns true if both are same. It returns 0 if both operands have bit 0 and vice versa.
 (Bitwise OR)	Performs OR operation on every bit of both operands and returns true if any of two-bit is 1. Returns 1 if one of the operands has bit 1.
^ (Bitwise XOR)	Performs XOR operation on every bit of both operands and return true if both bits are different. Returns 1 if one operand has bit 0 and the other one has bit 1, otherwise returns false.
<< (Left shift)	Bits of the first operand are shifted on the left for n times wherein n is the value of the second operand.
>> (Right shift)	Bits of the first operand are shifted on the right for n times wherein n is the value of the second operand.

Table 5.4: Bitwise operators

Example:

```
var a int = 50
var b int = 6
fmt.Println("a&b is:", a&b) // a&b is: 2
fmt.Println("a|b is:", a|b) // a|b is: 54
fmt.Println("a^b is:", a^b) // a^b is: 52
fmt.Println("a<<b is:", b<<2) // a<<b is: 24
fmt.Println("a>>b is:", a>>5) // a>>b is: 1
```

- **Assignment operator:** Assignment operators set or reset the value of a variable. They could be used to assign a value to a new variable or change the value of a variable by evaluating any specific expression.

These operators first evaluate an expression given on the right side of the assignment operator and then assign the final result to the variable written on the left of the assignment operator. We have multiple assignment operators in the Go language. These are as follows:

Operator	Description
=	Assigns the value from the right side to the left side variable.
+=	Adds the value of the left side variable to the value calculated on the right side and assign the final result to the left side variable.
-=	Subtracts the value calculated on the right side from the value of the left side variable and assigns the final result to the left side variable.
*=	Multiplies the value of the left side variable and the value calculated on the right side. Then assigns the final result to the left side variable.
/=	Divides the value of the left side variable from the value calculated on the right side. Then assigns the final result to the left side variable.
%=	Gets modulus on performing the above described divide operation and assigns the final result to the left side variable.
<<=	‘Shifts left’ the value of the left side variable by the value calculated on the right side and assigns the final result to the left side variable.
>>=	‘Shifts right’ the value of the left side variable by the value calculated on the right side and assign the final result to left side variable.
&=	Performs bitwise AND operation on the value of the left side variable and the values calculated on the right side. Then assigns the final result to the left side variable.
^=	Performs bitwise XOR operation on the value of the left side variable and the values calculated on the right side. Then assigns the final result to the left side variable.
=	Performs bitwise OR operation on the value of the left side variable and the values calculated on the right side. Then assigns the final result to the left side variable.
:=	Assigns the right side value to the left side variable that is previously not defined.

Table 5.5: Assignment operators

Example:

```
var a int = 50 //simple assignment
var b int = 5
a += b
fmt.Println("ex. of '+=' operator:", a)    // ex. of '+' operator: 55
a -= b
fmt.Println("ex. of '-=' operator:", a)    // ex. of '-' operator: 50
a *= 3
fmt.Println("ex. of '*=' operator:", a)    // ex. of '*' operator: 150
a /= b
fmt.Println("ex. of '/=' operator:", a)    // ex. of '/' operator: 30
a %= 7
fmt.Println("ex. of '%=' operator:", a)    // ex. of '%' operator: 2
b <<= 2
fmt.Println("ex. of '<<=' operator:", b)// ex. of '<<=' operator: 20
b >>= 2
fmt.Println("ex. of '>>=' operator:", b)// ex. of '>>='
operator: 5
a &= 10
fmt.Println("ex. of '&=' operator:", a)    // ex. of '&='
operator: 2
a ^= 10
fmt.Println("ex. of '^=' operator:", a)    // ex. of '^='
operator: 8
a |= 10
fmt.Println("ex. of '|=' operator:", a)    // ex. of '|=' operator: 10
c := "hello world"
fmt.Println("ex. of ':=' operator:", c)    // ex. of ':=' operator:
```

hello world

- **Miscellaneous operator:** Go provides some other language-specific operators. These are as follows:

Operator	Description
& (Address operator)	The address operator is a unary operator that provides the address of a variable at which that specified variable is stored in the memory.
* (Indication operator)	This is also a unary operator that provides the value of a variable stored at the address specified by an operand in the memory.
<- (Receive operator)	This operator is used for channel communication, either for sending to (channel <- value) or receiving from (value := <-channel) a channel.

Table 5.6: Miscellaneous operators

Example:

```
package main
import ("fmt"
"unsafe"
"reflect"
)
func main(){
    var a int64 = 100
    fmt.Println("size of variable a:", unsafe.Sizeof(a))
        // size of variable a: 8
    fmt.Println("size of variable a:", reflect.TypeOf(a).
    Size()) // size of variable a: 8
    fmt.Println("address of variable a:", &a) // address of
variable a: 0x1184e068
    fmt.Println("value of a at specified address:", *(&a))
```

```

        // value of a at specified address: 100
    }

```

Operator precedence

Operator precedence decides the execution priority of every operator so that expression can be evaluated properly. Operators that have high precedence are evaluated first while evaluating an expression. Associativity comes when we have two or more expressions having the same operator or operators of the same precedence. In this case, the expression is evaluated either left to right or right to left as per their associativity rules.

In the following table, associativity is well-described for every operator:

Category	Operator	Associativity
Postfix	() [] -> . ++ --	Left to right
Unary	+ - ! ~ ++ -- (type)* &sizeof	Right to left
Multiplicative	* / %	Left to right
Additive	+ -	Left to right
Shift	<<>>	Left to right
Relational	<<= >>=	Left to right
Equality	== !=	Left to right
Bitwise AND	&	Left to right
Bitwise XOR	^	Left to right
Bitwise OR		Left to right

Category	Operator	Associativity
Logical AND	&&	Left to right
Logical OR	 	Left to right
Assignment	= += -= *= /= %= >>= <<= &= ^= =	Right to left

Conclusion

In this chapter, we gained an understanding of the operators in a programming language. We learned the use of arithmetic, relational, logical, bitwise, assignment, and miscellaneous operators supported by Go language. We saw how the operators are parsed concerning each other with the understanding of operator precedence. It decides which operator needs to be evaluated first in an expression. After getting an idea of operators, it is time to understand the control flow in a program. Let us cover it in the next chapter.

Questions

1. What type of operators do we have in Go language?
2. What do unary operators mean?
3. Which operator performs equality/inequality operations?
4. Specify logical operators with an example.
5. How do bitwise operators work with Go language?
6. Which operator is used to get a value from a channel?
7. How do we get the address of a variable stored in memory?
8. What is the associativity of shift operators?

Join our book's Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

<https://discord.bpbonline.com>



CHAPTER 6

Control Structures

Introduction

While programming, it is necessary to understand the control flow of the program. It reduces the complexity and also helps in debugging. Our program consists of blocks that perform different types of tasks. A block that works on variables and transfers the control to a specific block on a specific condition is called a control structure. In this chapter, we will be discussing the decision-making process that we need to perform in our programs.

Structure

In this chapter, we will discuss the following topics:

- Introduction to control structures
- Branching statements
- Looping statements
- Jumping statements

Objectives

In this chapter, we will look at control structures in Go (Golang) to learn how they control a program's flow of execution. We will start by going over

branching statements, including if, else, and switch, using practical Go examples to show you how to write code that makes decisions. Next, we will look into looping and jumping statements, such as break and continue. You will obtain a comprehensive understanding of how to implement and manage program flow in Go by going through these examples.

Introduction to control structures

Control structures define the flow of source code. By analyzing certain conditions/ logics, we often need to alter the flow of the execution of a program. Every control statement defines a specific flow of source code. In the Go language, we have three types of control statements. They are covered in the subsequent sections.

Branching statements

These are decision-making statements consisting of one or multiple conditions. Based on these conditions, the flow of source code is as follows:

- **If/Else:** The **if** statement has a specific condition. If the condition given within **if**, the statement is true, then a block of **if** statement is followed; otherwise the block of **else** statement is followed. In any program, it is not necessary to have else case. Go program can consist of multiple if-else statements by using an else-if statement. The Go language is strict with unexpected newlines. It gives an error on giving a line break before curly brackets at the start of the block.

Example:

```
package main
import "fmt"
func main() {
var a int = 100
var b int = 20
if a==b {
    fmt.Println("a is equal to b")
}
```

```

    } else if a % b == 0 {
        fmt.Println("a is divisible by b")
    } else {
        fmt.Println("no match")
    }
}

```

Output:

a is divisible by b

- **Switch:** The **switch** statement is similar to multiple **if-else** statements where multiple cases replace these multiple conditions. The conditional expression that we need to compare is defined within the switch statement, and the evaluated result is compared with every case value. The case block whose value matches the evaluated expression is further followed by the compiler. We have an optional default case here, which is followed if no case value matches the conditional expression.

Example:

```

package main
import "fmt"
func main() {
    var a int = 100
    switch a {
    case 20:
        fmt.Println("value of a is 20")
    case 50:
        fmt.Println("value of a is 50")
    case 100:
        fmt.Println("value of a is 100")
    default:
        fmt.Println("switch loop processed")
    }
}

```

```
}
```

Output:

value of a is 100

In Go, a switch statement's fallthrough statement compels execution to go from one case to the next, even if the subsequent cases do not match. By default, Go exits the switch after the first match.

Example:

```
switch x {  
case 1:  
    fmt.Println("Case 1")  
    fallthrough  
case 2:  
    fmt.Println("Case 2")  
}
```

Output:

Case 1

Case 2

Looping statements

Looping statements are used for the repetition of a specific block of code based on a specific condition. The looping condition decides how many times that specific block of code would be repeated. Unlike some other languages, Go does not feature conventional while or do-while loops; however, the for loop can handle all looping requirements as follows:

- **for:** The for loop is used to execute a block of code multiple times, either indefinitely, for a predetermined number of iterations, or until a condition is satisfied.
- The for loop consists of three parts: initialization of a variable, the condition that defines the number of times iterations will take place, and the post statement that is executed after each iteration. At first, the variable gets initialized, and checks the condition. If the condition is

true, it follows the block of code within the for loop, then the post statement is executed. That statement changes the value of the variable which is checked again by the given condition on the next iteration. The iteration is stopped if it finds the condition false. Parentheses are not required in the for loop here. To define the code block, braces{ } are always used.

Example:

```
package main
import "fmt"
func main() {
    result := 1
    for i := 1; i < 10; i++ {
        result *= i
    }
    fmt.Println(result)
}
```

Output:

362880

Jumping statements

Jumping statements are used to skip a specific code or to jump a specific block of code in a program. We will explore each of these jumping statements in detail:

- **break:** The **break** statement is used for immediate termination from the loop at a specific condition. The break statement is used with **for** loop or **switch** statement. In nested loops, the **break** statement stops the execution on a given condition and jumps to the next statement out of the loop.

Example:

```
package main
import "fmt"
```

```

func main() {
/ Label the outer loop
OuterLoop:
for i := 1; i <= 3; i++ {
    for j := 1; j <= 3; j++ {
        fmt.Printf("i = %d, j = %d\n", i, j)

        if i == 2 && j == 2 {
            fmt.Println("Breaking from outer loop...")
            break OuterLoop
        }
    }
}
fmt.Println("Exited the loop.")
}

```

Output:

```

value of a: 0
value of a: 1
value of a: 2
value of a: 3
value of a: 4

```

- **Continue:** The **continue** statement is similar to the **break** statement. Instead of terminating from the loop, it skips the current iteration and jumps to the next one. The **continue** statement is also used at specific conditions within a loop.

Example:

```

package main
import "fmt"
func main() {

```

```

var a int = 0
for a < 5 {
    if a == 3 {
/* skip the iteration */ a = a + 1 continue
    }
    fmt.Printf("value of a: %d\n", a)
    a++
}
}

```

Output:

```

value of a: 0
value of a: 1
value of a: 2
value of a: 4

```

- **exit:** The **exit** statement is used to terminate the program execution by forcing a return to the operating system.

Example:

```

package main
import "fmt"
import "os"
func main() {
var a string = "hello"
fmt.Println("string value is: ", a) os.Exit(0)
a = a+" world"
fmt.Println("updated string value is: ", a)
}

```

Output:

```

string value is: hello

```


Conclusion

In this chapter, we discussed various control statements used to transfer the control to a specific block of code. These control structures are machine independent and easy to maintain. They provide the flexibility to control the statement as needed. By the looping statement, we learned how to run a specific block of code repeatedly. In the next chapter, we will see the execution of a specific code block by using functions.

Questions

1. What is the branching statement in the Go language?
2. What is the difference between a switch and a nested if-else statement?
3. What is the default case in a switch statement?
4. What do you mean by a looping statement?
5. What is the use of the break statement?
6. Which statement is used to skip the execution of a code block for a specific value in a loop?

Join our book's Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

<https://discord.bpbonline.com>



CHAPTER 7

Functions

Introduction

While programming, sometimes we use the same code again and again for different values at different places. In such scenarios, functions come into the role. A function is a segment of clean, reusable code that executes a single connected operation. The `main()` function has already been in earlier examples. In this chapter, we will explore how to define and use functions in Go, pass arguments, return values, and understand their role in making code modular and efficient. Let us dive in and discover the power of functions in Go.

Structure

In this chapter, we will discuss the following topics:

- Introducing functions
- Declaring functions
- Function arguments and return types
- Calling a function
- Call by value
- Call by reference
- Named return values

- Blank identifier
- Anonymous functions
- Closures
- Recursive functions
- Defer keyword
- Variadic functions

Objectives

By the end of this chapter, you will be able to efficiently perform repetitive tasks using a single block of reusable code. You will learn how functions help reduce complexity in your programs, how to pass multiple values to a function, and how to return specific types of values. Additionally, you will explore various ways to use functions, gaining flexibility and control in your Go programming.

Introducing functions

Functions play a major role in any programming language. Dividing a program into multiple functions reduces the complexity and the size of the program and makes the source code easy to understand. A function is a bunch of statements that perform a specific task and produce an output. The reason for using a function is the reusability of any specific code. We can use the code defined within the function recurrently with varying input values to get the desired output. The execution of every Go program starts with the main function that is defined in the main package, which is as follows:

- **main function:** In Go language, we have a package named main that tells the compiler that the package is not a shared library and it should be compiled as an executable program. The **main** package has a main function which is the entrance door for any executable program. The **main** function neither takes any argument nor returns any value. The initialization of the **main** function is done by the **init** function that is executed only once with the execution of the **main** function. We can easily execute all the tasks that we want to perform once throughout the

program at the initial phase with **init** function.

Example:

```
package main
import "fmt"
func init(){
    fmt.Println("initialization of main function")
}
func main() {
    fmt.Println("Program execution begins")
}
```

Output:

**initialization of main function Program
execution begins**

Declaring functions

The first line in function declaration provides the basic information of any function. It is called the signature of the function. It contains **func** keyword, name of the function, an optional list of comma-separated input values called arguments (or parameters), and return values with their specific types. Then, there is a pair of braces that specifies the body of a function performing a specific task.

Syntax:

```
func [function_name] (optional_inputval1 type, optional_inputval2 type..) (optional_returnval1 type, optional_returnval2 type){
    //body of function
}
```

Example:

```
func first_function(){
    fmt.Println("Hi....I am introducing a function")
}
```

Function arguments and return types

In the Go language, we can pass zero or a large number of arguments to a function. Similarly, the function can also return zero or multiple values. On calling the function, passed parameters to a function are called **actual parameters** while the values received by the function in function declaration are called **formal parameters**. On the basis of the signatures mentioned previously, let us create a function of two integers. Here is an example that takes two integer input values and returns the sum of them, which is also an integer value:

Example:

```
package main

import "fmt"

func sum(a int,b int) int{
    sum_value := a+b
    fmt.Println("sum is:", sum_value)
    return sum_value
}

func main() {
    a := 90
    b := 30

    sum(a, b)
}
```

Output:

sum is: 120

If we have multiple parameters of the same type, then we can write the type of parameter with the last parameter value to avoid repetition. This can be done as follows:

```
func sum(a, b int) int{
    sum_value := a+b
    fmt.Println("sum is:", sum_value)
    return sum_value
}
```

```
}
```

It can also be done as follows:

```
func sum(a, b int, c string) int{  
    sum_value := a+b  
    fmt.Println("sum is:", sum_value)  
    fmt.Println("value of c is:", c)  
    return sum_value  
}
```

Go functions have the capability of returning multiple values. We can also return error-type values in the Go language which will be covered in the later chapters. All possible return types are specified inside the parentheses with comma separated which is shown as follows:

```
func sum(a, b int, c string) (int, string){  
    sum_value := a+b  
    fmt.Println("sum is:", sum_value)  
    fmt.Println("value of c is:", c)  
    return sum_value, c  
}
```

Calling a function

We have declared the function so far. To use the function in a program, we call them within the **main** function with the required parameters. On calling the function, the control is transferred from the main function to the desired called function. The called function executes the statements defined within the function and returns to main when it gets the closing brace or any immediate terminate statement from the function. To call a function, we can use its name, followed by the required parameters. We can assign the values in a variable that is returned by the function.

Example:

```
package main  
  
import "fmt"
```

```

func sum(a, b int, c string) (int, string){
    sum_value := a+b
    return
    sum_value, c
}

func main() {
    a := 90
    b := 30
    var c string
    result, c := sum(a, b, "Hello Go")
    fmt.Println("Values returned from the function are: ", result, c)
}

```

Output:

Values returned from the function are: 120 Hello Go

In the Go language, arguments can be passed to a function in two ways, call by value and call by reference.

Call by value

In this method, the actual value of the variables is passed to a function. These values are copied to formal parameters defined with the function signature. So, the changes performed in formal parameters do not reflect the values of actual parameters/variables.

Example:

```

package main

import "fmt"

func call_by_value(x int, y int) {
    x = x-20
    y = y-30

    fmt.Printf("In function, value of a : %d\n", x ) // 80
    fmt.Printf("In function, value of b : %d\n", y ) // 170
}

```

```

}

func main() {
    var a int = 100
    var b int = 200

    fmt.Printf("Before calling function, value of a : %d\n", a )
    // 100

    fmt.Printf("Before calling function, value of b : %d\n", b )
    // 200

    call_by_value(a, b)

    fmt.Printf("After calling function, value of a : %d\n", a )
    // 100

    fmt.Printf("After calling function, value of b : %d\n", b )
    // 200
}

```

Output:

Before calling function, value of a : 100

Before calling function, value of b : 200

In function, value of a : 80

In function, value of b : 170

After calling function, value of a : 100

After calling function, value of b : 200

Call by reference

In this method, the address of a variable is passed as an argument while calling a function. In this case, format parameters have an address of a variable instead of a value. When any changes occur within the function, the changes at that give address are reflected. In this way, changes are also made with actual parameters.

Example:

```
package main
```



```

import "fmt"

func call_by_reference(x *int, y *int) {
    *x = *x-20
    *y = *y-30
}

func main() {
    var a int = 100
    var b int = 200

    fmt.Printf("Before calling function, value of a : %d\n", a )
    // 100

    fmt.Printf("Before calling function, value of b : %d\n", b )
    // 200

    call_by_reference(&a, &b)

    fmt.Printf("After calling function, value of a : %d\n", a )
    // 80

    fmt.Printf("After calling function, value of b : %d\n", b )
    // 170
}

```

Output:

Before calling function, value of a : 100

Before calling function, value of b : 200

After calling function, value of a : 80

After calling function, value of b : 170

Named return values

The Go function can return the name values as defined with the signature of function. These named return values are themselves defined by the Go that can be used in a function. The name of the return value should be different and not declared earlier in the program. Following is an example:

```
func sum(a, b int, c string) ( sum_value int, c_val string){
    sum_value = a+b
    c_val = c
    return sum_value, c_val
}
```

Golang facilitates “naked return” which does not have any value with the return statement but, by default, returns all values specified with the signature of the function. Naked return is good in use, but is not required in long functions as it is not necessary to return all the values mentioned at the top of the function.

Example:

```
package main
import "fmt"
func sum(a, b int, c string) ( sum_value int, c_val string){
    sum_value = a+b
    c_val = c
    return
}
func main() {
    a := 90
    b := 30
    var c string
    result, c := sum(a, b, "Hello Go")
    fmt.Println("Values returned from the function are: ", result, c)
}
```

Output:

Values returned from the function are: 120 Hello Go

Blank identifier

There can be cases where we get some values returned by the function that we do not want to use further in our program. Golang does not allow such unused values and throws an error. In such cases, blank identifiers are used. They tell the compiler that this particular variable is not going to be used in the program. The **blank identifier** is dictated by the underscore (_), which stores the value in a blank variable.

In the following example, we are calling the function sum as declared in the preceding example. Here, we do not need the variable c; so we are using the blank identifier for it.

Example:

```
package main

import "fmt"

func sum(a, b int, c string) (sum_value int, c_val string){

    sum_value = a+b

    c_val = c

    return

}

func main() {

    a := 90

    b := 30

    result, _ := sum(a, b, "Hello Go")

    fmt.Println ("Values returned from the function are:", result)

}
```

Output:

Values returned from the function are: 120

Using a function as a type

In the Go language, we can create our function type. The syntax for creating the function type is as follows:

Syntax:

```
type sub func(a int, b int) int
```

The preceding syntax creates a function type `sub` that accepts two integer arguments and returns an integer value.

Example:

```
package main

import "fmt"

type sub func(a int, b int) int

func main() {
    var result

    sub = func(a int, b int) int {
        return a-b
    }

    fmt.Println("Result is: ", result(100, 70))
}
```

Output:

Result is: 30

Anonymous functions

A function without any name is called **lambda** or **anonymous function**. These functions are assigned to a variable and that variable is used to call the function. These functions can be defined inside the main function as follows:

```
func main() {
    a := func(){
        fmt.Println("Introducing anonymous function")
    }

    a()
}
```

In the preceding example, we are not using any function name or variable to call the function. Following is another example of calling a function without any name:

```
func main() {
    func(){
        fmt.Println("Introducing anonmyous function")
    }()
}
```

Or

```
func main() {
    a := func() string{
        value := "Introducing anonymous function"
        return value
    }
    value := a()
    fmt.Println(value)
}
```

Both will print the same output: **Introducing anonymous function.**

Return anonymous functions

There can be some functions that return an anonymous function. Let us understand them with the help of the following example:

```
package main

import "fmt"

func return_anon() func(string){ //declare type of return value as function
    return func(message string){
        fmt.Println(message)
    }
}

func main() {
    return_anon_variable := return_anon()
    fmt.Println(return_anon_variable) //prints the address of function returned by 'return_anon'
```

```
function
    return_anon_variable("Hello")
}
```

Output:

0x47d880

Hello

Here, we have a function **return_an** on that returns a value of **function** type. In the body of the function, we return an anonymous function that prints the value passed in its argument. Now, in the **main** function, we capture the value returned by `return_anon` function that is of type function. **Return_anon_variable** is representing a function. That is why we can use the variable **return_anon_variable** to call the function and pass the required arguments.

Closures

A closure is a function or function value that references variables from out of its body. These functions can access and assign the values to a variable that are defined somewhere outside the function body. These functions are bound with that referenced variable.

Example:

```
package main
import "fmt"

func return_anon() func() int{
    count := 0
    return func() int{
        count++;
        return count
    }
}

func main() {
    return_anon_variable := return_anon()
```

```

    fmt.Println("value returned by 'return_anon' is:", return_anon_
variable) //prints '0x47da70'
    fmt.Println("count is:", return_anon_variable()) //prints "count is 1"
    fmt.Println("count is:", return_anon_variable()) //prints "count is 2" return_anon_variable2 :=
return_anon() //new instance
    fmt.Println("count is:", return_anon_variable2()) //prints "count is 1"
}

```

Output:

value returned by 'return_anon' is: 0x47da70

count is: 1

count is: 2

count is: 1

In the preceding example, the value returned by the function **return_anon** is a closure. Here, the anonymous function is capable of accessing the variable **count**. This anonymous function preserves the value of the **count** variable, and, thus keeps track of the number of times the function is being called. In the main function, **return_anon_variable** has a value returned by the function that is also a function. That is why when we print its value, it shows the address of that function. Whenever we call the function **return_anon_variable**, the value of the count variable is modified by the anonymous function. Here, **return_anon_variable2** works as a new instance of that function. That is why on calling through this variable, it returns the new value of count. Referencing the variable outside of its body is the key point in closures.

Recursive functions

A recursive function can be defined as a function that calls itself directly or indirectly at a finite number of times until the recursive condition becomes false. The recursive functions should have a termination point at some stage.

Example

```

func recursive_function(){

```

```
    recursive_function()
}
```

The preceding example is of infinite recursion. No condition will stop the loop and terminate the program. There must be a conditional statement to come out this of infinite recursion to continue further execution.

Example:

```
package main

import "fmt"

func factorial(num int) int {
    if num > 1 {
        return num * factorial(num-1)
    } else {
        return 1
    }
}

func main() {
    num := 10
    result := factorial(num)
    fmt.Println("Factorial result is:", result)
}
```

Output:

Factorial result is: 3628800

Defer keyword

The defer keyword is used to execute a function at the end after executing all the statements specified in the main function. As the compiler gets the defer keyword, the argument values passed within a function are evaluated with the execution of the deferring keyword, but function execution gets delayed. When the control returns from the main function, all the deferred functions are executed in the **Last In First Out** order.

Example:

```
package main
```



```
import "fmt"

func defer_fun() {
    fmt.Println("Hi I am introducing deferred function")
}

func main() {
    fmt.Println("Before defer statement")
    defer defer_fun()
    fmt.Println("After defer statement")
}
```

Output:

Before defer statement After defer statement

Hi I am introducing deferred function

Variadic function

Variadic functions are flexible with passing arguments. These functions can take an arbitrary number of arguments. It is necessary to specify the type of the last parameter. The symbol specifies that the function can take any number of arguments, whether zero or more.

Example:

```
func add(args ...int) int {
    total := 0
    for _, v := range args {
        total += v
    }
    return total
}

func main() {
    fmt.Println(add(1, 2, 3))
}
```

Output:

6

Here, **args** is a list of passed arguments while calling the function. We can pass any number of integer values as needed.

Conclusion

In this chapter, we learned how to reduce the lines of code by using functions. Concepts like “blank identifier,” “closures,” and “named return values” are new in the Go language. We saw that by using the `defer` keyword, some of the necessary tasks can be executed at the end. With the use of a variadic function, we get the flexibility with the number of arguments in a function. We gain an understanding of functions that call themselves and reduce complexity. In this chapter, we bound the code into a block using the function. Let us learn the function binding in a package in the next chapter.

Questions

1. What is the use of functions in Go language?
2. What is the importance of the main function in Go language?
3. What is the difference between call by value and call by reference?
4. What do you understand by named return values in a function?
5. In what scenario can blank identifiers be used?
6. What function is used to pass an arbitrary number of arguments?
7. How can we create a function type in Go language?
8. Which keyword is used to call a function at the end of the main function?
9. What is the use of the anonymous function in Go language?

CHAPTER 8

Packages in Go

Introduction

In the last chapter, we discussed the function that provides the feature of maintainability and reusability of code. The package facilitates the same functionality. The package is a set of functions or variables that can be accessed in any program by importing the specific package. Packages control the function definition and visibility. Some functions can be accessed outside that package, and some cannot be accessed. Let us understand packages in the Go language.

Structure

In this chapter, we will discuss the following topics:

- Introduction to packages
- Main package
- fmt package
- Importing a package
- Creating a custom package
- Nested package
- Installing third-party package
- Circular import

- Top Go libraries

Objectives

After studying this chapter, you will be able to use/import packages in your Go program and Install 3rd party packages in Go. You will learn the importance of main & fmt package in Go programs and create custom packages as per our requirement You will also work with nested packages in Go program.

Introduction to packages

A **package** is simply a directory/folder that organizes multiple **.go** files. These Go files/source code can be reused in multiple Go applications. As a package represents a directory, the naming convention for a package is the same as a system directory. An underscore is used for naming conventions with the files that contain test code or are bound to a specific platform. Each imported package has its unique name. The package is the starting point for any Go program. When we install Go, a workspace is reserved in the system directory for Go applications. This workspace is divided into multiple subdirectories and files. **src**, **pkg**, and **bin** directories are some of them at its root. In the **src** directory, we create Go programs that are organized as packages. So, it is clear that every Go program is a part of the package. The **bin** directory contains the executable Go programs. The **pkg** directory contains Go package objects. In the **pkg** directory, we have a standard package of Go language that is used for real-world applications.

Main package

As we have discussed, every Go program is a part of the package. Therefore, every program is declared within the main package to make it executable. Every program that is a part of the main package creates an executable binary file that calls the **main** function of the program. The main function is the key to any go program. The **main** package tells the compiler that the source code is an executable program and should not be treated like a shared library. While creating a shared library, there will not be any main

package or the main function. For an executable program, package main is the first statement and thus, the entry point.

fmt package

fmt stands for **format**. The format package implements formatted I/O with various functions. **fmt** packages make it easy to display data to the console. The package or **fmt** directory has various Go files that are used to scan or display data to the console.

The key concept is the same as the C language's **printf/scanf** functions. In Go, we use the symbol %, followed by a letter to indicate a placeholder as %v is a default placeholder while %s represents a string.

The Go standard library consists of pre-written packages included in the Go distribution. Developers can use these packages' key functionality to complete tasks like input/output, string manipulation, networking, and more. The **fmt** package in the Go standard library is one of the most commonly used packages.

Importing a package

Declaration of the import keyword tells the compiler that the defined packages need to be imported in the running source code. The path of the package is defined as a string literal within the import statement called **import path**.

Go searches for the import path in the following order: first, it looks in the local directory, then in the vendor directory, followed by **GOROOT/src**, and finally, if not found, it checks **GOPATH/src**.

The imported package can be accessed anywhere in the file, but it cannot be used throughout the package in which the source code is defined. In Golang, packages, functions, or variables can be imported from other packages.

The following are the two ways to import multiple packages within the same source code:

```
import "fmt"

import "math"
```

or

```
import(  
    "fmt"  
    "math"  
    "github.com/mlowicki/package_name" //importing custom package  
)
```

Imported packages can only refer to exported identifiers from the importing package. Unexported identifiers cannot be accessed, as they are the private members of the package. An identifier should follow two basic rules to be an exported identifier. These are as follows:

- The first letter of an identifier should be in upper case.
- The identifier should be defined in a package block (outside the function), or it should be any function or field name.

Let us begin with a simple executable Go program defined inside the **src** directory. First, create a directory within **src**, say **first**. It is not necessary to keep the name of Go file same as the directory name; it can be different. Create a file **first.go**, as shown:

```
//first.go  
  
package main // represents that the file belongs to main package  
  
import "fmt" // import a package named fmt that contains 'Println' method  
  
func main(){  
    fmt.Println("This is Go Program")  
}
```

Compile the program by go **build first.go**. An executable file of the source code within the same directory will be created. To run the source code, we can also run its executable file by **first.exe**.

In the case of a long import path, we can override the import path by an identifier to access the exported identifiers of a package. This is called an alias of a package.

Example:

```
package main  
  
import(  
    "github.com/mlowicki/package_name" //importing custom package  
)
```

```

    "fmt"

    m "math"
)

func main(){

    fmt.Println(m.Pow(6, 2)) // 36
}

```

We can use `_` to refer to a package. As we discussed in the previous chapters, `_` acts as a null container, so we can just refer to a package by an underscore but we cannot use it. The compiler throws an error if we declare a package but do not use it in our program. In such scenarios, we can refer to it by an underscore, as shown:

```
import _ "math"
```

An **init** function is defined by many Go packages and is invoked upon package import. By using “**import _ package_name**”, you’re only executing the **init** function of the imported package, even though you don’t use any of the package’s functions or types.

Using dot (`.`) as an alias of a package allows access to all the exported identifiers without using any qualifiers. We can access the exported identifiers directly in our program. Such an import statement is called **dot import** in the Go language.

Example:

```

package main

import (

    "fmt"

    ."math"
)

func main() {

    r := Pow(-6, 2)

    fmt.Println("example of dot import:", r)
}

```

Output:

example of dot import: 36

Dot imports should be used with caution as they can cause naming conflicts and make the code less clear. You won't be able to tell two packages apart if their function names are the same.

Creating a custom package

We have discussed how and why we use packages. Other than the standard library, we can create our package as per our needs. If we are creating an application and we need to perform some specific task again and again, then we can define such a task as a function/Go source file in a package. It is simple to import a package, access that function, and execute that task.

Consider a program in which we need to create a **Fibonacci series** and the factorial of a number. Instead of performing it in a single program, we will create a package having functions for both of them.

We have two environment variables in the Go language, **\$GOROOT** and **\$GOPATH**, at which the Go tool searches for any package. By default, this variable has a specific path, but we can set it to another path as per our requirements. We can check all default variables by using the command: **go env**

It will list all the variables with their default values.

Let us understand what is **\$GOROOT** & **\$GOPATH**:

- **\$GOROOT**: When installing Go, **GOROOT** is the default path where all the installed Go files take place. It is used by the Go tool to find the compiler and standard library of the Go language. **GOROOT** is similar to the **JAVA_HOME** variable in Java. When we compile Go from the source, the value of the **GOROOT** variable is embedded into the Go tool. We can also set the value of the **GOROOT** variable as per our wish, but it is not advisable because, in such cases, it will override the value in the Go tool. In this case, the Go tool will point the compiler from one version and standard libraries from another one.
- **\$GOPATH**: Outside **\$GOROOT**, the **GOPATH** variable is used to specify the workspace directory for all Go projects or third-party libraries. For every program, the Go tool searches the imported

package in the **\$GOROOT** or **\$GOPATH/src** directory. By default, **GOPATH** is set to the user's home directory in any operating system. We can set multiple paths to the **\$GOPATH** variable. In Unix, the values of **GOPATH** variables are separated by a colon, while in Windows, these values are separated by a semicolon.

To create a custom package in Go, first initialize a Go module inside your project directory through the following command:

go mod init myproject

Now, let us make a directory named **custom_package**. Within **custom_package** directory, create a package (directory) named **factFib**. Within this package, we will create two Go files, one for finding the factorial and the other one for the Fibonacci series, which is defined as follows:

```
//fact.go

package factFib

func Factorial(num int)int{
    if num> 1 {
        return num * Factorial(num-1)
    } else {
        return 1
    }
}
```

Here is the code for the Fibonacci series:

```
// fib.go

package factFib

import "fmt"

func Fibonacci(num int){
    f1 := 0
    f2 := 1
    nextnum := 0
    for i := 1; i <= num; i++ {
```

```

        fmt.Println(f1)
        nextnum = f1 + f2
        f1 = f2
        f2 = nextnum
    }
}

```

Now, create an executable Go program to use this package in our workspace. Let us create a file name **myprogram.go**, as shown:

```

// myprogram.go
package main //indicates this is an executable file
import "fmt"
import "custom_package/factFib" //importing custom package 'factFib'
func main(){
    number := 10
    fmt.Println(factFib.Factorial(number)) //calling exported function 'Factorial'
    fmt.Println("Fibonacci series is:")
    factFib.Fibonacci(10) //calling exported function 'Fibonacci'
}

```

Here, while importing the package **factFib**, the tool will look for two directories, **default_path_of_GOROOT/src/custom_package/factFib** either for **\$GOROOT** or **default_path_of_GOPATH/src/custom_package/factFib** for **\$GOPATH**. Let us execute the program and see the output:

3628800

Fibonacci series is:

```

0
1
1
2
3

```

5
8
13
21
34

Nested packages

In the Go language, we can create a package inside a package. With the nested package, we provide a relative path of the package at the place of the import path.

Let us begin with creating a nested package named **nested_package** within the package **factFib** that we created previously. Within **nested_package**, create a file **nestedfile.go**, as shown:

```
// nestedfile.go  
  
package nested_package  
  
import "fmt"  
  
func Nested_function(){  
    fmt.Println("Here is the example of nested package")  
}
```

Use the following package in the Go program as shown:

```
// myprogram.go  
  
package main  
  
import "custom_package/factFib/nested_package"  
  
func main(){  
    nested_package.Nested_function()//prints 'Here is the example of  
    nested package'  
}
```

In Go (before 1.17), you had to make distinct **go.mod** files for every submodule in order to handle its dependencies separately if you had nested packages or submodules in subdirectories. Only the dependencies for the

module being executed would be cleaned up by running `go mod tidy`; other submodules would remain unaffected.

Go 1.17 and later versions eliminate the need for distinct `go.mod` files for every nested package. `Go mod tidy` will remove unnecessary dependencies from the entire project, including nested packages, making dependency management easier. A single `go.mod` at the project root can handle all dependencies.

Installing third-party package

Go facilitates cloning remote code in Go's `src` package directory. We can get any package from GitHub or anywhere else and use it in our program directly. There is a need to specify the import path of the third-party package as GitHub if we are using the package from GitHub.

When someone publishes a code/package on GitHub, they must provide two things: user-name and repository, or, we can say, package path. To use the package in our code, we can import files from http://github.com/user_name/repo_name and save them in the `src/github.com/user_name/repo_name` directory. We can also import the package by using the link `github.com/user_name/repo_name` (import `github.com/user_name/repo_name`). Here, `repo_name` is the package that we are going to use in our program.

Circular import

Circular import is importing such packages where two or more modules depend on each other. Circular imports are not allowed in the Go language. If the compiler detects any import cycles, it throws a compile-time error.

Let us consider a case of circular import. Suppose we have a package **one** that is accessing an exported member of another package **two**. In package **two**, we are importing and accessing the exported member of package **one**. So here, both the packages are dependent on each other.

Let us create package **one**, as shown:

```
// one/one.go package one
```

```
import (
```

```

    "fmt"
    "custom_package/circular_import/two"
)
func One_function(){
    One_val := "Variable of package one"
    fmt.Println(One_val) two.Two_function()
}

```

Here is the code for package **two**:

```

// two/two.go package two
import (
    "fmt"
    "custom_package/circular_import/one"
)
func Two_function(){
    Two_val := "Variable of package two"
    fmt.Println(Two_val)
}

```

Now, create an executable Go file to use package one, as follows. In this file, we are calling an exported method of package one.

```

// circularimport.go
package main
import (
    "custom_package/circular_import/one"
)
func main(){
    one.One_function()
}

```

On executing this program, the Go compiler will show the following error:

```

C:\Workspace\src\use_circularimport>go run circularimport.go
import cycle not allowed
package main
    imports custom_package/circular_import/one
    imports custom_package/circular_import/two
    imports custom_package/circular_import/one
C:\Workspace\src\use_circularimport>_

```

Figure 8.1: Circular import

In **circularimport.go**, on importing package one, it finds the statement to import package two. While importing package two, it again finds the statement to import package one. In this way, cyclic dependency exists here.

An analysis of circular dependency concluded that there is no need to have these dependencies in any Go program. We can create more or less number of packages to avoid such situations.

To overcome such a problem, we can create a new package as an interface and define all the members of package **two** that are accessed by package **one**. By doing so, we can import that new package in package **one** rather than package **two**. However, in some cases, this will not be the proper solution. To completely avoid import dependency, we can merge both the packages in a single one. In such a case, all the members can be accessed easily and no circular dependency would be there. Here, we are merging both packages:

```

// one/one.go package one

import (
    "fmt"
    // "custom_package/circular_import/two"
)

func One_function(){
    One_val := "Variable of package one"
    fmt.Println(One_val)
    Two_function()
}

func Two_function(){
    Two_val := "Variable of package two"

```

```
    fmt.Println(Two_val)
}
```

Here is the output on executing **circulatimport.go**:

Variable of package one

Variable of package two

Top Go libraries

Go has a rich set of libraries. Here is a list of some of the top libraries that we have in the Go language:

- **Go kit:**
 - o The solutions for Go-related microservices in terms of system observability, infrastructure integration, program design, etc.
 - o Avoids common pitfalls.
 - o Builds robust, reliable, and maintainable microservices.
- **GORM:**
 - o Best ORM library for Golang.
 - o Converts data between incompatible type systems.
 - o Deals with relational database.
 - o Has a feature of callbacks, transactions, logger, associations, composite primary key, and many others.
- **cli:**
 - o A part of the **Data Plane Development Kit (DPDK)**.
 - o Helps to build command-line applications in Go.
 - o Makes it easy to create flags, bash-completion, routines, and even generated help text.
 - o Provides support for colors and cursor movement.
 - o Has the ability to add and remove commands, files, and directories dynamically.
- **Vegeta:**

- o A versatile HTTP load testing tool and library.
- o Can also be used as a command-line utility.
- **Fuzzy:**
 - o This package includes a fuzzy matching algorithm.
 - o Provides the results in milliseconds with quality matches.
 - o It is even Unicode aware.
- **Kubernetes:**
 - o An open-source container orchestration system for application deployment, scaling, and management.
 - o Has automatically generated clients.
 - o Contains a minimal client for the Kubernetes API.
- **Testify:**
 - o A set of multiple packages that are used to test the code.
 - o The **assert** package in it provides assertion functions for testing.
 - o The **http** package checks for Http activity.
 - o The **mock** package makes it possible to mock the object and verify calls.
 - o The suite package provides a basic structure of structs as test suites.
- **gocheck:**
 - o Helps in error-catching logic with proper error reports.
 - o Provides support for expected failures.
 - o Manages temporary directories.
 - o Facilitates comments surrounding checks on failure reports.
- **glog:**
 - o Facilitates the use of flags at the command line.
 - o Implements logging analogous to the Google-internal C++ INFO/ERROR/V setup.
 - o Provides functions for warning, error, fatal, info, and others.

- **color:**
 - o Provides the multiple color option for Go applications.
 - o Provides supports for mixing colors.
 - o Facilitates background colors, font style as bold, italic, and many others.

Conclusion

In this chapter, we learned the use of packages in the Go program. We saw how public and private members are declared in a package and how to access them in code outside the package. In every Go program, the main package is imported first. Here, we understand the need for the main package. We have acquired the concept of nested package and circular import (package dependency) in Go. We learned all about packages in this chapter. In the next few chapters, we will thoroughly discuss various data types.

Questions

1. What is the importance of the main package in Go?
2. What is the need for the fmt package in Go language?
3. Which path does a Go compiler look in when importing a package?
4. How do we declare private members in a package?
5. What is dot import in Go?
6. How can we create custom packages in Go? Explain with an example.

Join our book's Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

<https://discord.bpbonline.com>



CHAPTER 9

Arrays and Slices

Introduction

In this chapter, we will explore the foundational data structures of Go: arrays and slices. We will look at array functions and iteration, as well as how to declare and initialize arrays, including multi-dimensional arrays. We will then explore slices, which provide more versatility than arrays. Along with exploring practical slice functions, you will learn how to generate slices from arrays and structs. Let us get started!

Structure

In this chapter, we will discuss the following topics:

- Introduction to array
- Declaring an array
- Initializing an array
- Multidimensional array
- Functions of an array
- Iteration over an array
- Introduction to slices
- Declaring slices
- Slices from an array

- Slice from slice
- Slice from structs
- Slice internals
- Slice using the make function
- Two-dimensional slices
- Slice functions

Objectives

You will be proficient with Go's arrays and slices by the end of this chapter. You will gain knowledge about array definition, including how to declare, initialize, and work with multi-dimensional arrays. You will look into array-specific built-in methods and learn how to efficiently iterate over its elements. You will also eventually be able to use these ideas in your Go programs to efficiently manage data gathering.

Introduction to array

Array and slices are the first two collection types in the Go language. An array is a set of similar elements, such that an element can be accessed by its index value. The index always starts with zero in the array. Arrays are of fixed length in the Go language. Arrays are value types in Golang. On assigning an array to a variable or passing to a function, a complete array will be copied rather than its reference.

Declaring an array

An array of type **T** has **n** no. of values as declared:

```
var name_of_array[n]T
```

or

```
var a [3] int
```

Declare an array that has three integer values.

By default, all array elements are initialized with zero if they are of the type integer. If an array is declared of the string type, then it will be initialized

with an empty string.

Example:

```
package main

import "fmt"

func main() {
    var array_int[10] int
    var array_float[10] float32
    var array_string[10] string
    var array_complex[10] complex128
    fmt.Println("Int type array:", array_int)
    fmt.Println("Float type array:", array_float)
    fmt.Println("String type array:", array_string)
    fmt.Println("Complex type array:", array_complex)
}
```

Output:

Int type array: [0 0 0 0 0 0 0 0 0 0]

Float type array: [0 0 0 0 0 0 0 0 0 0]

String type array: []

Complex type array: [(0+0i) (0+0i) (0+0i) (0+0i) (0+0i) (0+0i) (0+0i) (0+0i) (0+0i) (0+0i)]

Initializing an array

Assigning values to an array is called an **array initialization**. After declaring an array, we can initialize it with predefined values by accessing its index, which is shown as follows:

```
var arr [5] int = [5] int {1, 2, 3, 4, 5}

fmt.Println(arr) // [1 2 3 4 5]
```

Or

```
var arr [5]int

arr[0] = 1
```

```
arr[1] = 2
arr[2] = 3
arr[3] = 4
arr[4] = 5

fmt.Println(arr) // [1 2 3 4 5]
```

We can declare and initialize an array simultaneously with syntax:

```
name_of_array := [size] type_of_values {values}

arr := [5] int {1, 2, 3, 4, 5}

fmt.Println(arr) // [1 2 3 4 5]
```

It is not necessary to initialize an array with the same number of values that we define at the time of declaration. We can initialize an array with lesser values; the remaining indexes will have their default values.

Example:

```
var arr [5]int

arr[1] = 2
arr[3] = 4

fmt.Println(arr) // [0 2 0 4 0]
```

Or

```
arr := [5]int{1, 2, 3}

fmt.Println(arr) // [1 2 3 0 0]
```

In Golang, we can use an ellipsis to specify the implicit length. By using an ellipsis, we can initialize an array with as many values as we want. An ellipsis is dictated by three dots within square brackets [...]. If we are using an ellipsis, the size of an array is equal to the number of values that we are initializing an array with.

Example:

```
arr := [...] int {1, 4, 7, 2, 7, 9, 3, 5}

fmt.Println(arr) // [1 4 7 2 7 9 3 5]
```

Need for an array

Let us consider an application in which we need to use the names of all the students of a class. Let us say that we write it as the following:

```
package main

import "fmt"

func main() {

var name1 string = "aman"

var name2 string = "rahul"

var name3 string = "geet"

var name4 string = "preeti"

fmt.Printf("Names are %q, %q, %q and %q", name1, name2, name3,
name4) //Names are "aman", "rahul", "geet" and "preeti"

}
```

Here, we are keeping the record of every name, but the problem is that in a large application, we will not know how many students are there. What if we need to add some more names here at run time or delete some records? To overcome such problems, arrays/slices are a great solution. They decrease the program size. Also, we can grow or shrink an array as per our need. Here is the program with arrays:

```
func main() {

names:[...] string{"aman", "rahul", "geet", "preeti"}

fmt.Printf("Names are %q", names)

}
```

Multidimensional arrays

Till now, we have discussed single dimensional arrays. Multidimensional arrays can be defined as arrays of an array. A **multidimensional array** can be broken down into multiple single-dimensional arrays. In such arrays, the data is stored in the tabular form (row-major order). The size of a multidimensional array is defined by its dimensions that is equal to the multiplication of the sizes of all dimensions.

These arrays are declared as the following:

Name_of_array := [dim1][dim2]..type_of_array{values_of_array}

The number of square brackets specifies the number of dimensions of the array.

Example:

```
arr := [3][2] int {  
    {1, 2},  
    {4, 5},  
    {7, 8},  
}  
  
fmt.Println(arr) // [[1 2] [4 5] [7 8]]
```

Preceding is a two-dimensional array of size 3x2. The declaration of an array represents that it contains three single-dimensional arrays of the length of two. Like 1D arrays, there is no need to initialize an array with all the elements. Uninitialized arrays will take their default values.

Example:

```
arr := [3][2] int {  
    {1, 2},  
    {7},  
}  
  
fmt.Println(arr) // [[1 2] [7 0] [0 0]]
```

We can initialize multidimensional in another way as well:

```
var arr [3] [3] int  
arr[0] = [3] int {1, 2, 3}  
arr[1] = [3] int {4, 5, 6}  
arr[2] = [3] int {7, 8, 9}  
  
fmt.Println(arr) // [[1 2 3] [4 5 6] [7 8 9]]
```

Functions of an array

Here are some functions that work with arrays in Go.

Length of an array

Go provides a built-in function to get the length of an array. It tells the number of elements an array can store. As arrays are of fixed size in Go, the compiler throws an error (out of bound) on accessing/inserting an element out of its size.

Example:

```
func main() {  
    arr1D := [...] int {1, 4, 7, 2}  
    arr2D := [3][2] int {  
        {1, 2},  
        {7},  
    }  
    fmt.Println("Length of 1D array", len(arr1D)) fmt.Println("Length of 2D array", len(arr2D))  
    //tells number of rows it can have.  
}
```

Output:

Length of 1D array 4 Length of 2D array 3

Array comparison

Two arrays are called equal if they both are of the same type and size, have the same elements, and all elements are in the same order in both arrays. The comparison operator is dictated as `==`. It returns the Boolean result true if both the arrays are equal, else false. It first checks for the data type of both arrays and then matches the values by index.

Example:

```
func main() {  
    arr1 := [4] int {1, 4, 7, 2}  
    arr2 := [4] int {1, 4, 7}  
    arr3 := [3][2] int {  
        {1, 2},
```

```

{7},
}
arr4 := [3][2] int {
{1, 1},
{7},
}
arr5 := [4] int{1, 4, 7, 0}
fmt.Println("arr1 and arr2", arr1==arr2)
fmt.Println("arr3 and arr4", arr3==arr4)
fmt.Println("arr2 and arr5", arr2==arr5)
}

```

Output:

arr1 and arr2 false

arr3 and arr4 false

arr2 and arr5 true

arr1 and **arr2** are of the same size but **arr2** takes default value 0 at its last index. So, the result is **false**.

arr3 and **arr4** are both of the same size, but the elements are different. So, the result is false.

arr2 and **arr5** both have the same size and elements. So, **arr2** and **arr5** are equal and the result is true.

Copying an array

Copying an array is just assigning an array into another one. Since in Golang, an array behaves like a value instead of reference, so changing any value of one array does not have any effect on the other one.

Example:

```

func main() {
arr1 := [4] int {1, 4, 7, 2}
arr2 := arr1 arr2[2] = 90

```

```
fmt.Println("arr1 is", arr1) // arr1 is [1 4 7 2]
fmt.Println("arr2 is", arr2) // arr2 is [1 4 90 2]
}
```

Passing an array to a function

An array can be passed as an argument to any function. In a function declaration, the parameter should be defined as an array that tells the compiler that an array is being passed as an argument for a function call.

Example:

```
func main() {
    arrval := [4] int {1, 4, 7, 2}
    fmt.Println("sum of array elements is:", sum(arrval))
}

func sum(arr [4]int) int{
    sum := 0
    for i:=0; i<len(arr); i++){
        sum = sum + arr[ i ]
    }
    return sum
}
```

Output:

sum of array elements is: 14

Iteration over an array

When creating an application, there can be many cases in which we might need to iterate over an array. For example, to find the sum of all the elements of an array, we have to access every element of an array. We can easily iterate an array by using the for loop, which is shown as follows:

Example:

```
func main() {
```

```

arr :=[...] int {1, 4, 7, 2, 7, 9, 3, 5}

sum := 0

for i:=0; i<len(arr); i++){
    sum = sum +arr[i]
}

fmt.Println("sum of values:", sum) // sum of values: 38
}

```

Here, the **len** function is used to get the length of an array, but the **range** keyword can also be used to directly iterate over the elements and their indices, which is shown as follows: Example:

```

func main() {
    arr := [...] int {1, 4, 7, 2}
    sum := 0
    for index, value := range arr{
        fmt.Println("index is:", index)
        fmt.Println("value is:", value)
        sum = sum + value
    }
    fmt.Println("sum of values:", sum)
}

```

Output:

index is: 0

value is: 1

index is: 1

value is: 4

index is: 2

value is: 7

index is: 3

value is: 2

sum of values: 14

Here, **range arr** always returns two values, index and the value at that index. It is compulsory to write both the variables on the left side. If we write **value:= range arr**, by default, it will return the indexes, not the values. In case we do not need indexes in our program, we can assign them to the blank identifier. Otherwise, the compiler will give an error.

```
package main

import "fmt"

func main() {
    arr := [...] int {1, 4, 7, 2}
    sum := 0
    for index, value:= range arr{
        sum =sum + value
    }
    fmt.Println("sum of values:", sum)
}
```

Figure 9.1: Array program

The error is shown in the following figure:

```
C:\Workspace\src\first>go run p1.go
# command-line-arguments
.\p1.go:10:6: index declared and not used
C:\Workspace\src\first>
```

Figure 9.2: Index not used error

The solution of the preceding issue is shown as follows:

```
func main() {
    arr := [...] int {1, 4, 7, 2}
    sum := 0
    for _, value := range arr{
        sum = sum + value
    }
    fmt.Println("sum of values:", sum)// sum of values: 14
}
```

So far, we have discussed arrays, but a few points are missing with arrays in

the Go language. As arrays are of fixed size, we cannot append any element at run time. We have to define the size of the array at the compile time. This increases time complexity. With arrays, we are not allowed to use a variable to define its array size. If we declare an array as shown, the compiler will give an error of '**invalid array length size**':

```
func main() {  
    var size int = 4  
  
    arrval := [size] int {1, 4, 7, 2}  
  
    fmt.Println("sum of array elements is:", arrval)  
}
```

Introduction to slices

To overcome the problems, we face with arrays, Go provides the more powerful, dynamically data type slices. **Slice** is a segment of an array that holds some elements of the same data type. Slices are typed to the elements they contain instead of their size.

Declaring slices

Just like an array, slices are indexable and declared in the same way as arrays without specifying their size:

Syntax: `var s [] int`

It declares a nil slice of integer type as a slice is called nil if it does not refer to an array.

Initializing slices

A slice can be initialized with a slice literal as we do with an array without defining its size. As slices are the reference of arrays, while creating slices using literals, it first creates an array and then returns a reference to the slice. Basically, slices do not contain any data. Data is stored by an array, and the reference of that array is defined as a slice.

Example:

```
func main() {
```

```

var slice1 [] int = [] int {1, 2, 3, 5}

slice2 := [] int {1, 2, 3, 5}

fmt.Println("slice with literal", slice1) // slice with literal [1 2 3 5]

fmt.Println("slice short hand declaration", slice2) // slice short hand declaration [1 2 3 5]

}

```

Slices from an array

A slice is a segment of an array. We can create a slice from an array also. To create a slice from an array, we need to specify two indices, ‘low’ and ‘high’, where low indicates the starting index of an array and high indicates the next of last index. This means that we are getting all array elements from index ‘low’ to ‘high-1.’ These two indices are separated by a colon at the time of the slice declaration. By default, low has 0 indexes, and high contains the length of the array. If these indices are not specified, the compiler assumes their default values to create a slice.

Syntax:

```
var name_of_slice [] type_of_slice := name_of_array[low:high]
```

Example:

```

func main() {

var arrayval [10] int = [10] int {1, 2, 3, 4, 5, 6, 7, 8, 9, 10}

slice1 := arrayval[3:7]

slice2 := arrayval[:6]

slice3 := arrayval[5:]

slice4 := arrayval[:]

fmt.Println("slice from array:", slice1)

fmt.Println("slice with default low indice:", slice2)

fmt.Println("slice with default high indice:", slice3)

fmt.Println("slice with default low and high indice:", slice4)

}

```

Output:

slice from array: [4 5 6 7]

slice with default low indice: [1 2 3 4 5 6]

slice with default high indice: [6 7 8 9 10]

slice with default low and high indice: [1 2 3 4 5 6 7 8 9 10]

Slice from slice

We can create a slice from an existing slice in the same way that we are creating it from an array. Re-slicing does not copy the data, but a new slice is created which refers to the original array also.

Example:

```
func main() {  
    var arrayval [10] int = [10] int {1, 2, 3, 4, 5, 6, 7, 8, 9, 10}  
    slice1 := arrayval[1:8]  
    slice2 := slice1[:6]  
    fmt.Println("slice from array:", slice1)  
    fmt.Println("slice from existing slice", slice2)  
}
```

Output:

slice from array: [2 3 4 5 6 7 8]

slice from existing slice [2 3 4 5 6 7]

Here is an example that shows slices are just reference of an array:

```
func main() {  
    var arrayval [10] int = [10] int {1, 2, 3, 4, 5, 6, 7, 8, 9, 10}  
    slice1 := arrayval[1:8]  
    slice2 := slice1[:6]  
    fmt.Println("slice from array:", slice1)  
    fmt.Println("slice from existing slice", slice2) arrayval[2] = 90  
    fmt.Println("slice from array after modifying an array:", slice1)  
    fmt.Println("slice from existing slice after modifying an array",  
    slice2)
```



```
}
```

Output:

slice from array: [2 3 4 5 6 7 8]

slice from existing slice [2 3 4 5 6 7]

slice from array after modifying an array: [2 90 4 5 6 7 8]

slice from existing slice after modifying an array [2 90 4 5 6 7]

Slice from structs

A **struct** is a set of a different type of fields. We will discuss it in depth later. Just like an array, slices can be initialized with structs also. These are **struct** type of slices.

Example:

```
func main() {  
    record := []struct {  
        roll_no int  
        name string  
        present bool  
    }{  
        {1, "Rahul", true},  
        {3, "Akansha", false},  
        {5, "Ram", true},  
    }  
    fmt.Println("structs type slice is:", record)  
    //structs type slice is: [{1 Rahul true} {3 Akansha false} {5 Ram true}]  
}
```

Slice internals

Generally, a slice consists of a pointer to an array segment, the length of the array segment and the maximum to which the slice can be expanded is

shown as follows:

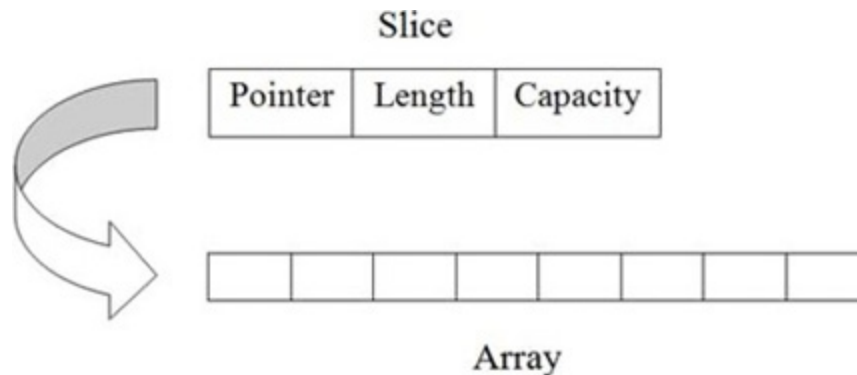


Figure 9.3: Slice structure

Let us discuss the segments in detail:

- **Pointer:** We have already discussed that slices do not store any data; they only refer to an array or a section of an underlying array. Initializing a slice from an array and modifying the array elements changes the slice value as well. We have seen an example of this. Similarly, changing the element value of a slice reflects its referred array.

Example:

```
func main() {  
    var arrayval [10] int = [10] int {1, 2, 3, 4, 5, 6, 7, 8, 9, 10}  
    slice1 := arrayval[1:8]  
    slice2 := slice1[:6]  
    fmt.Println("original array", arrayval)  
    fmt.Println("slice from array:", slice1)  
    fmt.Println("slice from existing slice", slice2)  
    slice1[0] = 178  
    fmt.Println("array after modifying slice", arrayval)  
    fmt.Println("slice from array after modifying slice:",
```

```

slice1)

fmt.Println("slice from existing slice after modifying
slice", slice2)

}

```

Output:

original array [1 2 3 4 5 6 7 8 9 10]

slice from array: [2 3 4 5 6 7 8]

slice from existing slice [2 3 4 5 6 7]

array after modifying slice [1 178 3 4 5 6 7 8 9 10]

slice from array after modifying slice: [178 3 4 5 6 7 8]

slice from existing slice after modifying slice [178 3 4 5 6 7]

- **Length:** Length is the number of elements referred by the slice. As arrays, we use the **len** function to get the length of a slice. It returns the total number of elements that a slice contains.

Example:

```

func main() {

var arrayval [10] int = [10] int {1, 2, 3, 4, 5, 6, 7, 8,
9, 10}

slice1 := arrayval[1:8]

fmt.Println("length of slice", len(slice1)) //length of slice 7

}

```

- **Capacity:** For the same slice, its length and capacity could be the same or different. Length is coupled with a slice, while capacity is linked with an array. The **capacity** of a slice is the number of elements in the underlying array from the first element of the slice. This means the capacity of a slice refers to the size up to which a slice can be extended.

In the preceding example, the slice contains elements from index 2 to 4 (3 elements). So the length of the slice is 3. Slice can be extended to the maximum index of an array which is 9 here. So, the capacity of a slice is the number of the element from index 2 to 9; which is 8 here.

A slice cannot be extended beyond its capacity. Also, a slice cannot be re-sliced below zero.

Example:

```
func main() {  
    var arrayval [10] int = [10] int {1, 2, 3, 4, 5, 6, 7, 8,  
    9, 10}  
    slice1 := arrayval[2:5]  
    fmt.Println("length of slice", len(slice1))  
    // length of slice 3  
    fmt.Println("capacity of slice", cap(slice1))  
    // capacity of slice 8  
}
```

Slice using the make function

Go provides a built-in function that is used to create empty slices. An empty slice refers to an empty array. To create such slices, it is necessary to define its size at the time of the slice declaration. To create an empty slice of non-zero length, we declare slices in the following way:

```
name_of_slice := make( [], type_of_slice, length_of_slice, capacity_of_  
slice )
```

The **make** function creates an array of a specified type and assigns its reference to a slice. Here, the capacity of the slice is optional. By default, the size of a slice to be considered is its capacity.

```
s := make([] int, 5, 10)
```

This creates an integer-type array of size ten and assigns a reference to slice s till the 5th index.

Example:

```

func main() {
    slice1 := make([] int, 2,4)

    fmt.Println("length of slice", len(slice1)) // length of slice 2
    fmt.Println("capacity of slice", cap(slice1)) // capacity of slice 4
    fmt.Println("value of slice", slice1) // value of slice [0 0]

    slice1[0] = 194
    slice1[1] = 18

    fmt.Println("value of slice", slice1) // value of slice [194 18]

    slice1[2] = 78 // error as length of slice is 2
}

```

In the preceding example of assigning the value at the 2nd index, it exits because we have defined its length as 2. After that, we can extend the slice until the 3rd index by appending more elements as its capacity is 4.

Extending slices

In the Go language, we can extend the capacity of slices. There is a simple process to increase the capacity of a slice. To increase it, create a new slice (that must be larger than the old slice) and copy the content of the slice to a new one. Now, the capacity of the new slice will be greater than the old one. Then, assign this new slice to the old one.

Example:

```

func main() {
    slice1 := []int {12, 65, 34, 78, 23, 67}

    fmt.Println("capacity of slice1:", cap(slice1))

    slice2 := make([]int, len(slice1), (cap(slice1)+1)*2)

    for i:= range slice1{
        slice2[i] = slice1[i]
    }

    slice1 = slice2

    fmt.Println("capacity of slice1:", cap(slice1))
}

```

```
}
```

Output:

capacity of slice1: 6 capacity of slice1: 14

In Go, **slices.Grow** function from 'slices' package in standard library offers a simple alternative.

Two-dimensional slices

Like arrays, we can also create 2D slices. 2D slices can be defined as slices of a slice. We can declare 2D slices in the same way that we do in arrays, without defining their size:

```
slice1 := [][] int{  
    [] int {1, 2},  
    [] int {3, 4},  
    [] int {5, 6},  
}
```

While declaring 2D slices using the make function, we define the number of rows as the length of the slice. In 2D slices, the length of the inner slice can vary as required. For uninitialized rows or elements, it takes default values. Here is an example that explains 2D slices.

Example:

```
func main() {  
    slice1 := [][] int{  
        [] int {1, 2, 78, 54},  
        [] int {0, 3, 4},  
        [] int {5, 6},  
    }  
  
    fmt.Println("slice1", slice1) // slice1 [[1 2 78 54] [0 3 4] [5 6]]  
  
    slice2 := make([][]string, 4)  
  
    slice2[0] = []string{"College", "School"}  
  
    slice2[1] = []string{"University", "Institute"} fmt.Println("slice2 using make function:", slice2)
```


slice>>>>>>>>>>

"sort"

```

)

func main() {
    slice1 := []int{23, 0, 105, 58}
    slice2 := []string{"Ram", "Akansha", "Mayank"}
    slice3 := []float64{75.84, 0.007, 85.62, 0.74637}
    fmt.Println("slice1 before sorting:", slice1)
    sort.Ints(slice1)
    fmt.Println("slice1 after sorting:", slice1)
    fmt.Println("slice1 before sorting:", slice2)
    sort.Strings(slice2)
    fmt.Println("slice1 after sorting:", slice2)
    fmt.Println("slice1 before sorting:", slice3)
    sort.Float64s(slice3)
    fmt.Println("slice1 after sorting:", slice3)
}

```

Output:

slice1 before sorting: [23 0 105 58]

slice1 after sorting: [0 23 58 105]

slice1 before sorting: [Ram Akansha Mayank] slice1 after sorting: [Akansha Mayank Ram]

slice1 before sorting: [75.84 0.007 85.62 0.74637]

slice1 after sorting: [0.007 0.74637 75.84 85.62]

The **slices.SortFunc** function in the standard library allows custom sorting of slices using a comparator, providing an alternative to the standard sort package.

Compare slices

Two slices are said to be equal if they are both of the same type and contain the same elements. Golang provides a function `DeepEqual` to compare the slices, whether they are equal or not. We can access this function by using the **reflect** package.

Example:

```

package main

import ("fmt"
"reflect"
)

func main() {
slice1 := []int{23, 0, 105, 0}
slice2 := []string{"Ram", "Akansha", "Mayank"}
slice3 := []float64{23, 0, 105, 0}
slice4 := make([]int, 4)
slice4[0] = 23
slice4[1] = 0
slice4[2] = 105

fmt.Println("Is slice1 equal to slice2:", reflect.DeepEqual(slice1, slice2))
fmt.Println("Is slice1 equal to slice3:", reflect.
DeepEqual(slice1, slice3))
fmt.Println("Is slice1 equal to slice4:", reflect.DeepEqual(slice1, slice4))
}

```

Output:

Is slice1 equal to slice2: false Is slice1 equal to slice3: false Is slice1 equal to slice4: true

Searching in slice

In Golang, we can search whether a specific element exists in a slice or not. Go provides a search function that finds an element and returns its position in a particular integer, float, or string slice. The search function performs a binary search on a sorted slice in the ascending order. If the element is found in a sorted slice, the search function returns its index; if not, it returns the index where the element would fit in the slice.

Example:

```

package main

import ("fmt"

```

```

"sort"
)
func main() {
slice1 := [] int{0, 9,16,105, 345}
slice2 := [] string{"Akansha", "Mayank", "Ram"}
slice3 := [] float64{0, 0.001, 5.7}
fmt.Println(sort.SearchInts(slice1, 105)) // 3
fmt.Println(sort.SearchInts(slice1, 78)) // 3
fmt.Println(sort.SearchInts(slice1, 0.0)) // 0
fmt.Println(sort.SearchStrings(slice2, "Ram")) // 2
fmt.Println(sort.SearchStrings(slice2, "ram")) // 3
fmt.Println(sort.SearchFloat64s(slice3, 0.000)) // 0
fmt.Println(sort.SearchFloat64s(slice3,5.7)) // 2
}

```

Reverse slice

The reverse function returns a slice in the reverse/descending order. It works well with sorted or unsorted slices.

Example:

```

package main
import ("fmt"
"sort"
)
func main() {
slice1 := [] int{0, 888, 9,16,105, 345}
slice2 := [] string{"Soni", "Akansha", "Charu", "Mayank",
"Ram"}
sort.Sort(sort.Reverse(sort.IntSlice(slice1)))
fmt.Println("reverse of slice1:", slice1)

```

```
sort.Sort(sort.Reverse(sort.StringSlice(slice2)))  
fmt.Println("reverse of slice2:", slice2)  
}
```

Output:

reverse of slice1: [888 345 105 16 9 0]

reverse of slice2: [Soni Ram Mayank Charu Akansha]

Appending slice element

The **append** function adds a new element at the end of the slice. We can append multiple elements, at the end of the slice, at once. If a slice has less capacity than the number of items we are appending, then a new slice is allocated of a large capacity and all the elements are copied and appended into the new slice.

Example:

```
func main() {  
    slice1 := []int{0, 888, 9, 16, 105, 345}  
    slice1 = append(slice1, 345, 646, 0)  
    fmt.Println("slice1 after appending few elements:", slice1)  
    slice3 := make([]int, 2)  
    slice3 = append(slice3, 56, 67, 48, 90, 46, 26, 57)  
    fmt.Println("slice3 after appending elements more than  
    it's capacity:", slice3)  
    slice4 := []int{}  
    slice4 = append(slice4, 45, 67)  
    fmt.Println("slice4 after appending elements in nil slice:", slice4)  
}
```

Output:

**slice1 after appending few elements: [0 888 9 16 105 345 345 646
0]**

slice3 after appending elements more than it's capacity: [0 0 56

67 48 90 46 26 57]

slice4 after appending elements in nil slice: [45 67]

We can append a slice into a slice of the same type using ... operator. This operator helps in increasing the capacity of a slice up to the length of the appended slice.

Example:

```
func main() {  
    slice1 := []int{0, 888, 9, 16, 105, 345}  
    slice2 := []int{78, 34, 9, 57}  
    slice1 = append(slice1, slice2...)  
    fmt.Println("slice1 after appending slice2", slice1)  
}
```

Output:

slice1 after appending slice2 [0 888 9 16 105 345 78 34 9 57]

Conclusion

In this chapter, we learned the declaration and initialization of arrays in Go. We learned about various functions such as length, comparison, copy, and iteration with arrays. Here, we understood the need for an array and, thus, slices. We saw the use of some functions that are used to get a slice from an array, slice, or struct. We also saw that some in-built functions are used to analyze an array. To get a continuous understanding of data types in Go, we will discuss another data type called “strings” in the next chapter.

Questions

1. How can we declare an array in Go?
2. How can we initialize multidimensional arrays in Go?
3. What is the need of a slice over an array?
4. How can we add elements in a slice?
5. How can we pass an array in a function?

6. How can we get a slice from an array for some specific contiguous value?
7. How can we compare two slices in Go?

Join our book's Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

<https://discord.bpbonline.com>



CHAPTER 10

Strings

Introduction

In this chapter, we are going to discuss another non-primitive data type 'String' that is used to represent a text (set of characters, special characters, numbers, etc.). It illustrates various operations that we can perform on a string in Go language.

Structure

In this chapter, we will discuss the following topics:

- Introduction to strings
- Rune data type
- Multiline strings
- Converting a slice of runes to string
- Converting a slice of bytes to string
- Immutable nature of the string
- Strings package
- Functions with strings package

Objectives

By the end of this chapter, you will have a comprehensive understanding of how strings work in Go programs. You will be able to utilize various built-in functions that streamline your coding process, saving time and reducing complexity. Additionally, you will grasp the immutable behavior of strings and learn how to convert strings from other data types as needed. Finally, you will gain valuable knowledge of the strings library that Go provides, empowering you to handle string manipulation effectively in your applications.

Introduction to strings

The **string** is one of the important categories of Golang data types. In Go, strings are quite different as compared to other languages. In other programming languages, the string is defined as an array of characters, but here, the **string is a slice of bytes**. In JavaScript, Python, or any other language, we can define strings within single quote, but Golang does not allow multiple characters or strings to be defined within single quotes. We can define a character with single quotes ('...') but strings are always defined with double quotes ("...").

By default, strings are UTF-8 encoded in Go language. This is a key factor of Go's string data type. UTF-8 is supported by many languages, so it facilitates easy interaction with other languages. While creating web pages, we need to display the Unicode content; here, strings work well with the Go language. UTF-8 takes 8 bits block to represent a character. The number of blocks can vary from 1 to 4. The Unicode character set maps each character to a unique number called code points. It is capable of encoding 1,112,064 code points.

Here is a basic example to explain the declaration and initialization of strings:

```
func main() {  
    var str string  
    fmt.Printf("By default strings are:%q\n", str)  
    str = "Hello Go"  
    fmt.Println("After assigning a value string is:", str)
```

```

str_new := "Hello world"

fmt.Printf("After shorthand declaration, double quoted value of str_new is:%q\n", str_new)

fmt.Printf("Uninterpreted bytes of str_new is:%s\n", str_new)

fmt.Printf("base16 notation of str=%x", str)

}

```

Output:

By default strings are: ""

After assigning a value string is: Hello Go

After shorthand declaration, double quoted value of str_new is: "Hello world"

Uninterpreted bytes of str_new is: Hello world

base16 notation of str=48656c6c6f20476f

The preceding example tells us that by default, strings are empty.

Functions with strings

Here are some functions that work with strings in Go.

- **Length of string:** Golang provides the **len** function to find the length of the string or the number of characters that a string contains. This function encodes the string and then returns the total number of bytes taken by each character.

Example:

```

func main() {

var str string

str = "Hello Go"

fmt.Println("Length of string 'str' is:", len(str))

str_new := "世Hello Go世"

fmt.Printf("Length of string 'str_new' is:", len(str_new))

```

```
}
```

Output:

length of string 'str' is: 8

length of string 'str_new' is:%!(EXTRA int=14)

In the preceding example, the length of **str** simply counts the number of characters and return the length as 8. In **str_new**, the length is 14. In this string, character 世 is encoded with 3 bytes in UTF-8. So, the length is 14.

- **Iterate through a string:** As the string is a slice of bytes, we can access each byte/character individually. We can get an individual as a character or their Unicode UTF-8 encoded value.

While iterating, it covers each byte, not character.

Example:

```
func main() {  
    var str string = "Hello Go"  
    fmt.Println("characters of str")  
    for itr:=0; itr<len(str); itr++){  
        fmt.Printf("%c ", str[itr])  
    }  
    fmt.Println("\nUTF-8 encoded values of str")  
    for itr:=0; itr<len(str); itr++){  
        fmt.Printf("%x ", str[itr])  
    }  
    str_new := "世Hello Go世"  
    fmt.Println("\ncharacters of str_new")
```

```

for itr:=0; itr<len(str_new); itr++){
    fmt.Printf("%c ", str_new[itr])
}

fmt.Println("\nUTF-8 encoded values of str_new")
for itr:=0; itr<len(str_new); itr++){
    fmt.Printf("%x ", str_new[itr])
}
}

```

Output:

characters of str H e l l o G o

UTF-8 encoded values of str

48 65 6c 6c 6f 20 47 6f

characters of str_new

ä , – H e l l o G o ä , –

UTF-8 encoded values of str_new

e4 b8 96 48 65 6c 6c 6f 20 47 6f e4 b8 96

In the preceding example, everything works fine but on getting the character of string **str_new**, it prints the character incorrectly. When the character 世 is encoded, it takes 3 bytes. Let us break down the output where we get UTF-8 encoded values of this string:

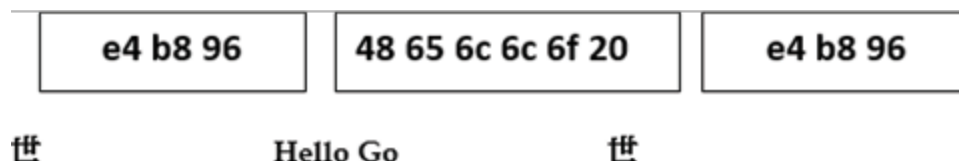


Figure 10.1: Output

Here, we can clearly see that 世 occupies three bytes, e4, b8 and 96. On

printing the characters, we are assuming that it occupies only one byte. Here, things can go wrong. To overcome such problems, Golang provides a 'rune' data type.

Rune data type

Rune is a built-in data type in the Go language. It is an alias of int32 as we have seen that Unicode UTF-8 encodes the character to a unique number or code point. The **rune** represents these Unicode code points in Go. It is not bound with the number of bytes that a character occupies.

In the previous example, we were iterating through the string that iterates each byte as the string is a slice of bytes, but it leads to an unexpected result. To avoid this, we will iterate through the slice of runes by converting a slice into runes.

Example:

```
func main() {  
    str_new := "世Hello Go世"  
    rune_slice := []rune(str_new)  
    fmt.Println("\ncharacters of str_new")  
    for itr:=0; itr<len(rune_slice); itr++){  
        fmt.Printf("%c ", rune_slice[itr])  
    }  
    fmt.Println("\nUTF-8 encoded values of str_new")  
    for itr:=0; itr<len(str_new); itr++){  
        fmt.Printf("%x ", str_new[itr])  
    }  
}
```

Output:

characters of str_new

世 H e l l o G o 世

UTF-8 encoded values of str_new

e4 b8 96 48 65 6c 6c 6f 20 47 6f e4 b8 96

Iterate using range keyword

Instead of using for loop or runes, we can use the **range** keyword to iterate over the items of a slice or array. It returns an index as integer and the item on that index. For loop covers each byte whereas range covers each index separately. So, the problem we had with for loop never occurs with the range keyword.

```
for index, item := range name_of_string{  
  
//body  
  
}
```

On writing one variable on the left side, it only returns the index and not the item at that index. If we only need the item and not the index, we can use `_` at the place of the index variable.

Example:

```
func main() {  
    str_new := "世Hello Go世"  
    fmt.Println("\nCharacters of str_new")  
    for _, item := range str_new{  
        fmt.Printf("%c ", item)  
    }  
}
```

Output:

```
Characters of str_new  
世 H e l l o   G o 世
```

Multiline string

There is another way to define strings, which is by using backtick (`). With double quotes, we need to define newline character (\n) for a new line but with backticks, we can simply give a line break as needed. On putting a line

break with the backtick string, it is considered as a `\n` character. There is no need to specify escape characters with backtick as it interprets them as a normal character and prints with the strings.

Example:

```
str := "Hello  
Go"  
  
fmt.Printf("string with double quotes:%s\n", str) // error
```

Example:

```
func main() {  
    str := "Hello\n Go"  
  
    fmt.Printf("String with double quotes:%s\n", str)  
  
    str_new := `Hello\n Go`  
  
    fmt.Printf("String with backtick:%s\n", str_new)  
  
    //treats \n as normal character  
  
    str_new = `Hello Go`  
  
    fmt.Printf("Linebreak in string with backtick:%s\n", str_new)  
}
```

Output:

String with double quotes:Hello

Go

String with backtick: Hello\n Go

Linebreak in string with backtick: Hello

Go

Converting a slice of runes to string

We have already understood how to convert string to rune. In Go, we can also convert the slice of runes to a string. In the preceding example, the rune contains a few code points that we are converting to string.

Example:

```
func main() {
```

```

rune := [] rune{0x4e16, 0x0048, 0x0065, 0x006c, 0x006c, 0x006f,
0x0047, 0x006f, 0x4e16}

str := string(rune)

fmt.Println("string from slice of rune:", str)
}

```

Output:

string from slice of rune: 世HelloGo世

Converting a slice of bytes to string

We can convert a slice of byte to string like we converted slice of the rune. Here is an example:

```

func main() {

rune := [] byte{228, 184, 150, 72, 101, 108, 108, 111, 71, 111,
228, 184, 150}

str := string(rune)

fmt.Println("String from a slice of bytes:", str)

}

```

Output:

String from a slice of bytes: 世HelloGo世

Immutable nature of the string

In the Go language, we cannot change the value of string once it is defined. That is why they are called immutable strings. We can say that the string is a read-only slice of bytes. We can access any character of a string but on changing its value or extending its length, the compiler throws an error.

Example:

```

func main() {

str := "Hello Go"

fmt.Printf("%c", str[3]) // l

```



```
str[3] = 'y'    // cannot assign to str[3]
}
```

There are certain cases where we need to change the value of the strings. Here, rune helps. We can convert the string to rune type, change the value as required, and then assign the rune to the string. Here is a short example:

Example:

```
func main() {
    str := "Hello Go"

    fmt.Printf("Character at third index: %c\n", str[3])

    rune := []rune(str)
    rune[3] = 'M'

    str = string(rune)

    fmt.Printf("Character at third index after changing its value:
%c", str[3])
}
```

Output:

character at third index: l

character at third index after changing its value: M

In the preceding example, we saw that the rune represents a character as a Unicode code point (int-32). Thus, each character can be defined as a decimal value and so, it can be compared. We can also perform a few other operations with the help of a character's code point.

Example:

```
func main() {
    if('a'>'A'){
        fmt.Println("a is greater tha A")
    }

    fmt.Println("a is greater than z:", 'a'>'z')

    for itr:='a'; itr<'m'; itr++{
        fmt.Print(itr)
    }
}
```

```
fmt.Printf(" "+string(itr)+"\n")
}
}
```

Output:

a is greater than A

a is greater than z: false

97 a

98 b

99 c

100 d

101 e

102 f

103 g

104 h

105 i

106 j

107 k

108 l

Strings package

In Golang, we have standard library strings that provide multiple functions to work with UTF-8 encoded strings. The strings package is in Go's **src** directory that has multiple Go files. The package facilitates several functions to modify and manipulate the strings.

To use a package in a program, we need to import it first. After importing the package, we can access its exported functions with the dot operator. The section given describes most of the functions that are provided in the strings package. To avoid accessing the **Println** method again and again, we are assigning it a variable **fp** and using this variable in the program.

Functions with strings package

Here are some functions available in the strings package in Go:

- **Converting string in uppercase and lowercase:** The **ToUpper** function converts each character of a string into uppercase, whereas the **ToLower** function is used to convert each character of a string into lowercase. With both the functions, the conversion is done with each character, whether it is in lowercase or uppercase. If any character is in uppercase and the **ToUpper** function is applied, it remains in uppercase and vice versa.

Example:

```
package main

import ("fmt"
"strings"
)

func main() {
    fp := fmt.Println str := "Hello Go"
    fp("string in uppercase:", strings.ToUpper(str))
    // string in uppercase: HELLO GO
    fp("string in lowercase:", strings.ToLower(str))
    // string in lowercase: hello go
}
```

- **Search for string prefix/suffix:** The **strings** package provides a function called **HasPrefix** that checks whether a string starts with a specified string or not. Similarly, the function **HasSuffix** checks if a string ends with a specified string. Two arguments are passed while calling these functions. The first is the string on which we are checking the prefix/suffix and the second is the string that we are searching for.

Example:

```

func main() {
    fp := fmt.Println str := "Hello Go"
    fp("has prefix He", strings.HasPrefix(str, "He"))
    // has prefix He true
    fp("has prefix he", strings.HasPrefix(str, "he"))
    // has prefix he false
    fp("has suffix go", strings.HasSuffix(str, "go"))
    // has suffix go false
    fp("has suffix Go", strings.HasSuffix(str, "Go"))
    // has suffix Go true
}

```

- **Search for specific text:** We have a function called **Contains** that checks if a string exists in another string or not. On the other hand, we have the **ContainsAny** functions, which return true if any character of a string exists in another string.

Example:

```

func main() {
    fp := fmt.Println str := "Hello Go"
    fp("contains ellO", strings.Contains(str, "ellO"))
    // contains ellO false
    fp("contains o G:", strings.Contains(str, "o G"))
    // contains o G: true

    fp("containsany welcome:", strings.ContainsAny(str, "welcome")) //
containsany welcome: true

    fp("containsany shubh:", strings.ContainsAny(str, "shubh"))

```

```
// containsany shubh: false
```

```
}
```

- **Number of occurrences in a string:** We have a function to check if a specific string exists in another string or not. The count function tells the number of times a specific string appears within another string. It returns zero if the specified string does not exist in another string.

Example:

```
func main() {  
  
    fp := fmt.Println  
  
    str := "Grow And Grow With Golang"  
  
    fp("occurences of Gr:", strings.Count(str, "Gr"))  
  
    // occurences of Gr: 2  
  
    fp("occurences of welcome:", strings.Count(str, "welcome"))  
  
    // occurences of welcome: 0  
  
}
```

- **Find the position of a string:** In the **strings** package, the **Index** function is used to get an index of the specified string in another string at the first occurrence. It returns - 1 if the specified string doesn't exist in another string.

Example:

```
func main() {  
  
    fp := fmt.Println  
  
    str := "Grow And Grow With Golang"  
  
    fp("position of Gr:", strings.Index(str, "Gr"))  
  
    // position of Gr: 0  
  
    fp("position of welcome:", strings.Index(str, "welcome"))
```

```
// position of welcome: -1
```

```
}
```

- **Find the position of any Unicode codepoint:** The **IndexAny** function searches for the Unicode codepoints (character) of the specified string in another string and returns the first index of the first appearing Unicode codepoint. It returns -1 if no Unicode code points appear in another string.

Example:

```
func main() {
```

```
    fp := fmt.Println
```

```
    str := "Grow And Grow With Golang"
```

```
    fp("position of first appearing Unicode codepoint in Ground:",  
strings.IndexAny(str, "Ground"))
```

```
// position of first appearing Unicode codepoint in Ground: 0
```

```
    fp("position of first appearing Unicode codepoint in sub:",  
strings.IndexAny(str, "sub"))
```

```
// position of first appearing Unicode codepoint in sub: -1
```

```
}
```

- **Find the position of a string from last:** As the **Index** function checks for the first occurrence, the **LastIndex** function checks for the last occurrence of a string in another string.

Example:

```
func main() {
```

```
    fp := fmt.Println
```

```
    str := "Grow And Grow With Golang"
```

```
    fp("last position of Gr:", strings.LastIndex(str, "Gr"))
```

```
// last position of Gr: 9
```

```
fp("last position of welcome:", strings.LastIndex(str,
"welcome")) // last position of welcome: -1

}
```

- **Find the position of any Unicode codepoint from last:** The function **LastIndexAny** works the same as **IndexAny**. The difference is that **IndexAny** returns the first occurrence, whereas **LastIndexAny** returns the last occurrence of the Unicode code point if it exists in another string. This function finds a codepoint that appears last in another string and then returns the index of its last occurrence. It returns -1 if no Unicode codepoint exists in another string.

Example:

```
func main() {

fp := fmt.Println

str := "Grow And Grow With Golang"

fp("position of last appearing Unicode codepoint in Ground: ",
strings.LastIndexAny(str, "Ground")) // position of last appearing Unicode
codepoint in Ground: 23

fp("position of last appearing Unicode codepoint in sub: ",
strings.LastIndexAny(str, "sub")) // position of last appearing Unicode
codepoint in sub: -1

}
```

- **Convert array to string:** In the **strings** package, we have a function called **Join** that concatenates all the elements of the array (string type) with the specified separator and creates a single string.

Example:

```
func main() {

fp := fmt.Println

str := [] string {"my", "name", "is", "rahul"}
```

```
fp(strings.Join(str, "-")) // my-name-is-rahul
}
```

- **Replacing specified string from the given string:** The **strings** package provides a **Replace** function that returns the copy of the string by replacing a given string with another string. The **Replace** function takes four values: the main string on which we are performing this action, the old string that we need to replace, the new string that we need to replace with, and an integer number that tells the number of occurrences of that old string that we need to replace. If an integer value is zero or an old string does not exist in a string, then no replace action is performed. If an integer value is greater than the number of occurrences, it replaces all the occurrences of an old string with a new one.

We have another function called **ReplaceAll** that works the same as the **Replace** function. The difference between them is that **Replace** works on a given number of occurrences, while **ReplaceAll** replaces all the occurrences of an old string with a new one.

Example:

```
func main() {
    fp := fmt.Println
    str := "my-name-is-rahul"
    fp(strings.Replace(str, "-", " ", 0)) // my-name-is-rahul
    fp(strings.Replace(str, "-", " ", 1)) // my name-is-rahul
    fp(strings.Replace(str, "-", " ", 9)) // my name is rahul
    fp(strings.Replace(str, "welcome", " ", 9)) // my-name-is-rahul
    fp(strings.ReplaceAll(str, "-", " ")) // my name is rahul
}
```

- **Convert string to array by splitting strings:** The **split** function returns an array of substrings from a string, split by a given separator. If a separator does not exist in a string, then it simply converts a string

into an array.

We have another function called **Fields** that splits a string by the whitespace or newline characters and returns an array.

Example:

```
func main() {  
    fp := fmt.Println  
  
    str := "my-name-is-rahul"  
  
    fp(strings.Split(str, "-"))      // [my name is rahul] fp(strings.Split(str,  
"welcome")) // [my-name-is-rahul] fp(strings.Split(str, ""))      // [m y - n  
a m e - i s - r a h u l] fp(strings.Fields("welcome \t in the world of \n Go"))  
  
    // [welcome in the world of Go]  
}
```

- **Removing specific Unicode code points:** We have a function called **Trim** that removes all the leading and trailing Unicode code points contained in a specific given string from another string. We have many other functions that give support to trim the specific Unicode point as needed. Here is an example that demonstrate this well:

Example:

```
func main() {  
    fp := fmt.Println  
  
    str := "-my-name-is-rahul-"  
  
    fp(strings.Trim(str, "-")) // my-name-is-rahul  
  
    fp(strings.TrimLeft(str, "-")) // my-name-is-rahul-  
    fp(strings.TrimRight(str, "-")) // -my-name-is-rahul  
  
    fp(strings.TrimFunc(str, func(c rune) bool {  
        return c >= 23 && c <= 50  
    }))) // my-name-is-rahul
```

```

fp(strings.TrimSpace(str, func(c rune) bool {
return c >= 23 && c <= 50
}))      // my-name-is-rahul-

fp(strings.TrimSpace(str, func(c rune) bool {
return c >= 23 && c <= 50
}))      // -my-name-is-rahul

fp(strings.TrimSpace(str, "-my-"))      // name-is-rahul-
fp(strings.TrimSpace(str, "-rahul-"))  // -my-name-is
fp(strings.TrimSpace(str, "welcome"))  // -my-name-is-rahul-
(Unchanged)

str = "      -my-name-is-rahul- "

fp(strings.TrimSpace(str))      // -my-name-is-rahul-
}

```

Conclusion

In this chapter, we saw how a string works as a code point in Go. Golang provides various functions to analyze and modify the strings. We have seen multiple ways to get a string from other data types. The concept of rune data type is new in Golang. Golang provides a rich set of libraries and strings package is one of them that facilitates multiple functions for the string. In the next chapter, we will discuss another data type called pointer in depth.

Questions

1. How is a string declaration different in Go from other languages?
2. What is a rune in Go?
3. How can we get a string from rune?

4. Which function is used to find the index of the string in another one?
5. How can we access the exported function of the strings package in Go?
6. How are strings immutable in Go?

Join our book's Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

<https://discord.bpbonline.com>



CHAPTER 11

Pointers

Introduction

In this chapter, we will discuss the data type pointers that play an important role in memory management and manipulation. Pointer allows dynamic memory allocation and deallocation, data sharing and makes a difference between the default (null or zero) and undefined value of a variable. Let us have a look at and understand the concept of pointers in Go.

Structure

In this chapter, we will discuss the following topics:

- Introduction to pointers
- Need for pointers
- Declare pointers
- Initializing pointers
- Dereferencing pointers
- Manipulating pointers
- Creating pointers with new
- Nested pointers
- Array of pointers
- Passing pointers to a function

Objectives

This chapter will cover pointers in Go, with an emphasis on how they are used for data manipulation and memory management. In addition to using the new keyword for dynamic memory allocation, you will learn how to declare, initialize, and dereference pointers. Additionally, we will discuss arrays of pointers, nested pointers, and the advantages of sending pointers to functions. You will comprehend pointers and their useful uses in Go programming by the end.

Introduction to pointers

Pointers are a key concept in the Go language. The pointer is one of the derived data types. When we declare a variable, the Go compiler allocates some space in the system memory according to the type of that variable. That memory space has a unique address to identify that variable. The address is represented by a hexadecimal value. The pointer refers to the data by storing the address of any variable located in the system memory. It makes the data handling task easy by accessing the data directly from its memory. The type of pointer is the same as the type of that variable whose address is stored by the pointer. Therefore, an integer type of pointer holds the address of an integer type of variable. A pointer refers by accessing the value, and this is called dereferencing a pointer.

Need for pointers

The pointer is a variable that stores the address of another variable. Now the question is, why should we use it? Although we can access the variable by its name in a large application, this process takes additional space in the system memory. If we need to pass a large chunk of data through a function, then we can pass only its address instead of passing the complete data. Pointers are used to assign as well as release the memory space. So, pointers do great memory utilization. Also, the execution time is faster with pointers because data is directly accessed by the memory location. Thus, we can conclude the following points as the benefits of pointers:

- Faster execution time

- Memory utilization
- Easy management of variables
- Dynamic memory allocation

As Go is a concurrent language, we can run multiple tasks simultaneously with the help of Goroutine and channels. While using pointers, it is easy to read data by multiple processes, but modifying the data using an address is quite dangerous. Go facilitates pointers, but the use of pointers is totally in our hands. The wise use of pointers is necessary for the Go language.

Declare pointers

As the pointer is a variable, the way to declare a pointer is the same as other variables. Here, the asterisk (*) makes the difference. It tells the compiler that the variable is a pointer. To declare a pointer, we use * before the type of pointer variable.

Syntax:

```
var name_of_pointer *type_of_pointer
```

Example:

```
func main() {  
    var pointerv *int  
  
    fmt.Println("Default value of pointer is:",pointerv)  
  
    // Default value of pointer is: <nil>  
}
```

Here, nothing is assigned to a pointer. So, the default value is **nil** of integer type of pointer. Any uninitialized pointer has a zero value that is nil here.

Initializing pointers

Initializing pointers means assigning an address of a variable to the pointer. Here, an address is denoted by the & operator. We will use two operators: * to declare a pointer or access the value at a specific address and & to get the address of a specific variable.

Syntax:

name_of_pointer = &name_of_variable

Example:

```
func main() {  
    a:= 10  
  
    var pointerv *int  
  
    pointerv = &a  
  
    //1  
  
    fmt.Println(pointerv) // 0x1184e068  
  
    //2  
  
    fmt.Println(&pointerv) // 0x1183e0f8  
  
}
```

In the preceding example, the statement 1 shows the address of variable a, and the value stored in the pointer variable. Statement 2 shows the address of the pointer in memory.

Dereferencing pointers

Getting the value stored in an address pointed by a pointer is called **dereferencing the pointer**. The operator * is used for accessing the value. This operator is called dereference or indirection operator.

Example:

```
func main() {  
    a:= 10  
  
    var pointerv *int  
  
    pointerv = &a  
  
    fmt.Println(pointerv) // 0x1184e068  
  
    fmt.Println(*pointerv) // 10  
  
}
```

Manipulating pointers

We can manipulate pointers using the indirection operator. We can access the data by using pointers, and we can use it as required. The value at a specific memory location can also be changed by pointers.

Example:

```
func main() {  
    a:= 10  
  
    var pointerv *int  
  
    pointerv = &a  
  
    fmt.Println(*pointerv) // 10  
  
    *pointerv = 30  
  
    fmt.Println("Afer assigning new value")  
  
    fmt.Println(*pointerv) // 30  
}
```

Creating pointers with new

Go provides a built-in function called **new** that is used to allocate memory space at runtime. This function allocates memory according to the passed data type and returns its address to a pointer. The syntax of the new function is as follows:

Syntax:

```
func new(Type) *Type
```

The argument type indicates the data type for which we need to allocate space in memory. The new function returns an address that can be assigned in any pointer variable.

Example:

```
func main() {  
    var a int  
  
    pointerv:= &a  
  
    fmt.Println(pointerv) // 0x1184e068  
  
    pointerv = new(int)
```



```

fmt.Println(pointerv) // 0x1184e09c
fmt.Println(*pointerv) // 0
*pointerv = 10
fmt.Println(*pointerv) // 10
}

```

In the preceding example, **pointerv** first contains the address of a variable “a” that is **0x1184e068**. Then, we allocate runtime memory space for an integer variable and assign its address to **pointerv**. Now, **pointerv** has an address of this integer type data, created by a new function that is **0x1184e09c**. After that, we get the value that is referred by pointer. Since nothing is assigned at this address, the default value is zero. Now, we dereference the pointer and assign the value to it, which is 10.

Nested pointers

With the use of the new function, it is clear that a pointer does not only hold an address of a variable, but simply holds an address of anything that is allocated in memory. When we declare a pointer, the compiler allocates memory space for the pointers too, depending on the type of pointers. Thus, we can assign an address of a pointer to another pointer easily. We can say it as **pointer to a pointer** also. Here is an example of nested pointers:

```

func main() {
var a int
pointerv:= &a
fmt.Println("value of pointerv:", pointerv)
// value of pointerv: 0x1184e068
fmt.Println("Address of pointerv:", &pointerv)
// Address of pointerv: 0x1183e0f8
pointerv1 := &pointerv
fmt.Println("value of pointerv1:", pointerv1)
// value of pointerv1: 0x1183e0f8
fmt.Println("Address of pointerv1:", &pointerv1)
}

```

```
// Address of pointerv1: 0x1183e108
pointerv2 := &pointerv1
fmt.Println("value of pointerv2:", pointerv2)
// value of pointerv2: 0x1183e108
fmt.Println("Address of pointerv2:", &pointerv2)
// Address of pointerv2: 0x1183e110
}
```

In the preceding example, **pointerv1** and **pointerv2** work as **pointer to a pointer**. They both hold an address of a variable that already holds an address of some other variable.

Array of pointer

In the Go language, we can create an array of pointers that hold an address of the same type of multiple variables.

Example:

```
func main() {
    var a, b int
    var pointerv[4] *int
    pointerv[0] = &a
    pointerv[3] = &b
    fmt.Println(pointerv) // [0x11810088 <nil> <nil> 0x1181008c]
}
```

Passing pointers to a function

In the previous chapters, we discussed functions, passing arguments to functions,

etc. While using functions, we can pass a pointer as an argument; or we should say, instead of passing a value, we can pass a reference. Let us understand why there is a need to pass a reference and how can we do it. In the following example, we need to change the value of the variable **a**.

Example:

```
//Passing by value
func main() {
a := "Welcome world"
pointerv := &a
fmt.Println("value of a is:", *pointerv)
changeVal(a)
fmt.Println("After modification, value of a is:", *pointerv)
}
func changeVal(y string){
y = "Hello Go!"
fmt.Println("changed value is:", y)
}
```

Output:

value of a is: Welcome world

changed value is: Hello Go!

After modification, value of a is: Welcome world

Here, it is clear that the value is to be changed in function **changeVal**, but in the main function, it does not reflect. The value in memory is the same as it was. Now, we will pass the reference of “a”.

Example:

```
func main() {
a := "Welcome world"
pointerv := &a
fmt.Println("Value of a is:", *pointerv)
changeVal(pointerv)
fmt.Println("After modification, value of a is:", *pointerv)
}
func changeVal(y *string){
```

```
*y = "Hello Go!"  
fmt.Println("Changed value is:", *y)  
}
```

Output:

Value of a is: Welcome world

Changed value is: Hello Go!

After modification, value of a is: Hello Go!

In the preceding example, we are passing the address of “a” that **pointer** holds. In the function **changeVal**, we are modifying the value directly at the address of that variable. Here, the pointer works well.

Note: If someone knows C or C++, they would be familiar with pointer operations, increments, or decrements. The Go language does not support arithmetic pointers. On performing such operations, the Go compiler will give an error (invalid operation).

Conclusion

In this chapter, we learned about one of the derived data type pointers that refer to the data stored in a memory. We saw how to declare, initialize, and manipulate the pointers with or without the new keyword. We understood the concept of nested pointers, array of pointers, and others to optimize our code. Till now, we have seen a few data types individually. In the next chapter, we will see the arrangement or relation between various data types that we have in the Go language by using structures.

Questions

1. Why do we need pointers?
2. What is the difference between * and & with pointers in Go?
3. How can we create pointers with a new keyword?
4. How can we pass pointers to a function?
5. Is it possible to create an array of pointers?
6. Why do we need nested pointers?

CHAPTER 12

Structures

Introduction

In this chapter, we will learn about structures in Go, which allow us to group different types of data into a single variable. While arrays are useful for storing multiple items of the same type, structures enable us to hold various data types together. This is essential for managing complex data more effectively. Let us cover other aspects of structure that we have in the Go language.

Structure

In this chapter, we will discuss the following topics:

- Introducing structures
- Declaring struct
- Initializing struct
- Accessing struct's field
- Pointer to structure
- Struct with a new function
- Anonymous struct fields
- Nested struct and embedded struct
- Passing structs to functions or copying structs

- Comparing structs

Objectives

This chapter on Go structures will cover the process of combining various data types into a single variable. The creation of anonymous and nested structs, accessing their fields, using pointers, and declaring and initializing structures will all be covered. Additionally, we will go over copying, comparing instances, and sending structs to functions. At the end, you will have a firm grasp on how to use structures in Go.

Introducing structures

A **struct** is a user-defined composite data type that defines a list of different types of variables in a block of memory. By using struct, a collection of multiple data types combined and create a single type. An entity that has various properties can be represented by struct. In the Go language, struct works in the same way as classes do in object-oriented programming with some differences. Struct supports composition as we can create a complex data type by combining different data types in one.

Declaring struct

A **struct** is nothing but a collection of multiple types of data that is defined under one name and one type. Here is the syntax to declare **struct**:

Syntax:

```
type name_of_struct struct{  
    name_of_variable1 type_of_variable1  
    name_of_variable2 type_of_variable2  
    name_of_variable3 type_of_variable3  
    ...  
    ...  
}
```

Example:

```
type student struct{  
    name string  
    rollno int  
    city string  
}
```

Or

```
type student struct{  
    name, city string  
    rollno int  
}
```

In the preceding code, the **type** keyword tells the compiler that the followed is a new data type. Here, we declared a struct named student. The student struct has various fields like name, city, etc., which is of string type, and rollno, which is of type integer. In the second one, we combined the variables of the same data type. Such structs are called named struct because here, we create a new struct type student.

It is not necessary to declare a new type while declaring structs. We can also create an anonymous structure in the Go language, as shown in the following code:

```
var student struct{  
    name, city string  
    rollno int  
}
```

Here, instead of creating a new type, we are declaring a variable **student** of the **struct** type. Let us create a variable of type **student** (named **struct**).

Syntax:

```
var type_of_variable name_of_struct
```

Example:

```
func main() {  
    type student struct{
```

```

name, city string
rollno int
}

var student_var student

fmt.Println(student_var) // { 0}

var student1 struct{
name, city string
rollno int
}

fmt.Println(student1) // { 0}
}

```

In the preceding example, we have created a variable **student_var** of type **student**. By default, it has default values for every field: 0 for integer type of variables and blank for string type variables. The same goes for anonymous struct **student1**.

Initializing struct

Using the struct literal, we can initialize struct variables. While initializing struct, you can either maintain the same order of values as the fields are declared in the struct or use named field declarations for clarity and ease of understanding.

Example:

```

func main() {
type student struct{
name, city string
rollno int
}

var student_var student

student_var = student{"rahul", "jaipur", 67}

fmt.Println(student_var) // {rahul jaipur 67}

```



```
}
```

In the preceding example, values are passed in the same order as they are declared. If we do not pass the values in the same order, the compiler will give an error. To avoid such an issue, we can pass the values with the field's name. On passing the values with the field's name, we can skip the value of any field. In such a case, the default value is taken by that field.

Example:

```
func main() {  
    type student struct{  
        name, city string  
        rollno int  
    }  
  
    var student_var student  
  
    student_var = student{rollno:67, city:"jaipur"} // { jaipur 67}  
  
    fmt.Println(student_var)  
  
    // Anonymous struct  
  
    student1 := struct{  
        name string  
        city string  
        rollno int  
    }{  
        name : "rahul", rollno :67,  
    }  
  
    fmt.Println(student1) // {rahul 67}  
}
```

Accessing struct's field

We can access the particular field of a struct by using the dot operator. By accessing fields, we can modify their values and so, struct is called mutable. There is another way to initialize an anonymous struct using the dot

operator. Here is an example of using the dot operator.

Example:

```
func main() {  
    var student struct{  
        name, city string  
        rollno int  
    }  
    student.city = "jaipur"  
    student.rollno = 67  
    fmt.Println(student)    // { jaipur 67}  
    type student1 struct{  
        name, city string  
        rollno int  
    }  
    var student_var student1  
    student_var.name = "rahul"  
    student_var.rollno = 67  
    fmt.Println(student_var) // {rahul 67}  
}
```

Pointer to struct

We can use a pointer with a struct like a normal variable. There are two ways through which a pointer can refer structs. Either create a struct and then assign its address to a pointer first or directly create a pointer that points to a struct. The second way skips the step of creating the **struct** type of the variable and creates a pointer directly, which can be useful when working with heap-allocated structs. With pointers, we can access the struct's field using the dot operator. The following example explains the use of pointers with struct:

Example:

```

func main() {
type student struct{
name, city string
rollno int
}

student_var := student{"rahul", "jaipur", 67}

p1 := &student_var

fmt.Println(p1) // &{rahul jaipur 67}
fmt.Println(*p1) // {rahul jaipur 67}
fmt.Println((*p1).city) //jaipur
fmt.Println(p1.city) //Jaipur

//Directly create a pointer

p2 := &student{"rahul", "jaipur", 67}

fmt.Println(p2) // &{rahul jaipur 67}
fmt.Println(*p2) // {rahul jaipur 67}
fmt.Println((*p2).city) //jaipur
fmt.Println(p2.city) //Jaipur

//Pointer with anonymous struct

var student1 struct{
name, city string
rollno int
}

annony_point := &student1
annony_point.name = "rahul"
annony_point.rollno = 67

fmt.Println(annony_point) // &{rahul 67}
fmt.Println(*annony_point) // {rahul 67}
}

```

Struct with a new function

A struct can be created with a new function also. This function allocates memory space according to the size of the struct fields and returns a pointer to it. Initially, all fields have their default values if they are not given.

Example:

```
func main() {  
    type student struct{  
        name, city string  
        rollno int  
    }  
    p1 := new(student)  
    fmt.Println(*p1) // { 0}  
    p1.city = "jaipur"  
    p1.rollno = 67  
    fmt.Println(*p1)    // { jaipur 67}  
    //anonymous struct with new  
    annony_point := new(struct{ name, city string rollno int  
    })  
    fmt.Println(*annony_point) // { 0}  
    annony_point.name = "rahul"  
    fmt.Println(*annony_point) // {rahul 0}  
}
```

Anonymous struct fields

In the Go language, it is not necessary to define the name of struct fields. If we mention the field type only, it will consider the type as the name of that field.

Example:

```
func main() {
```

```

type student struct{
    string
    int
}
a := student{"rahul", 67}
fmt.Println(a) // {rahul 67}
}

```

If we have two or more fields of the same data type, we can use only one field as anonymous. For the others, we have to give the field names. Otherwise, the compiler will give an error (duplicate field), as shown in the following code:

```

type student struct{
    string
    string
    int
}
a := student{"rahul", "jaipur", 67}
fmt.Println(a)

```

Output:

Error: string redeclared

Example:

```

func main() {
    type student struct{
        string
        city string
        int
    }
    a := student{"rahul","jaipur", 67}
    fmt.Println(a) // {rahul jaipur 67}
}

```

```
}
```

Nested struct and embedded struct

In Go, **nested structs** are defined as named fields within another struct, and their fields are accessed using the outer struct's field name. **Embedded structs**, on the other hand, are included without a field name, which promotes their fields to the outer struct for direct access. Embedded structs offer cleaner syntax but may cause naming conflicts. Example – Nested struct:

```
package main

import "fmt"

func main() {
    type result struct {
        math    int
        science int
        english  int
        isPass   bool
    }

    type student struct {
        name, city string
        rollno    int
        result     result // Nested struct as a field
    }

    a := student{
        name: "Rahul",
        city: "Jaipur",
        rollno: 11,
        result: result{
            math: 45,
```

```

        science: 78,
        english: 90,
        isPass: true,
    },
}

fmt.Println("Final Result is: ", a.result.isPass) //true
}

```

Example – Embedded struct:

```

func main() {
    type result struct{
        math int
        science int
        english int
        isPass bool
    }

    type student struct{
        name, city string
        rollno int
        result
    }

    a := student{"rahul", "jaipur", 11, result{45, 78, 90, true}}

    fmt.Println(a) // {rahul jaipur 11 {45 78 90 true}}

    //anonymous embedded struct
    result_annon := struct{
        math int
        science int
        english int
        isPass bool
    }{

```

```

math: 45,
science: 78,
english: 90,
isPass: true,
}

student_annon := struct{
name, city string
rollno int
result
}{
name : "rahul",
city : "city",
rollno : 11,
result : result_annon,
}

fmt.Println(student_annon) // {rahul city 11 {45 78 90 true}}
}

```

Passing structs to functions or copying structs

While copying one struct to another, a new copy is created and then assigned to another variable. In the same way, while passing a struct to another function, a copy of the struct is passed to a function.

Example:

```

package main

import "fmt"

func getValue(student_copy1 *student) string{
fmt.Println(student_copy1)      // &{rahul jaipur 67}

    student_copy1.name = "Manish"

    return student_copy1.name
}

```



```

}

type student struct{
name, city string
rollno int
}

func main() {
a := student{"rahul", "jaipur", 67}
student_copy := a
fmt.Println(student_copy.rollno) // 67
fmt.Println(student_copy) // {rahul jaipur 67}
// Passing address of struct to function
fmt.Println(getValue(&student_copy)) // Manish
}

```

Comparing structs

Since a struct can be copied from one to another variable, it is not wrong to say that a struct is of the value type. Like values, we can also compare two structs. Two structs are said to be equal if their fields and corresponding values are the same.

Example:

```

func main() {
student := struct{
name, city string
rollno int
}{
name : "rahul",
city : "jaipur",
rollno : 11,
}

```

```
student1 := struct{
name, city string
rollno int
}{
name : "rahul",
city : "jaipur",
rollno : 11,
}

fmt.Println(student==student1) // true

student3 := struct{
name, city string
rollno int
}{
name : "rahul",
city : "jaipur",
}

fmt.Println(student==student3) // false

student4 := struct{
name, city string
rollno int
}{
name : "rahul",
city : "jaipur",
rollno : 0,
}

fmt.Println(student3==student4) // true
}
```

Conclusion

From the preceding examples, we have learned how to declare, initialize, and perform various operations with structures. In the context of object-oriented programming, structures in Go are similar to classes, with the key distinction that while classes support inheritance, structures support composition. In the next chapter, we will explore the concept of composition in more detail.

Questions

1. How can we declare and initialize a structure in Go?
2. What is the use of a new keyword in structs?
3. How can we compare two structs in Go?
4. Which operator is used to access the struct fields in Go?
5. What do you understand by anonymous fields in structure?
6. How can we pass structs in the Go functions?

Join our book's Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

<https://discord.bpbonline.com>



CHAPTER 13

Composition

Introduction

If you understand programming languages like C++ or Java, you are likely familiar with the concepts of classes and inheritance. Unlike these languages, Go does not support traditional class-based inheritance. Instead, Go uses structs to define data structures and composition (via struct embedding) to enable behavior reuse.

In simple terms, composition in Go means combining or embedding one struct into another to share its methods and fields. While this is not inheritance in the classical sense (there is no parent-child hierarchy), it achieves similar outcomes by allowing one struct to reuse the functionality of another.

In this chapter, let us cover the concept of composition.

Structure

In this chapter, we will discuss the following topics:

- Introducing composition
- Embedding pointers
- Embedding slices of structs

Objectives

After studying this chapter, you will be able to reuse the code by composition and embed multiple structs, a slice of structs, and pointers to struct. You will also learn to build large and efficient APIs by merging multiple smaller parts.

Introducing composition

In Go language, the **composition** is the embedding of structs and composing data structures. Composition helps to create a large application by merging multiple, single-purpose type or methods. It provides readability and flexibility with a Go application. Let us create a simple program that will explain composition.

Example:

```
package main

import "fmt"

type websiteinfo struct{
    name string
    url string
    ageinyear int
}

func main() {
    type youtube struct{
        websiteinfo
        ytFirstVideo string
        ytlongestvideo string
    }

    type amazon struct{
        websiteinfo
        departments string
```

```

can_sell bool
}

youtube_var := youtube{websiteinfo{"Youtube", "www.youtube.com", 14}, "me at the zoo", "571
hours, 1 minute and 41 seconds"}

amazon_var := amazon{websiteinfo{"Amazon", "www.amazon.com", 25},
"Automotive, computers, electronics etc", true}

fmt.Println(youtube_var, amazon_var)

fmt.Println("youtube first video:", youtube_var.ytFirstVideo)

fmt.Println("youtube url:", youtube_var.websiteinfo.url)

fmt.Println("youtube url:", youtube_var.url)

fmt.Println("can we sell on amazon:", amazon_var.can_sell)

fmt.Println("Amazon url:", amazon_var.websiteinfo.url)

fmt.Println("Amazon url:", amazon_var.url)

}

```

Output:

```

{{Youtube www.youtube.com 14} me at the zoo 571 hours, 1 minute and 41 seconds} {{Amazon
www.amazon.com 25} Automotive, computers, electronics etc. true}

```

```

youtube first video: me at the zoo youtube url: www.youtube.com youtube url:
www.youtube.com

```

```

can we sell on amazon: true Amazon url: www.amazon.com Amazon url: www.amazon.com

```

In the preceding example, we have created a struct **websiteinfo** that has three fields. This struct dictates the basic information of any website. Another is struct **youtube**, which has three fields. An anonymous field, **websiteinfo**, represents the basic information about YouTube, and the other two fields represent the first and longest video on YouTube.

Struct **amazon** also has three fields. One of them is an anonymous field **websiteinfo**, like struct **youtube** has. Here, struct **websiteinfo** is embedded in both the other structs. It represents that structs **youtube** and **amazon** are composed of **websiteinfo**.

We are creating a literal **youtube_var** of type **youtube**. It is a composite literal and a combination of two literals, **websiteinfo** and **youtube**. The

same goes with literal **amazon_var**. Struct **youtube** and **amazon** have full access to all the methods of struct **websiteinfo** as **websiteinfo** is defined as a field in struct **youtube** and **amazon**.

We can access the fields of **websiteinfo** in two ways. First, we can access struct **websiteinfo** with struct literals (**youtube_var** or **amazon_var**) and then access its field using the dot operator. Second, we can directly access the fields of **websiteinfo** with struct literals using the dot operator.

Now, create a method on **websiteinfo** struct with the preceding program through the following:

```
func (webinfo websiteinfo) basicInfo(){  
    fmt.Println("Name of website:", webinfo.name)  
    fmt.Println("Url of website:", webinfo.url)  
    fmt.Println("How old is this website:", webinfo.ageinyear)  
}
```

The method **basicInfo** has **websiteinfo** as a receiver type that provides basic information on any website. We can access such a method by the composite literals **youtube_var** and **amazon_var** using the dot operator. There are two ways to access this method, either first access the struct **websiteinfo** and then access the method **basicInfo**; or directly access the method **basicInfo** by composite literals.

Here is the code to access method through **websiteinfo** struct:

```
youtube_var.websiteinfo.basicInfo() amazon_var.basicInfo()
```

Since struct like **youtube** and **amazon** have access to all the fields and methods of the **websiteinfo** struct, we can define a method for the **youtube** struct as follows:

```
func (yt youtube) youtubeInfo(){  
    fmt.Println("Name of website:", yt.name)  
    fmt.Println("Url of website:", yt.websiteinfo.url)  
    fmt.Println("How old is this website:", yt.ageinyear)  
    fmt.Println("First video on youtube:", yt.ytFirstVideo)  
    fmt.Println("Longest video on youtube:", yt.ytlongestvideo)
```

```
}
```

We can access this method by the **youtube** literal using the dot operator.

```
youtube_var.youtubeInfo()
```

If required, we can define a method by calling the base method of the embedded struct through the following method:

```
func (azn amazon) amazonInfo(){ azn.basicInfo()
fmt.Println("Departments of amazon:", azn.departments)
fmt.Println("Can we sell on amazon:", azn.can_sell)
}
```

```
amazon_var.amazonInfo()
```

In the preceding method, we are accessing the **basicInfo** method by the **amazon** literal, and the same is done in the method **amazonInfo** as in **youtubeinfo**. A complete example of **composition** is shown as follows:

Example:

```
package main
import "fmt"
type websiteinfo struct{
name string
url string
ageinyear int
}
type youtube struct{
websiteinfo
ytFirstVideo string
ytlongestvideo string
}
type amazon struct{
websiteinfo
departments string
```



```

can_sell bool
}

func (webinfo websiteinfo) basicInfo(){
fmt.Println("Name of website:", webinfo.name)
fmt.Println("Url of website:", webinfo.url)
fmt.Println("How old is this website:", webinfo.ageinyear)
}

func (yt youtube) youtubeInfo(){
fmt.Println("Name of website:", yt.name)
fmt.Println("Url of website:", yt.websiteinfo.url)
fmt.Println("How old is this website:", yt.ageinyear)
fmt.Println("First video on youtube:", yt.ytFirstVideo)
fmt.Println("Longest video on youtube:", yt.ytlongestvideo)
}

func (azn amazon) amazonInfo(){ azn.basicInfo()
fmt.Println("Departments of amazon:", azn.departments)
fmt.Println("Can we sell on amazon:", azn.can_sell)
}

func main() {
youtube_var := youtube{websiteinfo{"Youtube", "www.youtube.com", 14}, "me at the zoo", "571
hours, 1 minute and 41 seconds"}
amazon_var := amazon{websiteinfo{"Amazon", "www.amazon.com", 25},
"Automotive, computers, electronics etc", true}
fmt.Println(youtube_var, amazon_var)
fmt.Println("youtube first video:", youtube_var.ytFirstVideo)  fmt.Println("youtube url:",
youtube_var.websiteinfo.url) fmt.Println("youtube url:", youtube_var.url)
fmt.Println("can we sell on amazon:", amazon_var.can_sell)  fmt.Println("Amazon url:",
amazon_var.websiteinfo.url)
fmt.Println("Amazon url:", amazon_var.url) youtube_var.websiteinfo.basicInfo()

```

```
amazon_var.basicInfo() youtube_var.youtubeInfo() amazon_var.amazonInfo()
}
```

Output:

```
{{Youtube www.youtube.com 14} me at the zoo 571 hours, 1 minute and 41 seconds} {{Amazon
www.amazon.com 25} Automotive, computers, electronics etc true}
```

```
youtube first video: me at the zoo youtube url: www.youtube.com youtube url:
www.youtube.com
```

```
can we sell on amazon: true Amazon url: www.amazon.com Amazon url: www.amazon.com
Name of website: Youtube
```

```
Url of website: www.youtube.com How old is this website: 14 Name of website: Amazon
```

```
Url of website: www.amazon.com How old is this website: 25 Name of website: Youtube
```

```
Url of website: www.youtube.com
```

```
How old is this website: 14
```

```
First video on youtube: me at the zoo
```

```
Longest video on youtube: 571 hours, 1 minute and 41 seconds
```

```
Name of website: Amazon
```

```
Url of website: www.amazon.com
```

```
How old is this website: 25
```

```
Departments of amazon: Automotive, computers, electronics etc Can we sell on amazon: true
```

Embedding pointers

In **Go**, **pointer embedding** refers to embedding a pointer to a struct inside another struct. This is an extension of regular **struct embedding** and can provide additional flexibility. The embedded pointer allows you to reference a specific instance of the embedded struct rather than copying its value.

Example:

```
package main

import "fmt"

type websiteinfo struct {
```

```

        name    string
        url      string
        ageinyear int
    }

    type youtube struct {
        *websiteinfo // Pointer embedding
        ytFirstVideo string
        ytlongestvideo string
    }

    func (webinfo *websiteinfo) BasicInfo() {
        fmt.Println("Name of website:", webinfo.name)
        fmt.Println("Url of website:", webinfo.url)
        fmt.Println("How old is this website:", webinfo.ageinyear)
    }

    func main() {
        webinfo := &websiteinfo{"YouTube", "www.youtube.com", 14}
        youtube_var := youtube{
            websiteinfo: webinfo, // Pointer embedding
            ytFirstVideo: "Me at the zoo",
            ytlongestvideo: "571 hours, 1 minute and 41 seconds",
        }
        // Passing the pointer to basicInfo
        youtube_var.BasicInfo()
    }

```

Output:

Name of website: YouTube

Url of website: www.youtube.com

How old is this website: 14

Embedding slice of structs

Embedding a **slice of structs** in Go means including a slice of a struct type inside another struct. This allows the parent struct to manage or group multiple instances of the embedded struct as a collection.

Example:

```
package main

import "fmt"

type websiteinfo struct {
    name    string
    url     string
    ageinyear int
}

type websitelist struct {
    websites []websiteinfo // Embedding a slice of structs
}

// Function to modify a specific website by index
func (wl *websitelist) updateWebsiteAge(index int, newAge int) {
    if index >= 0 && index < len(wl.websites) {
        wl.websites[index].ageinyear = newAge
    } else {
        fmt.Println("Index out of range")
    }
}

func main() {
    websiteData := []websiteinfo{
        {"YouTube", "www.youtube.com", 14},
        {"Google", "www.google.com", 25},
        {"Facebook", "www.facebook.com", 20},
    }
```

```

    }

    websiteList := websitelist{
        websites: websiteData,
    }

    fmt.Println("Before Update:")

    for i, website := range websiteList.websites {
        fmt.Printf("Index %d - Name: %s, URL: %s, Age: %d\n", i, website.name, website.url,
            website.ageinyear)
    }

    websiteList.updateWebsiteAge(0, 15)

    fmt.Println("\nAfter Update:")

    for i, website := range websiteList.websites {
        fmt.Printf("Index %d - Name: %s, URL: %s, Age: %d\n", i, website.name, website.url,
            website.ageinyear)
    }
}

```

Output:

Before Update:

Index 0 - Name: YouTube, URL: www.youtube.com, Age: 14

Index 1 - Name: Google, URL: www.google.com, Age: 25

Index 2 - Name: Facebook, URL: www.facebook.com, Age: 20

After Update:

Index 0 - Name: YouTube, URL: www.youtube.com, Age: 15

Index 1 - Name: Google, URL: www.google.com, Age: 25

Index 2 - Name: Facebook, URL: www.facebook.com, Age: 20

Conclusion

In this chapter, we have learned how to get code flexibility and readability by embedding structs in Go. These structures can be embedded by values or

by-passing reference to a struct. By composition, a struct can contain instances of another struct and thus, provide flexibility. Just as we have composed structs in this chapter, we can do the same for interfaces. In the next chapter, we will discuss interfaces and their role in the Go language.

Questions

1. What is a composition in Go?
2. What is the need for composition in Go?
3. Does the Go language support inheritance?
4. How can we achieve composition in Go? Specify with an example.
5. How can we embed structs using pointers?

Join our book's Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

<https://discord.bpbonline.com>



CHAPTER 14

Interfaces and Polymorphism

Introduction

In this chapter, we will explore interfaces as a way to connect different objects and define behavior in Go. Unlike Java, Go's interfaces offer greater flexibility and enable polymorphism, allowing the same operation to be applied to various types. We will cover how to declare and implement interfaces, use empty interfaces, implement multiple interfaces, and work with type assertion and type switches.

Structure

In this chapter, we will discuss the following topics:

- Introduction to interface and polymorphism
- Declaring interface
- Implementing interface
- Empty interface
- Implementing multiple interfaces
- Type assertion
- Type switch
- Embedding interfaces
- Interfaces as polymorphism

Objectives

In this chapter on interfaces and polymorphism in Go, we will explore how to declare and implement interfaces. By the end, you will be able to organize complex compositions for code reusability, use functions with different argument types, perform type assertions, store various types of data in a single variable, and implement functions with multiple versions effectively.

Introduction to interface and polymorphism

An **interface** is a reflection of the method declaration that only defines an action, not the implementation of that action. It can be defined as a *collection of multiple method names, types of input arguments, and return types*. No variable declaration or implementation is performed in the method body. In the Go language, structs or non-struct types implement the interfaces and define how the action is being performed.

In Go language, polymorphism is achieved by interfaces. In interfaces, no method has a specific implementation. It depends on the type that is implementing an interface and how it defines the method. Multiple types can implement an interface by their desired methods.

Declaring interface

Interfaces are declared in the same way as structs are. With structs, we use the keyword **struct**. Here, we use the keyword **interface** to declare interfaces. Instead of declaring multiple fields, we declare various methods with their names, arguments, and return types in interfaces.

Syntax:

```
type name_of_interface interface{  
    //method declaration  
}
```

Example:

```
type AirthOperation interface{
```



```
Add() float32
Subtract(float32, float32) float32
Multiply(int, int) int
}
```

In the preceding example, we created a variable **AirthOperation** of the type interface. This interface defines three methods. The **Add** method does not accept the argument, but it returns a float-type value. The **Subtract** methods accept two values of float type and return a float type value. The method **Multiply** accepts two integer-type arguments and returns an integer type value.

Like structs, we can create a variable of any interface type. Here, we can create a variable of the type **AirthOperation**:

```
var interface_var AirthOperation
```

Example:

```
func main() {
type AirthOperation interface{
Add() float32
Subtract(float32, float32) float32
Multiply(int, int) int
}
var interface_var AirthOperation
fmt.Println(interface_var) // <nil>
}
```

This type of interface does not have any value; it is nil as we cannot identify who is implementing this interface. Such a value, where we are creating only a variable of an interface value is called the **static value** of the interface. The dynamic value of an interface is obtained by implementing an interface using a type.

Implementing interface

An interface can be implemented by defining the functions provided by an

interface.

We can implement an interface by using structs which is shown as follows:

Example:

```
package main

import "fmt"

// step 1

type AirthOperation interface{
    Add() float32
    Subtract(float32, float32) float32
    Multiply(int, int) int
}

// step 2

type Operandset1 struct{
    op1 float32
    op2 float32
}

//step 3

type Operandset2 struct {
    op1 int
    op2 int
}

//step 4 - Define all abstract methods

func (operandset1_var Operandset1) Add() float32 {
    return operandset1_var.op1 + operandset1_var.op2
}

func main() {

// step 5

operandset1_var := Operandset1{3.6, 6.9}
operandset2_var := Operandset2{45, 67}
```

```
// step 6
```

```
fmt.Println("Return value from Add method with Operandset1: ", operandset1_var.Add())
```

```
}
```

In the preceding example, we are creating two **struct Operandset1** and **Operandset2**. We defined the **Add** method two times, with the receiver of **Operandset1** and **Operandse2** types, and use it accordingly.

The breakdown of each step from code outlining the purpose to enhance the understanding of the implementation is as follows:

1. Create the interface **AirthOperation** and declare various methods.
2. Create **struct Operandset1** that have two fields of type float32.
3. Create another **struct Operandset2** that have two fields of type int.
4. Define all methods that are part of interface. Method should be defined with the same receiver type and return type as declared within interface.
5. Create a variable of type **Operandset1** and **Operandset2** as required.
6. Call **Add** method that has a receiver type **Operandset1** with struct field values.

We can implement the **Subtract** and **Multiply** methods in the same way. A complete example of implementing **AirthOperation** interface is shown as follows:

Example:

```
package main
```

```
import "fmt"
```

```
// step 1
```

```
type AirthOperation interface {
```

```
    Add() float32
```

```
    Subtract(float32, float32) float32
```

```
    Multiply(int, int) int
```

```
}
```

```
// step 2
```

```
type Operandset1 struct {
```

```

    op1 float32
    op2 float32
}
// step 3
type Operandset2 struct {
    op1 int
    op2 int
}
// step 4 - Define all abstract methods
func (operandset1_var Operandset1) Add() float32 {
    return operandset1_var.op1 + operandset1_var.op2
}
func (operandset1_var Operandset1) Subtract(v1 float32, v2 float32) float32 {
    fmt.Println("Passed values in Subtract method: ", v1, v2)
    return operandset1_var.op1 - operandset1_var.op2
}
func (operandset2_var Operandset2) Multiply(v1 int, v2 int) int {
    fmt.Println("Passed values in Multiply method: ", v1, v2)
    return operandset2_var.op1 * operandset2_var.op2
}
func main() {
    // step 5
    a := Operandset1{3.6, 6.9}
    b := Operandset2{3, 6}
    // step 6
    fmt.Println("Return value from Add method with Operandset1: ", a.Add())
    fmt.Println("Return value from Subtract method with Operandset1: ", a.Subtract(2, 3))
    fmt.Println("Return value from Multiply method with Operandset2: ", b.Multiply(2, 3))
}

```

```
}
```

Output:

Return value from Add method with Operandset1: 10.5

Passed values in Subtract method: 2 3

Return value from Subtract method with Operandset1: -3.3000002

Passed values in Multiply method: 2 3

Return value from Multiply method with Operandset2: 18

Empty interface

An interface that does not have any method is called an **empty interface**. Since every type implements zero methods, an empty interface can hold any value. In programming, many cases occur where there is no surety of the type of value. To handle such cases, empty interfaces are used. The **print** function is the best example of empty interfaces that are used to print any value.

In the following example, we are creating two structs of integer and string-type field values. The **empty_interface** interface does not have any method. The method **show_value** accepts empty interface **empty_interface** that can hold any value. The value of such an empty interface is called its dynamic or concrete value.

Example:

```
package main

import "fmt"

type empty_interface interface{
}

func show_value(interface_var empty_interface){ fmt.Println(interface_var)
}

func main() {
type string_struct struct{
a string
}
```

```

string_struct_var := string_struct{"Hello world"}
type float_struct struct{
a float32
}
float_struct_var := float_struct{77.88}
show_value(string_struct_var) // {Hello world}
show_value(float_struct_var) // {77.88}
}

```

Implementing multiple interfaces

In the Go language, multiple interfaces can be implemented by a type. A variable of a specific type is assigned to an interface variable to implement each interface. Here is an example of implementing multiple interfaces with **struct Operandset1**.

Example:

```

package main

import "fmt"

type AirthOperation interface{
Add() float32
}

type CompareOperation interface{
Compare(float32, float32) bool
}

type Operandset1 struct{
op1 float32
op2 float32
}

func (operandset1_var Operandset1) Add() float32 {
return operandset1_var.op1 + operandset1_var.op2
}

```

```

func (operandset1_var Operandset1) Compare(a float32, b float32) bool {
return a==b
}

func main() {
operandset1_var := Operandset1{3.6, 6.9}
fmt.Println(operandset1_var.Add())    // 10.5
fmt.Println(operandset1_var.Compare(operandset1_var.op1, operandset1_var.op2)) // false
var AirthOperation_var AirthOperation = operandset1_var fmt.Println(AirthOperation_var.Add())
// 10.5
var compareOperation_var CompareOperation = operandset1_var
fmt.Println(compareOperation_var.Compare(operandset1_var.op1, operandset1_var.op2)) //
false
}

```

Type assertion

In general language, to assert means to **claim something**. Type assertion claims for a type. It provides access to the interface value's underlying concrete value. In the Go language, the type assertion checks for a specific type whether an interface value holds that type or not. If an interface value holds that specific type, it returns true and the value of that particular type; otherwise, it returns false and zero as a value.

Syntax:

```
t, ok := i.(T)
```

The preceding statement checks an interface **i** for a concrete value of type **T**. Type assertion returns two values, **ok** and **t**. If an interface contains the value of type **T**, it returns true to ok and assigns that value of type **T** to variable **t** on the left side. If an interface does not contain any value of type **T**, then ok will be false and zero will be assigned to variable **t**.

Example:

```

package main

import "fmt"

```

```

type AirthOperation interface{
    Add() float32
}

type Operandset1 struct{
    op1 float32
    op2 float32
}

type Operandset2 struct{
    op1 float32 op2 float32
}

func (operandset1_var Operandset1) Add() float32 {
    return operandset1_var.op1 + operandset1_var.op2
}

func main() {
    var interface_var AirthOperation
    interface_var = Operandset1{3.6, 6.9}
    t, ok := interface_var.(Operandset1)
    fmt.Println(t, ok)
    // {3.6 6.9} true
}

```

In the preceding example, we are checking whether the interface **AirthOperation** is implemented by a specific type or not. We create an interface variable **interface_var**. Here, the type assertion checks whether the interface is implemented by type **Operandset1** or not. In the preceding example, we can clearly see that **struct Operandset1** is implementing the interface's **Add** method. So, type assertion returns true to ok and assigns the struct value to 't'. If we check for the following lines of code, it triggers panic as **struct Operandset2** is not implemented on the interface **AirthOperation**:

```

var a AirthOperation

```



```
a = Operandset2{5.6, 1.9}
t, ok = a.(Operandset2)
fmt.Println(t, ok) // error: impossible type assertion
```

Type switch

A type **switch** allows multiple type assertions in a series. It is similar to a **switch** statement. The **switch** statement compares the value and type **switch** compares the type. It runs the case to see if the type specified in the case statement is held by the interface's value. There is a point in remembering that the “nil” interface always has type **nil**.

Syntax:

```
switch v := i.(type){
case T1:
// v is of type T1
case T2:
// v is of type T2
default:
// no case matched
}
```

In the preceding syntax, i is an interface like we used in type assertion. The keyword T in type assertion is replaced by type here but the purpose is the same. On the left side, value **v** is the interface's value of **type**. The **switch** statement checks whether the type of **v** is **T1** or **T2** and executes that particular case. It executes default if it does not get any matching case.

Example:

```
package main

import "fmt"

type AirthOperation interface{
Add() float32
}
```

```

type Operandset1 struct{
op1 float32
op2 float32
}

type Operandset2 struct{
op1 int
op2 int
}

type Operandset3 struct{
op1 string
op2 string
}

func main() {
check(Operandset1{3.7, 6.7})
check(Operandset2{37, 67})
check(Operandset3{"abc", "def"})
}

func check(i interface{}) {
switch v := i.(type) {
case Operandset1:
fmt.Println("Interface has a value of type Operandset1 and the value is", v)
case Operandset2:
fmt.Println("Interface has a value of type Operandset2 and the value is", v)
default:
fmt.Println("Default case, Interface doesn't have a
value of type", v)
}
}

```

Output:

Interface has a value of type Operandset1 and the value is {3.7 6.7} Interface has a value of type Operandset2 and the value is {37 67} Default case, interface doesn't have a value of type {abc def}

Embedding interfaces

Go language does not support inheritance, but it allows the embedding of interfaces. An interface can embed multiple interfaces or any specific method's signature of an interface. Any changes made in the method of interfaces are reflected in the embedded interface also. This somehow works as an inheritance as we are acquiring the method implementation from another interface.

The following is an example of embedding interfaces:

```
package main

import "fmt"

type AirthOperation interface{
    Add() float32
}

type CompareOperation interface{
    Compare() bool
}

type Operation interface{ AirthOperation Compare() bool
}

type Operandset1 struct{
    op1 float32
    op2 float32
}

func (operandset1_var Operandset1) Add() float32 {
    return operandset1_var.op1 + operandset1_var.op2
}

func (operandset1_var Operandset1) Compare() bool {
```

```

return operandset1_var.op1 == operandset1_var.op2
}

func main() {
var Operation_var Operation
var CompareOperation_var CompareOperation
var AirthOperation_var AirthOperation

Operation_var = Operandset1{5.78, 6.8}
fmt.Println(Operation_var, Operation_var.Add(), Operation_var.
Compare())
CompareOperation_var = Operandset1{6.8, 6.8}
fmt.Println(CompareOperation_var, CompareOperation_var.Compare())
AirthOperation_var = Operandset1{5, 7.8}
fmt.Println(AirthOperation_var, AirthOperation_var.Add())
}

```

Output:

{5.78 6.8} 12.58 false

{6.8 6.8} true

{5 7.8} 12.8

Interfaces as polymorphism

An interface can be implemented by a type that defines the methods declared within an interface. An interface type variable can hold values of different types which implements an interface. Acquiring dynamic values by an interface variable is defined as polymorphism since in a single program, the behavior of interface changes according to the type that is implementing the interface.

In the following example, interface variable **interface_var** can have two types of values **Operandset1** and **Operandset2**:

Example:

```
package main
```

```

import "fmt"

type AirthOperation interface{
    Add() float32
}

type Operandset1 struct{
    op1 float32
    op2 float32
}

type Operandset2 struct{
    op1 int
    op2 int
}

func(operandset1_var Operandset1) Add() float32 {
    return operandset1_var.op1 + operandset1_var.op2
}

func(operandset2_var Operandset2) Add() float32 {
    return float32(operandset2_var.op1 + operandset2_var.op2)
}

func main() {
    var interface_var AirthOperation

    interface_var = Operandset1{5.78, 6.8} fmt.Println(interface_var.Add()) // 12.58
    interface_var = Operandset2{5, 6} fmt.Println(interface_var.Add()) //11
}

```

Conclusion

In this chapter, we saw that an interface is a way of declaring functions. An object that implements the interface defines the functions declared by that specific interface. We learned the concept of type assertion and type switch that checks for specific types in an interface. Here, we see the composition

or embedding of interfaces and how these interfaces are used to achieve polymorphism. In the next chapter, we will discuss one of Golang's reference-type maps widely used for accessing values in an efficient way.

Questions

1. How do interfaces work in Go?
2. How can we declare and implement an interface? Specify with an example.
3. What is the advantage of polymorphism?
4. How is polymorphism achieved in Go?
5. Explain the difference between type assertion and type switch in Go.
6. How are interfaces embedded in Go?

Join our book's Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

<https://discord.bpbonline.com>



CHAPTER 15

Maps

Introduction

In large applications, we need to hold the data in an organized way so that the values can be accessed efficiently. The concept of the map is the same as a dictionary, hash tables, etc. These are all defined by the same data structure. In this chapter, let us go ahead with maps.

Structure

In this chapter, we will discuss the following topics:

- Introduction to maps
- Declaring a map
- Initializing a map
- Map operations
- Map as a reference type

Objectives

After studying this chapter, you will be able to understand the concept of maps and access the values by keys. You will also learn how to add, get, update, and delete the keys and values and iterate over maps.

Introduction to maps

The **map** is a type of data structure that stores value in the form of a key-value pair. Keys are associated with data values that can be changed as required. The map contains unique keys that cannot be changed. The type of map keys could be anything comparable. Keys can be deleted or added to the map data structure. In the map, values are accessed by using the corresponding key, similar to how we access an index in the array. Maps are also called hashes or dictionary in the general programming language.

Use of maps

The maps can be used for the following purposes:

- Fast lookups and retrieval of values
- Easy access to value by using keys
- Deletion of data based on keys
- Implementing a hash table

Declaring a map

A map can be declared in the same way as arrays. With map declaration, there is a need to declare the type of keys also. Here is the syntax to declare maps:

```
var name_of_map map [type_of_keys] type_of_values
```

Example:

```
func main() {  
    var phone_dict map [string] int64 fmt.Println(phone_dict) // map[ ]  
}
```

In the preceding example, variable **phone_dict** is a map that defines the string type of keys to **int** type values. By default, the map contains zero value or nil. Here, **phone_dict** is nil, as it does not have any key and so, no value.

Initializing a map

After declaring a map, its default value is nil. Reading from a nil map returns an empty result since it contains no keys, but attempting to write to a nil map causes a runtime error. Similar to slices, maps are reference types, and they must be initialized by assigning them to a variable before use. For example:

```
phone_dict1 := map[string] int64{}  
fmt.Println(phone_dict1) // map[]  
phone_dict2 := map[string] int64{  
    "Rahul" : 9876543201,  
    "Swati" : 9123456789,  
}  
phone_dict2["aayush"] = 9823165748  
fmt.Println(phone_dict2) // map[Rahul:9876543201 Swati:9123456789 aayush:9823165748]
```

phone_dict1 indicates an empty map, while **phone_dict2** has two keys, **Rahul** and **Swati**. We can get their number by accessing the associated key. In the Go language, the **make** function is also used to initialize a map. The **make** function allocates and initializes map data structure that allows adding elements to the map. It returns a map value that points to it.

Example:

```
phone_dict3 := make(map[string] int64)  
phone_dict3["Rahul"] = 9876543201  
phone_dict3["Swati"] = 9123456789  
fmt.Println(phone_dict3) // map[Rahul:9876543201 Swati:9123456789]
```

With arrays, if we access an index “out of an array limit,” the compiler gives an error. On the other hand, if we access the key that does not belong to that map, it will not give any error. In such a case, it returns the default value of the specified type. The following is a complete example of initializing a map:

```
package main  
import "fmt"  
func main() {
```

```

phone_dict1 := map [string] int64{}
phone_dict1["Rahul"] = 9876543201
fmt.Println(phone_dict1)

phone_dict2 := map [string] int64{
    "Rahul": 9876543201,
    "Swati": 9123456789,
}
phone_dict2["aayush"] = 9823165748
fmt.Println(phone_dict2["Swati"])

phone_dict3 := make(map [string] int64)
fmt.Println(phone_dict3) // map[ ]
phone_dict3["Rahul"] = 9876543201
phone_dict3["Swati"] = 9123456789
fmt.Println(phone_dict3)
fmt.Println(phone_dict3["aayush"])
}

```

The output is as follows:

```

map[Rahul:9876543201]
9123456789
map[Rahul:9876543201 Swati:9123456789]
0

```

Map operations

Various operations can be performed on maps in Go.

Length of map

The length of a map is defined by the number of keys contained by a map. In the

Go language, we have a function called **len** to get the length of a map.

Example:

```

func main() {
phone_dict3 := make(map [string] int64)
phone_dict3["Rahul"] = 9876543201
phone_dict3["Swati"] = 9123456789
fmt.Println(phone_dict3)
fmt.Println("length of phone_dict3:", len(phone_dict3))
}

```

Output:

```

map[Rahul:9876543201 Swati:9123456789]
length of phone_dict3: 2

```

Check for keys

In the Go language, we have a few functions to get all the keys of a map or to check whether a specific key exists in a map or not.

A two-value assignment test is done to check for a key in a map. The syntax to test if a key exists in a map or not is as follows:

```

i, ok := map_name[key_name]

```

It checks for a key **key_name** in map **map_name**. If a key exists, it returns true to **ok**, and the value of that key is assigned to variable **i**. If the key does not exist, it returns false to **ok**, and the default value of the value's type is assigned to variable **i**. This is quite similar to type assertion in the interface.

Example:

```

package main

import (
    "fmt"
    "reflect"
)

func main() {
phone_dict3 := make(map [string] int64)
phone_dict3["Rahul"] = 9876543201

```

```

phone_dict3["Swati"] = 9123456789
mapkeys := reflect.ValueOf(phone_dict3).MapKeys()
fmt.Println("Keys of phone_dict3", mapkeys)
val, ok := phone_dict3["Swati"]
fmt.Println(ok, val)
val, ok = phone_dict3["Suman"]
fmt.Println(ok, val)
}

```

Output:

Keys of phone_dict3 [Rahul Swati]

true 9123456789

false 0

Iteration over maps

We can iterate over a map as slices using map keys. At the same time, for iterating slices, a specific order of iteration is followed depending on the index. With maps, no order is specified. It randomly works on map records. If we want to work on map records in a specific order, we can sort the slice of keys accordingly.

Example:

```

package main

import (
    "fmt"
    "sort"
)

func main() {
    phone_dict3 := make(map[string] int64)
    phone_dict3["Rahul"] = 9876543201
    phone_dict3["Swati"] = 9123456789
    phone_dict3["Suman"] = 9000000000
}

```

```

phone_dict3["Mayank"] = 98000000000
phone_dict3["Deepak"] = 80000000000
var mapkeys [] string
for itr := range(phone_dict3){
fmt.Println(itr)
mapkeys = append(mapkeys, itr)
}
sort.Strings(mapkeys)
for _, k := range mapkeys {
fmt.Println("Key:", k, "Value:", phone_dict3[k])
}
}

```

Output:

Rahul Swati Suman Mayank Deepak

Key: Deepak Value: 80000000000

Key: Mayank Value: 98000000000

Key: Rahul Value: 9876543201

Key: Suman Value: 90000000000

Key: Swati Value: 9123456789

Delete an element from the map

We have Go's built-in function **delete** that is used to delete a specific key from a map. The delete function deletes the specified key and does not return anything. There is no harm if the specified key is not present in a map. It does not give any error in such case.

Syntax:

delete (map_name, key_to_be_deleted)

Example:

```

func main() {
phone_dict3 := make(map [string] int64)

```

```

phone_dict3["Rahul"] = 9876543201
phone_dict3["Swati"] = 9123456789
phone_dict3["Suman"] = 9675463768
phone_dict3["Mayank"] = 9342516789
phone_dict3["Deepak"] = 8765453678
fmt.Println("Before deleting a key:", phone_dict3)
delete(phone_dict3, "Suman")
fmt.Println("After deleting a key:", phone_dict3)
}

```

Output:

```

Before deleting a key: map[Deepak:8765453678 Mayank:9342516789
Rahul:9876543201 Suman:9675463768 Swati:9123456789]
After deleting a key: map[Deepak:8765453678 Mayank:9342516789
Rahul:9876543201 Swati:9123456789]

```

Comparison of maps

In the Go language, maps can be compared in two ways. The first is by checking whether a map is nil or not. The second is by comparing two maps by differentiating each one's element.

We can use the operator `==` to check if a map is nil or not.

To compare two maps individually, Go provides the function **DeepEqual** in the **reflect** package. It returns true if both maps have the same keys and the same key's associated values, otherwise false.

Example:

```

package main

import (
    "fmt"
    "reflect"
)

func main() {

```

```

phone_dict := make(map [string] int64)
phone_dict["Rahul"] = 9876543201
phone_dict["Swati"] = 9123456789
phone_dict1 := make(map [string] int64)
phone_dict1["Rahul"] = 9876543201
phone_dict1["Swati"] = 9123456789
fmt.Println(phone_dict==nil)
fmt.Println(reflect.DeepEqual(phone_dict, phone_dict1))
phone_dict1["Swati"] = 1987654321
fmt.Println(reflect.DeepEqual(phone_dict, phone_dict1))
}

```

Output:

false true false

Map as a reference type

Like slices, maps are also of referenced type data structure. When we initialize a map and assign it to a variable, it points to the internal structure of a map. Any changes made in the map record is reflected in its memory. On assigning the map variable to another variable, both of them refer to the same data structure of the map. Any changes made in one variable reflects another one as well, as both are referring to the same memory location.

In the following example, we are assigning map variable **phone_dict** to **phone_dict1**. When we change the value of the key Mayank in **phone_dict1**, it reflects the same in **phone_dict** too.

Example:

```

func main() {
phone_dict := make(map [string] int64)
phone_dict["Rahul"] = 9876543201
phone_dict["Swati"] = 9123456789
phone_dict["Suman"] = 9675463768

```

```

phone_dict["Mayank"] = 9342516789
phone_dict["Deepak"] = 8765453678
fmt.Println(phone_dict["Mayank"])
phone_dict1 := phone_dict
phone_dict1["Mayank"] = 1234567890
fmt.Println(phone_dict["Mayank"])
}

```

Output:

9342516789

1234567890

The same goes for function, when we pass a map as an argument to a function. Here is an example:

```

package main

import "fmt"

func change_value(phone_dict1 map[string]int64){
phone_dict1["Mayank"] = 1234567890
}

func main() {
phone_dict := make(map [string] int64)
phone_dict["Rahul"]    =    9876543201
phone_dict["Swati"] = 9123456789
phone_dict["Suman"] = 9675463768
phone_dict["Mayank"] = 9342516789
phone_dict["Deepak"] = 8765453678
fmt.Println("Before calling function, value of key 'Mayank' is:",
phone_dict["Mayank"])
change_value(phone_dict)
fmt.Println("After calling function, value of key 'Mayank' is:", phone_dict["Mayank"])
}

```


Output:

Before calling function, value of key 'Mayank' is: 9342516789 After calling function, value of key 'Mayank' is: 1234567890

Conclusion

Thus, we can say that maps are of reference type in Go and connect keys to value. For a fast lookup and retrieval of values, maps can be used. Here, we have learned the declaration, initialization, and various operations that can be performed on maps. After getting an idea of the Golang data types, we will understand one of the key concepts of Golang, which is concurrency. Let us understand it in the next chapter.

Questions

1. Specify the declaration and initialization of maps in Go with an example.
2. How can we use the map as a reference in Go?
3. What is the default value of the map?
4. Which function is used to get the length of a map in Go?
5. What do you understand by map comparison in Go?
6. How to check whether a specific key exists in a map or not

Join our book's Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

<https://discord.bpbonline.com>



CHAPTER 16

Concurrency with Go

Introduction

Concurrency is an extremely important feature of Go that enables the execution of multiple tasks at the same time, allowing developers to build highly efficient and responsive applications. In Golang, Goroutines, lightweight threads managed by Go runtime and channels, which facilitate communication and synchronization between these Goroutines, make concurrency possible. Together, they form the backbone of Go's concurrency model. Let us learn about it in this chapter.

Structure

In this chapter, we will discuss the following topics:

- Introduction to concurrency
- Parallelism versus concurrency
- Goroutines
- Waiting for Goroutine to finish

Objectives

After studying this chapter, you will be able to understand the key concept of Goroutines and run multiple processes simultaneously. You will also

learn to write efficient code in Go.

Introduction to concurrency

While creating large applications, it becomes necessary not to wait for any process to complete. Concurrency allows running different parts of a program in an overlapping period without affecting the outcome of a program. It prevents the issues that occur while updating the data by two different processes at a time. For example, banking allows more than one transaction at the same time, or it allows access to the same data from multiple sources at a time. Golang facilitates concurrency with Goroutines and channels. Concurrency is quite similar to the thread used in C++, Java, or Python.

Parallelism versus concurrency

Running multiple things in parallel at the same time is called **parallelism**. The multicore processor makes parallelism possible. The CPU's multiple cores run multiple processes at the same time. With concurrency, we run multiple processes but not at the same time. The CPU time is divided among multiple processes that need to run. The fluctuation between the processes is done so fast that it seems that all the different processes are running at the same time. In a single-core processor, concurrency is achieved by **context-switching** that divides the time for each process and executes it.

Thread

Thread is a sub-task of a process. A process can be divided into multiple threads that share the same address space and can be executed independently. Threads have their own register set and stack to perform various operations. Threads allow maximum CPU utilization and, thus, increase its performance. A thread belongs to the same process. The data within memory can be shared by all the threads. Threads are the sources of concurrency. In a single-core processor, each sub-task behaves like a thread, and concurrency is achieved by switching between the threads.

Goroutines

Goroutines are ordinary methods or functions with some additional features. Goroutines can be executed concurrently with other methods. Every concurrently running method or function is called a Goroutine in the Go language. Goroutines are called lightweight threads as they exist in Go runtime virtual space. Every Go program contains at least one Goroutine. As each Go program contains the main method, it is, itself, a Goroutine that runs independently. The main method is a root Goroutine; all other Goroutines are executed within the main. Once the compiler reaches the end of the **main** function, it terminates all the created Goroutines. Like threads, all Goroutines run in the same address space in any Go program. With Goroutines, the size of the stack grows and shrinks according to the need of the program, while in thread, the stack size is fixed (several megabytes), which can lead to inefficient memory usage or memory exhaustion if too many threads are created. Golang has the specific keyword **go** to declare a function as a Goroutine:

```
go name_of_function()
```

In the following example, we create a function **show_value** that prints the value up to a specific range, and after calling this function, we show a message that says that we have called the function.

Example:

```
func show_value(i int){  
    for i<10{  
        fmt.Println(i)  
        i = i+1  
    }  
}  
  
func main() {  
    show_value(0)  
    fmt.Println("We have called the function")  
}
```

Output:

0
1
2
3
4
5
6
7
8
9

We have called the function

Here, we see that it shows a message when it returns from a function. Suppose we do not want that, and we need to display a message early so that a user can understand that a function is being called. We can use that function as a Goroutine. In such a case, it would not wait for the function **show_value** to return and will execute the next statement, followed by a Goroutine.

Example:

```
package main

import "fmt"

func show_value(i int){
    for i<10{
        fmt.Println(i)
        i = i+1
    }
}

func main() {
    go show_value(0)

    fmt.Println("We have called the function")
}
```

```
}
```

Output:

We have called the function

We can see now that the only message we get is in output, but not the values that we are trying to display within a function. The function executed well but since we used Goroutines in the preceding example, it will call the Goroutines and execute the next statement defined after the Goroutine. So, after calling the Goroutine, it prints a message and gets the closing block of the main function. On terminating from the main function, all Goroutines are terminated, and thus, the execution of **show_value** also gets terminated. Here, to overcome this issue, we can use the **Scanln** function, which waits for an input value. Till that time, the execution of Goroutine will take place.

Example:

```
package main

import "fmt"

func show_value(i int){
    for i<10{
        fmt.Println(i)
        i = i+1
    }
}

func main() {
    go show_value(0)
    fmt.Println("We have called the function")
    fmt.Scanln()
}
```

Output:

We have called the function 0

1

2

3

4

5

6

7

8

9

We can use a function as a Goroutine or ordinarily in a single program. It depends on the way the function calls. In the following example, we call the method **show_value** as a Goroutine, and then the standard method is being called. The output explains how the execution is done.

Example:

```
package main

import (
    "fmt"
    "time"
)

func show_value(i int){
    for i<10{
        fmt.Println(i)
        time.Sleep(200 * time.Millisecond)
        i = i+1
    }
}

func main() {
    go show_value(0)
    fmt.Println("We have called a goroutine")
    show_value(0)
    fmt.Println("We have called a function")
}
```

Output:

We have called a goroutine 0

0

1

1

2

2

3

3

4

4

5

5

6

6

7

7

8

8

9

9

We have called a function

Multiple Goroutines

We can execute multiple Goroutines in a single program without affecting any other Goroutine. Here is an example of running multiple Goroutines in a program:

Example:

```
package main
```



```

import (
    "fmt"
    "time"
)

func show_value(i int){
    for i<10{
        fmt.Println(i)
        time.Sleep(200 * time.Millisecond)
        i = i+1
    }
}

func show_value1(c int){
    for c<110{
        fmt.Println(c)
        time.Sleep(200 * time.Millisecond)
        c = c+1
    }
}

func main() {
    go show_value(0)
    fmt.Println("We have called a goroutine show_value")
    go show_value1(100)
    fmt.Println("We have called a goroutine show_value1")
    fmt.Scanln()
}

```

Output:

We have called a goroutine show_value We have called a goroutine show_value1 100

0

1

101

102

3

103

104

4

105

5

6

106

107

7

8

108

9

109

Goroutines at multiple cores

In the Go language, we can fix the number of CPU cores to run Goroutines in a program. There are various ways to run Goroutines on multiple cores of a system. Go provides an environment variable **GOMAXPROCS**, that defines the number of cores to be used in a program. This variable takes control at runtime in the Go program. The default value of this variable is the number of cores available in the system. The system detects at start-up according to the running operating system.

Waiting for Goroutine to finish

In the previous example, we saw that the Goroutine gets terminated because it reaches the end of the main function. Go provides a synchronized mechanism for Goroutines by **WaitGroup**, defined in the **sync** package. **WaitGroup** is a struct that contains a field **counter** to indicate the number

of Goroutines that we need to wait for. It is used to wait for all the Goroutines to terminate. We have three basic methods used to track all the Goroutines that are called in a program:

- **Add:** This method tells us how many Goroutines need to be waited.
- **Done:** When a Goroutine terminates, it must call the “Done” method to indicate that the execution of the specific Goroutine has been done.
- **Wait:** The **Wait** method is used by the **main** Goroutine, until the counter become zero. When all the Goroutines get terminated and the counter is zero, the **Wait** function returns and continues the further execution.

Example:

```
package main

import (
    "fmt"

    "time"

    "sync"
)

func show_value(i int, wg *sync.WaitGroup){
    for i<10{
        fmt.Println(i)

        time.Sleep(200 * time.Millisecond)

        i = i+1
    }

    wg.Done()
}

func show_value1(c int, wg *sync.WaitGroup){
```

```

for c<110{
    fmt.Println(c)
    time.Sleep(200 * time.Millisecond)
    c = c+1
}

wg.Done()
}

func main() {
    var wg
    sync.WaitGroup
    wg.Add(2)
    go show_value(0, &wg)
    fmt.Println("We have called a goroutine show_value")
    go show_value1(100, &wg)
    fmt.Println("We have called a goroutine show_value1")
    wg.Wait()
    fmt.Println("Done")
}

```

Output :

We have called a goroutine show_value We have called a goroutine show_value1

100

0

1
101
102
2
103
3
104
4
5
105
6
106
107
7
8
108
109
9
Done

Conclusion

In this chapter, we saw how Golang provides flexibility to execute various tasks at the same time so that time complexity can be reduced. We understand Goroutines and the necessary functions that we have in the Go

language. After getting an idea of one of the concurrency constructs, that is, Goroutines, let us move on to another construct called channel in the next chapter that facilitates communication between Goroutines. This will help us understand how to coordinate tasks effectively.

Questions

1. How can we track Goroutines in Go?
2. Does the main function work as a Goroutine in Go programs?
3. What do you understand by WaitGroup?
4. What is the use of the wait method? Specify with an example.
5. What is GOMAXPROCS in Go?

CHAPTER 17

Mutex and Channels

Introduction

Channels are like pipelines that connect concurrently running Goroutines. In Go language, a channel has a specific type, that is, the type of values it can transport from one Goroutine to another. Go provides a channel operator through which we can send and receive the data through channels. A **channel operator** is used to sending and receive the data between two channels.

Structure

In this chapter, we will discuss the following topics:

- Declaring channels
- Channel operations
- Deadlock
- Closing a channel
- Direction of a channel
- Select keyword
- Data race in Go
- Race condition
- Check for data race

- Mutex

Objectives

After studying this chapter, you will be able to properly send and receive data across Goroutines. Using the select keyword with channels allows you to manage numerous channel operations effectively. The chapter will also cover Golang's lock and unlock techniques using mutexes, which will improve your ability to handle concurrency and synchronization in your programs.

Declaring channels

In the Go language, we can declare a channel in two ways, using **var** or **make** method. The **var** keyword reserves the memory for the named variables of a specified type and initializes them with the type's default value. If we declare a channel variable using the **var** keyword, it creates a nil channel. Declaring a channel with the **make** method also reserves space in storage, initializes memory, and creates an underlying header of a specific type.

Syntax:

```
var name_of_channel chan type_of_channel
```

Or

```
Name_of_channel := make(chan type_of_channel)
```

Example:

```
func main(){  
    var ch1 chan int  
  
    fmt.Println("channel with 'var' keyword:", ch1)  
  
    ch2 := make(chan string)  
  
    fmt.Println("channel with 'make' method:", ch2)  
}
```

Output:

channel with 'var' keyword: <nil> channel with 'make' method: 0x1180e180

In the preceding example, channels **ch1** and **ch2** transport integer and string type of data between channels. **ch1** is declared with **var** keyword that initializes it with the default value nil. We cannot transfer data using nil channels, as sending or receiving data on a nil channel will block forever. Therefore, in Go language, channels are mostly declared with the **make** method. The “**make**” method provides easy-to-use channels and returns a value of a specified type. By default, channels are pointers, but there is no need to dereference them to access the data.

Channel operations

Golang facilitates various channel operations, which are as follows:

Send operation with channels: The send operation sends the data of a specified type to a channel variable. The send operation blocks the channels until no channel is ready to receive the data. While sending data, it first checks for a channel that is ready to receive the data.

Example:

```
func main(){  
    ch := make(chan int)  
    ch<- 8 fmt.Println(ch)  
}
```

The preceding example will show an error. Statement **ch<- 8** blocks the channel and no channel is ready to receive the data:

- **Receive operation in channel:** By performing the receive operation, we can get the data from a channel to a variable. While receiving the data from channels, we get two values. The first is the data that a channel sends, and the other one indicates the status of whether an operation is successful or not. The idiom for the second value is **ok**. The value of **ok** variable is **true** if the data is received successfully from the channel and **false only if the channel is closed** and no more data is available to receive.

Example:

```
package main

import "fmt"

func show_value(c chan int){
    c <- 8
}

func main(){
    c := make(chan int)
    go show_value(c)
    a, ok := <-c
    fmt.Println("value received from channel:", a)
    fmt.Println("status of operation is:", ok)
}
```

Output :

value received from channel: 8

status of operation is: true

In the preceding example, we are using the Goroutine **show_value** that sends a value to a channel. After calling the Goroutine, a channel sends the data and on the next line, the channel is ready to receive the data in the main function. As the operation is successful, true is assigned to an ok variable. The sending and receiving channel should be of the same type otherwise the compiler will show an error.

If we execute the same program with the channels declared using the **var** keyword, it will give an error.

Example:

```
func show_value(c chan int){
```

```

c <- 8
}

func main(){
var c chan int
go show_value(c)
a, ok := <-c
fmt.Println("value received from channel:", a)
fmt.Println("status of operation is:", ok)
}

```

Output :

error " [chan send <nil chan>] "

We have observed that the send operation blocks a channel if no channel is ready to receive the data. So, we perform the receive operation to avoid such an issue. To overcome such scenarios, we can use buffered channels in Go programs. The following describes buffered and unbuffered channels briefly:

- o **Unbuffered channel:** By default, channels are **unbuffered** in the Go language. Such channels accept the send operation if there is a channel ready to receive the data. We saw in the previous examples that on sending data without any corresponding receiver, the compiler gives an error.

- o **Buffered channel:** Buffered channels can send a specified number of values without any corresponding receiver. They store the values sent by a channel in a buffer and return when a corresponding channel is ready to receive the data. The number of values that are allowed to send without any receiver is called the capacity of a channel. The following is the syntax to create buffered channels in the Go language.

Syntax:

```
name_of_channel      :=      make(chan      type_of_channel,  
capacity_of_channel  
)
```

Example:

```
func main(){  
  ch := make(chan int, 2)  
  ch<- 8  
  a := <-ch  
  fmt.Println(ch, a)  
}
```

Output:

0x1183c0c0 8

In the preceding example, the capacity of the buffered channel is 2 which indicates that it can send maximum two values without waiting for a receiver channel. We can send the second value as shown:

Example:

```
func main(){  
  ch := make(chan int, 2)  
  ch<- 8  
  ch<- 99  
  a := <-ch  
  b := <- ch  
  fmt.Println(a, b)  
}
```

Output:

8 99

In the preceding example, it stores the values in a buffer and returns to the corresponding receiver when the channel is ready to receive the data. We can receive the data from two or multiple channels in the same line which is shown as follows:

a, b := <-ch, <-ch

In the previous program, sending the third value without a corresponding receiver channel will give an error as the channel capacity is 2.

- **Find the capacity of the channel:** Golang provides a function **cap** to get the capacity of a channel. The following example illustrates the use of the **cap** function with channels.

Example:

```
func main(){  
    ch := make(chan int, 2)  
    cap_ch := cap(ch)  
    fmt.Println("Capacity of channel is:", cap_ch)  
    for itr := 1; itr<=cap_ch; itr++{  
        ch<- itr*2  
        fmt.Println("value received from channel is:", <-ch)  
    }  
}
```

Output:

Capacity of channel is: 2

value received from channel is: 2 value received from channel is: 4

Deadlock

In the previous examples, we saw that on sending data, it waits for a

channel that is ready to receive the data. If it does not find any receiver channel, it blocks that Goroutine and passes the control to other Goroutines to run. When it finds the receiver channel in another Goroutine, it unblocks the send operation and runs that Goroutine.

In such cases, if we do not get any receiver channel in another Goroutines or there is no other Goroutine available in the program, the compiler imagines that all the Goroutines are asleep. The send operation remains blocked until the end of the program. Such a situation is called a deadlock.

The deadlock can occur in two ways depending on the operation that is being blocked.

On sending the data to a channel, it blocks the current Goroutine and schedules other Goroutines with the hope of getting a receiver channel. Here, the “sending operation is getting blocked.” On receiving the data from a channel, it checks for an existing value in a channel. If a channel contains no value, it blocks the receive operation until it does not get any value in that channel. So here, the “receive operation is getting blocked.” In the Go language, a deadlock occurs if we have at least one blocked operation and no other Goroutine is available to schedule in a program. In such a case, the program crashes, and the compiler shows an error. The following example explains a deadlock.

Example:

```
package main

import "fmt"

func show(){
    fmt.Println("Hello")
}

func main(){
    str_ch := make(chan string)
    go show()
    str_ch<- "Hello world"
    fmt.Println("Send operation is getting blocked")
}
```

Output:

Hello

fatal error: all goroutines are asleep – deadlock!

goroutine 1 [chan send]

In the previous example, we are calling Goroutines and then trying to send a value to a channel. It blocks the send operations as there is no other channel to receive the data. After blocking the main Goroutines, there is no other Goroutine to schedule. Hence, the deadlock occurs.

Example:

```
package main

import "fmt"

func show(){
    fmt.Println("Hello world")
}

func main(){
    str_ch := make(chan string)
    go show()
    a := <- str_ch
    fmt.Println("Receive operation is getting blocked", a)
}
```

Output:

Hello

fatal error: all goroutines are asleep – deadlock!

goroutine 1 [chan receive]

In the preceding example, after calling **show** Goroutine, we are trying to receive a value from the channel **str_ch**, but no value has been assigned to a channel till now. So, it blocks the current main Goroutine. As it continues, it finds that there is no other Goroutine to schedule. In such a case, a deadlock occurs and the compiler gives a fatal error.

Using multiple Goroutines

A Go program can have multiple Goroutines with sharing values. Go defines the concept as **Do not communicate by sharing memory. Instead, share memory by communicating.**

Example:

```
package main

import "fmt"

func square(input chan int, output chan int) {
    value := <-input
    output <- value * value
}

func cube(input chan int, output chan int) {
    value := <-input
    output <- value * value * value
}

func main() {
    num := make(chan int)
    squareOutput := make(chan int)
    cubeOutput := make(chan int)
    // Start goroutines for square and cube
    go square(num, squareOutput)
    go cube(num, cubeOutput)
    // Send input to both functions
    num <- 3 // Send to square
    sqrVal := <-squareOutput // Receive from square
    num <- 3 // Send to cube
    cubeVal := <-cubeOutput // Receive from cube
    fmt.Println("Square of value:", sqrVal)
    fmt.Println("Cube of value:", cubeVal)
}
```


Output:

Square of value: 9

Cube of value: 27

Following is the explanation of the code:

- First, we are creating a channel called “num” that passes an integer type of value.
- In the second line, we call the “square” goroutine. In this Goroutine, we are trying to get the value from a channel. However, since no value has been assigned to a channel till now, it blocks the receive operation and control is passed to the “main” Goroutine.
- In the third line, we pass “3” to a channel that further executes the receive operation in the square Goroutine. Then, the final value is passed to a channel called ‘num_ch’ in the Goroutine.
- In the fourth line, the channel is ready to receive a value that is sent by the “square” goroutine. Thus, we get a square of 3.
- In the same way, we get a cube of value in line 5, 6, and 7.

Closing a channel

While programming, there may be an instance where after creating a channel, we do not want to use it further in our Go program. In such a case, Go provides a method to close a channel that indicates that no value will be sent through a channel. The close function is used to close a channel.

While receiving the value from a channel, we can get the status of a channel. It will return true if a channel is open and the receive operation is successful; otherwise, it will return false if a channel is closed.

Example:

```
package main

import "fmt"

func cube(value chan int){
    fmt.Println("In cube goroutine")
    val := <-value
```

```

        value <- val*val*val
    }
func square(value chan int){
    fmt.Println("In square goroutine")
    val := <-value
    fmt.Println(val)
    value <- val*val
}
func main(){
    num := make(chan int)
    go square(num)
    num<- 3
    sqr_val := <-num
    go cube(num)
    num<- 3 close(num)
    cube_val, ok := <-num
    fmt.Println("Square of value:", sqr_val)
    fmt.Println("Cube of value:", cube_val, ok)
}

```

Output:

In square Goroutine

3

In cube goroutine Square of value: 9 Cube of value:0 false

Panic: send on closed channel

In the preceding example, after calling the cube Goroutine, we passed a value to the num channel. Before we received the final value from the channel sent by the cube Goroutine, we closed that channel. This situation lead to a runtime panic. On receiving a value from a closed channel, it returns zero without any error and assigns false to an ok variable.

Direction of a channel

In the previous examples, we have seen that we can perform, read and write both operations on a channel. By default, Go provides a bidirectional nature of channels. We can also create unidirectional channels in the Go language that either allow the read operation or receive operation.

The syntax for declaring the unidirectional channel is as follows:

- Receive only channel:

```
name_of_channel := make(<- chanint)
```

- Send only channel:

```
name_of_channel := make(chan <- int)
```

Here is an example of send only channel:

Example:

```
package main

import (
    "fmt"
    "sync"
)

var wg sync.WaitGroup

func square(value chan<- int){
    fmt.Println("defining send only goroutine")

    val := value * value

    fmt.Println(val)

    wg.Done()
}

func main(){
    send_ch := make(chan<- int, 1)
    wg.Add(1)
    go square(send_ch) send_ch<- 3
    wg.Wait()
}
```

```
}
```

Output:

defining send only Goroutine 0x1183c0c0

In the previous example, we have defined a send only unidirectional channel **send_ch** that only sends a value to a channel. We cannot receive value from such channels. On receiving value from such channels, the compiler gives an error like the following:

Example:

```
package main import ("fmt"
"sync"
)
var wg sync.WaitGroup
func square(value chan<- int){
    fmt.Println("defining send only goroutine")
    val := value
    fmt.Println(val)
    val<- value wg.Done()
}
func main(){
    send_ch := make(chan<- int, 1)
    wg.Add(1)
    go square(send_ch) send_ch<- 3
    wg.Wait()
}
```

Output:

Cannot use value <type chan<- int> as type int in send

In the same way, we can declare receive only channels in the Go language. An example of receiving only channels is shown as follows:

Example:

```
package main
```

```

import (
    "fmt"
    "sync"
)

var wg sync.WaitGroup

func square(value <-chan int){
    fmt.Println("defining receive only goroutine")
    val := <-value
    fmt.Println(val)
    wg.Done()
}

func main(){
    receive_ch := make(<-chan int)
    wg.Add(1)
    go square(receive_ch)
    wg.Wait()
}

```

In the preceding example, we created a receive-only channel **receive_ch** and called squareGoroutine. Here, we are trying to receive a value from a channel. As there is no value in the channel, it blocks the receive operation, and the compiler gives an error “deadlock.” We can see the solution to this issue in the next program. We can convert a bidirectional channel to unidirectional, for a specific Goroutine.

Example:

```

package main

import "fmt"

func receive(receive_ch chan<- int, val int) {
    receive_ch<- val
}

func square(receive_ch<-chan int, send_ch chan<- int){
    fmt.Println("defining send only goroutine")
}

```

```

val := <- receive_ch send_ch<- val
}

func main(){
send_ch := make(chan int, 1)
receive_ch := make(chan int,1)
receive(receive_ch, 6)
square(receive_ch, send_ch)
fmt.Println(<-send_ch)
}

```

Output:

defining send only Goroutine

6

Select keyword

Select keyword in Go is used to wait on multiple channel operations simultaneously. The select statement is like a switch that works with multiple cases. With the select statement, the case refers to the channel's send/receive operation. Select waits for a specific case until it is ready to perform the operation specified in a particular case statement. If no case is defined within a select statement, then it will lead to a deadlock since it did not get any case for channel communication. The select statement gets blocked if no case statement is ready to execute. In the case of blocking, it waits for a case ready to perform a channel operation. To avoid select blocking, it can have a default case like a switch statement. If no case statement is ready to perform the communication, it proceeds with the default case. If the select statement has multiple cases that are ready to perform the specified channel operation, it selects any random case statement and executes it.

The syntax of the **select** statement is the same as the **switch** statement except for the case statement. Here, the **case** statement defines a channel operation.

Syntax:

```

select{
case send_receive_operation1:
// body of case statement 1
case send_receive_operation2:
// body of case statement 2
.....
.....
}

```

Multiple examples of the **select** statement that describe the use of the **select** statement in different ways are as follows.

Example:

```

package main

import (
    "fmt"
    "time"
)

func square(num_ch chan int){
    value := <-num_ch
    sqr_val := value*value
    time.Sleep(5*time.Second)
    num_ch<- sqr_val
}

func cube(num_ch chan int){
    value := <-num_ch
    cube_val := value*value*value time.Sleep(10*time.Second)
    num_ch<- cube_val
}

func main(){
    sqr_ch := make(chan int)

```

```

go square(sqr_ch)

    sqr_ch<- 3

cube_ch := make(chan int)

go cube(cube_ch)

cube_ch<- 5

select{

case sqr_val := <-sqr_ch:

    fmt.Println("square of a value is:", sqr_val)

case cube_val := <-cube_ch:

    fmt.Println("Cube of a value is:", cube_val)

}

}

```

Output:

square of a value is: 9

In the preceding example, we have two Goroutines, **square** and **cube**. We are calling both of them by passing a channel value from the main Goroutine. The select statement has two cases. In the first case, we receive the value from the channel **sqr_ch** that is sent by the **square** Goroutine. In the second case, we receive the value from channel **cube_ch** that is sent by the cube Goroutine. In the square goroutine, we have given the sleep time of five seconds and ten seconds for the cube goroutine, so that the square Goroutines will finish its execution first. As the square Goroutine finishes its execution, it will send the value to a channel and **case1** would be ready to execute in the **select** statement. The **select** statement executes that case and comes out of its block.

In the preceding example, we saw that only one case was ready to execute at a time. The following is an example where both cases are ready to execute at the same time.

Example:

```

package main

import ("fmt"

```



```

"time"
)
func square(num_ch chan int){
    value := <-num_ch
    sqr_val := value*value
    time.Sleep(1*time.Second)
    num_ch<- sqr_val
}
func cube(num_ch chan int){
    value := <-num_ch
    cube_val := value*value*value
    time.Sleep(1*time.Second)
    num_ch<- cube_val
}
func main(){
    sqr_ch := make(chan int)
    go square(sqr_ch)
    sqr_ch<- 3
    cube_ch := make(chan int)
    go cube(cube_ch)
    cube_ch<- 5
select{
    case sqr_val := <-sqr_ch:
        fmt.Println("square of a value is:", sqr_val)
    case cube_val := <-cube_ch:
        fmt.Println("Cube of a value is:", cube_val)
}
}

```

Output:

Cube of a value is: 125

Or square of a value is: 9

Here, the sleep time for both the Goroutines is the same (one second). So, both cases would be ready to execute at the same time. In such a case, the **select** statement randomly picks the case and executes it. The output of this program varies on executing it again and again.

Now, it could be possible that the **select** statement did not find any case statement to execute.

Example:

```
package main

import (
    "fmt"
    "time"
)

func square(num_ch chan int){
    _ = <-num_ch
    time.Sleep(1*time.Second)
}

func cube(num_ch chan int){
    _ = <-num_ch
    time.Sleep(1*time.Second)
}

func main(){
    sqr_ch := make(chan int)
    go square(sqr_ch)
    sqr_ch<- 3
    cube_ch := make(chan int)
    go cube(cube_ch)
    cube_ch<- 5
```

```

select{
    case sqr_val := <-sqr_ch:
        fmt.Println("square of a value is:", sqr_val)
    case cube_val := <-cube_ch:
        fmt.Println("Cube of a value is:", cube_val)
}
}

```

Output:

Fatal error: all goroutines are asleep – deadlock!

goroutine 1 [select]

In this case, we send the value to channels that are received by Goroutines. Now, in both case statements, we are trying to receive the value from a channel, but the channel does not have any value. As the **select** statement does not find any ready **case** statement here, it blocks the **select** statement and also leads to a deadlock.

To avoid a deadlock, we can add the default statement in **select** that proceeds the execution if **select** does not find any ready **case** statement.

Example:

```

package main

import ("fmt"
    "time"
)

func square(num_ch chan int){
    _ = <-num_ch
    time.Sleep(1*time.Second)
}

func cube(num_ch chan int){
    _ = <-num_ch
    time.Sleep(1*time.Second)
}

```

```

func main(){
    sqr_ch := make(chan int)
    go square(sqr_ch)
    sqr_ch<- 3
    cube_ch := make(chan int)
    go cube(cube_ch)
    cube_ch<- 5
    select{
        case sqr_val := <-sqr_ch:
            fmt.Println("square of a value is:", sqr_val)
        case cube_val := <-cube_ch:
            fmt.Println("Cube of a value is:", cube_val)
        default:
            fmt.Println("No case statement is ready to execute!")
    }
}

```

Output:

No case statement is ready to execute!

Here is an example where the **select** statement does not have any case for channel communication.

Example:

```

func main(){
    select{
    }
}

```

Output:

Fatal error: all goroutines are asleep – deadlock!

goroutine 1 [select <no cases>]

With this blank **select** statement, the compiler will give a fatal error and

lead to a deadlock.

We saw that a maximum of one case is executed with a **select** statement at a time. We can execute multiple cases by repeatedly checking if a case is ready to execute.

Example:

```
package main

import ("fmt"
"time"
)

func square(num_ch chan int){
    value := <-num_ch
    sqr_val := value*value
    time.Sleep(5*time.Second)
    num_ch<- sqr_val
}

func cube(num_ch chan int){
    value := <-num_ch
    cube_val := value*value*value time.Sleep(10*time.Second)
    num_ch<- cube_val
}

func main(){
    sqr_ch := make(chan int)
    go square(sqr_ch)
    sqr_ch<- 3
    cube_ch := make(chan int)
    go cube(cube_ch)
    cube_ch<- 5
    for itr := 1; itr<= 2; itr++){
        select{
```

```

        case sqr_val := <-sqr_ch:
            fmt.Println("square of a value is:", sqr_val)
        case cube_val := <-cube_ch:
            fmt.Println("Cube of a value is:", cube_val)
    }
}

```

Output:

square of a value is: 9

Cube of a value is: 125

Here, we are executing the **select** statement two times as we have two cases in the **select** statement. Since the square Goroutine has less sleep time, it will finish its execution first, and case1 would be ready to execute. So, a select statement first executes **case1**. The second time, it finds **case2** that is ready to execute. In this way, the **select** statement waits for the Goroutine to finish its execution and proceeds with the specified case.

Golang provides the facility to build concurrent programs that sometimes lead to unexpected behavior. Concurrency comes with multiple bugs that are quite hard to debug. The following defines some possible unexpected situations and how to detect and solve them.

Data race in Go

Data race can be defined by a situation where two or multiple threads concurrently access the same memory location and perform at least one write operation. The following example defines data race in the Go language:

Example:

```

package main

import "fmt"

var wg sync.WaitGroup

var value = 45

```

```

func update_value(){
    if(value==45){
        value++
    }
    fmt.Println(value)
    wg.Done()
}

func main(){
    wg.Add(2)
    go update_value() //gr1
    go update_value() //gr2
    wg.Wait()
}

```

In the preceding example, we have two Goroutines (**gr1** and **gr2**), and both are accessing the same variable value to update its value. There could be three cases with this program. It depends on the order of execution of the Goroutines **gr1** and **gr2**.

Case1: **gr1** first reads the data and then updates the value according to a specific condition. After the execution of **gr1**, **gr2** reads the data, checks for the conditions, and updates the data. No concurrent process is running here; the program will provide an accurate result. Here, the order of the Goroutines is synchronized is as follows:

Read_gr1	Write_gr1	Read_gr2	Write_gr2
----------	-----------	----------	-----------

Case2: Here, **gr1** first reads the data and then **gr2** reads the data. So, both the Goroutines will read the value 45 in this case. After that **gr1** and **gr2** write a value as the condition is satisfied with both the Goroutines. Here, the execution goes wrong and gets an unexpected result. The execution will proceed in the following order:

Read_gr1	Read_gr2	Write_gr1	Write_gr2
----------	----------	-----------	-----------

Case3: Here, **gr1** first reads the data, and then **gr2** reads the data. So, both the Goroutines will read the value 45 in this case. After that, **gr2** and **gr1** write a value as the condition gets satisfied with both the Goroutines. Here, the execution goes wrong and there is an unexpected result. The sequence of execution is outlined as follows:

Read_gr1	Read_gr2	Write_gr2	Write_gr1
----------	----------	-----------	-----------

Race condition

A race condition is a feature of a program or system that can lead to an unexpected result. A race condition occurs when two or multiple operations are performed at the same time in a program or system. Due to incorrect timing or ordering of operation, it leads to the crash of a program. A race condition is different from a data race. It is not necessary that if a program has a race condition, it must be because of a data race. A program can have a race condition but not a data race.

The following is an example where a data race does not exist, but it defines a race condition.

For example:

Thread 1 Thread2

lock(1) lock(1)

value = 1 value = 2

unlock(1) unlock(1)

In this example, we used a lock before updating the value of a variable to synchronize the process. There would never be any concurrent process, so data race doesn't exist here.

We can get race conditions with such examples. As we do not know the order of execution for these threads, it is hard to determine which process updates the value of a variable. The final value of a variable can be 1 or 2.

Check for data race

While working with multi-threading, it becomes necessary to protect a

program from abnormal behavior to make the program reliable and scalable. Go language provides a “data race detector” tool that tracks all the operations related to static/dynamic memory access and how and when the memory is accessed in a program. Dynamic tracking is done for unsynchronized access to shared variables. This tool looks for a race condition. If a program has a race condition because of a data race, it prints a warning message. The Go race detector tool was introduced with the v1.1 version of Go, integrated with the Go toolchain. This tool supports Linux/amd64, Linux/ppc64le, linuxarm64, FreeBSD/amd64, NetBSD/amd64, Darwin/amd64, and windows/amd64.

To enable the race detector in a program, **-race** flag is raised while executing a program. We can raise the **race** flag in the following ways:

- **go test -race package_name**
- **go run -race program_name**
- **go build -race command_name**
- **go install -race package_name**

It is advisable to use a race detector only if it necessary, as it consumes ten times the CPU and memory. Also, we can say that with concurrency, data sharing between Goroutines must be properly synchronized to avoid such erroneous conditions.

Mutex

In the Go language, **Mutex** provides a mechanism to synchronize the data access between multiple Goroutines. Mutex follows the concept of a mutual exclusion lock that allows only one Goroutine to access the memory at a time by the locking and unlocking mechanism.

Till now, we have used channels for synchronized communication between Goroutines. Although channels are also good as they have the built-in property of thread safety and prevent a race condition, it is advisable not to use channels in every situation. With a large application, a channel allows single-thread access to memory, but the performance penalty occurs with channels if fewer resources are shared between them. Channels and Mutex both have their pros and cons depending on the requirement and the way of programming.

Here, we are going to use the concept of **mutual exclusion lock**, provided by a data structure Mutex. Mutex is defined in the **sync** package which is part of the standard library. It has an interface **Locker** that encapsulates the functionality of locking and unlocking by declaring two methods, **Lock** and **Unlock**. These two methods are implemented by the **mutex** struct type. Mutex defines these two methods to lock and unlock a block of code that is executed by only one Goroutine at a time. Data access is synchronized with Mutex by keeping a lock when a shared resource is being used so that no other Goroutine can use it. After using that shared resource, the lock is removed. It prevents the race condition between multiple Goroutines if they are accessing the same memory space at a time.

Here is an example that shows how Mutex locks and unlocks a block of code and enables synchronized data access.

Example:

```
package main

import ("fmt"
"sync"
)

type initial_amount struct{
    amount int
    sync.Mutex
}

func (ia *initial_amount) withdraw_amount(withdraw_val int){
    ia.Lock()
    ia.amount = ia.amount - withdraw_val
    defer ia.Unlock()
}

func (ia *initial_amount) deposit_amount(deposit_val int){
    ia.Lock()
    ia.amount = ia.amount + deposit_val
    defer ia.Unlock()
}
```

```

}

func (ia *initial_amount) get_balance() int {
    val :=ia.amount
    return val
}

func main(){
    ia := initial_amount{amount: 2000}
    for itr:= 1; itr<=10; itr++){
        go ia.deposit_amount(500)
        go ia.withdraw_amount(300)
    }

    final_amount := ia.get_balance()

    fmt.Println("Final balance is:",final_amount)
}

```

Output:

Final balance is: 3900

In the preceding program, we create a struct **initial_amount** that has two fields, **amount** and **Mutex**. The amount variable is shared between two Goroutines, **withdraw_amount** and **deposit_amount**. Here, **Mutex** is defined as a struct field that locks and unlocks the access to amount variable. **withdraw_amount** withdraws an amount and updates amount variable. Similarly, **deposit_amount** deposits an amount as specified and updates the same amount variable. Both Goroutines are called from the main Goroutine; they will run concurrently. In the **deposit_amount** Goroutine, before updating the value of the “**amount**” variable, we use the lock method so that any other Goroutine will not be able to access the amount variable at the same time. When **deposit_amount** updates the value, it unlocks the resources to make them available for other Goroutines. In such a way, the access to the amount variable is synchronized.

When we run the preceding program with the race detector, it prints the output message with a **data race** warning that indicates the data race exists

in the program. In the **get_balance** method, we are accessing the same shared variable **amount** to read its value. While accessing this variable, the lock is being acquired either by **deposit_amount** or **withdraw_amount** method. We can prevent this warning by acquiring a lock before reading the value in **get_balance** method.

```
func (ia *initial_amount) get_balance() int {  
    ia.Lock()  
    val := ia.amount  
    ia.Unlock()  
    return val  
}
```

Now, run the program with the same race detector tool.

Output:

Final balance is: 4300

This time, it is quite different as it waits for some time to acquire a lock for the shared variable.

Also, it did not get any data race warning here. Now the question is of not getting the correct result.

The **get_balance** method is being called before the termination of all Goroutines, so it returns the intermediate value of a variable. We can resolve this issue by using **WaitGroup** as shown in the following example:

Example:

```
package main  
  
import ("fmt"  
    "sync"  
)  
  
var wg sync.WaitGroup  
  
type initial_amount struct{  
    amount int  
    sync.Mutex  
}  
  
func (ia *initial_amount) withdraw_amount(withdraw_val int){
```

```

    ia.Lock()

    ia.amount = ia.amount - withdraw_val

    ia.Unlock()

    wg.Done()
}

func (ia *initial_amount) deposit_amount(deposit_val int){

    ia.Lock()

    ia.amount = ia.amount + deposit_val

    ia.Unlock()

    wg.Done()
}

func (ia *initial_amount) get_balance() int {

    ia.Lock()

    val := ia.amount ia.Unlock()

    return val
}

func main(){

    ia := initial_amount{amount: 2000}

    for itr:= 1; itr<=10; itr++){

        wg.Add(1)

        go ia.deposit_amount(500)

        wg.Add(1)

        go ia.withdraw_amount(300)

    }

    wg.Wait()

    final_amount := ia.get_balance()

    fmt.Println("final balance is:", final_amount)
}

```

Output:

final balance is: 4000

RWMutex

Any thread either reads the data or modifies it. On performing a write operation, data sharing is not allowed with multiple Goroutines, whereas with a read operation, multiple Goroutines can read the data from the same memory space without any issue. Based on several operations, Golang provides a specific reader/writer Mutex that multiple readers and writers can behold. Like Mutex, **RWMutex** is also available in a sync package. RWMutex is a struct that contains various fields to track the memory access by multiple threads. RWMutex implements multiple methods, like the following, that provide a specific type of lock/unlocking service:

- **RLock():** It is used to acquire a lock rw for reading purposes. Multiple readers can acquire this lock in a single program. RLock is not bound with a specific thread.
- **RUnlock():** This method releases the acquired lock for a single RLock call that does not affect another running read operation. In the Go language, one thread may RLock another thread, and it can release that.
- **Lock():** It locks rw for writing purposes. This type of lock is acquired by only one thread at a time. If another thread acquires a lock for the reading/writing purpose, Lock blocks until the lock is available to acquire.
- **Unlock():** This method unblocks rw to make it available for another writer thread. On executing Unlock without acquiring the lock, the compiler will give an error at runtime.

There is a point to remember, which is that we should avoid contain Mutex and RWMutex both as struct fields in a single program. Doing so may lead to the compiler error “ambiguous selector.” The following example defines thread-safe maps with the use of RWMutex.

Example:

```
package main  
  
import ("fmt"
```

```

"sync"
"time"
)
type Map_struct struct{
    map_val map[int] int
    sync.RWMutex
}

func (map_data *Map_struct) update_map(keyval int){
    map_data.Lock()
    map_data.map_val[keyval] = keyval * 10
    map_data.Unlock()
}

func (map_data *Map_struct) read_map(){
    map_data.RLock()
    val := map_data.map_val
    map_data.RUnlock()
    fmt.Println(val)
}

func main(){
    ia := Map_struct{map_val:make(map[int]int)}
    for itr:= 1; itr<10; itr++){
        go ia.update_map(itr)
        go ia.read_map()
    }
    time.Sleep(5 * time.Second)
}

```

Output:

map[1:10 2:20]

map[1:10 2:20]

map[1:10 2:20 3:30]

map[1:10 2:20 3:30 4:40]

map[1:10 2:20 3:30 4:40 5:50]

map[1:10 2:20 3:30 4:40 5:50 6:60]

map[1:10 2:20 3:30 4:40 5:50 6:60 7:70]

map[1:10 2:20 3:30 4:40 5:50 6:60 7:70 8:80]

map[1:10 2:20 3:30 4:40 5:50 6:60 7:70 8:80 9:90]

In the preceding program, we create a struct **Map_struct** that has two fields, **map_val** and **RWMutex** object. Embedding of **RWMutex** object allows accessing multiple methods for **map_val**. In the main Goroutine, we initialize the struct and call Goroutines in a loop. Here, we have two Goroutines. **read_map** reads the map values and **update_map** modifies the map. **read_map** Goroutine puts RLock to perform read operation, and after reading a map, it releases the lock using the **RUnlock**. Goroutine **update_map** acquires the lock for a write operation and after updating the map, it releases the lock using the **Unlock** method. We give time sleep for 5 seconds at the end of the main Goroutine so that all the running Goroutines can finish their execution till that time.

Conclusion

In this chapter, we got an overview of the channels in Go. We have learned how send/receive operations are performed by using channels in Golang. To avoid some restrictions, Go provides buffered and unbuffered channels that can be used as required. We have grasped here the idea of deadlock, channel direction, select keyword, and others. We have understood the lock/unlock mechanism with Mutex in Go. After getting a deeper understanding of Go, we know that there are multiple cases where we need to handle the errors that we get in the Golang program. Let us see in the next chapter how to handle errors in Go.

Questions

1. What are the channels used in Go?

2. What are the various ways to declare a channel in Go?
3. How do channels work in Golang?
4. Explain the use of select keyword with an example.
5. What do you understand by the capacity of a channel?
6. What is the difference between buffered and unbuffered channels?
7. How can we create unidirectional channels?
8. What is a deadlock in Golang?
9. How can we close a channel in Go?
10. What do you understand by Mutex in Go?

Join our book's Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

<https://discord.bpbonline.com>



CHAPTER 18

Error Handling

Introduction

In this chapter, we learn about error handling as a process of detecting and handling an erroneous situation that can lead to the crash of a program. We learn about various error types and packages. Compile-time errors occur with the compilation of the program. Errors that occur at the execution of the program are called runtime errors. Opening a file that does not exist in a directory is the best example of a runtime error. Error handling detects such runtime errors and represents and handles them as specified.

Structure

In this chapter, we will discuss the following topics:

- Introduction to Golang errors
- Error types
- Errors package
- Defer statement
- Panic
- Recovery

Objectives

After studying this chapter, you will be able to identify abnormal conditions in your code and manage potential errors using the **error** data type. You will learn effective techniques to tackle errors in Go and gain expertise in performing error wrapping, unwrapping, and formatting using the **errors** package. These skills will help you handle errors gracefully and write more robust and maintainable Go programs.

Introduction to Golang errors

The Go language has a built-in **error** type to indicate an abnormal condition. The error type is used like other data types—store them in memory, pass them to any function, and many others. In other languages, we use try-catch or try-except for error handling, but there is nothing like that here, as the Go language does not have exceptions. In Go, errors are handled either by multiple return values or by panic. The following is an example of trying to open a file that does not exist, which gives a runtime error.

We can open a file in the Go language with the following syntax:

```
func Open(name string) (file *File, err error)
```

Example:

```
package main

import (
    "fmt"
    "os"
)

func main(){
    file, err := os.Open("./example.json")

    fmt.Println("file status:", file)

    fmt.Println("error status:", err)
}
```

Output:

file status: <nil>

error status: open ./example.json: The system cannot find the file specified.

In the preceding example, since the file does not exist in the current directory, it returns **nil** instead of the file handler and an error message that occurs while opening a file. Let us create a file named **example.json** in the current directory and run the program again. In this case, it finds the file and returns the file handler and **nil** to **err**. Here is the updated output after creating a specified file:

```
file status: &{0x1185c420}
```

```
error status: <nil>
```

To handle an error, we can compare the last return value **err** to **nil**. If the value of **err** is not **nil**, there is an error. If **err** is **nil**, the operation has been done successfully.

Error types

error is an interface type that contains a single method **Error** of return type string.

The default value of the error type is **nil**. The error type is defined as shown:

```
type error interface{
    Error() string
}
```

Any type that implements the error interface is considered as an error.

Example:

```
package main

import "fmt"

type My_error struct{}

func (error_object *My_error) Error() string{
    return "Unexpected outcome"
}

func main(){
    fmt.Println(&My_error{})
}
```

```
}
```

Output:

Unexpected outcome

In the preceding program, we create a **struct My_error** that has no field. **Struct My_error** is used to implement the **Error** method that returns an error message as a string. We can call an **Error** method directly in case of any error to find its output.

Example:

```
package main

import "fmt"

type My_error struct{}

func (error_object *My_error) Error() string{
return "Unexpected outcome"
}

func main(){
    obj := My_error{}
    fmt.Println(obj.Error()) // Unexpected outcome
}
```

Error package

Go has a built-in package called **errors** in the **standard library**. It implements various functions to manipulate errors. It has a function called **New** that is used to return an error of a specified format.

Example:

```
package main

import (
    "fmt"
    "os"
    "errors"
)
```

```

func main(){
    file, err := os.Open("./example.json")
    if(err!=nil){
        error_msg := errors.New("file operation failed") fmt.Println(error_msg)
    }
    if(err==nil){
        fmt.Println("file opened successfully", file)
    }
}

```

Output:

file operation failed

In the preceding program, in the case of an error, we called the **New** function by passing an error message. This function accepts the error message as a string and returns it as an error type value. Here, the type of **error_msg** is ***errors. errorString**, which points to **errors.errorString**. **errorString** is a struct type that has a string type of a field in the errors package. The struct type **errorString** implements the **Error** method and returns a string type of value. For a better understanding, you must have a look at the errors package in the **src** directory.

Defer statement

Execution of a specific code can be deferred until the surrounding function returns by using the **defer** statement. An expression in the **defer** statement must be a function call. The arguments passed in the **defer** statement are evaluated once the compiler finds a **defer** keyword, but the execution of the **defer** statement takes place at the end when the function returns. Multiple clean-up operations are performed in the **defer** statement, such as closing the files if any file is open.

The **defer** statement works in the following ways:

- The **defer** statement pushes a function into the stack that is executed in last in the first out order. When the surrounding function returns, all functions are called that are stored in the stack. The function that is

stored at the end in the stack is executed first.

The values passed in the **defer** function arguments are evaluated as the compiler finds the defer statement.

- In **defer** functions, both the read and write operations can be performed.
- Return values can be modified in defer functions.
- Defer statement ensures deferred functions run even in case of an error or panic, aiding in resource cleanup.

Example:

```
package main

import ("fmt"
"os"
)

func open_file(){
    defer file.Close()

    file, err := os.Open("./example.txt")

    fmt.Println("file status:", file)

    fmt.Println("error status:", err)
}

func main(){
    open_file()
}
```

Output:

Error: undefined: file

The program has a function called **open_file**. When the compiler starts the execution of **open_file**, it gets a **defer** statement. While evaluating the values, it does not get the file object, so it gives an error. We can resolve this issue which is shown as follows:

Example:

```
package main
```

```

import (
    "fmt"
    "os"
)

func open_file(){
    file, err := os.Open("./example.txt")
    defer file.Close()
    fmt.Println("file status:", file)
    fmt.Println("error status:", err)
}

func main(){
    open_file()
}

```

Output:

file status: &{0x1186c420}

error status: <nil>

The following is an example that explains that defer function calls are executed in the **Last In, First Out (LIFO)** order. We can also execute the defer statement in a loop.

Example:

```

package main

import "fmt"

func show_value(){
    for itr:=0; itr<=10; itr++){
        defer fmt.Println("value:", itr)
    }
}

func main(){
    show_value()
}

```



```
}
```

Output:

value: 10

value: 9

value: 8

value: 7

value: 6

value: 5

value: 4

value: 3

value: 2

value: 1

value: 0

Here is another example in which defer is used to update the return value:

Example:

```
package main
```

```
import "fmt"
```

```
func update_return_value() (i int) {
```

```
    defer func() {
```

```
        i = i+10
```

```
    }()
```

```
    return 1
```

```
}
```

```
func main(){
```

```
    fmt.Println("updated return value:", update_return_value())
```

```
}
```

Output:

updated return value: 11

Panic

Panic is another way of error handling. It is used to check for unexpected errors throughout the program. It is somehow similar to exception as it also occurs during runtime. It skips the execution of a function if it returns an error that shows unexpected behavior. Whenever a **Panic** function is called in any function, it stops the execution of that function, executes the defer statement which is given to handle that panic, returns to the caller statement, and continues with further execution.

In Go, a runtime error also causes panic in some cases, like in out-of-bound accesses, divide by zero, and many others.

Syntax:

```
Panic(interface {})
```

Panic accepts any value as an argument because the empty interface can accept any type of value.

Example:

```
package main

import "fmt"

var result int

func op(a, b int) int {
    result = a / b
    return result
}

func main(){
    result = op(10, 0) fmt.Println("Completed")
}
```

Output:

panic: runtime error: integer divide by zero

On executing the preceding example, it gives a panic that is caused by a runtime error on dividing a number by zero. When panic occurs, the program gets terminated immediately. We can call a **panic** function on a specific case when a program is showing unexpected behavior and set an

error message as required.

Example:

```
package main

import "fmt"

var result int

func op(a, b int) int {
    if(b==0){
        panic("Can't divide a number by zero!")
    }
    result = a/b
    return result
}

func main(){
    result = op(10, 0) fmt.Println("Completed")
}
```

Output:

panic: Can't divide a number by zero!

Here, everything remains the same, but it shows an error message which is what we required. In the same way, panic can be initiated by calling a **panic** function with any possible erroneous condition or a situation that can show unexpected behavior.

Example:

```
package main

import "fmt"

func show_value(max int){
    for itr:=1; itr<=max; itr++){
        if(itr==5){
            panic("Not acceptable")
        }
    }
}
```

```

        fmt.Println(itr)
    }
}

func main(){
    show_value(10)
    fmt.Println("Completed")
}

```

Output:

```

1
2
3
4

```

panic: Not acceptable

In the preceding program, we do not want to show values more than 4. If a value greater than 4 is passed in a parameter, it prints the values up to 4; and when it comes to 5, it shows panic and stops further execution. In some cases, we might execute some clean-up or other necessary operation, before the program terminates. The same can be done by using the defer function that executes after getting the panicking situation.

Example:

```

package main

import "fmt"

var result int

func op_defer(){
    fmt.Println("op defer function executed")
}

func main_defer(){
    fmt.Println("main defer function executed")
}

func op(a, b int) int {

```

```

defer op_defer()
if(b==0){
    panic("Can't divide a number by zero!")
}
result = a/b return result
}

func main(){
defer main_defer()
result = op(10, 0)
fmt.Println("Completed")
}

```

Output:

```

op defer function executed main defer function executed
panic: Can't divide a number by zero!

```

Recovery

Go provides a built-in function called **recover** that controls the panicking situation. In the Go language, errors handling is done by using the **recover** function like try-catch, try-except, or other similar methods are used in other languages. In the previous example, we saw that the execution of a program terminates when a panic occurs. The **recover** function is used inside the defer functions because only defer statements are executed after panicking. The value passed to panic is returned by the **recover** function. If there is no panic in the program, the **recover** function returns nil.

Example:

```

package main

import "fmt"

var result int

func op_defer(){
    fmt.Println("op defer function executed")
}

```

```

}

func main_defer(){
fmt.Println("main defer function executed")
}

func op(a, b int) int {
defer func() {
if r := recover(); r != nil {
fmt.Println("Panic recovered", r)
op_defer()
}
}()
if(b==0){
panic("Can't divide a number by zero!")
}
result = a/b return result
}

func main(){
defer main_defer()
result = op(10, 0)
fmt.Println("Completed")
}

```

Output:

Panic recovered Can't divide a number by zero! op defer function executed

Completed

main defer function executed

In the preceding program, we called the “op” method from the main Goroutine. When it gets panicked, the program should terminate. However, before the termination, it executes the list of defer calls. While executing the defer function, it gets the **recover** function that takes control of the panic and returns the non-nil value passed in the panic. Here, the recover

function resumes the execution of the program and after executing **op_defer** function, it comes back to **function caller statement** in main and continues the further execution. In the end, the defer function of the main Goroutine is called, and thus, the program executes smoothly.

Conclusion

It can be concluded that we can recognize the errors by multiple return values of error type in Go. Golang provides **errors** packages that facilitate various functions to manipulate and analyze the errors in code. All the operations that need to be performed at the end can be done by the **defer** statement. The built-in function, **panic** controls the flow of the program and starts panicking. This panic situation is controlled by the **recover** function. Let us discuss another feature of Go called reflection in the next chapter.

Questions

1. Explain error handling in Go with an example.
2. What is the purpose of the defer statement?
3. What is the use of the “New” function for error handling?
4. In what order are the defer statements executed in Go?
5. What is the role of panic in error handling?
6. How do you catch panic with the recover function?

CHAPTER 19

Reflection

Introduction

Reflection is a key feature of a programming language that can be defined as examining the program execution and modifying it at runtime. It is particularly important in object-oriented and dynamic programming, allowing developers to write flexible and adaptive code. In this chapter, we will delve into the concept of reflection in Go.

Structure

In this chapter, we will discuss the following topics:

- Introduction to reflection
- Reflect package
- Type
- Value
- Kind
- Numfield()
- Field
- String(), Int(), and Float()
- Laws of reflection

Objectives

After studying this chapter, you will be able to inspect the type of a variable at runtime, create and modify variables and functions dynamically, and analyze struct fields in Go. These skills will enable you to leverage Go's reflection capabilities for advanced programming tasks.

Introduction to reflection

Reflection allows the exploration of any data at runtime without knowing its name defined at compile time. By using reflection, we can inspect the behavior of any method, interface, struct, or any data. Inspecting the source code at runtime is called reflection.

By using reflection, we can get the type, field, and the value of any type. Although in the Go language, every variable is declared with its type but still, there could be many cases where we may need to know the type of a variable at runtime. Here, reflection comes in.

Example:

```
func main(){
    slice_val := []string {"one", "two", "three", "four", "five", "six"}
    map_val := map [string] int64{}
    for itr:=0; itr<len(slice_val); itr++){
        map_val[slice_val[itr]] = 0
    }
}
```

In the preceding example, we initialize a slice of string type. We need to use the items of the slice like a key of a map. Here, we have declared a map that has the string type of keys. What if we need to initialize that map at runtime or a slice is of any type? In that case, we would need to find the type of slice at runtime. For that, we could use reflection.

The reflect package should be used for generic programming or frameworks, but avoided in performance-critical code or where type safety is essential. It introduces performance overhead, reduces type safety, and

makes code harder to maintain.

Reflect package

Go provides a built-in package called **reflect** that provides the functionality of reflection at runtime. We can get the **reflect** package in standard library. The **reflect** package contains a few variables and methods to manipulate the object of various types. It is used to return the dynamic type of a variable that is static or interface type. As an empty interface can accept any value, we can get the exact type of value that is accepted by an interface with the use of the **reflect** package. The **reflect** package is mainly used to get the type, value, and kind of variable.

Type

The **reflect** package provides the method **TypeOf** to get the type of a variable. This function returns the value of the type **reflect.Type**. The type has various methods to define the property of that type. The signature of this function is as follows:

```
func TypeOf(i interface{}) Type
```

This function takes an interface as an argument. Any value is accepted by an interface and then analyzed by the **TypeOf** function.

Syntax:

```
variable_type := reflect.TypeOf(variable)
```

Example:

```
package main

import (
    "fmt"
    "reflect"
)

func main(){
    type student struct{
        name string
```

```

        age int
    }

    stu := student{"Akansha", 17}

    fmt.Println(stu)

    stu_type := reflect.TypeOf(stu)

    fmt.Println("type of 'stu' variable:", stu_type)

    fmt.Println("type of 'stu_type' variable:", reflect.TypeOf(stu_
type))

    fmt.Println("name of 'stu' variable type:", stu_type.Name())
}

```

Output:

{Akansha 17}

type of 'stu' variable: main.student

type of 'stu_type' variable: *reflect.rtype name of 'stu' variable type: student

Value

The reflect package has a method to evaluate the value of a variable.

Syntax:

value := reflect.ValueOf(variable)

Example:

```

package main

import (
    "fmt"
    "reflect"
)

func main(){
    type student struct{
        name string
        age int
    }
}

```

```

}
stu := student{"Akansha", 17}
fmt.Println(stu)
stu_type := reflect.TypeOf(stu)
fmt.Println("value of 'stu' variable:", reflect.ValueOf(stu))
fmt.Println("value of 'stu_type' variable:", reflect.ValueOf(stu_
type))
}

```

Output:

```
{Akansha 17}
```

```
value of 'stu' variable: {Akansha 17} value of 'stu_type' variable: main.student
```

Kind

Kind is quite a different method of the **reflect** package. The type defines the type of a variable, while “kind” defines the source type of a variable type. This function is used with a type to get its source, not with variables.

Example:

```

package main

import (
    "fmt"
    "reflect"
)

func main(){
    type student struct{
        name string
        age int
    }
    stu := student{"Akansha", 17}
    fmt.Println(stu)
}

```

```

stu_type := reflect.TypeOf(stu)
stu_kind := stu_type.Kind()
fmt.Println("type of 'stu' variable:", stu_type.Name())
fmt.Println("kind of 'stu_type' variable:", stu_kind)
}

```

Output:

```
{Akansha 17}
```

type of 'stu' variable: student kind of 'stu_type' variable: struct

Here, we can see that student is a type of struct. So, the source type of the **student** type is a struct.

NumField()

This method is used to get the number of fields contained by a struct. As this function is defined in the reflect package, it only works with type **reflect.Value** or **reflect**.

Example:

```

package main

import (
    "fmt"
    "reflect"
)

func main(){
    type student struct{
        name string
        age int
    }

    stu := student{"Akansha", 17}

    stu_value := reflect.ValueOf(stu) num_fields := stu_value.NumField()

    fmt.Println("number of fields in 'stu':", num_fields, reflect.TypeOf(stu_value))
}

```

```

    stu_type := reflect.TypeOf(stu)

    fmt.Println("number of fields in 'stu':", num_fields, reflect.TypeOf(stu_type))
}

```

Output:

number of fields in 'stu': 2 reflect.Value number of fields in 'stu': 2 *reflect.rtype

Field

This function is used to get the value of struct fields. It returns the value of type **reflect.Value**.

Example:

```

package main

import ("fmt"
    "reflect"
)

func main(){
    type student struct{
        name string
        age int
    }

    stu := student{"Akansha", 17}

    stu_value := reflect.ValueOf(stu)

    num_fields := stu_value.NumField()

    for itr:=0; itr<num_fields; itr++){
        fmt.Printf("value of %dth field is:%v of type %T\n", itr+1, stu_value.Field(itr),
stu_value.Field(itr))
    }
}

```

Output:

value of 1th field is:Akansha of type reflect.Value value of 2th field is:17 of type reflect.Value

String(), Int() and Float()

In the **reflect** package, the value function returns the data of type **reflect.Value**. We can convert such values to string or any other type as required.

Example:

```
package main

import (
    "fmt"
    "reflect"
)

func main(){
    type student struct{
        name string
        age int
        income float32
    }

    stu := student{"Akansha", 17, 40000.565}
    stu_value := reflect.ValueOf(stu)

    fmt.Printf("Type of name is %T and value is %v\n", stu_value.Field(0), stu_value.Field(0))
    fmt.Printf("Type of age is %T and value is %v\n", stu_value.Field(1), stu_value.Field(1))
    fmt.Printf("Type of income is %T and value is %v\n", stu_value.Field(2), stu_value.Field(2))

    name := reflect.ValueOf(stu.name).String()
    age := reflect.ValueOf(stu.age).Int()
    income := reflect.ValueOf(stu.income).Float()

    fmt.Printf("Type of name is %T and value is %v\n", name, name)
    fmt.Printf("Type of age is %T and value is %v\n", age, age)
    fmt.Printf("Type of income is %T and value is %v\n", income, income)
}
```

Output:

Type of name is reflect.Value and value is Akansha Type of age is reflect.Value and value is 17

Type of income is reflect.Value and value is 40000.566 Type of name is string and value is Akansha

Type of age is int64 and value is 17

Type of income is float64 and value is 40000.56640625

Laws of reflection

Go defines three laws of reflection that define how reflection works and the points that should be remembered while working with reflection. The three laws of reflection are as follows:

Reflection goes from interface value to reflection object: Since an empty interface accepts any value, it makes it quite hard to understand what type of value is stored by an interface. Here, reflection works to get the type and value contained by an interface. The type and value of a variable are evaluated by using the functions **TypeOf** and **ValueOf**, as we have already seen in the previous example. The first law of reflection is all about extracting a reflection object from an interface.

Example:

```
package main

import (
    "fmt"
    "reflect"
)

func show_value(i interface{}){
    value_type := reflect.TypeOf(i)
    value := reflect.ValueOf(i)
    fmt.Printf("Passed value is %v of type %v\n", value, value_type.Kind())
    fmt.Println(reflect.TypeOf(value)) fmt.Println(reflect.TypeOf(value_type))
}
```



```

func main(){
    type student struct{
        name string
        age int
        income float32
    }
    stu := student{"Akansha", 17, 40000.565}
    show_value(stu)
}

```

Output:

passed value is {Akansha 17 40000.566} of type struct

reflect.Value

***reflect.rtype**

Reflection goes from reflection object to interface value: The second law is the inverse of the first law. The second law says the type **reflect.Value** and **reflect.Type** can be converted to interface value. The following is an example of reflection's second law.

Example:

```

package main

import (
    "fmt"
    "reflect"
)

func show_value(i interface{}){
    value_type := reflect.TypeOf(i)
    value := reflect.ValueOf(i)
    fmt.Println(reflect.TypeOf(value))
    fmt.Println(reflect.TypeOf(value_type))
    interface_val := value.Interface()
    fmt.Println("Interface value is:", interface_val)
}

```

```

        fmt.Println("Interface value type is:", reflect.TypeOf(interface_val).Kind())
    }
}

func main(){
    type student struct{
        name string
        age int
        income float32
    }

    stu := student{"Akansha", 17, 40000.565}

    show_value(stu)
}

```

Output:

reflect.Value

***reflect.rtype**

Interface value is: {Akansha 17 40000.566} Interface value type is: struct

To modify a reflection object, the value must be settable: On getting the value of a variable with the use of the **ValueOf** function, it returns the value of type **reflect.Value**. The third law says that to assign a value to **reflect.Value** type variable, the value must be of **reflect.Value** type. If it is not of the same type, it will show panic.

Example:

```

package main

import (
    "fmt"
    "reflect"
)

func main(){
    var str string = "Hello"

    val := reflect.ValueOf(str) fmt.Println(val)
}

```

```
    val.SetString("World ") // will throw panic
}
```

Output:

panic: reflect: reflect.Value.SetString using unaddressable value.

In the preceding example, we cannot set the value for type **reflect.Value** as **World** is not addressable. To get rid of such issues, we can first check if a variable is settable or not by using the function **CanSet**.

Example:

```
package main

import (
    "fmt"
    "reflect"
)

func main(){
    var str string = "Hello"
    val := reflect.ValueOf(str)
    fmt.Println(val.CanSet())
}
```

Output:

Hello false

Conclusion

This chapter provided a comprehensive overview of reflection in Go, equipping with the knowledge of key concepts such as the reflect package and its fundamental components: Type, Value, and Kind. Practical functions like NumField(), Field(), and methods like String(), Int(), and Float() were also discussed. Additionally, the laws of reflection were covered to ensure a deeper understanding of the principles governing reflection in Go. In the next chapter, we will dive into web scraping in Go, exploring how to extract and process data from the web efficiently.

Questions

1. What is a reflection in Go?
2. How do you get the type of any variable?
3. What function is used to get several fields in a structure?
4. How can we change the type of reflection object?
5. What do you understand by “kind” property of reflection object?
6. How to get the value of the struct field in Go?
7. What are the laws of reflection in Go?

Join our book's Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

<https://discord.bpbonline.com>



CHAPTER 20

Web Scraping in Go

Introduction

The method of obtaining data from webpages is called web scraping. In order to extract usable information, it first accesses web pages and then parses the HTML or other structured data from those pages. This information may be gathered for a number of reasons, including study, analysis, data mining, or the creation of databases or websites.

Structure

In this chapter, we will discuss the following topics:

- Introduction to web scraping
- Web scraping libraries in Go
- Overview of Colly
- Scrap website in Go
- Headless scraping
- How to avoid blocking

Objectives

After studying this chapter, you will be equipped to collect data from websites by ensuring ethical scraping practices and protecting scraping

operations from blocking. You will gain an understanding of open-source libraries and frameworks specifically designed for web scraping. Additionally, this knowledge will enable you to generate potential leads for sales and marketing teams and efficiently monitor market trends.

Introduction to web scraping

Web scraping has become a need of today's business. Monitoring a rival's pricing, offerings, and customer feedback can help to remodel their strategies in order to stay competitive in the market. Scraping plays a key role in finding potential leads and other business-related data.

Go's excellent features, concurrency, and parallelism provides a suitable environment to create large-scale web scraping applications where vast amount of data needs to be scrapped. It becomes painless to create scraping algorithms that meet your specific needs.

Web scraping

Web scraping can be defined as an automated process to retrieve bulk data from web. This data could be structured or unstructured. Web scraping is a process but in this process, multiple different data extraction ways could be involved. It includes leveraging specific APIs, online services, or even writing your own code from scratch for web scraping.

A full web scraping procedure can be broken down into different stages, which are as follows:

- **Web request:** Sending a request to a particular URL is the first step in web scraping. The web server that hosts the website is contacted and asked for the page's content. To make a web request, the HTTP protocol is used, which is a standard protocol for data transmission on the web.
- **HTML parsing:** Since web pages are written in a markup language called HTML, we must parse the HTML after sending the request to the web sites. In simple words, HTML parsing is the process of reading raw HTML code and creating a DOM tree object structure from it.
- **Data extraction:** The needed data is subsequently found and extracted from the HTML by the scraper. Text, images, links, tables, and any other information on the website can be included in this data.

- **Data transformation:** Data transformation is converting raw data into something else so that it can be used for an analysis or a particular purpose. To fulfill the requirements of a certain activity or system, data transformation can comprise a number of processes to clean, enrich, reshape, or aggregate the data.
- **Storing data:** Once we collect all the required data, it needs to be stored in some CSV, Excel, Database, cloud, or warehouse for later use. The amount of data, its format (structured or unstructured), how you intend to use it, your infrastructure, and analysis needs, all play a role in the decision of how to store it. Data security and privacy must also be taken into account, particularly when working with sensitive or **Personally Identifiable Information (PII)**.
- **Automation:** To collect the updated data every time, all the preceding scraping steps are automated by using some programming languages. Automation makes it possible to scrape numerous pages from one or more websites in a systematic manner. You could, for instance, use loops or functions to scrape information from paginated results or click on links to acquire further information. We will understand the concept of automation in-depth in the next chapter.

Web Scraping libraries in Go

The ability to perform web scraping in Go depends on the libraries and packages available for web scraping. There are a number of Go libraries and tools that you may use for web scraping, even though it might not have as many as some other languages like Python.

The following are some prominent Go web scraping tools and libraries:

- **Colly:** Colly offers clean APIs to extract structured data from websites, which can then be utilized for a variety of purposes, including data mining, processing, and archiving. It controls request latency and the maximum concurrent users for each domain with the support of parallel, sync, and async scraping. The Colly package's import path is:

github.com/gocolly/colly

- **GoQuery:** GoQuery is a Go package that provides jQuery-like syntax and features for parsing and manipulating HTML documents. In

Golang, net/html package returns nodes. However, this package extracts and alters data from the HTML and XML material since it enables you to work with structured documents using techniques and selectors that are familiar from CSS. You can get GoQuery package here:

github.com/PuerkitoBio/goquery

- **Goose:** This Go package is used to extract article content, metadata, and other pertinent data from web pages, especially news items. It is a specialized tool made specifically for parsing and collecting structured data from articles, not a general-purpose web scraping library. You can install the goose package through the following command:

`go get github.com/advancedlogic/goose`

- **Pholcus:** Pholcus is a powerful, distributed web crawler with higher concurrency support, purely written in Go language. It offers three operation interfaces **Web, GUI, and command line** and three operating modes **stand-alone, server, and client**. It runs batch tasks concurrently, and generates a variety of output ways **mysql, mongodb, kafka, csv, excel, etc.**; Additionally, it supports number of sophisticated features like simulated login, task suspension, and task cancellation. Here is the git repository for this package:

<https://github.com/henrylee2cn/pholcus>

- **Chromed:** Chromedp library provides a way to control and interact with the Chrome browser by using the **Chrome DevTools Protocol (CDP)**. It can be used to scrape dynamic webpages with a lot of JavaScript content. With the use of the package, we get the flexibility to automate web interaction as click on button, typing, post some content and other variety of actions on web pages. Headless browsing, asynchronous operation, control and navigation, data extraction are some key features of this library. The following can be used to import **chromed** in your code:

github.com/chromedp/cdproto/cdp

github.com/chromedp/chromedp

Overview of Colly

Colly is a Golang framework that has been created by the Go community to simplify the process of web scraping. For creating web crawlers and scrapers in Go, Colly offers clear and expressive APIs. It takes care of several common tasks related to web crawling and data extraction and provides specific functions for the same. Following are the main functions and components of Colly:

- **Collector:** Collector is a core component of the Colly framework that specifies the rules of how to navigate the website and what action needs to be performed. The Collector uses goroutines to support concurrent scraping and can scrape multiple pages simultaneously.
- **HTTP requests:** You can send HTTP requests to websites using Colly. You can change the request headers, set cookies, and specify different options like timeouts and user agents.
- **HTML parsing:** Colly comes with a powerful HTML parser that makes it easier to extract data from the web. It allows you to locate and extract particular HTML components using XPath and CSS selectors.
- **Event – Driven model:** In Colly, Events are essential in determining how your web scraper interacts with and extracts data from web pages. You can modify the behavior of your scraper by adding handlers to the various events that Colly provides. A callback parameter is accepted by all Colly's functions that gets activated when the corresponding event is being called.

Here are some of the common events in Colly:

- **OnHTML:** When Colly comes across an HTML element that matches a given selector, this event handler is called. It enables you to extract data from the HTML element and process it.
- **OnXML:** Like OnHTML, this event is triggered when an XML element matches the specified selector.
- **OnHTMLDetach:** After the HTML selector has been matched and executed once, this event is used to separate the event handler from it. For scraping paginated content, it is helpful.
- **OnRequest:** Before performing an HTTP request with **Visit()**, this event is initiated. It allows you to change any request-related configuration, including cookies and request headers.

- **OnResponse:** When a response is received from a request, this event is started. It can be used to examine the HTTP response, look for errors, or take actions based on the response.
- **OnError:** During scraping process, when an error, parsing issue, network error or timeout occurs, this event is triggered. This is where error-handling logic can be defined.
- **OnScraped:** Once the web page has been scraped, this event is called, enabling you to carry out cleanup or must to do tasks.
- **Request queuing:** The request queue is a crucial idea in the Colly framework for Golang that enables you to manage and control the sequence in which requests are performed during web scraping. By default, requests are handled in the same order as they are added by using the **Visit** or **Request** method, and Colly handles the management of the request queue for you. However, Colly lets you define the priority of the URL. Then, the processing of requests with higher priority takes precedence over those with lower priority values.
- **Rate limiting and throttling:** Strategies like rate limiting and throttling are crucial for managing the speed and frequency of your requests to prevent overloading a website's server or getting blocked. Colly does not have a built-in feature for this but you can use an external Go library to implement rate limiting. One well-liked library for this is: **golang.org/x/time/rate**.
- **Middleware:** The request/response of web scraper can be enhanced with extra functionality using the sophisticated middleware architecture in Colly. Middleware can be used for a number of activities, including altering requests, processing responses, logging, and more.
- **Web UI:** You may visualize and manage your scraping jobs with Colly's optional web-based **user interface (UI)**. When monitoring and debugging jobs, the UI might be helpful.

Some highlights of Colly's features are:

- Fast speed, handles more than 1K requests/second per core.
- Sync/async/parallel scraping.
- Support for **robots.txt**.
- Manages request delays and maximum concurrency per domain.

- Automatic cookie and session handling.
- Support for Cache of visited URLs.
- Automatic encoding of non - unicode responses.
- UI support for monitoring and debugging.
- Configuration via environment variables.
- Support for middleware.

Scrap websites in Go

After getting enough understanding on Colly framework, let us create a web scraper in Go.

To create a project, you can open any Golang supported IDE and now create a project directory. Let us say project directory is **GoScraper**.

Within the project directory GoScraper, run the following command:

```
go mod init webScraper
```

The preceding command initializes the project as Go module where **webScraper** is the module name. Instead of this, you can provide GitHub or any other domain path if want to store the project there. It will create the module file with name **go.mod** that contains project's direct or indirect dependencies.

For web scraping, we are going to use the Colly framework. Let us get it install by running the following command:

```
go get github.com/gocolly/colly
```

After installing Colly, **go.mod** will look like this:

```
≡ go.mod  ×
GoScraper > ≡ go.mod
1  module webScraper
2
3  go 1.20
4
   Check for upgrades | Upgrade transitive dependencies | Upgrade direct dependencies
5  require (
6      github.com/PuerkitoBio/goquery v1.8.1 // indirect
7      github.com/andybalholm/cascadia v1.3.1 // indirect
8      github.com/antchfx/htmlquery v1.3.0 // indirect
9      github.com/antchfx/xmlquery v1.3.18 // indirect
10     github.com/antchfx/xpath v1.2.4 // indirect
11     github.com/gobwas/glob v0.2.3 // indirect
12     github.com/gocolly/colly v1.2.0 // indirect
13     github.com/golang/groupcache v0.0.0-20210331224755-41bb18bfe9da // indirect
14     github.com/golang/protobuf v1.5.2 // indirect
15     github.com/kennygrant/sanitize v1.2.4 // indirect
16     github.com/saintfish/chardet v0.0.0-20230101081208-5e3ef4b5456d // indirect
17     github.com/temoto/robotstxt v1.1.2 // indirect
18     golang.org/x/net v0.16.0 // indirect
19     golang.org/x/text v0.13.0 // indirect
20     google.golang.org/appengine v1.6.8 // indirect
21     google.golang.org/protobuf v1.26.0 // indirect
22 )
```

Figure 20.1: go.mod

We have set up the path for the scraping project. Create **webScraping.go** and start writing the code. In this project, we are going to extract the data from <https://erraticcoder.com> website:

```
package main

import (
    "fmt"

    "github.com/gocolly/colly" // importing Colly package
)

func main() {
    fmt.Println("Start Web Scraping!")

    c := colly.NewCollector()

    c.OnRequest(func(r *colly.Request) {
        fmt.Println("Visiting: ", r.URL)
```

```

}))

    c.Visit("https://erraticcoder.com")

}

```

In the preceding code, first, we imported the Colly package and then created an instance of Colly Collector with a default configuration that is responsible for all scraping functionalities. In the next line, we defined **OnRequest** callback function that will be called before placing any request. You can change request headers, print information about the URL being accessed, or take any other necessary actions inside this callback. Finally, we use **Visit** function with the required URL to start the scraping process. Here is the output of the preceding code:

Start Web Scraping!

Visiting: <https://erraticcoder.com>

Similar to **OnRequest** callback, we can define other events also. These callbacks need to be customized before calling **Visit** function which is shown as follows:

```

func main() {
    fmt.Println("Start Web Scraping!")
    c := colly.NewCollector()
    c.OnRequest(func(r *colly.Request) {
        fmt.Println("Visiting: ", r.URL)
    })
    c.OnResponse(func(r *colly.Response) {
        fmt.Println("Request completed for ", r.Request.URL)
    })
    c.OnError(func(_ *colly.Response, err error) {
        log.Println("Something went wrong: ", err)
    })
    c.Visit("https://1.com")
}

```

Output:

Start Web Scraping!

Visiting: `https://.com`

Something went wrong: Get "https://.com": dial tcp: lookup .com: no such host

In the preceding output, the **OnResponse** callback function is called once the web server responds. You can access details about the response, including the URL, status code, response body, and headers, inside of this callback.

In this code, since we have given an incorrect URL, we will not get any response from the server, and thus, it is triggering the **OnError** callback. This callback can be used to log error details or to perform other appropriate actions. Next, we will try to extract the data from the webpage by using "OnHTML" callback that is called whenever the specified selector matches elements in the HTML response.

```
c.OnHTML("a", func(e *colly.HTMLElement) {  
    // prints all the available URLs  
    fmt.Println(e.Attr("href"))  
})
```

Output:

`https://erraticcoder.com/`

`https://erraticcoder.com/blog/`

`https://erraticcoder.com/golang/`

`https://erraticcoder.com/tech/`

`https://erraticcoder.com/python/`

`https://erraticcoder.com/2021/07/31/free-email-validation/`

`https://erraticcoder.com/2021/07/31/install-go-on-windows-mac-linux/`

`https://erraticcoder.com/2021/07/31/what-is-matplotlib/`

`https://www.instagram.com/erratic__coder`

In the preceding code, searches for all anchor tags and will get the **href** attribute of this. As an output, it will print all the hyperlinks found in <https://erraticcoder.com> webpage.

In the following code, we are extracting the URL from the anchor tag that

contains specific text **Python guide**:

```
c.OnHTML("a:contains('Python Guide')", func(e *colly.HTMLElement) {  
    fmt.Println(e.Attr("href"))    // https://erraticcoder.com/python/  
})
```

After getting the URL, we will visit this Python Guide and will extract text and URL of all the listed Python blogs:

```
c.OnHTML("a:contains('Python guide')", func(e *colly.HTMLElement) {  
    // prints URL where anchor tag contains 'Python Guide'  
    pythonURL := e.Attr("href")  
    fmt.Println("Python Guide URL: ", pythonURL)  
    c.OnHTML("li.listing-item a", func(e *colly.HTMLElement) {  
        fmt.Println(e.Text, ": ", e.Attr("href"))  
    })  
    c.Visit(pythonURL)  
})
```

Output:

Python Guide URL: <https://erraticcoder.com/python/>

Where do we use decision tree algorithm : <https://erraticcoder.com/2022/01/20/where-do-we-use-decision-tree-algorithm/>

Sources of Machine learning dataset : <https://erraticcoder.com/2022/01/20/sources-of-machine-learning-dataset/>

Reinforcement ML algorithms : <https://erraticcoder.com/2022/01/12/reinforcement-ml-algorithms/>

Unsupervised ML algorithm : <https://erraticcoder.com/2022/01/12/unsupervised-ml-algorithm/>

Supervised Machine Learning Algorithm : <https://erraticcoder.com/2022/01/11/supervised-learning/>

Type of Machine Learning : <https://erraticcoder.com/2022/01/10/machine-learning-types/>

Text Summarization in NLP : <https://erraticcoder.com/2022/01/06/text-summarization-in-nlp/>

Lemmatization in NLP : <https://erraticcoder.com/2021/12/31/lemmatization-in-nlp/>

Stemming in NLP – Python : <https://erraticcoder.com/2021/12/30/stemming-in-nlp-python/>

Stopwords in NLP – Python : <https://erraticcoder.com/2021/12/29/stopwords-in-nlp-python/>

To check the HTML tags on any webpage, you can use web development tools provided by web browsers. The majority of contemporary web browsers come with built-in developer tools that let you examine a webpage's HTML code. To open the developer tool on Chrome or Firefox, either right-click on any part of the webpage and select **Inspect** or press *Ctrl + Shift + I*.

Let us define a data structure, struct, to store an extracted data:

```
type PythonBlogs struct {  
    subject, url string  
}
```

The complete code, where all the Python blogs are stored in the preceding struct, is as follows:

```
package main  
  
import (  
    "fmt"  
    "log"  
    "github.com/gocolly/colly"  
)  
  
type PythonBlogs struct {  
    subject, url string  
}  
  
func main() {  
    var blogs []PythonBlogs  
    fmt.Println("Start Web Scraping!")  
    c := colly.NewCollector()  
    c.OnRequest(func(r *colly.Request) {  
        fmt.Println("Visiting: ", r.URL)  
    })  
    c.OnResponse(func(r *colly.Response) {
```



```

        fmt.Println("Request completed for ", r.Request.URL)
    })
    c.OnError(func(_ *colly.Response, err error) {
        log.Println("Something went wrong: ", err)
    })
    // c.OnHTML("a", func(e *colly.HTMLElement) {
    //     // prints all the available URLs
    //     fmt.Println(e.Attr("href"))
    // })
    c.OnHTML("a:contains('Python guide')", func(e *colly.HTMLElement) {
        // prints URL where anchor tag contains 'Python Guide'
        pythonURL := e.Attr("href")
        fmt.Println("Python Guide URL: ", pythonURL)
        c.OnHTML("li.listing-item a", func(e *colly.HTMLElement) {
            pythonBlog := PythonBlogs{}
            pythonBlog.subject = e.Text
            pythonBlog.url = e.Attr("href")
            blogs = append(blogs, pythonBlog)
        })
        c.Visit(pythonURL)
    })
    c.Visit("https://erraticcoder.com")
    fmt.Println(blogs)
}

```

Output:

Start Web Scraping!

Visiting: <https://erraticcoder.com>

Request completed for <https://erraticcoder.com>

Python Guide URL: <https://erraticcoder.com/python/>

Visiting: <https://erraticcoder.com/python/>

Request completed for <https://erraticcoder.com/python/>

[{Where do we use decision tree algorithm <https://erraticcoder.com/2022/01/20/where-do-we-use-decision-tree-algorithm/>} {Sources of Machine learning dataset <https://erraticcoder.com/2022/01/20/sources-of-machine-learning-dataset/>} {Reinforcement ML algorithms <https://erraticcoder.com/2022/01/12/reinforcement-ml-algorithms/>} {Unsupervised ML algorithm <https://erraticcoder.com/2022/01/12/unsupervised-ml-algorithm/>} {Supervised Machine Learning Algorithm <https://erraticcoder.com/2022/01/11/supervised-learning/>} {Type of Machine Learning <https://erraticcoder.com/2022/01/10/machine-learning-types/>} {Text Summarization in NLP <https://erraticcoder.com/2022/01/06/text-summarization-in-nlp/>} {Lemmatization in NLP <https://erraticcoder.com/2021/12/31/lemmatization-in-nlp/>} {Stemming in NLP – Python <https://erraticcoder.com/2021/12/30/stemming-in-nlp-python/>} {Stopwords in NLP – Python <https://erraticcoder.com/2021/12/29/stopwords-in-nlp-python/>}

Get scraped data to CSV

To store scraped data in CSV format, you can use **encoding/csv** package in Go. First, you need to create/open a file for writing. You can use the **os** package to create the file. A writer object needs to be initialized and associated with this file to facilitate writing data into the CSV file. After writing the data in file, writer object should get cleaned up. Similarly, file must be closed, once all operations are done. We have defined these mandatory tasks with defer statement that will execute at the end of main function. Here are the basic steps or structure to follow when working with a file:

```
file, _ := os.Create("blogs.csv")
```

```
defer file.Close()
```

```
writer := csv.NewWriter(file)
```

```
defer writer.Flush()
```

In the preceding code, we got scraped data in **blogs** array. We will iterate over this array and write each record in the CSV file:

```
headers := []string{"subject", "url"}
```

```
writer.Write(headers)
```

```

for _, blog := range blogs {
    rec := []string{blog.subject, blog.url}
    writer.Write(rec)
}

```

Here is the complete code of the web scraper:

```

package main

import (
    "encoding/csv"
    "fmt"
    "log"
    "os"
    "github.com/gocolly/colly"
)

type PythonBlogs struct {
    subject, url string
}

func main() {
    var blogs []PythonBlogs
    fmt.Println("Start Web Scraping!")
    c := colly.NewCollector()
    c.OnRequest(func(r *colly.Request) {
        fmt.Println("Visiting: ", r.URL)
    })
    c.OnResponse(func(r *colly.Response) {
        fmt.Println("Request completed for ", r.Request.URL)
    })
    c.OnError(func(_ *colly.Response, err error) {
        log.Println("Something went wrong: ", err)
    })
}

```

```

// c.OnHTML("a", func(e *colly.HTML_Element) {
//     // prints all the available URLs
//     fmt.Println(e.Attr("href"))
// })

c.OnHTML("a:contains('Python guide')", func(e *colly.HTML_Element) {
    // prints URL where anchor tag contains 'Python Guide'
    pythonURL := e.Attr("href")
    fmt.Println("Python Guide URL: ", pythonURL)
    c.OnHTML("li.listing-item a", func(e *colly.HTML_Element) {
        pythonBlog := PythonBlogs{}
        pythonBlog.subject = e.Text
        pythonBlog.url = e.Attr("href")
        blogs = append(blogs, pythonBlog)
    })
    c.Visit(pythonURL)
})

c.Visit("https://erraticcoder.com")

fmt.Println(blogs)

// Store data in CSV
file, _ := os.Create("blogs.csv")
defer file.Close()

writer := csv.NewWriter(file)
defer writer.Flush()

headers := []string{"subject", "url"}
writer.Write(headers)

for _, blog := range blogs {
    rec := []string{blog.subject, blog.url}
    writer.Write(rec)
}

```

```
}
```

Headless scraping

Headless scraping enables users to execute automated scripts in headless mode, which hides the browser window. For browsers built on the Chrome and Firefox platforms, the headless mode is used. By using a headless browser, we avoid the needless rendering of graphics and images that can conserve time and resources. It is the best practice to crawl the data from web pages.

In headless scraping, the web page is loaded and processed by scraping the script, but in the complete process, the browser is not displayed to the user. The following are some essential features and benefits of headless scraping:

- It has no GUI interaction, which makes the script suitable to run on a server in the background.
- Consumes less system resources.
- Since browser does not appear, it protects privacy while dealing with sensitive data.
- Due to its ability to run JavaScript, it can scrape dynamic webpages.
- Click links, submit button, scrolling, and extracting data is possible through script.
- It can be used for screen scraping as well as website testing.

In Go, “chromedp” is the most common tool for headless scraping over chrome browser. You can install chromedp library by using the following command:

```
go get -u github.com/chromedp/chromedp
```

Let us write a headless script with **chromedp**. Create a file **headless_scraping.go** in same project as created previously:

```
package main
```

```
import (
```

```
    "context"
```

```
    "fmt"
```

```
    "time"
```

```

        "github.com/chromedp/chromedp"
    )
    func main() {
        ctx, cancel := chromedp.NewContext(context.Background())
        defer cancel()
        // Visit to the website
        url := "http://books.toscrape.com"
        err := chromedp.Run(ctx,
            chromedp.Navigate(url),
            chromedp.Sleep(5*time.Second),
        )
        if err != nil {
            fmt.Println("Error navigating to the website:", err)
            return
        }
    }
}

```

In the preceding code, script will navigate a given webpage and wait for five seconds so that the whole content can be loaded:

```
ctx, cancel := chromedp.NewContext(context.Background())
```

Here, we are using the **context** library that offers a robust collection of capabilities for handling concurrent activities, such as request timeout, cancellation, passing information to downstream handlers, etc. A new context is initiated by using the **chromedp.NewContext** function, which can then be used to control browser automation operations. A separate context is created to start each browser automation session and activities/processes are managed within that respective session. **Context.Background()** represents a clean, empty context that does not have any associated values or deadlines.

The resources linked to a particular context can be terminated and cleaned up using the **cancel** function in chromedp which is shown as follows:

```
defer cancel()
```

When a task or activity is finished, it is customary in the Go programming

language to terminate the context which ensures appropriate resource release and prevent resource leaks. By using defer, **cancel** function is called when the context has finished, even in situations when there is an error or an early return.

Next, the **chromedp.Run** function in the **chromedp** package is used to carry out a series of operations or tasks in a browser automation context. Here, we are navigating a webpage and waiting for five seconds to load the whole page.

Now, the webpage has loaded, and it is time to perform some activity on browser. In chromedp, **chromedp.ActionFunc** allows to define all the custom actions that needs to be performed for browser automation tasks. A browser associated context is passed as a parameter that regulate and manages the action's lifecycle. This function returns an error that helps to identify whether defined actions were completed successfully or not. If an execution fails, you can return an error and handle it appropriately.

Now, in the following code, we are going to extract the data from **books.toscrape** website where a variety of books are listed. We will fetch the book title and price for a predefined number of pages using web scraping. Since we have given page count as 5, the data for the first five pages will be retrieved. Here is the complete headless scraping code:

```
package main

import (
    "context"
    "encoding/csv"
    "fmt"
    "os"
    "time"

    "github.com/chromedp/cdproto/cdp"
    "github.com/chromedp/chromedp"
)

// A struct that will hold scraped data
type Books struct {
```

```

        title, price string
    }
}

func main() {
    // Number of pages you want the data to be taken from
    pageCount := 5
    books := []Books{}

    // the csv file containing the data as an output
    file, _ := os.Create("books.csv")
    defer file.Close()
    writer := csv.NewWriter(file)
    defer writer.Flush()
    headers := []string{"title", "price"}
    writer.Write(headers)

    ctx, cancel := chromedp.NewContext(context.Background())
    defer cancel()

    // Navigate to the website
    url := "http://books.toscrape.com"
    err := chromedp.Run(ctx,
        chromedp.Navigate(url),
        chromedp.Sleep(5*time.Second),
        chromedp.ActionFunc(func(ctx context.Context) error {
            var err error
            for count := 0; count < pageCount; count++ {
                var articles []*cdp.Node
                err = chromedp.Run(ctx, chromedp.Nodes("article.product_pod", &articles))
                // Iterate over the listed books
                for _, article := range articles {
                    var title, price string
                    // Scrap the required data

```



```

        err = chromedp.Run(ctx, chromedp.Text("h3", &title, chromedp.ByQuery,
chromedp.FromNode(article)))

        err = chromedp.Run(ctx, chromedp.Text("p.price_color", &price,
chromedp.ByQuery, chromedp.FromNode(article)))

        // Add detail of book to scraped data list
        books = append(books, Books{title: title, price: price})

        // Write Scraped data to CSV file
        writer.Write([]string{title, price})

    }

    // Click on next button available on webpage
    err = chromedp.Run(ctx, chromedp.Click(".next > a"))
    time.Sleep(10 * time.Second)

}

return err

}),

)

if err != nil {

    fmt.Println("Error navigating to the website:", err)

    return

}

}

```

In the preceding code, we are referring HTML tags, class name, etc. to read the required data. However, it can be different over time whenever web pages undergo the changes. In this case, update your scraping code in accordance with any changes you find on the target website on a regular basis. Overall, you may need to adjust the web scraping script and set up Chrome or Chromium for headless operation, depending on the specific structure of the website you are scraping.

In this way, you can scrap the data without using GUI or rendering the web page in any browser. Since no graphics, CSS, or other non-essential elements must load in headless scraping, it can be more efficient than scraping with a

visible browser.

How to avoid blocking

Web scraping has many advantages, but it is crucial to remember that it should only be done in compliance with the law and ethical norms. To keep the activity morally and legally acceptable, it is imperative to abide by the terms of service of the websites being scraped, refrain from overloading servers, and manage data appropriately. Here are some key points to remember while scraping:

- **Identify yourself:** User agent or particular headers in your HTTP queries could be needed by certain websites in order to recognize your scraping bot. In order to have a more realistic browsing experience, make sure you include the required headers.
- **Respect copyright and privacy laws:** Avoid using data scraping and any other practices that compromise user privacy or violate intellectual property rights.
- **Avoid scraping sensitive information:** Refrain from collecting private or sensitive personal information through scraping.
- **Do not overwhelm servers:** To prevent overburdening the server, keep your request volume and frequency to a minimum. Think about extending the duration between requests and avoiding scraping during periods of high traffic.
- **Follow robots.txt:** Websites use the robots.txt standard to interact with web crawlers and scrapers. It indicates which areas of a website are accessible to crawlers and which are not.

Conclusion

In this chapter, we explored the fundamentals of web scraping in Go, covering key tools like the Colly library and techniques such as headless scraping for dynamic content. With these concepts, you can now efficiently extract data from websites using Go. In the next chapter, we will dive into automation in Go, where we will explore how to automate tasks and workflows to streamline processes and improve productivity.

Questions

1. Which libraries can be used for headless scraping in Go?
2. How do you find an HTML path that contains the data you need?
3. What is the purpose of the robots.txt file?
4. How to avoid overloading on server while scraping.
5. How do we to install Colly in Go?

Join our book's Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

<https://discord.bpbonline.com>



CHAPTER 21

Automation with Golang

Introduction

Automation is primarily used to boost productivity by decreasing manual work and increasing accuracy and efficiency. Automation has applications in a wide range of fields, including information technology, manufacturing, healthcare, finance, and more.

In information technology, automation is the use of technology to carry out operations, procedures, or tasks, reduce errors and enhance overall productivity without the need for human intervention.

Structure

- Introduction to automation
- Automation key areas

Objectives

This chapter provides an in-depth examination of the various ways in which the Go programming language may be utilized to efficiently automate certain operations. The reader will gain knowledge of particular automation areas, including network automation, database automation, and infrastructure as a code. It also covers third-party libraries and the built-in

testing package when it comes to testing automation in Golang.

Introduction to automation

Automation is using Go more and more, especially in fields like network programming, cloud computing, and system tools. Go is easy to integrate with other software and services, enabling the automation of pipelines for data extraction and analysis. All in all, Go's attributes—including its ease of use, effectiveness, and robust concurrency support—make it a flexible language for automation tasks. Its growth in the DevOps area is proof of its ability to meet the demands of modern automated workflows.

Automation key areas

Go is a fast and efficient system-level operator that may be used to automate infrastructure provisioning, configuration management, and system administration activities. Here are some key areas where automation can be implemented with Go:

- Infrastructure as Code (IaC)
- Container orchestration
- Workflow Automation/Automation Scripts
- Cloud services development
- Network automation
- Testing
- Database tools
- Continuous Integration/Continuous Deployment (CI/CD)

Let us explore each automation area and see how Golang can be used to implement it.

Infrastructure as Code

IaC controls and provisions computer infrastructure using machine-readable script files rather than interactive setup tools or actual hardware configuration. Automating the deployment and maintenance of infrastructure is the aim of IaC, which enables more scalable, reliable, and

efficient IT operations.

Infrastructure as Code's essential features and elements include the following:

- **Version control:** The most common place to keep IaC scripts is in version control systems (like Git). This makes it possible to collaborate with team members, track changes over time, and, in case something goes wrong, roll back to earlier versions.
- **Declarative syntax:** IaC scripts employ a declarative syntax that places more emphasis on the final state of the infrastructure than on the steps necessary to get there. Here, the system executes what needs to happen to achieve that desired state.
- **Scalability:** IaC facilitates the infrastructure's effective scalability. Since code is used to specify the intended state, it is simple to adapt to changes in demand or to scale up or down resources as necessary.
- **Automation:** The IaC tool may install and configure the infrastructure automatically after the necessary configuration is defined in code. This reduces the need for manual intervention and lowers the possibility of human error.
- **Reusability:** IaC encourages code and configuration reuse. Reusable modules or templates can be created by encapsulating common infrastructure patterns, which will increase consistency and reduce development time.
- **Cloud support:** Organizations can use IaC to build and manage infrastructure across different cloud providers. Businesses looking to maximize resource allocation and avoid vendor lock-in will find this flexibility helpful.

IaC with Golang

When discussing IaC, or IaC, a number of IaC tools come into role, including Terraform, Chef, Puppet, Ansible, and others. However, these tools have several drawbacks, one of which is that configuration is given rather than programming. Unlike these IaC tools that use declarative languages, Pulumi enables us to define, deploy, and manage cloud infrastructure through imperative languages such as JavaScript, TypeScript,

Python, Go, and .NET. Let us explore Pulumi by building some AWS infrastructure examples in the Go programming language.

In order to use Pulumi with Golang, you can use the following steps to set up a simple Pulumi project, define infrastructure code in Golang, then deploy that infrastructure to a cloud provider. This is a basic AWS example. However, Pulumi is compatible with many cloud providers:

1. **Install Pulumi:** Pulumi is available for download and installation on the official website: <https://www.pulumi.com/docs/install/>.
2. You can find Pulumi documentation here <https://www.pulumi.com/docs/>.
3. **Configure cloud access:** Since Pulumi supports a variety of cloud platforms, you must set up the credentials for your project in order to access the services.
4. **Create Pulumi project:** Create a new directory for your Pulumi project by terminal. To start a new Pulumi project, navigate to the project directory and execute the following command. With the help of the Golang template, a new Pulumi project is started with this command.

pulumi new project_name

5. We can see some configuration files and **main.go** once the project has been created in the project directory.
6. **Write Code:** Open the **pulumi.dev.yaml** file that was generated, then modify this to provide the infrastructure resources you want using Golang code. Actual code may vary, depending on the cloud provider and the resources you want to create. Here is the sample code to create an AWS S3 bucket:

```
package main
```

```
import (
```

```
    "github.com/pulumi/pulumi-aws/sdk/v4/go/aws/s3"
```

```
    "github.com/pulumi/pulumi/sdk/v4/go/pulumi"
```

```

)
func main() {
    pulumi.Run(func(ctx *pulumi.Context) error {
        // Create an S3 bucket
        bucket, err := s3.NewBucket(ctx, "myBucket", nil)
        if err != nil {
            return err
        }
        // Export the bucket name
        ctx.Export("bucketName", bucket.ID())
        return nil
    })
}

```

7. **Deploy/ View / Destroy an infrastructure:** To deploy your infrastructure, save your code and type the following command into the terminal:

pulumi up

8. After deployment, Pulumi will provide the details of created resources on console. By using the following command you can destroy the resources if needed:

pulumi destroy

So, this is an overview to implement Pulumi using Golang. Here, we have created s3 bucket, but you can define and create E2 instances, RDS instances, Lambda functions, etc. Go generics are also supported by Pulumi, enabling programmers to create code that is clear, reusable, and type-safe. Remember that you can alter the infrastructure code to suit your needs and the needs of the cloud provider you want to use.

Container orchestration

The automated process of networking, scaling, deploying, and managing containerized applications without the need for human involvement is known as container orchestration. An application and its dependencies are packaged into lightweight, portable, and consistent components called containers, which allow the application to execute consistently and dependably in a variety of computing environments. On production, container orchestration plays a crucial role in the deployment, scaling, and management of microservices-based containerized applications.

Some features of container orchestration are:

- **Cluster management:** In order to provide high availability and scalability, containerized applications are usually distributed across several hosts or nodes. Managing and organizing these nodes together to ensure their smooth cooperation is known as cluster management.
- **Service discovery:** Containers can locate and communicate with each other dynamically with the aid of service discovery techniques.
- **Load balancing:** Load balancing is a feature that helps to distribute incoming traffic among containers equally. This guarantees that no single container becomes a bottleneck and that the workload is dispersed effectively.
- **Health monitoring:** The orchestration platform can automatically restart a container or service or reroute traffic to healthy instances in the event of a failure.
- **Rollback updates:** It allows steady deployment of new versions without downtime and has the capability to rollback the changes in case of any issues.
- **Scaling:** It has the ability to dynamically scale the number of containers up or down in response to changes in resource consumption or traffic volume.
- **Security and configuration:** It makes it simpler to maintain consistency and make changes throughout the entire application by enabling centralized control of container configuration parameters. It has security capabilities to guarantee container isolation, manage rights, regulate access, and store sensitive data securely.

Container orchestration platform

Several platforms offer a variety of features and functionalities, such as networking, load balancing, scaling, and deployment, for managing containerized applications. It includes:

- Kubernetes
- Docker Swarm
- Red Hat OpenShift
- Apache Mesos
- Hashicorp Nomad
- Rancher

Out of these, Kubernetes has become incredibly popular due to its extensive feature set, active community, and widespread industry support.

Kubernetes with Golang

Golang is a popular choice because of its compatibility with cloud-native principles and ease of use for developers building Kubernetes-enabled applications. Although Kubernetes is written in Go, it offers a Go client library that lets programmers use Go to communicate with the Kubernetes API and carry out tasks.

The general procedure to use Kubernetes Go client to carry out container orchestration with Golang is as follows:

- **Configure Kubernetes:** The Kubernetes documentation offers resources for customizing Kubernetes objects, such as Pods, Services, and Deployments, and can be used to setup Kubernetes. Tools such as *kubeadm*, the officially supported tool for Kubernetes deployment, can be used to install Kubernetes. Once Kubernetes has been installed., *kubectl*, is a command-line tool, that allows you to communicate with your cluster and manage Kubernetes objects.
- **Install the Kubernetes Go Client Library:** By using the following command you can install required packages to communicate with Kubernetes APIs to deploy PODs, creating services and managing other Kubernetes resources:

go get k8s.io/client-go/...

- **Write Golang code:** Here is a short example that shows you how to use Go to make a deployment. Using the Kubernetes Go client library generates a basic Kubernetes deployment that may be altered to suit your unique requirements:

```
package main
```

```
import (
```

```
    "context"
```

```
    "fmt"
```

```
    "os"
```

```
    "k8s.io/client-go/kubernetes"
```

```
    "k8s.io/client-go/tools/clientcmd"
```

```
    "k8s.io/client-go/util/homedir"
```

```
    "k8s.io/client-go/util/rest"
```

```
)
```

```
func main() {
```

```
    home := homedir.HomeDir()
```

```
    kubeconfig := filepath.Join(home, ".kube", "config")
```

```
    // use the current context in kubeconfig
```

```
    config, err := clientcmd.BuildConfigFromFlags("", kubeconfig)
```

```
    if err != nil {
```

```
        panic(err.Error())
```

```
    }
```

```
    // create the clientset
```

```
    clientset, err := kubernetes.NewForConfig(config)
```

```
if err != nil {  
    panic(err.Error())  
}  
  
// Define a simple Deployment  
deployment := &appsv1.Deployment{  
    ObjectMeta: metav1.ObjectMeta{  
        Name: "example-deployment",  
    },  
    Spec: appsv1.DeploymentSpec{  
        Replicas: int32Ptr(3),  
        Selector: &metav1.LabelSelector{  
            MatchLabels: map[string]string{  
                "app": "example-app",  
            },  
        },  
        Template: corev1.PodTemplateSpec{  
            ObjectMeta: metav1.ObjectMeta{  
                Labels: map[string]string{  
                    "app": "example-app",  
                },  
            },  
            Spec: corev1.PodSpec{  
                Containers: []corev1.Container{
```

```

        {
            Name: "example-container",
            Image: "nginx:latest",
        },
    },
    },
},
}

// Create the Deployment

result, err :=
clientset.AppsV1().Deployments("default").Create(context.TODO(),
deployment, metav1.CreateOptions{})

if err != nil {
    panic(err.Error())
}

fmt.Printf("Created Deployment %q.\n",
result.GetObjectMeta().GetName())
}

```

- func int32Ptr(i int32) *int32 { return &i }**
- Run and verify the code:** Ensure that the Kubernetes cluster is running. Save the preceding code and run with simple Go command **go run file_name.go** . You can use kubectl to verify the successful deployment.

An example of a simple use case is shown here. In addition, you may automate the management of stateful apps by using tools like the operator-

sdk, which facilitates the creation of Kubernetes operators in Golang.

Workflow automation or automation scripts

When implementing process automation or automation scripts in Golang, programmatic task execution is made possible by the language's features. At first, you need to understand the process or job that you wish to automate and then divide it up into discrete acts or steps.

While creating an automation script or workflow, do not forget to consider error handling, logging, and other required validations into account. Additionally, input parameters and configuration settings need to be properly planned.

The following example shows how a script reads content from an **input.txt** file and writes it to a backup.txt file. In every five minutes, this process examines the two files and, if any changes have been made, replicates them to the other file:

```
package main

import (
    "fmt"
    "io/ioutil"
    "time"
)

const (
    sourceFilePath    = "input.txt"
    destinationFilePath = "backup.txt"
    interval          = 5 * time.Minute
)

func readFileContent(filePath string) ([]byte, error) {
    content, err := ioutil.ReadFile(filePath)
    if err != nil {
        return nil, err
    }
}
```

```

    return content, nil
}

func writeFileContent(filePath string, content []byte) error {
    err := ioutil.WriteFile(filePath, content, 0644)

    if err != nil {
        return err
    }

    return nil
}

func main() {
    for {
        sourceContent, err := readFileContent(sourceFilePath)

        if err != nil {
            fmt.Println("Error reading source file:", err)
            time.Sleep(interval)
            continue
        }

        destinationContent, err := readFileContent(destinationFilePath)

        if err != nil {
            fmt.Println("Error reading destination file:", err)
            time.Sleep(interval)
            continue
        }

        if string(sourceContent) != string(destinationContent) {
            err := writeFileContent(destinationFilePath, sourceContent)

            if err != nil {
                fmt.Println("Error writing to destination file:", err)
            } else {

```

```

        fmt.Println("File content updated successfully.")
    }
    } else {
        fmt.Println("No changes detected.")
    }
    time.Sleep(interval)
}
}

```

In this way, you can perform any automated task as per need. Keep in mind that this might not be the most effective way to poll for changes continuously, particularly for large files or frequent updates.

Cloud services development

There are multiple steps involved in deploying a Golang application to a cloud service, and the exact procedure may change based on the cloud provider you select. When deploying an application, there are a few things to take into account, such as:

- **Environment configuration:** Make sure the environment variables for configuration in your Golang application can be read.
- **Networking and security:** Set up load balancers, firewall rules, and HTTPS settings, as well as other networking and security settings, according to your application's requirements.
- **Logging and monitoring:** Configure the cloud provider's logging and monitoring tools to keep track of the functionality and condition of your deployed application.
- **Scaling:** If necessary, set up auto-scaling parameters so that your application can grow in response to demand.

You can use Golang to automate the deployment of cloud services by writing a script or program that communicates with the CLI or APIs of the cloud provider to launch your application. Here is a basic example using the AWS SDK for Go (v2):

Using your AWS login credentials, install and set up the Amazon CLI:

1. Install AWS SDK for Go using **go get -u [github.com/aws/aws-sdk-go@v2](https://github.com/aws/aws-sdk-go)**
2. Write a code that builds the Golang application, create a ZIP for the same and deploy the ZIP to AWS by creating a new application version and update an environment:

```
package main

import (
    "context"
    "fmt"
    "os"
    "os/exec"
    "path/filepath"
    "strings"
    "time"
    "github.com/aws/aws-sdk-go-v2/aws"
    "github.com/aws/aws-sdk-go-v2/config"
    "github.com/aws/aws-sdk-go-v2/service/elasticbeanstalk"
)

const (
    applicationName = "YourApplicationName"
    environmentName = "YourEnvironmentName"
    zipFileName     = "app.zip"
)

func main() {
```

```

// Build the Golang application
if err := runCommand("go", "build", "-o", "myapp"); err != nil {
    fmt.Println("Error building Golang application:", err)
    os.Exit(1)
}

// Create a ZIP file with the application code
if err := runCommand("zip", "-r", zipFileName, "myapp"); err != nil
{
    fmt.Println("Error creating ZIP file:", err)
    os.Exit(1)
}

// Deploy to Elastic Beanstalk
if err := deployToElasticBeanstalk(); err != nil {
    fmt.Println("Error deploying to Elastic Beanstalk:", err)
    os.Exit(1)
}

fmt.Println("Deployment successful!")
}

func deployToElasticBeanstalk() error {
    cfg, err := config.LoadDefaultConfig(context.TODO())
    if err != nil {
        return err
    }

```

```

    client := elasticbeanstalk.NewFromConfig(cfg)

    // Create a new Elastic Beanstalk application version
    createAppVersionOutput, err :=
        client.CreateApplicationVersion(context.TODO(),
            &elasticbeanstalk.CreateApplicationVersionInput{

                ApplicationName: aws.String(applicationName),
                VersionLabel: aws.String(time.Now().Format("20060102150405")),

                SourceBundle: &elasticbeanstalk.S3Location{

                    S3Bucket: aws.String("your-s3-bucket-name"),

                    S3Key:  aws.String(zipFileName),

                },

            })

    if err != nil {

        return err

    }

    fmt.Printf("Application version created: %s\n",
        *createAppVersionOutput.ApplicationVersion.VersionLabel)

    // Update the Elastic Beanstalk environment to use the new version
    _, err = client.UpdateEnvironment(context.TODO(),
        &elasticbeanstalk.UpdateEnvironmentInput{

            ApplicationName: aws.String(applicationName),

            EnvironmentName: aws.String(environmentName),

            VersionLabel:

                createAppVersionOutput.ApplicationVersion.VersionLabel,

        })

    if err != nil {

```

```

        return err
    }

    fmt.Printf("Environment updated: %s\n", environmentName)

    return nil
}

func runCommand(command string, args ...string) error {
    cmd := exec.Command(command, args...)

    cmd.Stdout = os.Stdout

    cmd.Stderr = os.Stderr

    return cmd.Run()
}

func cleanUp() {
    // Clean up temporary files if needed

    if err := os.Remove("myapp"); err != nil {
        fmt.Println("Error cleaning up:", err)
    }

    if err := os.Remove(zipFileName); err != nil {
        fmt.Println("Error cleaning up:", err)
    }
}

func init() {
    // Ensure clean-up is performed even if the script exits with an error

    cleanUp()
}

```

```

}

func cleanupOnExit() {
    if r := recover(); r != nil {
        // Recovered from panic, perform clean-up
        cleanUp()
    }
}

func main() {
    defer cleanupOnExit()

    automation

    // Your main function code here
}

```

In this way, the Golang application can be deployed on the cloud and you can automate the process.

Network automation

Network automation is the process of automating computer network configuration, management, and operation through the use of software tools, scripts, and procedures. This covers both contemporary **software-defined networking (SDN)** and virtualized network functions, as well as more conventional networking hardware like routers, switches, and firewalls. Numerous facets of network operations can be automated, such as:

- **Device provisioning:** Automating the addition, modification, or removal of network devices is called device provisioning.
- **Configuration management:** Configuring access control lists, routing protocols, VLANs, and other things.
- **Scaling resources:** It refers to the automatic allocation and removal of

network resources in response to variations in demand.

- **Monitoring and troubleshooting:** The process of gathering information about network performance and analyzing logs to find and fix problems.
- **Security policy enforcement:** Automated deployment and monitoring of security protocols throughout the network, such as firewall rule updates.

Golang is ideal for creating dependable and effective network automation solutions. A wide range of libraries are available for Golang that can be used for network automation. Several libraries are as follows:

- **net:** Go comes with a net package in its standard library that serves as the basis for networking functions. Working with TCP, UDP, and other network protocols is one of its features.
- **gnet:** Gnet is a lightweight, non-blocking, high-performance network framework that allows you to design your own protocols with TCP/UDP on the transport layer.
- **github.com/go-expect/go-expect:** **go-expect** automates CLI interactions by sending commands and receiving output. It is useful in network automation for automating configuration tasks with CLI-based network devices like routers or switches.
- **goPacket:** This is a library for manipulating and decoding packets that enables you to operate at a lower level with network packets.
- **github.com/go-resty/resty:** Resty is a straightforward Golang client library for HTTP and REST. It can be used to communicate with RESTful APIs and submit HTTP requests.
- **gorilla/websocket:** A well-liked option for developing WebSocket apps in Go is the **gorilla/websocket** package, which offers a quick, reliable, and extensively utilized WebSocket implementation.
- **github.com/go-redis/redis:** If your network automation involves interacting with Redis databases, this library provides a Golang client for Redis.
- **go-netconf:** The purpose of this library is to operate with NETCONF, a network management protocol that enables the **network management system (NMS)** to add, modify, and remove network

device configurations.

- **gRPC Network Management Interface (gNMI):** The gnmi package may be helpful if you work with network devices that enable gRPC-based management interfaces. The gRPC framework is used by gNMI to facilitate device configuration management and telemetry.

Recall that Golang is ideally suited for interfacing with network devices and management systems due to its strong support for working with APIs. Even though Golang may not have as many specialized libraries as other languages, you can still design efficient network automation solutions by utilizing and expanding upon the language's standard libraries.

Testing

Numerous testing frameworks and tools are available with Golang that enable automated testing:

- **testing:** Following are powerful and widely used built-in testing package that facilitates the generation of benchmarks, examples, and unit tests:
 - o **testify:** It is a testing toolkit for Go that adds features like assertions, mocks, and test suite creation on the top of built-in testing package, making the output of failed tests easier to interpret. This package can be imported with the command: github.com/stretchr/testify
 - o **Goblin:** Goblin is a straightforward and expressive Go testing framework that offers a clear and uncomplicated syntax for test writing. You can import this package with github.com/franela/goblin

Database automation

Database automation is the process of managing, deploying, and maintaining databases more efficiently and automatically through the use of tools, scripts, and procedures. Through automation, it becomes easy to find errors or issues in the database, including data corruption, missing, erroneous, and duplicate data. Aspects of database administration that can benefit from this automation include the following:

- **Deployment:** Automating the deployment of database instances, including replication of existing databases and installation of new database servers.
- **Configuration management:** It includes database setup, user access configuration, and schema structure initialization are all part of this.
- **Monitoring and alerts:** It describes responsibilities related to alerting, optimization, and automated database performance monitoring. To preserve peak performance, this entails identifying bottlenecks, refining queries, and modifying setups.
- **Backup and recovery:** It includes recovery procedures and automating routine database backups to address data loss or system failure issues.
- **Data migration:** Automated data transfer process between several database versions or platforms.
- **Data masking:** Automating the critical step of masking sensitive data in non-production environments which is necessary to preserve data privacy.
- **Compliance and security:** Automating procedures linked to security, like encryption, compliance checks, and access control.

Golang's database drivers and libraries, which offer an interface for interacting with different database systems, can be used to automate database-related operations. Many databases, including PostgreSQL, MySQL, SQLite, MongoDB, and others, are supported by Golang. You can use the following libraries to automate database operations in Golang:

- **pq:** Go programs can communicate with PostgreSQL databases with the popular and extensively used **pq** package. Connection pooling, prepared statements, parameterized queries, SSL connections, transactions, error handling, and other features are supported by this package.
- **mattn/go-sqlite3:** Using this package, you may automate tasks like opening SQLite databases, creating tables, inserting and querying data, and more. It is a SQLite driver that enables Go programs to communicate with SQLite databases.
- **go-sql-driver/mysql:** Go has this MySQL driver that is used to

communicate with MySQL databases and perform various functions such as connection pooling, statement preparation, error handling, and more.

- **mongo-driver/mongo:** This is the official Go driver for MongoDB, which enables Go applications to communicate with MongoDB databases. It handles timeouts and cancellations in addition to connection pooling, authentication information, and data searching.
- **database/sql:** It gives developers the ability to use alternative database drivers without having to make large changes to their application code by providing a general interface to **Structured Query Language (SQL)** databases that defines a set of common database activities.
- **GORM:** A collection of methods for standard **Create, Read, Update, Delete (CRUD)** operations, associations, transactions, hooks, and more are provided by this **Open Relational Mapping (ORM)** library, known as GORM. It can be imported with “**gorm.io/gorm**”
- **ent:** Ent is a strong and effective Golang entity framework that facilitates the creation of database schema corresponding to written Golang code. It is compatible with multiple databases, such as PostgreSQL, MySQL, and SQLite. Moreover, ent has good GraphQL integration.
- **golang-migrate/migrate:** In order to maintain your database structure in line with your application’s evolution over time, this Golang database migration tool handles database schema modifications, versioning, and helps for data migrations.
- **pressly/goose:** Similar to **golang-migrate/migrate**, this is another Golang database migration utility. Select the tool that best meets the needs and preferences of your project.

You can automate a range of database-related processes in Golang, from simple CRUD operations to more complex features like connection pooling and migrations, by integrating these packages.

Continuous Integration and Continuous Deployment

For a Go (Golang) project, automating the build, testing, and deployment procedures is necessary to set up **Continuous Integration (CI)** and

Continuous Deployment (CD). In later chapters, we will see more about this.

Conclusion

In this chapter, we explored the key areas where Go excels as a powerful tool for automation. From IaC and container orchestration to workflow automation and cloud services development, we examined how Go can streamline processes across diverse domains. We also delved into its applications in network automation, testing, database tools, and CI/CD pipelines, showcasing its versatility and efficiency. **In the next chapter, we will dive deeper into building a CI/CD pipeline using Go.**

Questions

1. What is the use of Golang in network automation?
2. Which library is commonly used for testing in Golang?
3. What is GORM in Golang?
4. How can database automation be done in Golang?
5. What is automation for Infrastructure.

CHAPTER 22

Data Analysis and Machine Learning

Introduction

Data science and machine learning are transforming industries by optimizing operations, advancing research, and solving complex problems. By analyzing large datasets, businesses gain insights into consumer behavior, market trends, and efficiency. Machine learning, a branch of AI, develops algorithms enabling computers to learn from data and make predictions or decisions without explicit programming.

In this chapter, we will explore the core concepts of data science and machine learning, including supervised, unsupervised, and **reinforcement learning (RL)**.

Structure

- Machine learning and data analysis overview
- Application of machine learning
- Data handling and manipulation
- Exploratory Data Analysis (EDA) in Go
- Machine learning ecosystem with Go
- Golang machine learning libraries

- Time series analysis

Objectives

This chapter introduces the fundamental concepts and terminology of data analysis and machine learning, including **exploratory data analysis (EDA)** for insights, preprocessing techniques like cleaning and normalization, and the distinctions between supervised, unsupervised, and RL. It also includes a practical example of time series analysis using Go.

Machine learning and data analysis overview

Machine learning is a field of artificial intelligence that enables computers to learn from data and make decisions or predictions without explicit programming. It involves building models and algorithms that identify patterns and insights from data.

Data analysis, on the other hand, is the process of inspecting, cleaning, transforming, and modeling data to extract useful information and support decision-making.

In Golang, machine learning and data analysis are increasingly used for building fast, efficient, and scalable applications. With libraries and tools like Gorgonia, GoLearn, and Gonum, developers can perform data manipulation, apply machine learning algorithms, and create data-driven applications while benefiting from Go's speed and concurrency features.

Application of machine learning

Machine learning is crucial as it can analyze large datasets, identify patterns, and make predictions without explicit programming. Here is why it is so important:

- **Data mining and analysis:** Machine learning algorithms can extract valuable insights from large, complex datasets, helping organizations identify trends, correlations, and patterns that traditional methods may miss.
- **Predictive analytics:** Machine learning enables predictive modeling,

allowing companies to forecast trends, actions, and results. It helps businesses anticipate consumer preferences, optimize inventory, and make informed decisions across marketing, finance, and healthcare.

- **Personalization and recommendation systems:** Machine learning systems increase user engagement and satisfaction by delivering personalized content, goods, and services based on analysis of user behavior and preferences.
- **Fraud detection and security:** Machine learning algorithms are able to identify patterns of identity theft attempts, cybersecurity threats, and fraudulent activity in financial transactions. These algorithms detect abnormalities and suspect activity by evaluating massive amounts of data in real-time, assisting businesses in averting financial losses and safeguarding confidential data.
- **Healthcare innovations:** Machine learning aids drug discovery, personalized treatment, and early disease detection in healthcare. By analyzing genetic data, health records, and medical images, it helps diagnose conditions, predict outcomes, and improve treatment efficacy.
- **Natural language processing (NLP):** Natural language processing advances due to machine learning, allowing machines to comprehend, interpret, and produce human language. NLP is used to improve accessibility and communication in a variety of fields through applications like sentiment analysis, language translation, chatbots, virtual assistants, and content summary.
- **Scientific research and exploration:** Machine learning helps scientists study complicated phenomena, model experiments, and create ground-breaking findings in a variety of fields, from astronomy to environmental science.

As algorithms evolve and datasets grow, machine learning opens up new possibilities for solving complex problems, driving progress, and shaping the future of technology and society.

Data handling and manipulation

Working with different data structures, reading and writing data to and from files, and carrying out operations like filtering, sorting, and altering data,

are all part of data handling and manipulation in **Golang (Go)**. This is a summary of how to handle and manipulate data in Go:

- **Data structure:** Golang supports several data structures, including:
 - o **Arrays:** Arrays are fixed-size, homogeneous collections of elements.
 - o **Slices:** Slices are similar to arrays but are dynamically sized and can hold elements of different types.
 - o **Maps:** Maps in Go are collections of key-value pairs where each key is unique. The key type must be comparable, and the value can be of any type.
 - o **Structs:** Structs are custom data types that hold and pass complex data structures, similar to classes in object-oriented programming languages.
 - o **Sets:** Sets are collections of unique elements, similar to Python's set data structure.
 - o **Queues:** Queues are synchronized **First-In-First-Out (FIFO)** collections of elements, similar to Python's queue data structure.

These data structures were covered in previous chapters. While Golang doesn't natively support advanced structures like heaps and linked hash maps, they are available in third-party libraries.

- **Reading and writing files:** Golang provides several ways to read and write files. Here are some ways:
 - o **Using OS package:** Functions for opening, reading, and writing files are provided by the OS package. For instance, you can use the **os.OpenFile()** and **os.Create()** functions to read and write data to and from files, respectively. Once you have a file object, you can read and write data to the file using the **ReadFile()** and **Write()** methods:

```
package main
```

```
import (
```

```
    "fmt"
```

```

        "os"
    )
    func main() {
        // Open the file
file, err := os.OpenFile("example.txt", os.O_RDWR|os.O_CREATE,
0755)
        if err != nil {
            fmt.Println("Error opening file:", err)
            return
        }
        defer file.Close()
        // Write to the file
        newData := []byte("Hello, world!")
        _, err = file.Write(newData)
        if err != nil {
            fmt.Println("Error writing to file:", err)
            return
        }
        // Read the file
        data, err := os.ReadFile("example.txt")
        if err != nil {
            fmt.Println("Error reading file:", err)
            return
        } else {
            fmt.Println("File content:", string(data))
        }
    }
}

```

- o **Using ioutil package:** There is an easier approach to read and write files with the **ioutil** package. To read a file, you can use the

ioutil.ReadFile() function, and to write to a file, you can use the **ioutil.WriteFile()** function.

The **ioutil** package is obsolete as of Go 1.16. Its functions like, **ioutil.ReadFile**, **ioutil.WriteFile**, and **ioutil.ReadDir**, are replaced with their equivalents in the **io** or **os** packages. The following are the new functions that should be used instead:

- **ioutil.ReadFile** -> **os.ReadFile** or **io.ReadAll**
 - **ioutil.WriteFile** -> **os.WriteFile**
 - **ioutil.ReadDir** -> **os.ReadDir**
- o **Using bufio package:** Buffered I/O operations for reading and writing files are provided by the **bufio** package. For instance, you can use **bufio.NewScanner()** function to read a file line by line and to write to a file you can use a **bufio.NewWriter()** function. Here is an example:

```
package main
import (
    "bufio"
    "fmt"
    "os"
)
func main() {
    // Create or open the file
    file, err := os.OpenFile("example.txt", os.O_RDWR|os.O_CREATE,
    0644)
    if err != nil {
        fmt.Println("Error opening file:", err)
        return
    }
    defer file.Close()
    // Create a buffered writer
```



```

writer := bufio.NewWriter(file)
// Write data to the file
_, err = writer.WriteString("Hello World!")
if err != nil {
    fmt.Println("Error writing to file:", err)
    return
}
// Flush the buffer to ensure all data is written to the file
writer.Flush()
// Reading from a file using bufio.NewScanner
readFile, err := os.Open("example.txt")
if err != nil {
    fmt.Println("Error opening file:", err)
    return
}
defer readFile.Close()
// Create a scanner to read from the file
scanner := bufio.NewScanner(readFile)
// Read the file line by line
for scanner.Scan() {
    fmt.Println(scanner.Text())
}
// Check for scanner errors
if err := scanner.Err(); err != nil {
    fmt.Println("Error scanning file:", err)
    return
}
}

```

- **Data manipulation/preprocessing:** Processing the data is a crucial

stage in machine learning and data analysis. You can use a variety of packages and techniques in Go to carry out data manipulation tasks like cleaning, transformation, and normalization.

- o **Data cleaning:** In Golang, data cleaning is the process of converting unprocessed data into a format that may be used for storage, analysis, or presentation. Cleaning the data involves handling missing values, removing duplicates, standardizing formats, mathematical operations, string manipulations, handling outliers and much more. Here is a basic example of how to clean the data:

```
func cleanData(data [][]string) [][]string {
    seen := make(map[string]struct{})
    var cleanedData [][]string
    for _, row := range data {
        // Merge data to create a unique identifier
        identifier := fmt.Sprintf("%v", row)
        // Check if the identifier has been seen before
        if _, ok := seen[identifier]; !ok {
            // If not seen, add to cleaned records and mark as
            seen

            // Skip empty rows
            if len(row) == 0 || (len(row) == 1 && row[0] == "")
            {
                continue
            }
            // Convert all values to lowercase and remove leading/trailing spaces
            cleanedRow := make([]string, len(row))
            for i, val := range row {
                cleanedRow[i] = strings.TrimSpace(strings.ToLower(val))
            }
            //Add to cleaned records
        }
    }
}
```

```

        cleanedData = append(cleanedData, cleanedRow)
        //Mark as seen
        seen[identifier] = struct{}{}
    }
}
return cleanedData
}

```

- o **Transformation:** In Golang, data transformation is the process of changing a data's format, structure, or representation to another. Golang comes with a number of built-in features and packages that make data transformation tasks easier to do. The following describes how to transform data in Golang:

```

package main
import (
    "encoding/xml"
    "fmt"
)
// Struct representing Person information
type Person struct {
    XMLName xml.Name `xml:"person"`
    Name    string `xml:"name"`
    Age     int   `xml:"age"`
    Location string `xml:"location"`
}
func main() {
    // Create an instance of the Person struct
    person := Person{Name: "Ashok", Age: 30, Location: "New
Delhi"}
    // Convert struct to XML

```

```

xmlData, err := xml.MarshalIndent(person, "", "  ")
if err != nil {
    fmt.Println("Error marshalling XML:", err)
    return
}
fmt.Println("XML representation:")
fmt.Println(string(xmlData))
// Convert XML back to struct
var newPerson Person
if err := xml.Unmarshal(xmlData, &newPerson); err != nil {
    fmt.Println("Error unmarshalling XML:", err)
    return
}
// Print the unmarshalled struct
fmt.Println("\nUnmarshalled struct:")
fmt.Println("Name:", newPerson.Name)
fmt.Println("Age:", newPerson.Age)
fmt.Println("Location:", newPerson.Location)
}

```

- o **Normalization:** Data normalization is the process of converting and normalizing a dataset's features or variables to a common scale or distribution. Larger scale or range variables may dominate the analysis and skew the findings. Normalizing the data puts all features on a level playing field. We have a number of methods for normalizing data, including decimal scaling, log transformation, Z-score standardization, and min-max scaling. Among all of these, we will talk about normalizing the data using min-max scaling, here is an example for the same:

```
package main
```

```

import (
    "fmt"
    "math"
)

// MinMaxScaler performs min-max scaling on a slice of floats
func MinMaxScaler(data []float64) []float64 {
    min, max := findMinMax(data)
    scaled := make([]float64, len(data))
    for i, val := range data {
        scaled[i] = (val - min) / (max - min)
    }
    return scaled
}

// Function to find min and max values in a slice of floats
func findMinMax(data []float64) (float64, float64) {
    min, max := math.Inf(1), math.Inf(-1)
    for _, val := range data {
        if val < min {
            min = val
        }
        if val > max {
            max = val
        }
    }
    return min, max
}

func main() {
    // Original data

```

```

data := []float64{100, 550, 280, 4000, 50}
// Min-Max scaling
scaledData := MinMaxScaler(data)
// Print original and scaled data
fmt.Println("Original Data:", data) // [100 550 280 4000 50]
fmt.Println("Scaled Data:", scaledData) // [0.012658227848101266
0.12658227848101267 0.05822784810126582 1 0]
}

```

In the preceding example, we have calculated the minimum (**min**) and maximum (**max**) values of the feature or variable across the entire dataset. It scales the data to a specified range, typically between 0 and 1, by ensuring that the minimum value in the dataset gets mapped to 0, the maximum value gets mapped to 1. For each data point x in the feature, the scaled value will be as follows:

$$x_{\text{scaled}} = (x - \min) / (\max - \min)$$

Exploratory Data Analysis in Go

In Go, the process of summarizing and displaying data to understand its characteristics and find underlying patterns is known as EDA. Even while Go might not have as many dedicated EDA libraries as Python, you can carry out EDA utilizing a variety of Go-based methods and packages. The following is a simple overview of EDA in Go:

- **Data loading and parsing:** Load your dataset into memory and parse it using various libraries, such as ‘**encoding/csv**’, ‘**encoding/json**’ for CSV/JSON file or. ‘**database/sql**’ for database interaction.
- **Descriptive statistics:** ‘**gonum/gonum/stat**’ library can be used for statistical computations as mean, mode, median, min, max, standard deviation, frequency count, etc.
- **Data visualization:** To identify patterns and trends, visualize your data using plots, charts, and graphs. Go includes a number of libraries that provide varying degrees of flexibility and complexity for data visualization like **gonum/plot** or **gonum/plot/plotter**.

- **Correlation analysis:** It is a statistical method for determining the direction and strength of a relationship between two variables. Several statistical packages can be used to carry out correlation analysis, such as **gonum/stat/corr** or **github.com/montanaflynn/stats**.
- **Missing values and outlier detection:** There are several approaches to deal with missing values, like removing rows or columns containing missing values, estimating missing values based on the values of nearby data points or substituting missing values with the mean, median, mode, or a value predicted by a regression model. Identify outliers in your dataset using statistical methods like Z-score, **Interquartile Range (IQR)**, or visualization techniques such as box plots.
- **Time series analysis:** You can use libraries like ‘**github.com/lytics/anomalyzer**’ or ‘**github.com/robaho/go-trader**’ to perform time series analysis techniques like trend analysis, seasonality decomposition, and forecasting if your dataset contains time-series data.
- **Data transformation and feature engineering:** Libraries like **gonum/mat** can help with matrix operations and transformations. There are few libraries as ‘**gota**’ and ‘**gdataframe**’ that offers data manipulation functionalities similar to Pandas in Python.

Let us take a simple example of a CSV dataset file named **dataset.csv** that we can use for the EDA example. Let us assume a dataset with two columns: “Age” and “Salary”, as shown in the following table:

Age	Salary
25	50000
30	55000
32	60000
34	62000

37	65000
40	70000
0	42000
45	75000
48	10000000000
50	90000

Table 22.1: *dataset.csv*

In the following example, we are parsing the dataset from a CSV file and preparing it for analysis by converting it into a slice of float64. A descriptive statistics calculation is done such as mean, median, variance, and standard deviation. We visualize the data using a histogram, perform correlation analysis, count missing values, and detect outliers:

```
package main
```

```
import (
    "encoding/csv"
    "fmt"
    "log"
    "os"
    "sort"
    "strconv"
    "strings"
    "gonum.org/v1/gonum/mat"
    "gonum.org/v1/gonum/stat"
    "gonum.org/v1/plot"
    "gonum.org/v1/plot/plotter"
```



```

    "gonum.org/v1/plot/vg"
)
func main() {
    // Step 1: Parse the dataset
    file, err := os.Open("dataset.csv")
    if err != nil {
        log.Fatal(err)
    }
    defer file.Close()
    reader := csv.NewReader(file)
    reader.FieldsPerRecord = -1
    records, err := reader.ReadAll()
    if err != nil {
        log.Fatal(err)
    }
    // Step 2: Prepare the data for analysis
    var data []float64
    for _, record := range records {
        for _, value := range record {
            value = strings.TrimSpace(value)
            val, err := strconv.ParseFloat(value, 64)
            if err == nil {
                data = append(data, val)
            }
        }
    }
    // Step 3: Descriptive statistics calculation
    //stat.Quantile function requires the input data to be sorted

```

```

sort.Float64s(data)

mean := stat.Mean(data, nil)

median := stat.Quantile(0.5, stat.Empirical, data, nil)

variance := stat.Variance(data, nil)

stdDev := stat.StdDev(data, nil)

fmt.Printf("Mean: %.2f\n", mean)

fmt.Printf("Median: %.2f\n", median)

fmt.Printf("Variance: %.2f\n", variance)

fmt.Printf("Standard Deviation: %.2f\n", stdDev)

// Step 4: Data visualization (Histogram)

p := plot.New()

histogram, err := plotter.NewHist(plotter.Values(data), 16)

if err != nil {
    log.Fatal(err)
}

p.Add(histogram)

p.Title.Text = "Histogram"

p.X.Label.Text = "Values"

p.Y.Label.Text = "Frequency"

err = p.Save(4*vg.Inch, 4*vg.Inch, "histogram.png")

if err != nil {
    log.Fatal(err)
}

// Step 5: Correlation analysis (Dummy example)

// Create a dummy correlation matrix

correlationMatrix := mat.NewDense(2, 2, []float64{1.0, 0.5, 0.5, 1.0})

fmt.Println("Correlation Matrix:")

fmt.Printf("%v\n", mat.Formatted(correlationMatrix))

```

```

// Step 6: Missing values handling (Dummy example)

// Check for missing values
missingValues := 0
for _, val := range data {
    if val == 0 { // Assuming 0 indicates missing value
        missingValues++
    }
}

fmt.Printf("Number of missing values: %d\n", missingValues)

// Step 7: Outlier detection (Dummy example)
outliers := make([]float64, 0)
for _, val := range data {
    if val > 3*stdDev { // Assuming anything beyond 3 standard deviations is an outlier
        outliers = append(outliers, val)
    }
}

fmt.Printf("Outliers: %v\n", outliers)
}

```

Output:

Mean: 500028467.05

Median: 50.00

Variance: 4999970035809240064.00

Standard Deviation: 2236061277.29

Correlation Matrix:

$$\begin{bmatrix} 1 & 0.5 \\ 0.5 & 1 \end{bmatrix}$$

Number of missing values: 1

Outliers: [1e+10]

You may efficiently analyze and understand datasets in Go by combining

these methods and resources, which will allow you to get insightful information and make justifiable judgments based on your data.

Machine learning ecosystem with Go

Although Golang's machine learning ecosystem is not as developed or wide-ranging as that of languages like Python, there are still libraries and tools available that enable machine learning development and deployment. Based on the characteristics of the learning process and the availability of labeled data, machine learning may be roughly divided into the following three basic types:

- **Supervised learning:** Supervised learning is a type of machine learning paradigm where the algorithm learns from labeled data, which means the input data is paired with corresponding output labels or target variables. The goal of supervised learning is to learn a mapping from input variables to output variables based on the labeled examples provided during training. Here is how supervised learning works:
 - o **Step 1:** An algorithm is used with a training dataset consisting of input-output pairs. The algorithm learns to map the input features to the output labels by adjusting its internal parameters during the training process.
 - o **Step 2:** During the training process, an algorithm prepares a model, which can be a function or set of rules that defines the relationship between the input features and the output labels.
 - o **Step 3:** The supervised learning algorithm uses optimization techniques and adjusts model parameters to minimize the difference between the predicted output values generated by the model and the actual output labels in the training data.
 - o **Step 4:** Once the model is trained on the labeled dataset, it can be used to make predictions on new, unseen data.

Depending on the type of output variable, supervised learning can be further divided into two primary categories:

- o **Classification:** Classification involves predicting a categorical label

for a given input, such as email spam detection, sentiment analysis, and image classification. Some popular classification algorithm used in GO includes Logistic Regression, Naïve Bayes, **K-Nearest Neighbors (KNN)**, Decision Trees, and **Support Vector Machines (SVM)**.

- o **Regression:** Regression involves predicting a continuous value for a given input like house prices, size, location, number of bedrooms, stock prices, etc. Some popular regression algorithms used in supervised learning, which can also be implemented in Go, include Linear Regression, Logistic Regression, Decision Trees, Random Forest, and Gradient Boosting.
- **Unsupervised learning:** Unsupervised learning is a type of machine learning where the algorithm is trained on unlabeled data, meaning the data does not have any pre-existing labels or categories. The goal of unsupervised learning is to discover patterns and relationships in the data without any human intervention. Unsupervised learning is often used in various applications, such as customer segmentation, fraud detection, and image analysis. Some common unsupervised learning techniques include:
 - o **Clustering:** Here, algorithm groups similar data points together into clusters or segments based on their feature similarity. Several well-known clustering techniques for unsupervised learning are k-means clustering, **Density-Based Spatial Clustering of Applications with Noise (DBSCAN)**, Gaussian Mixture Model, and hierarchical clustering.
 - o **Dimensionality reduction:** It reduces the number of input features while preserving as much of the relevant information as possible. Some popular dimension reduction algorithms are **Principal Component Analysis (PCA)**, **t-Distributed Stochastic Neighbor Embedding (t-SNE)** and others.
 - o **Anomaly detection:** It involves identifying data points that do not fit into any group or cluster. Algorithms used for detecting anomalies

are DBSCAN, Isolation Forest, K-Means, SVM, and **Local Outlier Factor (LOF)** are algorithms.

- **Reinforcement learning:** RL is a type of machine learning that focuses on training agents to make decisions in an environment to maximize a reward or minimize a cost. The agent learns through trial and error, receiving feedback in the form of rewards or punishments for its actions. Some key aspects of RL include the following:
 - **Agent:** An entity interacting with the surroundings is called an agent. It chooses what to do after observing the situation of the environment and getting feedback in the form of rewards or penalty.
 - **Environment:** The external system or situation that an agent interacts with is referred to as the environment.
 - **State:** The state represents the configuration or collection of all relevant information of the environment necessary for the agent to make decisions.
 - **Policy:** The policy defines the strategy or set of rules that govern the agent's decision-making process.
 - **Action:** Actions are the decisions or choices made by the agent at each time step.
 - **Rewards:** Rewards are numerical values provided by the environment to indicate the desirability of the agent's actions. To achieve this goal, the agent typically uses exploration-exploitation strategies to balance between exploring new actions and exploiting known actions that lead to high rewards.

Applications for RL can be found in many fields, such as resource management, autonomous systems, gaming, robotics, and recommendation systems. Popular algorithms for RL are Q-Learning, **Deep Q-Networks (DQN)**, Actor-Critic Methods, and Policy Gradient Methods.

Golang machine learning libraries

Go provides a number of machine learning libraries that they can integrate

in their solutions, even though it has a far smaller collection of tools and libraries than other languages that have been around for decades and have well-established ecosystems and libraries for ML. When coupled with Go's functionalities, the following libraries can facilitate quick, robust, and effective machine learning systems:

- **GoLearn:** Golearn is an open-source machine learning library offering tools for clustering, regression, classification, and data management. Similar to Python's Scikit-learn, it supports cross-validation, test splitting, and data splitting.
- **GoML:** The GoML library allows developers to integrate machine learning into their Go applications. It offers models like text classification, perception, clustering, and linear models, along with features for easy integration, including online and reactive learning.
- **Gonum:** Gonum is a scientific library inspired by NumPy and MATLAB, offering functions for linear algebra, statistics, optimization, and more. It supports vectors, matrices, and operations like multiplication and decomposition. Gonum is useful for deploying ML models in production or creating ML APIs for other applications.
- **Gorgonia:** Gorgonia, part of the Gonum package, provides a framework for exploring neural networks and advanced deep learning techniques like evolutionary algorithms and neo-Hebbian learning. Together with Gonum, Gorgonia is a valuable tool for scientific and machine learning computations in Go.
- **GoBrain:** The **gobrain** library offers the ability to build and train neural networks for a range of machine learning applications. Gomind is another library supported by Go, designed for neural networks.
- **Hector:** The goal of the Hector library is to implement all well-known Golang machine learning algorithms. It is compatible with all algorithms, that can resolve binary classification issues, including neural networks, random forests, Logistic Regression, and others.
- **libSVM:** "github.com/ewalker544/libsvm-go," is a complete conversion of LIBSVM in Go, provides the **svm-train** and **svm-predict** command line tools. These packages let programmers use the libSVM library directly for classification and regression tasks in their

Golang applications.

- **go-cluster**: The **go-cluster** package implements clustering algorithms like DBSCAN, OPTICS, and k-means++. It includes tools to estimate optimal cluster numbers and import data, offering high-performance support within Go's growing ML ecosystem.
- **GoBot**: The "GoBot" is a Go-based framework for physical computing, robotics, and the Internet of Things. GoBot, though not quite a machine learning library, offers support in interacting with physical devices and sensors that are often used in machine learning applications and experiments.

Time series analysis

Time series analysis in machine learning involves modeling and forecasting data collected over time. Common sources include finance, economics, weather, and signal processing. The choice of models and methods depends on the data's characteristics and analysis goals.

Following is a simplified example of time series analysis for stock price prediction implemented in Go, where we are using the **gonum** package's linear regression:

```
package main

import (
    "fmt"
    "gonum.org/v1/gonum/mat"
    "gonum.org/v1/gonum/optimize"
)

func main() {
    // Sample time series data for the last month (assuming daily data)
    // For example, you might fetch data from an API or a CSV file
    x := []float64{1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25,
        26, 27, 28, 29, 30}
    y := []float64{143.0, 143.2, 142.4, 142.6, 142.8, 143.5, 141.0, 141.4, 142.5, 142.3, 142.5, 143.0,
```


143.3, 143.4, 143.7, 144.0, 144.6, 144.3, 144.5, 144.6, 144.2, 144.6, 144.2, 144.5, 144.2, 144.7,
145.2, 145.0, 145.2, 144.8}

```
// Create the design matrix
X := mat.NewDense(len(x), 2, nil)
for i := 0; i < len(x); i++ {
    X.Set(i, 0, 1) // Bias term
    X.Set(i, 1, x[i]) // Feature
}
// Create the target vector
Y := mat.NewVecDense(len(y), y)
// Initial the parameters
theta := []float64{0, 0}
// Define the linear regression objective function
objective := func(theta []float64) float64 {
    thetaVec := mat.NewVecDense(len(theta), theta)
    var diff mat.VecDense
    diff.MulVec(X, thetaVec)
    diff.SubVec(&diff, Y)
    return mat.Dot(&diff, &diff)
}
// Create the problem for optimization
problem := optimize.Problem{
    Func: objective,
}
// Initialize optimization settings
settings := &optimize.Settings{}
// Minimize the objective function using the default BFGS method
result, err := optimize.Minimize(problem, theta, settings, nil)
if err != nil {
```

```

        fmt.Println("Error:", err)

        return
    }

    // Get the optimized parameters
    thetaOpt := mat.NewVecDense(len(result.X), result.X)

    // Predict the stock price for tomorrow
    tomorrow := 31.0 // Assuming the next day

    prediction := thetaOpt.At(0, 0) + thetaOpt.At(1, 0)*tomorrow

    fmt.Printf("Predicted stock price for tomorrow: %.2f\n", prediction)
}

```

Output:

Predicted stock price for tomorrow: 145.27

In the preceding example, we generated matrices and vectors for our data and model parameters using ‘**gonum/mat**’ package. An objective function for linear regression is given to compute the mean squared error between the actual and predicted values.

Create a **optimize.Problem** object that encapsulates your objective function.

The **Optimize.Minimize** function is used to minimize the objective function using the given setting and method. Here, we have passed nil as a method that indicates default optimization method which is usually the BFGS method. On the other hand, you can specifically specify an instance of the desired optimization technique if you have a special criteria or if you wish to utilize a different optimization method.

The stock price for the following day is then predicted using the optimized parameters.

Note: This is a very basic example and may not yield reliable results. Predicting stock prices in the real world requires feature engineering and highly developed models. Additionally, when creating predictive models for financial markets, it is critical to take into account elements like risk management, model evaluation, and data preprocessing.

Conclusion

This chapter covered key topics such as machine learning and data science fundamentals, their applications, and data manipulation techniques. We discussed performing EDA in Go and examined the machine learning ecosystem and available libraries. The chapter also introduced time series analysis, showcasing how Go can be effectively utilized for various data science tasks.

Questions

1. How do we read and write in a file with Golang?
2. What is the purpose of gonum package in Go?
3. What are the application areas of machine learning?
4. What is meant by “time series analysis”?
5. Which libraries are supported for data visualization in Golang?
6. What is the difference between supervised and unsupervised learning?

Join our book’s Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

<https://discord.bpbonline.com>



CHAPTER 23

Build CI/CD pipeline with Golang

Introduction

The incorporation of **Continuous Integration and Continuous Deployment (CI/CD)** pipelines has become a fundamental practice in today's dynamic software development landscape, where agility, security, and speed are essential. Golang has grown in popularity as a language for creating scalable and reliable systems because of its ease of use, performance, and concurrency features. In this chapter, we are going to discover ideas, approaches, and best practices for creating efficient CI/CD pipelines in Golang.

Structure

In this chapter, we will cover the following topics:

- Introduction to CI/CD
- Golang project structure
- Setting up CI/CD environment
- Integration of CI/CD in Go
- Deployment strategies for Golang applications
- Monitoring and logging

Objectives

In this chapter, you will gain familiarity with the concept of CI/CD in context of Golang projects. You will understand step-by-step instructions to define the build, test, and deployment stages. Additionally, the chapter introduces essential monitoring and logging practices to effectively track pipeline performance and automated testing strategies for Golang applications.

Introduction to CI/CD

Automating and optimizing the software development lifecycle is the goal of the CI/CD methodology. CI/CD pipelines consist of a series of phases, such as building, testing, and deploying code changes. Here is a concise overview of CI/CD:

- **Continuous Integration (CI):** CI involves frequently integrating code changes into a shared repository and automated tests, including unit tests, integration tests, and other types of tests, which are carried out to validate the merged code changes. In order to maintain the stability and dependability of the codebase, CI aims to identify and resolve integration problems early in the development process.
- **Continuous Delivery (CD):** CD is the process of automatically deploying code changes to a staging or production environment once changes pass all automated tests in the CI phase. The deployment process is managed by automated deployment pipelines and might involve stages like creating artifacts, carrying out extra testing, and delivering to different environments. The fundamental advantage of CD is that customers can get software updates in a timely, effective, and reliable manner while keeping a high standard of quality.
- **Continuous Deployment (CDep):** In software development and DevOps, CD and CDep are related ideas; however, they differ in the level of automation and control over the deployment process. With CDep, code changes are automatically deployed to production without human intervention as soon as they pass the required tests.

The tools Jenkins, Spinnaker, GoCD, Concourse, and GitLab are among

those that supports the CI/CD process. To summarize, continuous integration and continuous delivery, or CI/CD, is an essential collection of procedures that improves code quality, speed up feedback, lower deployment risk, enhance teamwork, boost productivity, and streamline development workflows.

Golang project structure

Learning the CI/CD pipeline with Golang usually focuses on structuring your Golang codebase in a way that follows best practices for scalability, maintainability, and simplicity of integration with CI/CD technologies. An example of a typical Golang project structure is as follows:

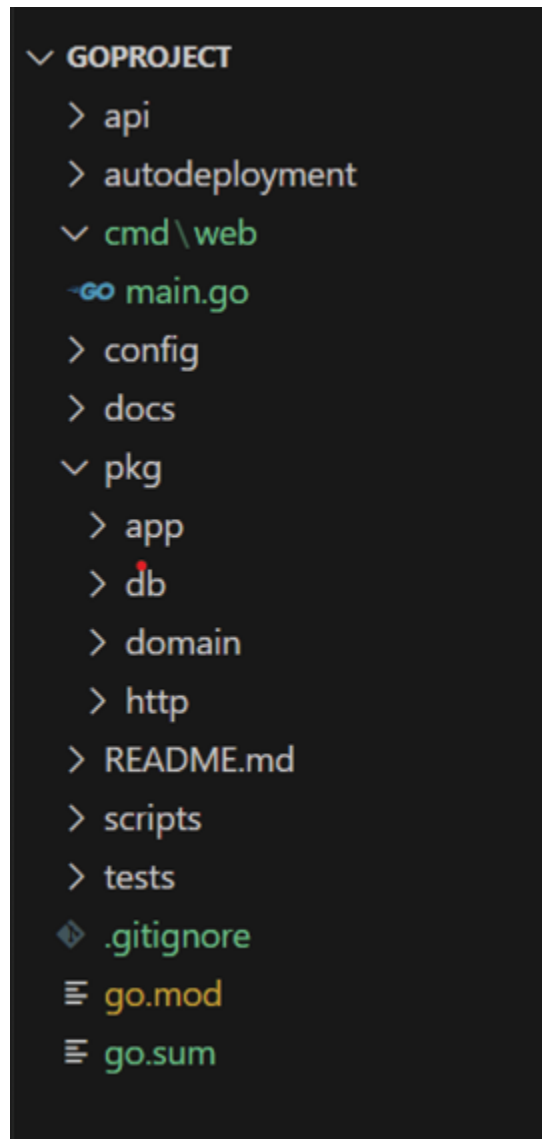


Figure 23.1: Golang project structure

The significant folders and files are broken down as follows:

- **cmd/**: It represents the main entry points for your Golang project, where each subfolder under **cmd/** corresponds to a specific application or service.
- **api/**: It includes code for creating HTTP APIs, including handlers, middleware, and other utilities.
- **autodeployment/**: This directory contains deployment files and scripts like Dockerfiles, Kubernetes manifests, and configuration files to streamline the application's deployment process.
- **config/**: It includes application configuration files with environment-

specific defaults and configurations.

- **docs/:** Documentation files such as design documents, or user guides can be kept here.
- **pkg/:** This directory holds reusable libraries with well-defined APIs, designed to be independent of your project's specifics.
- **scripts/:** It holds the scripts or utilities for your project's testing, development, and other uses.
- **tests/:** You can put test suites and test files in this directory. Correctness and stability of your codebase is possible with extensive test coverage.
- **README.md:** To assist other developers, users, or contributors, this file contains project-related information, guidelines, and documentation.
- **.git:** This hidden directory stores Git's version control data, including commits, branches, tags, and project history metadata.
- **.gitignore:** The **.gitignore** file specifies files, directories, or patterns (e.g., **.exe**, **.cache**, **bin/**) for Git to ignore in version control.
- **go.mod:** Go 1.11 introduced **go.mod** file to manage dependencies and versioning of Go modules within Golang project.
- **go.sum:** The **go.sum** file contains checksums for modules in **go.mod**, ensuring the integrity of downloaded module versions.

This project structure supports modular development and clear separation of concerns, simplifying maintenance, scaling, and integration with CI/CD processes for targeted testing and deployment.

Setting up CI/CD Environment

To setup CI/ CD environment for a Golang project, you have to configure CI/CD pipelines to automate building, testing and code quality checks. You can select appropriate CI/CD tool out of Jenkins, GitLab, Travis CI, Circle CI, GitHub Actions and more. Here, we are going to use Jenkins to host and execute CI pipelines. The basic steps to configure Jenkins for a Golang project are as follows:

1. **Install Jenkins:** Jenkins supports various installation methods across

operating systems, including package managers like apt or yum for Red Hat/CentOS and Debian/Ubuntu, or downloading the WAR or MSI files from its official site for easy setup.

2. **Accessing Jenkins:** Once Jenkins is installed and operational, open your web browser and go to **http://localhost:8080**, or the relevant port/IP address, if you are accessing it remotely.
3. **Install Required Plugins:** Further, you will need to install plugins on Jenkins, such as Go, Git, Artifactory, and others, that are necessary for developing and testing Golang applications. On Jenkins web interface, navigate to **Manage Jenkins | Manage Plugins | Available** tab, search and install the required plugin.
4. **Configure Jenkins Global tools:** In '**Manage Jenkins**', the '*Global Tool Configuration*' option lets you globally configure tools like compilers, interpreters, and build systems. You can add a Go installation, specify the version, and set parameters like directories or environment variables, then save the changes.
5. **Configure Jenkins credentials:** You might need to set up credentials in Jenkins and use them safely in your build scripts if your Golang project needs access to outside services (like Docker Hub or AWS).
6. **Create a new Jenkin Job:** A Jenkins job is a specific task in your CI/CD pipeline that Jenkins executes, such as freestyle, pipeline, or multi-configuration jobs.

To create a job, click **New Item** on the dashboard, name the job, choose its type, and go to the configuration page. Define the steps Jenkins needs to execute (e.g., compilation, testing, deployment), specify job parameters, version control, repository details, and post-build actions like archiving artifacts or triggering downstream jobs. After configuring the settings, save the job by clicking **Save**.

7. **Run the job:** You can now wait for the configured trigger to start the job execution, or you can directly start the job by clicking the **Build Now** or **Build with Parameters** button.

These steps lay the foundation for setting up a CI/CD environment and creating a Jenkins job. The specific configuration will depend on your requirements and the tools and services you use.

Integration of CI/CD in Go

To configure a Jenkins CI/CD pipeline in a Go project, define the stages and steps in a Jenkinsfile. You can create subdirectories in your project for Jenkins configuration files like **.yaml** and Jenkinsfile, which outline the pipeline steps, typically including the following:

- **Checkout:** This step retrieves source code from the repository from version control.
- **Build:** As part of the build process, a build step specifies a particular task that must be executed. This could involve creating documentation, packaging artifacts, executing tests, assembling source code, distributing software, and so on.
- **Linting and static analysis:** Linting and static analysis involve tools that examine code for errors, style issues, security flaws, and optimizations without executing it. Jenkins integrates various tools for this, including SonarQube, ESLint for JavaScript, PMD, and goLint for Go, via command-line execution or plugins.
- **Automated testing:** This stage validates the product's functionality by running various tests (unit, integration, end-to-end) on the latest code changes using the 'go test' command. Plugins like JUnit or xUnit can publish test results in Jenkins, especially for testing frameworks that generate XML reports.
- **Code coverage:** This step includes code coverage analysis and generate code coverage report. It helps developers identify parts of the code that do not have sufficient test coverage, enabling them to write more tests to improve the software's overall quality and reliability.
- **Artifact generation:** After developing and testing your Go project, you can generate artifacts (e.g., binaries, config files, documentation) for publishing or deployment. Use Jenkins' **archiveArtifacts** step to archive these files, making them available for download or further processing on the Jenkins webpage.
- **Deployment:** The deployment stage covers deploying the tested application to a target environment, such as a cloud platform or server, and may include tasks like database migrations and service restarts. A

rollback mechanism is recommended to revert to a previous version in case of deployment failure or issues.

- **Release orchestration:** This stage may involve a number of release-related activities, including packaging the release artifacts, deploying the release to target environments, and marking the release in version control. In order to automate deployment activities, you may also integrate with configuration management tools like Ansible or packaging technologies like Docker to produce container images.
- **Feedback and notification:** It includes notifying stakeholders and team members on test results, build status, and other important CI/CD pipeline information. To enable notifications via email, chat platforms, and other channels, Jenkins offers a variety of plugins such as the Email Extension, Slack Notification, and Microsoft Teams Notification, etc.

You must construct a Jenkinsfile at the project root in order to configure a Jenkins pipeline for a Golang project. Below are a few examples of steps that can be created:

```
pipeline {
    agent any
    stages {
        stage('Checkout') {
            steps {
                git 'your_repository_url' // Checkout source code from version control
            }
        }
        stage('Unit Tests') {
            steps {
                sh 'go test ./...' // Run unit tests
            }
        }
        stage('Code coverage') {
```

```

steps {
    sh 'go tool cover -html=coverage.out -o coverage.html' // Publish code coverage report
    publishHTML(target: [
        alwaysLinkToLastBuild: false,
        reportFiles: 'coverage.html' ])
    }
}

stage('Deploy') {
    steps {
        // Example: Deploy to a staging server
        sh 'scp your_binary user@staging_server:/path/to/deployment/directory/'
        // Additional steps such as restarting services or running migrations can be added here
    }
}

post {
    always {
        emailx ( // Send email notification on build completion
            subject: "Jenkins Build Notification: ${currentBuild.result}",
            body: "Build ${currentBuild.result}",
            to: "recipient@example.com")
        cleanWs() // Cleanup or other post-build actions
    }
}
}

```

To summarize, Jenkinsfile act as blueprints that define and automate the procedures associated with building, testing, and deploying Go projects within Jenkins pipelines. They are essential in facilitating CI/CD processes, improving teamwork, and ensuring pipeline reliability and consistency for

software delivery.

Integrating automated testing into CI/CD pipelines

The common testing frameworks and libraries used in Go projects, are **Ginkgo**, **testify**, **gocheck**, and the built-in **testing** package. Developers are free to select the Go testing framework or tool that best suits their needs and tastes, as each one has unique capabilities and testing methodologies. The following are some points that could be considered as a path when incorporating automated testing into a Go project:

- **Writing tests:** Make sure your Golang project has adequate test coverage before configuring CI/CD. Go comes with built-in support for the testing package. Tests can be written in files that ends with **_test.go**. The following is a simple test file example:

```
// add_test.go

package main

import (
    "testing"
)

func TestAdd(t *testing.T) {
    result := Add(1, 2)

    if result != 3 {
        t.Errorf("Add(1, 2) = %d; want 3", result)
    }
}
```

- **Test suites:** By combining related test cases into a single package, test cases can be arranged more efficiently. The **TestMain** method in the Go **testing** package lets you specify the setup and teardown logic for your test suite.

In the following code, test suite's starting point is the **TestMain** function. We can use **m.Run()** to run each test inside **TestMain**. This runs every test function specified within the package:

```
// calculator_test.go

package calculator

import (
    "testing"
)

func TestAdd(t *testing.T) {
    // Test cases for Add function
    // ...
}

func TestSubtract(t *testing.T) {
    // Test cases for Subtract function
    // ...
}

// This function is executed before any tests in the suite are run.
func TestMain(m *testing.M) {
    // Add setup logic, like initializing resources or the environment.
    // Run all tests
    exitCode := m.Run()

    // Teardown logic can go here if needed like clean up resources,
    restore test environment

    // Exit with the status of the test suite run
```

```
os.Exit(exitCode)
```

```
}
```

- **Parallel testing:** By default, Go's **testing** package runs tests in parallel across multiple CPU cores, with each test function in the same package running independently. This accelerates test execution, benefiting large projects. The **-p** flag lets you limit parallel runs if issues arise.
- **Enhancing test coverage and quality metrics:** Follow recommended standards and naming conventions to keep tests clear and manageable. Set minimum coverage levels to ensure the new code maintains or improves test coverage.

Code coverage reports can be generated with the built-in **go test** tool, which has the **-cover** flag enabled. You can get coverage statistics for the entire project with the **go test -cover** command. Apart from this, to enforce code style and best practices, you can integrate static analysis tools such as **golint**, **go vet**, and **gofmt** into your continuous integration and deployment process.

- **Mocking dependencies:** When you replace genuine dependencies during testing with simulated objects, or mocks, this is known as mocking dependencies in Go. Testing code modules that depend on other resources like databases, APIs, or external services is a great use case for this methodology. The **go.uber.org/mock/mockgen** library is great for generating mock implementations from interfaces and streamlining testing.

You may define a relation between your code and its dependencies in Go by using interfaces. Next, for testing purposes, you develop simulated implementations of these interfaces.

Here is a basic example:

```
// service.go
```

```
package service
```

```
type DataFetcher interface {
```

```
    FetchData() (string, error)
```

```

}

type MyService struct {
    DataFetcher DataFetcher
}

func (s *MyService) ProcessData() (string, error) {
    data, err := s.DataFetcher.FetchData()

    if err != nil {
        return "", err
    }

    // Process data...

    return data, nil
}

```

In the preceding code, **MyService** contains a field of type **DataFetcher**. For testing purpose, we can create a mock implementation of **DataFetcher**:

```

// mock_datafetcher.go

package service

type MockDataFetcher struct{}

func (m *MockDataFetcher) FetchData() (string, error) {
    // Simulate data fetching behavior

    return "Mocked Data", nil
}

```

You can make instances of your mocks in your test files and introduce them into the code being evaluated:


```
// service_test.go

package service_test

import (
    "testing"

    "your-package-path/service"
)

func TestMyService_ProcessData(t *testing.T) {
    mockDataFetcher := &service.MockDataFetcher{}

    myService := service.MyService{DataFetcher: mockDataFetcher}

    result, err := myService.ProcessData()

    // Perform assertions on the result and error
}
```

Mocks let you play out various scenarios, like successful data fetching, network problems, or timeout situations. This enables you to fully test how your code behaves in different scenarios.

- **Handling test failures:** Finding the main reason for a test's failure is essential. To find out why the test failed, examine the error messages, stack traces, and test output that the **testing** framework has given. Test failures can be reported with informative error messages by using the testing package's functions, such as **t.Errorf()**.

Take advantage of built-in Go debugger (**delve**) or external debuggers to step through your code and examine variables and states while the test is executing.

In summary, by using the Golang '**testing**' framework and incorporating automated testing techniques into your CI/CD workflows, you may streamline development processes, identify problems early, and deliver stable software releases.

Deployment strategies for Golang applications

The deployment strategies are methods for releasing and upgrading software in production. They ensure smooth transitions, minimize downtime, and reduce errors. The choice of strategy depends on the application's needs, infrastructure, and team preferences, each offering unique benefits.

Some common deployment strategies for Golang applications with CI/CD pipelines are:

- **CD:** In the CD release process, code changes are automatically deployed to production after passing automated tests and validations. Rollback techniques are included to revert to earlier versions if issues arise, ensuring the application remains accessible and functional.
- **Blue-green deployment:** Maintaining two identical environments is the task of blue-green deployment: the “green” environment is idle, and the “blue” environment is currently handling live traffic. Using CI/CD tools, blue-green deployment operates as follows:
 - o Developers use CI/CD to merge code changes into the main repository, running automated tests to ensure functionality and quality.
 - o A code merge, tag, or event triggers the CI/CD pipeline, preparing the new version of the application for deployment.
 - o In next step, CI/CD tool deploys the updated version of the application to the idle “green” environment rather than the live environment directly.
 - o Automated tests and validations are carried out once the new version of the program is deployed to the “green” environment to make sure it fulfills quality standards and operates as intended.
 - o Once the updated application passes tests, the load balancer gradually redirects user traffic from the “blue” to the “green” environment, either all at once or gradually, based on risk tolerance and requirements.

- If issues arise in the “green” environment, traffic can quickly be redirected back to the stable “blue” environment to minimize downtime and user impact.
- After validating the new version in the “green” environment, the “blue” environment is updated with the latest changes for the next deployment cycle.
- **Canary deployment:** Canary deployment, often used with CI/CD systems, reduces risks by releasing the updated application to a small user group or subset of servers, known as the “canary group.” The new version is gradually exposed to a small portion of traffic, allowing real-world testing. Once verified, the update is fully deployed across the production environment.
- **Rolling deployment:** Rolling deployment gradually replaces old instances with new ones, starting with a small portion of servers. Once verified, the new version is rolled out to more servers. A load balancer or proxy directs traffic between the old and new versions during the process.
- **A/B testing:** A/B testing involves releasing different versions of an application to determine which performs better. Users are randomly assigned to versions, and CI/CD tools help deploy them to various groups. By tracking KPIs like conversion rates or engagement, you can evaluate performance and select the best version.
- **Feature flag:** Feature flags allow you to deploy changes without immediately exposing them to all users, enabling runtime toggling of new features. Managed through a dashboard or configuration system, they let developers and stakeholders toggle functionality without altering code. Feature flags can also be used with A/B testing to compare feature variations across user segments.

Monitoring and logging

To ensure the reliability, effectiveness, and security of your software delivery pipeline, monitoring and logging in CI/CD solutions are essential. Here is how CI/CD for Golang projects could incorporate with monitoring

and logging:

- **Monitoring:** With CI/CD tools, monitoring entails tracking of availability, performance, and overall health of the CI/CD server as well as the services and application it deploys. The following are a few approaches to implementing monitoring:
 - **Infrastructure monitoring:** Infrastructure monitoring in CI/CD pipelines involves tracking components like servers, networks, and databases to ensure stability before making changes. Use APIs, plugins, or tools like Zabbix or Nagios to gather and analyze metrics within the CI/CD workflow.
 - **Application Monitoring:** To keep an eye on your Golang application's important metrics and performance indicators, use monitoring tools like Prometheus, Grafana, or Datadog. Response times, error rates, CPU and memory consumption, and other key performance indicators are a few examples of these measures.
 - **Custom metrics and dashboards:** CI/CD solutions provide customizable dashboards and reports to track trends, patterns, and key performance metrics. Custom metrics, charts, and alerts help monitor critical components, and dashboards can be tailored for specific stakeholders like operations teams, analysts, or developers.
 - **Alerting and notification:** During the CI/CD workflow, it facilitates the quick identification of problems, errors, or anomalies and allows for quick action and resolution. Notification channels can be set up to receive notifications through email, SMS, and chat services (Microsoft Teams, Slack, etc.). Organize alerts into several categories (e.g., informational, warning, and critical) according to how they might affect the pipeline for continuous integration and delivery.
 - **Incident management systems:** In order to automate problem detection, triage, escalation, and resolution process, several CI/CD platforms integrate with incident management systems such as PagerDuty, OpsGenie, or ServiceNow. During incidents and outages,

development, operations, and support teams can collaborate more easily when incident management systems are integrated. When an alert is generated during the CI/CD pipeline, it automatically creates incident tickets, assigns responsibilities, and monitors the resolution process.

- o **Security monitoring and Compliance:** CI/CD pipelines can incorporate security monitoring and compliance in the following ways:
 - o Automated security testing of deployed application can be done by integrating **Dynamic Application Security Testing (DAST)** tools into the CI/CD process. By checking running apps for security flaws like SQL injection, **cross-site scripting (XSS)**, and unsafe configurations, DAST tools simulate real-world incidents and vulnerabilities.
 - o In addition, a variety of dependency scanning tools are available to help find and fix security flaws in third-party libraries and dependencies that the application uses. Dependencies can be automatically scanned for known vulnerabilities using tools like **Open Web Application Security Project (OWASP)** dependency-check and snyk, which can also suggest repairing or upgrading susceptible components.
 - o Conduct evaluations and audits regularly to ensure adherence to standards including **System and Organization Controls 2 (SOC 2)**, **Health Insurance Portability and Accountability Act (HIPAA)**, **Payment Card Industry Data Security Standard (PCI DSS)**, and **General Data Protection Regulation (GDPR)**.
 - o Container images can be scanned for vulnerabilities, misconfigurations, and compliance breaches using tools like Clair, Anchore, and Twistlock.
 - o Handle and store private information, API keys, and other secrets used in the CI/CD pipeline effectively. A secure way to distribute

secrets to CI/CD processes could involve using HashiCorp Vault, AWS Secrets Manager, or Azure Key Vault.

- **Logging:** CI/CD logging tools collect and analyze log data from builds, tests, deployments, and runtime environments, aiding in problem diagnosis and performance monitoring. Here's how to implement logging:

- o **Integration with logging solutions:** CI/CD systems can integrate with logging solutions to collect and organize logs from applications, infrastructure, and pipeline components. Key factors in choosing tools include scalability, ease of integration, and retention policies. Examples include Splunk, Sumo Logic, Fluentd, ELK Stack, and CloudWatch Logs, which offer search, indexing, collection, analysis, and visualization features.

- o **Structured and centralized logging:** Structured logging uses key-value pairs (e.g., JSON or XML) to organize log messages in a machine-readable format, making it easier to search, filter, and detect issues. These logs include important context, such as timestamps and severity levels, providing insights into app behavior. Tools like Elasticsearch, Splunk, and Graylog help query structured logs.

Centralized logging aggregates log data from multiple sources (applications, microservices, containers, and infrastructure components) into a single repository, enabling real-time monitoring and alerts. This helps teams quickly identify and address issues across the CI/CD pipeline.

- o **Log retention and archiving:** Create log retention policies to keep logs for the required duration, based on operational and compliance needs. Consider scalability, data lifecycle, and storage costs. Log archiving moves older logs to secondary storage in formats like gzip, database archives, or cloud storage to optimize storage.

- o **Log analysis:** Log analysis tools enable real-time monitoring of application logs, helping teams identify issues, exceptions, root causes, and performance concerns during the build, test, deployment,

and runtime stages. Popular tools include ELK Stack, Splunk, Graylog, Datalog, Fluentd, and Sumo Logic.

Conclusion

In this chapter, we covered building a CI/CD pipeline with Golang, including project structure, environment setup, integration, automated testing, and deployment strategies. These practices help streamline development, improve code quality, and ensure efficient deployments. In the next chapter, we will explore data analysis and machine learning in Go.

Questions

1. What are some CI/CD tools available?
2. What steps are included in CI pipeline?
3. How do you ensure comprehensive test coverage in your Golang project?
4. How do you manage dependencies and versioning in your Golang projects?
5. How do you ensure fast feedback and early detection of issues in your CI pipeline for Go projects?
6. How do you incorporate security checks and vulnerability scanning into your CI/CD pipeline?
7. Explain blue-green deployment strategy in CI/CD.

Join our book's Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

<https://discord.bpbonline.com>



CHAPTER 24

Wrap-up and Takeaways

Introduction

Congratulations on finishing the comprehensive journey through the Go programming language! We have covered a lot of information in this book, from the language's foundations to more advanced concepts. As you consider what you have learned, let us detour some key points and talk about strategies to grow as a Go developer going forward.

Structure

In this chapter, we will learn the following topics:

- Summary of key learnings
- Common pitfalls
- Practical applications
- Next step

Summary of key learnings

We started by looking over the fundamental syntax and features of Go, such as functions, packages, variables, data types, and control structures. After that, we explored more complex subjects like error handling, interfaces, and concurrency with goroutines and channels. We pointed out Go's simplicity,

efficiency, and elegance as a programming language across. Let us review the following main points:

- **Basics of Go:** Go's basic syntax and structure includes variables, data types, functions, loops, conditional statements, package management, etc. Building applications in Go requires an understanding of these fundamental ideas, which enables developers to take full advantage of Go's simplicity, parallelism, and scalability features.
- **Simple Syntax:** Go has an easy-to-read, user-friendly syntax that is straightforward and intuitive. It uses minimal punctuation, logical naming conventions, and integrated formatting standards to help developers produce clear, concise code efficiently.
- **Static typing:** Static typing in Go refers to the explicit declaration and enforcement of variable types during the compilation process. This method finds type-related issues early in the development process, which promotes robustness and decreases runtime errors.
- **Concurrency and Goroutines:** Go's ability to perform numerous tasks at once is known as concurrency, and it is made possible by lightweight threads known as goroutines. Safe and synchronized data exchange amongst concurrent processes is made possible by the channels via which these goroutines communicate.
- **Interfaces:** Interfaces in Go specify collections of methods that a type needs to have in order to comply with the interface. This encourages abstraction and polymorphism, allowing programs to interact with many types as long as they comply with the interface contract.
- **Performance:** Due to its compiled nature, effective garbage collection, and concurrency support through goroutines and channels, Go is well known for its performance. Due to its fast execution speeds and statically typed architecture, it is well-suited for developing high-performance applications, particularly those involving distributed systems, online services, and networking.
- **Error handling:** In Go, creating and managing error instances often entails utilizing the **errors** package from the standard library. The **panic** function in Go is a feature for handling errors that stop regular execution and trigger panic. Using the **recover** function inside a

deferred function, developers can recover from panics and perform cleanup and error-handling tasks in a smooth manner.

- **Testing:** Go's built-in package “**testing**” facilitates test creation and execution. Every test function starts with “**Test**” and it can be executed with the command “**go test**” in project directory. By ensuring code accuracy and streamlining automated testing operations, this method encourages **test-driven development (TDD)**.
- **Tool support:** With the ‘**go**’ command-line tool, Go provides extensive tool support, including the ability to build, test, format, and manage dependencies. Furthermore, tools like **golint** and **go vet** provide static analysis and code quality checks, and **gofmt** provides consistent code formatting. Its capabilities are further expanded by the robust ecosystem of third-party libraries and frameworks.

Common pitfalls in Go programming

Let us look at some common pitfalls:

- **Ignoring errors:** One of the most common mistakes in Go is failing to handle errors returned by functions. Always check and handle errors appropriately.
- **Misusing Goroutines:** Poor synchronization can cause race conditions or resource leaks. Ensure proper synchronization using **channels** or sync primitives like **WaitGroup** and **Mutex**.
- **Neglecting defer statements:** Forgetting to use defer for cleanup tasks like closing files or releasing resources can lead to resource leaks.
- **Misusing interfaces:** Declaring overly broad interfaces or using empty interfaces (interface{}) excessively can lead to loss of type safety and clarity. Define precise, meaningful interfaces for better code design.
- **Neglecting contexts:** When working with APIs or long-running operations, failing to use context. The context for timeouts and cancellations can result in poor resource management and unresponsive programs.

Avoiding these pitfalls helps you write clean, reliable Go code.

Practical applications

Learning Go programming language facilitates practical applications in many different fields, making it an excellent option for developers. Here are some examples:

- Go's quick execution speed and effective concurrency make it a good choice for developing high-performance web servers and microservices in web development.
- Strong packages for managing HTTP requests are included in its standard library, facilitating the development of RESTful APIs and online applications.
- In addition, Go is a popular choice for developing distributed systems and cloud-native apps; in fact, Go is used to write tools like Docker and Kubernetes. This is because of its simplicity and ease of deployment.
- Its adaptability for DevOps tooling, networking chores, and data processing also shows how widely applicable it is in the software development industry.

All things considered, learning Go offers a wide range of job options and helps programmers build scalable, dependable, and efficient software for a variety of sectors.

Next step

Your next move in Go will depend on your interests and career goals. You can learn about web programming with Go by creating microservices or full-stack web apps using frameworks like **Gin** or **Echo**. For those who enjoy working with data, use libraries such as **Gonum** to process data in Go. Finally, put your newly acquired knowledge into effect, get experience, and establish yourself as a competent Go developer & immerse yourself in real-world projects.

Conclusion

In this chapter, we revisited the key learnings from the book and some common pitfalls that developers encounter and provided strategies to avoid

them.

As you come to the end of your Go trip, keep in mind that learning is an ongoing process. Remain attentive and flexible, embrace challenges, and recognize accomplishments.

We appreciate your participation in this educational journey with us. We hope your efforts at Go programming continue to be fruitful. Keep going and freely code with Go.

Join our book's Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

<https://discord.bpbonline.com>



Index

A

- A/B testing [322](#)
- actual parameters [63](#)
- anonymous function [69](#)
- anonymous functions
 - returning [70](#), [71](#)
- anonymous struct fields [146](#)
- append function [112](#)
- arithmetic operators [44](#)
- array [31](#), [90](#)
 - declaring [90](#), [91](#)
 - initialization [91](#), [92](#)
 - iteration [96-98](#)
 - need for [92](#), [93](#)
- array of pointer [136](#)
- assignment operator [47](#)
- Atom [8](#)
- automation [271](#), [272](#)
- automation key areas [272](#)
 - CI/CD [288](#)
 - cloud services development [281-285](#)
 - container orchestration [275](#)
 - database automation [286-288](#)
 - Infrastructure as Code [272](#), [273](#)
 - Kubernetes Go [276](#)
 - network automation [285](#), [286](#)
 - testing [286](#)
- automation scripts [278](#)

B

- BBedit [8](#)
- bitwise operators [47](#)
- blank identifier [23](#), [68](#)
- blocking
 - avoiding [269](#)
- blue-green deployment [321](#), [322](#)
- Boolean [28](#)
- Boolean constants [36](#)
- branching statements [54](#), [55](#)

break statement [57](#)

C

canary deployment [322](#)

CD release process [321](#)

channel operations [199-202](#)

channel operator [197](#)

channels [34](#), [187](#), [197](#)

- buffered channel [200](#)

- closing [205](#), [206](#)

- declaring [198](#)

- direction [206-209](#)

- unbuffered channel [200](#)

Chef [273](#)

Chrome DevTools Protocol (CDP) [254](#)

Chromedp [254](#)

CI/CD

- automated testing, integrating [317-321](#)

- environment, setting up [314](#), [315](#)

- integration, in Go [315-317](#)

- logging [324](#)

- monitoring [322-324](#)

- overview [312](#)

circular imports, packages [83-85](#)

cli [85](#)

closures [71](#), [72](#)

Cloud9 [9](#)

cloud-based IDEs [9](#)

CodeEnv [9](#)

Colly [253](#)

- Collector [254](#)

- Event – Driven model [254](#)

- events [255](#)

- features [256](#)

- HTML parsing [254](#)

- HTTP requests [254](#)

- overview [254](#)

color [86](#)

compiler [9](#)

composition [154-156](#)

concurrency [188](#)

constants [35](#)

- Boolean constants [36](#)

- Iota constants [37](#)

- numeric constant [35](#), [36](#)

- string constant [36](#)

container orchestration

- features [275](#)

- platform [275](#), [276](#)
- context-switching [188](#)
- continue statement [58](#)
- Continuous Delivery (CD) [312](#)
- Continuous Deployment (CDep) [312](#)
- Continuous Integration (CI) [312](#)
- control structures [53](#), [54](#)
- cross-site scripting (XSS) [323](#)
- CRUD operations [287](#)
- custom package
 - creating [79-81](#)

D

- data [25](#)
- data analysis
 - overview [290](#)
- data handling [291-293](#)
- data manipulation [291](#), [295](#)
 - data cleaning [295](#)
 - data normalization [297-299](#)
 - data transformation [296](#)
- Data Plane Development Kit (DPDK) [85](#)
- data race [217](#), [218](#)
 - checking for [218](#), [219](#)
- data types [28](#)
 - non-primitive data type [30](#)
 - primitive data types [28](#)
- deadlock [202](#), [203](#)
- Deep Q-Networks (DQN) [306](#)
- defer keyword [73](#)
- defer statement [231-233](#)
- Density-Based Spatial Clustering of Applications with Noise (DBSCAN) [305](#)
- Dynamic Application Security Testing (DAST) tools [323](#)

E

- Eclipse IDE [8](#)
- else statement [54](#)
- Emacs [8](#)
- embedded structs [147](#)
- empty interface [168](#), [169](#)
- error handling [227](#)
- error package [230](#)
- exit statement [58](#)
- Exploratory Data Analysis (EDA)
 - overview [299-304](#)

F

- feature flags [322](#)
- Field function [244](#), [245](#)
- First-In-First-Out (FIFO) [291](#)
- Float() [245](#)
- float32 [29](#)
- floating-point data type [29](#)
- fmt [18](#)
- for loop [56](#)
- formal parameters [63](#)
- format specifiers [37](#)
 - common format specifiers [37](#)
 - for Boolean [38](#)
 - for float [38](#)
 - for integer [38](#)
 - for pointers [39](#), [40](#)
 - for string [39](#)
- func [19](#)
- function arguments [63](#), [64](#)
- function caller statement [238](#)
- functions [62](#)
 - anonymous functions [69](#), [70](#)
 - call by reference [66](#)
 - call by value [65](#), [66](#)
 - calling [64](#), [65](#)
 - declaring [63](#)
 - defer keyword [73](#)
 - main function [62](#)
 - named return values [67](#), [68](#)
 - recursive functions [72](#)
 - return types [63](#), [64](#)
 - using as type [69](#)
 - variadic function [73](#), [74](#)
- functions, of array
 - array comparison [94](#), [95](#)
 - array, copying [95](#)
 - array, passing [96](#)
 - length [94](#)
- functions, with strings
 - iteration [117](#), [118](#)
 - length [117](#)
- Fuzzy [86](#)

G

- Gc [9](#)
- Gccgo [10](#)
- Gedit [8](#)

- General Data Protection Regulation (GDPR) [324](#)
- Gitpod [9](#)
- glog [86](#)
- gnet [285](#)
- Go1.20 [8](#)
- GoBot [307](#)
- GoBrain [307](#)
- gocheck [86](#)
- go-cluster [307](#)
- go-expect [285](#)
- Go kit [85](#)
- Goland [8](#)
- Golang applications, with CI/CD pipelines
 - deployment strategies [321](#)
- Golang code
 - writing [276](#)
- Golang errors [228](#)
 - types [229](#)
- Golang machine learning libraries [306](#)
- Golang project structure [312](#)
 - folders and files [313](#), [314](#)
- Go language [1](#)
 - analogy with other languages [4](#), [5](#)
 - applications [5](#), [6](#)
 - common pitfalls [329](#)
 - evolution [2](#)
 - features [3](#), [4](#), [328](#)
 - installing [10](#)
 - limitations [5](#)
 - Linux/macOS X and FreeBSD installation [15](#)
 - object-oriented language [4](#)
 - practical applications [329](#)
 - scrap websites [256-262](#)
 - system requirements [8](#)
 - uninstalling [15](#)
 - Windows installation [10-15](#)
- Golearn [306](#)
- GoML library [306](#)
- go-netconf [286](#)
- GOnpp plugin [8](#)
- Gonum [306](#)
- goose package [253](#)
- goPacket [285](#)
- GOPATH [79](#)
- Go program [18](#), [19](#)
- GoQuery [253](#)
- Gorgonia [307](#)
- GORM [85](#)

GOROOT [79](#)
Goroutines [187-192](#)
 at multiple cores [194](#)
 waiting to finish [194-196](#)
GoScraper [256](#)
gRPC Network Management Interface (gNMI) [286](#)

H

headless scraping [265-269](#)
Health Insurance Portability and Accountability Act (HIPAA) [324](#)
Hector library [307](#)

I

identifiers [23](#)
 blank identifier [23](#), [24](#)
if statement [54](#)
Infrastructure as Code [272](#)
 with Golang [273](#), [274](#)
Int() [245](#)
integer data types [28](#)
interfaces [163](#), [164](#)
 as polymorphism [174](#), [175](#)
 declaring [164](#), [165](#)
 embedding [173](#), [174](#)
 implementing [165-168](#)
Interquartile Range (IQR) [300](#)
Iota constants [37](#)
ioutil package [293](#)

J

JavaScript [273](#)
Jenkins [314](#)
jumping statements [57](#)

K

keywords [20](#), [21](#)
Kubernetes [86](#)
Kubernetes Go [276](#), [278](#)
Kubernetes Go Client Library
 installing [276](#)

L

lambda [69](#)
Last In, First Out (LIFO) [232](#)
Last In First Out order [73](#)

- laws of reflection [246-249](#)
- libSVM [307](#)
- Local Outlier Factor (LOF) [305](#)
- Lock() [223](#)
- logging, in CI/CD
 - integration with logging solutions [324](#)
 - log analysis [325](#)
 - log retention and achieving [324](#)
 - structured and centralized logging [324](#)
- logical operators [46](#)
- looping statements [56](#)

M

- machine learning [290](#)
 - application [290](#), [291](#)
- machine learning ecosystem [304](#)
 - reinforcement learning [305](#), [306](#)
 - supervised learning [304](#), [305](#)
 - unsupervised learning [305](#)
- map operations
 - element, deleting [182](#), [183](#)
 - iteration [181](#), [182](#)
 - keys [180](#), [181](#)
 - length [180](#)
- maps [33](#), [34](#), [178](#)
 - as reference type [184](#), [185](#)
 - comparison [183](#)
 - declaring [178](#)
 - initializing [178-180](#)
 - uses [178](#)
- Micro [9](#)
- miscellaneous operator [49](#)
- monitoring, in CI/CD
 - alerting and notification [323](#)
 - application monitoring [323](#)
 - automated security testing [323](#)
 - custom metrics and dashboards [323](#)
 - incident management systems [323](#)
 - infrastructure monitoring [323](#)
 - security monitoring and compliance [323](#)
- multidimensional array [93](#)
- multiline string [120](#), [121](#)
- multiple Goroutines [192](#), [193](#)
 - using [204](#), [205](#)
- multiple interfaces
 - implementing [169](#), [170](#)
- Mutex [219-221](#)
- mutual exclusion lock [219](#)

N

- natural language processing (NLP) [291](#)
- nested package [82](#)
- nested pointers [135](#), [136](#)
- nested structs [147](#)
- network automation [285](#)
- network management system (NMS) [286](#)
- non-primitive data type [30](#)
 - array [31](#)
 - channels [34](#)
 - maps [33](#), [34](#)
 - pointer [30](#)
 - slice [32](#), [33](#)
 - structures [31](#), [32](#)
- Notepad++ [8](#)
- numeric constant [35](#)
- numeric data type [28](#)
- NumField() method [243](#), [244](#)

O

- Open Relational Mapping (ORM) library [287](#)
- Open Web Application Security Project (OWASP) [324](#)
- operator precedence [50](#)
- operators [43](#), [44](#)
 - arithmetic operators [44](#)
 - assignment operators [47](#)
 - bitwise operators [47](#)
 - logical operators [46](#)
 - miscellaneous operator [49](#)
 - relational operators [45](#)
 - types [44](#)
- Optimize.Minimize function [309](#)

P

- packages [76](#)
 - circular import [83](#), [84](#)
 - custom package, creating [79-81](#)
 - fmt package [76](#), [77](#)
 - importing [77-79](#)
 - main package [76](#)
 - nested packages [82](#)
 - third-party package, installing [82](#), [83](#)
 - top Go libraries [85](#), [86](#)
- Panic function [234-236](#)
- parallelism
 - versus concurrency [188](#)

- Payment Card Industry Data Security
 - Standard (PCI DSS) [324](#)
- Personally Identifiable Information (PII) [253](#)
- Pholcus [253](#)
- pointer embedding [159](#)
- pointers [132](#)
 - creating, with new [134](#), [135](#)
 - declaring [133](#)
 - dereferencing [134](#)
 - initializing [133](#), [134](#)
 - manipulating [134](#)
 - need for [132](#)
 - nested pointers [135](#), [136](#)
 - passing, to function [136-138](#)
- polymorphism [163](#), [164](#)
- primitive data types [28](#)
 - Boolean [28](#), [29](#)
 - float32 [29](#)
 - floating [29](#)
 - integer [28](#)
 - numeric [28](#)
 - String [30](#)
- Principal Component Analysis (PCA) [305](#)
- process automation [278-280](#)
- Pulumi [273](#)
- Python [273](#)

R

- race condition [218](#)
- range keyword
 - using for iteration [119](#), [120](#)
- recover function [237](#), [238](#)
- recursive functions [72](#)
- reflection [239](#), [240](#)
- reflect package [240](#)
 - kind [242](#), [243](#)
 - value [242](#)
- reinforcement learning [305](#), [306](#)
- relational operators [45](#)
- Resty [285](#)
- RLock() [223](#)
- rolling deployment [322](#)
- rune data type [119](#)
- RUnlock() [223](#)
- RWMutex [223-225](#)

S

- scraped data
 - storing, in CSV [262](#), [263](#)
- scrap websites [256-260](#)
- select keyword [209-216](#)
- slice functions [106-108](#)
 - slice, comparing [110](#)
 - slice element, appending [112](#), [113](#)
 - slice, reversing [111](#), [112](#)
 - slice, searching [110](#), [111](#)
 - slice, sorting [108](#), [109](#)
- slice of bytes
 - converting, to string [121](#)
- slice of runes
 - converting, to string [121](#)
- slice of structs
 - embedding [160](#), [161](#)
- slices [32](#), [33](#), [99](#)
 - declaring [99](#)
 - extending [105](#)
 - from array [99](#), [100](#)
 - from existing slice [100](#), [101](#)
 - from structs [101](#)
 - initializing [99](#)
 - internals [102](#), [103](#)
 - make function, using [104](#)
 - two-dimensional slices [105](#)
- software-defined networking (SDN) [285](#)
- software essence [2](#)
- String() [245](#)
- string constant [36](#)
- strings [30](#), [116](#), [117](#)
 - functions [117](#)
 - immutable nature [122](#), [123](#)
 - slice of bytes, converting to [121](#)
 - slice of runes, converting to [121](#)
- strings package [124](#)
 - functions [124-129](#)
- struct [140](#)
 - anonymous struct fields [146](#)
 - comparing [150](#), [151](#)
 - copying [149](#)
 - declaring [140-142](#)
 - embedded structs [147](#)
 - field, accessing [143](#)
 - initializing [142](#)
 - nested structs [147](#)
 - passing, to functions [149](#)
 - pointer to struct [144](#)

- with new function [145](#)
- struct embedding [159](#)
- structures [31](#), [32](#), [139](#), [140](#)
- Sublime Text [9](#)
- supervised learning [304](#)
- switch statement [55](#)
- System and Organization Controls 2 (SOC 2) [324](#)

T

- t-Distributed Stochastic Neighbor Embedding (t-SNE) [305](#)
- Terraform [273](#)
- Testify [86](#)
- text editors [8](#)
- third-party package
 - installing [82](#), [83](#)
- thread [188](#)
- time series analysis [307-309](#)
- two-dimensional slices [105](#)
- type assertion [170](#), [171](#)
- typeof function [241](#)
- TypeScript [273](#)
- type switch [171-173](#)

U

- Unlock() [223](#)
- unsupervised learning [305](#)
- UTF- [116](#)

V

- variables [26](#)
- variadic functions [73](#), [74](#)
- Vegeta [86](#)
- VI improved (Vim) [9](#)
- Visual Studio [9](#)

W

- web scraping [251](#), [252](#)
 - automation [253](#)
 - data extraction [252](#)
 - data storing [253](#)
 - data transformation [252](#)
 - HTML parsing [252](#)
 - libraries [253](#), [254](#)
 - web request [252](#)
- Wide [9](#)